

回転機構のレーザー受光と楽器間無線 通信による同期・輪唱システム

情報工学科 2 年

24G1059

佐藤 涼菜

—— チーム 40 ——

宇井 祐真 (24G1021) 梅野 大誠 (24G1026)

山口 侑希 (24G1136) 薄井 悠希 (25G1020)

菊池 侑人 (25G1041)

2026 年 7 月 11 日

1 はじめに

1.1 社会的背景・課題・本稿の目的

近年、動画配信サイトやストリーミングサービスの普及によって、誰もが手軽に音楽コンテンツを視聴できる環境が整っている。特に日本では、ゲームやアニメなどのサブカルチャーを起点とした音楽コンテンツが数多く生み出されており、独自の文化として定着している。一方で、このようなデジタル配信の消費形態は、実際に会場へ足を運んで演奏を体感するライブイベントやコンサートとの相乗効果を生み、市場規模の拡大にも寄与している [1][2]。たとえば、博報堂の谷口による分析では、ストリーミングサービスの利用者がサービス内で新たに好みのアーティストを複数発掘し、そのアーティストたちを一度に鑑賞できるライブやフェスへ参加するという行動の流れが形成されつつあり、さらにライブでの体験を通じて別のアーティストと出会うことで、再びストリーミングでの視聴に戻るという音楽消費サイクルが生まれていると指摘されている [4]。すなわち、デジタル配信とライブイベントは互いの顧客を送客し合う関係にあり、この循環が音楽市場全体の拡大を後押ししていると考えられる。実際に、一般社団法人コンサートプロモーターズ協会が実施したライブ市場調査によれば、2025 年におけるライブ市場の年間売上高は 6443 億円、入場者数は 5999 万人にのぼり、デジタル配信の普及以降もライブ市場は堅調に拡大を続けている [3]。図 1 に示すように、国内のライブ・コンサート市場における年間売上額は 2010 年の 1280 億円から 2019 年の 3665 億円まで、10 年間でおよそ 2.9 倍に拡大している。2020 年には新型コロナウイルス感染症の影響により、会場に人を集めること自体が困難となった結果、年間売上額は 779 億円まで急落した。しかし、行動制限の緩和とともに市場は急速に回復し、2022 年には 3984 億円、2023 年には 5140 億円、そして 2025 年には過去最高となる 6443 億円を記録している。この推移から、ライブ会場に足を運ぶという体験に対する需要は、一時的な外的要因によって落ち込むことはあっても、長期的には一貫して拡大する傾向にあると言える。

デジタル配信が普及していく中で、なぜデジタル配信では代替できない価値がライブ会場に存在するのだろうか。その理由の一つとして、ライブ演出に用いられる音響や照明、ライブ参加者自身が感じる振動といった複数の物理的刺激が挙げられる。これらの刺激は、観客一人ひとりに個別に届けられるのではなく、同じ空間にいる観客全員に同時に共有されるという性質を持つ。その結果、観客同士が同じ体験を共有しているという感覚、すなわち空間全体としての一体感が形成される。換言すれば、ライブ会場における没入感とは、ユーザが音楽を聴くだけでとどまらない、物理的、視覚的な技術と同じ空間で同時に受け取ることによって生まれる体験だと考えられる。この仮説は、複数の研究によっても裏付けられている。ライブ演奏とデジタル配信を直接比較した実証研究によっても裏付けられている。Kreuzer ら (2025) は、同一のクラシック音楽コンサートを、会場で聴取する条件と映像配信で視聴する条件とで比較する実験を行った。その結果、没入感や社会的つながりの感覚を含む複数の評価軸において、ライブ条件の方が配信条件よりも有意に高い評価を得ることを示した [5]。すなわち、同じ演奏内容・同じ音質であっても、同じ空間を他の観客

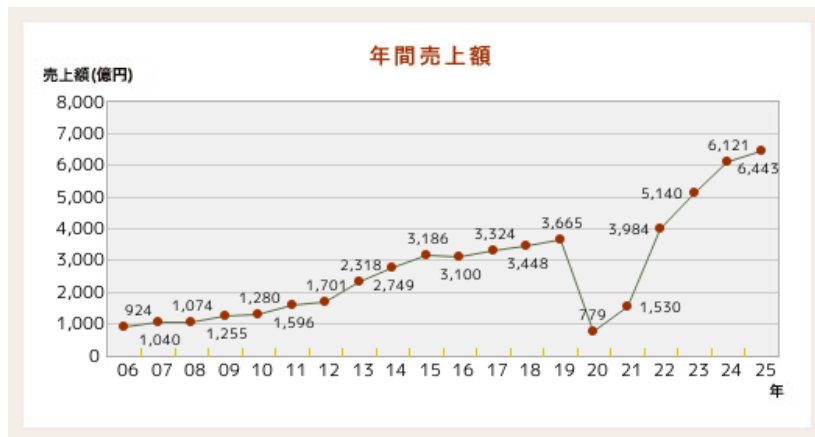


図1 国内ライブ・コンサート市場における年間売上額の推移（一般社団法人コンサートプロモーターズ協会「基礎調査推移表」より）

と共有しているという感覚そのものが、没入感を左右する独立した要因であり、これは録画・配信技術の高度化だけでは補えないことを意味する。したがって、音そのものの再現性だけでなく、ライブ体験に固有の空間的フィードバックを組み込むことで没入感の高い演奏体験の実現に有効であると考えられる。

また、Natalia Cooper ら（2018）は、仮想現実（VR）環境における体験の再現度を高めるため、視覚情報のみでは表現が難しい感覚を、音や触覚など別の感覚を用いた代替フィードバックとして提示する実験を行った。その結果、視覚刺激のみを提示した場合と比較して、音や触覚といった複数の異なる刺激を組み合わせると提示した場合の方が、利用者の没入感・関与感が有意に高まることを明らかにした [6]。つまり、複数の感覚情報を同時に提示する多感覚的フィードバックは、ユーザの体験における没入感の向上に有効であると考えられる。また、Human-Computer Interaction（HCI）分野の知見も同様の傾向を支持している。Martin ら（2024）は、ライブコンサート環境を対象とした研究において、観客が演奏を受動的に聴取するだけでなく、能動的に体験へ参加することが、会場全体の活気や観客の感情の高まりに寄与すると結論づけた [7]。よって、ユーザ自身が能動的に関与できる仕組みもまた、システムの没入感を高める要因として重要であると考えられる。

一方、音楽を自動で演奏する自動演奏技術の分野では、デジタル技術の発展により高精度な同期や楽曲再現が可能になっている。たとえば、Yamaha が提供する Disklavier ENSPIRE シリーズは、光学センサとサーボモータを用いて演奏情報を高精度に再現するとともに、鍵盤が実際に動作する様子をユーザへ視覚的にフィードバックを行う機能を備えている [8]。しかし、この視覚的フィードバックは、ピアノの鍵盤という限られた範囲でのみ提示されるものであり、その場に居合わせたユーザ1人のみが確認できる情報にとどまる。したがって、複数人が同じ空間にいる状況を想定すると、自動演奏技術によって提示される視覚的フィードバックは、空間全体には行き渡らないという限界を抱えていると言える。

以上を踏まえると、現状の自動演奏技術は、演奏そのものの精度は高い一方で、前述した「多感覚的フィードバック」と「空間全体への共有」という、没入感を高めるうえで重要な要素を十分に

は満たせていないという課題が生じる。特に、Kreuzer らの知見が示すとおり、同じ場を共有しているという空間的なフィードバックは、音質や演奏精度をいくら高めても代替できない、ライブ体験に固有の要素である。したがって、デジタル配信や高精度な自動演奏技術がどれほど発展しても、この空間的フィードバックを欠いたままでは、ライブ会場に匹敵する高い没入感を得ることはできないと結論づけられる。そこで本稿では、自動演奏技術に、ライブ体験に共通するこの空間的フィードバックを付加した楽器間通信システム「回転機構のレーザー受光と楽器間無線通信による同期・輪唱システム」を提案する。本システムは、ユーザの動作を検知する指揮デバイス、指揮デバイスからの情報に応じ回転する観覧車型の中継機デバイス、そして中継機からのレーザー光を受光し演奏を行う楽器デバイスによって構成される。指揮デバイスによるユーザの能動的な動作、観覧車型デバイスによる空間全体への視覚的なフィードバックの共有、そして高精度な音色の演奏による聴覚的な刺激を組み合わせることで、従来の自動演奏技術では実現が難しかった、会場全体を巻き込む没入感の高い演奏体験の提供を目指す。

1.2 類似技術・先行研究

本節では、本システムの独自性を明確にするため、音楽に視覚的なフィードバックを付加する既存技術を整理し、それぞれの特徴と限界を検討する。

第一に、DMX512 信号（以下、DMX 信号と表記する）を用いた照明制御システムが挙げられる。DMX 信号とは、舞台照明の操作卓と調光機との間でデータをやり取りするための通信規格であり、現在では舞台演出やコンサートにおいて、音楽に合わせた光量制御を行う目的で広く利用されている [9]。このシステムは、音楽という聴覚的な刺激に加えて光という視覚的な刺激を提示することで、観客の没入感を高めることができる。しかし、DMX 信号による照明制御は、あらかじめプログラムされた演出を再生する仕組みであるため、観客はその光の変化を一方的に受け取る、すなわち受動的に認識する立場にとどまる。前節で述べたとおり、能動的な関与が没入感を高める要因の一つであることを踏まえると、観客が演奏そのものに関与できない DMX 信号による演出には限界があると言える。この課題に対し、本提案では指揮デバイスを導入し、利用者が指揮という身体動作を通じて演奏そのものを能動的に制御できるようにすることで、身体的な関与に基づく没入感の向上を狙う。

第二に、物理的な操作によって音楽を制御するタンジブル・インターフェースの例として、ReacTable が挙げられる。ReacTable は、卓型のディスプレイ上に複数の物理モジュールを配置し、その配置や動きに応じて音を合成するシンセサイザの一種であり、利用者は身体的な物理操作を通じて演奏を制御するとともに、ディスプレイに表示される視覚的な情報によって演奏状態のフィードバックを受け取ることができる [10]。しかし、この視覚的フィードバックはディスプレイという限られた表示領域内で完結しており、操作を行う利用者本人以外の第三者や、会場全体へフィードバックを共有する仕組みとしては設計されていない。そこで本提案では、指揮デバイスに加えて観覧車型の中継機デバイスを導入する。観覧車型デバイスが演奏情報に応じて回転する様子を、利用者本人だけでなく周囲にいる第三者も視認できるようにすることで、演奏状態の把握や共有を空間全体に広げ、多感覚的なフィードバックを実現することを狙う。

以上のように、DMX 信号による照明制御は観客の能動的な関与を欠き、ReacTable のような既存のタンジブル・インターフェースは視覚的フィードバックが個人の操作範囲内にとどまるという、それぞれ異なる限界を抱えている。本提案は、指揮デバイスによる能動的な演奏制御という既存研究の知見と、観覧車型デバイスによる空間共有性という要素を組み合わせることで、これら二つの課題を同時に解決し、空間全体への多感覚的なフィードバックを通じて、利用者の没入感および関与感を高めることを目指す。

2 作成したシステムの概要

本節では、作成したシステムの概要について説明する。本システムは、ユーザの能動的な動作と連動した観覧車型中継機デバイスの回転およびレーザーの発光を通じて、演奏空間全体に多感覚なフィードバックを与えることで、従来の自動演奏技術における演奏インターフェースでは得られない高い没入感をユーザに提供することを目的として設計されている。本システムは、指揮デバイス、観覧車型の中継機デバイス、楽器デバイスという3つのデバイスから構成される。演奏開始後、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、中継機デバイスが

ここからは各デバイスの概要について説明する。

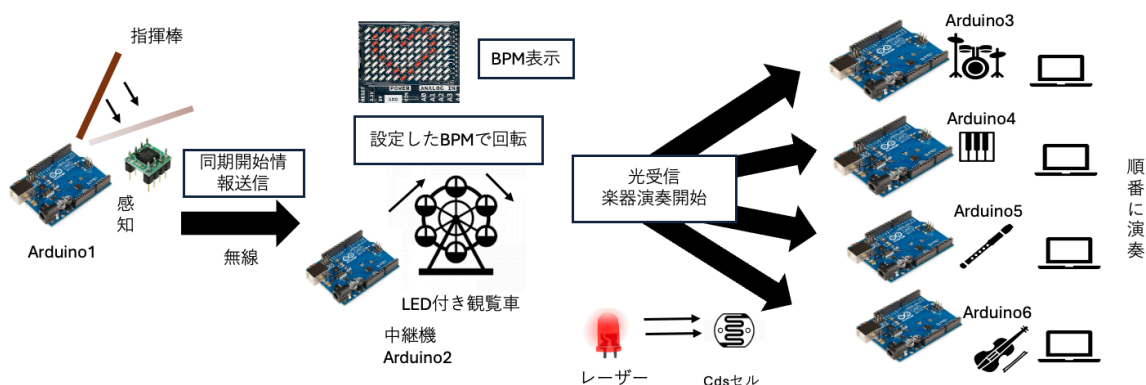


図2 全体の構成図

2.1 指揮デバイス

2.1.1 概要

まず、指揮デバイスについて説明する。指揮デバイスのシステム構成図を図3に示す。本デバイスは、演奏全体を統括するデバイスであり、主に以下の3つの機能を有する。

第一の機能は、演奏開始トリガーの送信である。本機能の設計について図4に示す。ユーザが指揮デバイスに搭載されたタクトスイッチを押下すると、中継機デバイスと楽器デバイスへ演奏開始トリガーを送信する。さらに、中継機デバイスの回転が開始される。

第二の機能は、楽曲のBPM変更である。本機能の設計について図5に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると新しい設定BPM情報が中継機デバイスへ送信される。これにより、中継機デバイスのモータ回転速度がこの値に追従して変化する。そして、その回転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、BPMの変更される。また、楽曲のBPMは5段階に分けて変更できる。

第三の機能は、楽曲の音量変更である。本機能の設計について図6に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると音量情報が各楽器デバイスへマルチキャスト送信されることで、音量が変更される。また、BPMと同様に音量も5段階に分けて変更できる。ここからは、外部設計と内部設計について説明をする。

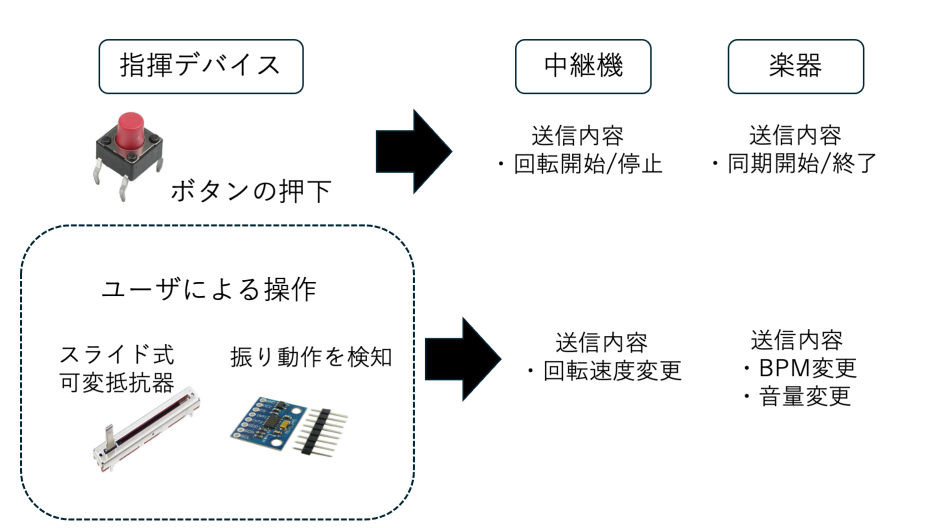


図3 指揮デバイスのシステム構成図

2.1.2 外部設計

本節では指揮デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表1に示す。デジタル3軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器はBPMおよび音量変更に使用される。当初はデバイスの電力供給源として9Vのアルカリ電池を外部電源として使用することを検討していたが、電池なしでも正常動作が確認できたため、軽量化のためPCとのUSB接続から電力を供給するように変更した。Arduino内蔵の電圧レギュレータにより5Vおよび3.3Vを生成し、各モジュールへ安定した電力を供給する。また、指揮デバイスの回路図を図7に示す。加速度センサにはGY-291（ADXL345）を用いており、I2C通信によりArduinoと接続する。CSピンをArduinoの3.3Vに接続することでI2Cモードに設定し、さらにSDOピンをGNDに接続することでI2Cアドレスを0x53に固定している。また、SDAピンおよび

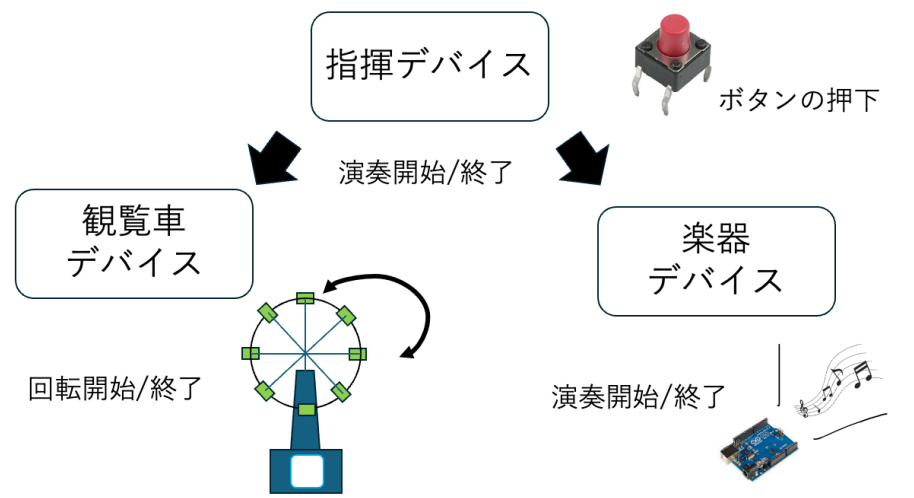


図 4 演奏開始トリガー送信機能設計

SCL ピンはそれぞれデータ通信線およびクロック信号線として Arduino の A4, A5 ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1 ピンから Arduino の D2 ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用 Arduino との UDP 通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため 10k Ω のプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピン D3 はプルアップ抵抗を介して 5V に接続されるため HIGH 状態となる。一方、ボタン押下時には回路が GND に短絡され、入力電位が LOW に遷移する。指揮デバイスのプログラムでは D3 ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

表 1 指揮デバイスの構成部品

Arduino UNO R4 WiFi	-	1
スライド式可変抵抗器 10 k Ω	-	2
デジタル 3 軸加速度センサ	GY-291	1
タクトスイッチ	DTS-63	1
抵抗器 10k Ω	-	1
ジャンパーワイヤー	-	適宜

図～～～に実際に作成した指揮デバイスを示す。実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用した。また、内部にはブレッドボードに加速度センサ、スライド式可変抵抗器、タクトスイッチを設置する。デバイスのユーザビリティを向上させるため、ユーザが操作するスライダー部分やボタンは外部へ露出させる構造を採用し、さらにデバイスの先端部に加速度センサを設置することで操作時の重心を安定させる。

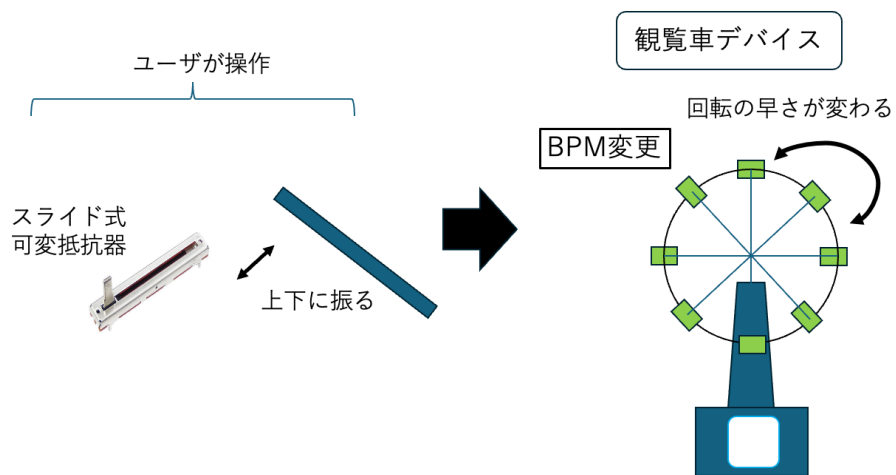


図 5 BPM 変更機能設計

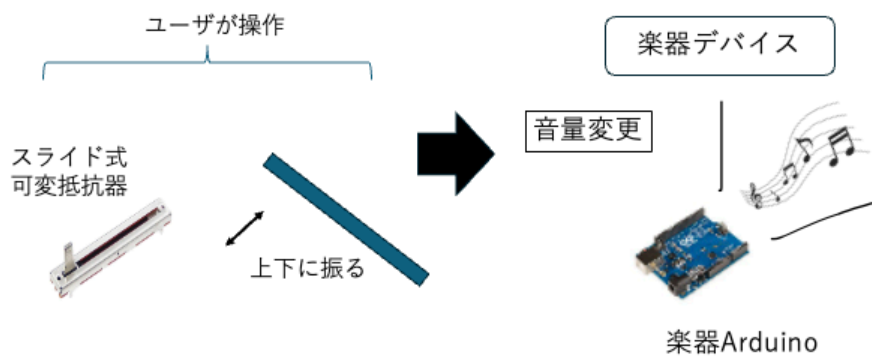


図 6 音量変更機能設計

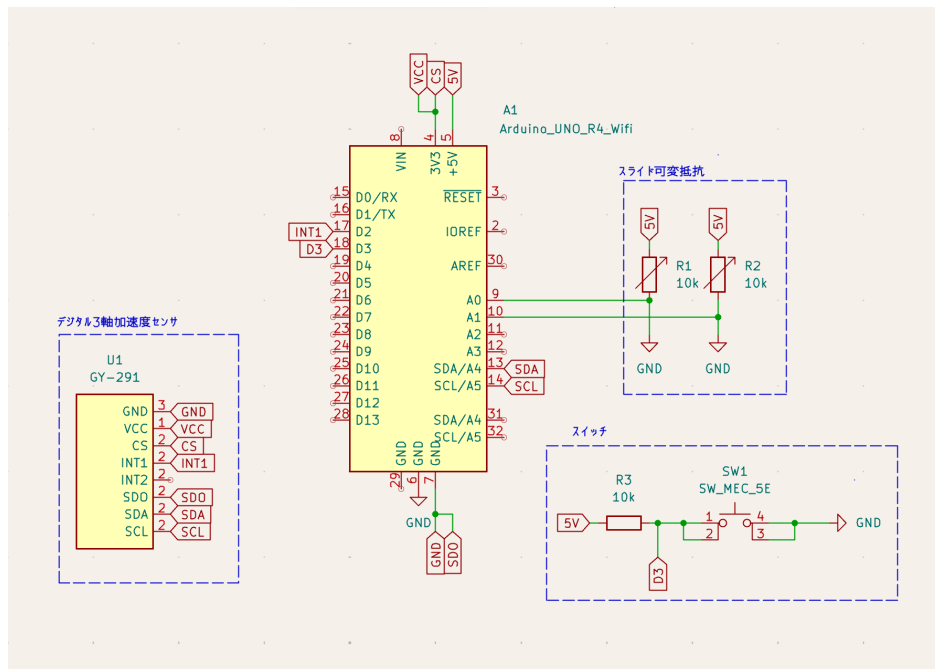


図 7 指揮デバイスの回路図

2.1.3 内部設計

本節では、指揮デバイスの内部設計について説明する。まず、指揮デバイス全体の内部動作を明確にするため、シーケンス図を図 8 に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知し Arduino に送信される。この時、値が閾値を超過した場合、観覧車デバイスと無線通信を開始し観覧車デバイスが送信した演奏情報を基に回転する。また、ボタンを押下した時、各楽器デバイスへ楽曲の初期 BPM および音量情報が送信されることで演奏が開始される。

次に、指揮デバイスの制御プログラムのファイル構成を示す。表 2 の通り、各ファイルは機能ごとにクラス化されている。

表 2 指揮デバイスのファイル構成

ファイル名	概要
Controller_simultaneous.ino	setup/loop による全体制御，ボタン入力の判定
AccManager.h / AccManager.cpp	加速度センサ (ADXL345) の初期化，振り動作の検知，I2C バス復旧処理
PotManager.h / PotManager.cpp	スライド式可変抵抗器 2 系統の移動平均処理および BPM・音量段階の判定
SyncManager.h / SyncManager.cpp	Wi-Fi/UDP 通信の初期化，各デバイスへの冗長送信処理

Controller_simultaneous.ino の loop 関数は、ボタン判定を行う button 関数，可変抵抗器を処理する PotManager，加速度センサを処理する AccManager を毎ループ順に呼び出す構成となっている。loop 関数をコード 1 に示す。プログラム全文は付録に示す。

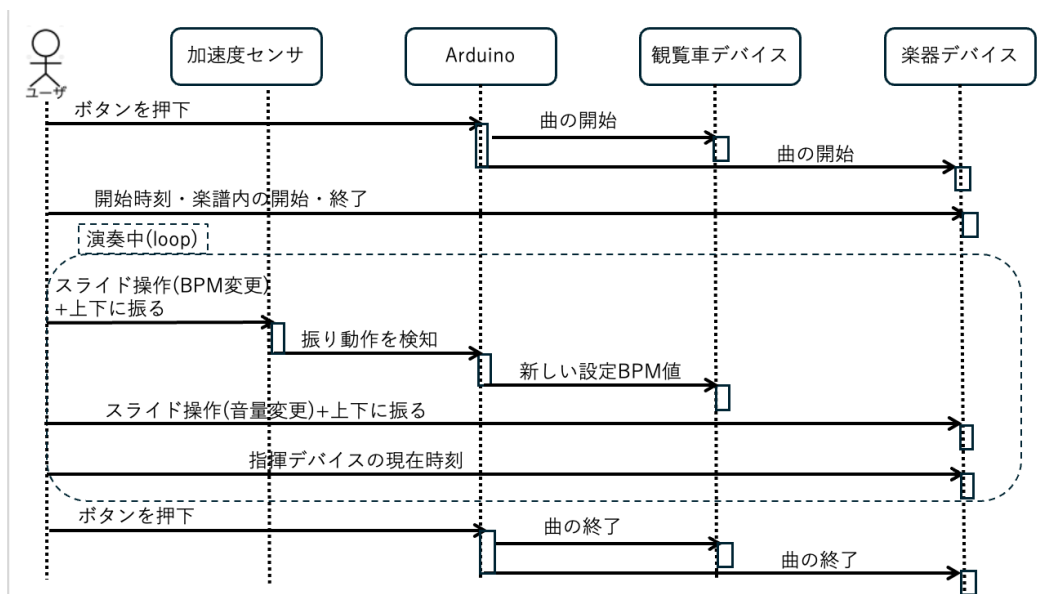


図 8 指揮デバイスのシーケンス図

コード 1 Controller_simultaneous.ino の loop 関数

```

1 void loop() {
2     button();
3     pot.update();
4     acc.update(sync, pot);
5 }

```

ここからは、ボタン入力の判定、加速度センサの制御、可変抵抗器を用いた BPM・音量読み取り機能の順に、使用する変数・関数の仕様と処理内容について説明する。

まず、指揮デバイスにおけるボタン入力の判定処理について説明する。ボタンの誤判定を防ぐため、プルアップ抵抗を用いた回路を構成しており、ボタンが押されていない状態ではデジタルピンに 5V が印加され、押下時には回路が GND に短絡されることでピンの電位が LOW へ遷移する。指揮デバイスのプログラムでは、このデジタルピンの電圧状態の変化を常時監視することでボタンの押下を判定する。さらに、チャタリングによる誤判定や冗長パケットによる誤動作を防止するため、押下検知後に一定時間の待機処理を設けるとともに、演奏状態管理フラグ (isButtonPlaying) を用いたトグル制御を導入している。これにより、同一の物理ボタンのみで演奏の開始と終了を交互に切り替えることが可能となる。ボタン入力の判定で使用する変数の仕様を表 3 に示す。

表 3 ボタン入力の判定で使用する変数の仕様一覧

変数名	型	説明
input_PIN	const uint8_t	ボタンの状態を読み取るデジタルピン番号を定義
switch_status	bool	現在読み取ったボタンピンの状態を格納
before_switch_status	bool	前回読み取ったボタンピンの状態を格納（立下り検知に使用）
isButtonPlaying	bool	演奏状態管理フラグ。トグル制御により演奏中(true)／停止中(false)を保持

続いて、ボタン入力の判定で使用する関数の仕様を表 4 に示す。

表 4 ボタン入力の判定で使用する関数の仕様一覧

関数名	戻り値	引数	説明
button	void	なし	input_PIN の立下り（switch_status が 0 かつ before_switch_status が 1）を検知すると isButtonPlaying をトグルし、開始時は現在の BPM 段階番号を付与して SyncManager の sendStart 関数を、終了時は sendEnd 関数を呼び出すことで、各楽器機・観覧車デバイスへ演奏開始・終了信号を送信する。送信後は 300 ms の待機によりチャタリングを防止する。

button 関数はこの 1 関数のみで構成されるため、以下では処理の流れに沿って実際のコードを 2 箇所に分けて説明する。まず、立下りエッジの検知部分について説明する。switch_status に現在のピン状態を読み込み、前回値 before_switch_status と比較することで、HIGH → LOW へ変化した瞬間（ボタンが押された瞬間）のみを検知する。該当部分をコード 2 に示す。

コード 2 button 関数（立下りエッジの検知部分）

```

1   switch_status = digitalRead(input_PIN);
2   if (switch_status == 0 && before_switch_status == 1) {
3       ...
4   }
5   before_switch_status = switch_status;

```

次に、立下りを検知した際の状態トグルと送信処理について説明する。isButtonPlaying が true（演奏中）であれば sync.sendEnd() を呼び出して演奏を終了し、false（停止中）であれば現在の BPM 段階（pot.getBpmStep()）を引数として sync.sendStart() を呼び出して演奏を開始する。

いずれの場合も `isButtonPlaying` の値を反転させたうえで、`delay(300)` により 300 ms 待機することでチャタリングを防止する。該当部分をコード 3 に示す。

コード 3 button 関数（状態トグルと送信処理部分）

```
1  if (isButtonPlaying) {  
2      sync.sendEnd();  
3      isButtonPlaying = false;  
4  } else {  
5      uint8_t step = pot.getBpmStep();  
6      sync.sendStart(step);  
7      isButtonPlaying = true;  
8  }  
9  delay(300); // チャタリング防止
```

なお、`sendStart・sendEnd` の内部では、`SyncManager` クラスが複数台のデバイスへラウンドロビン方式による冗長送信を行っている。この通信処理の詳細な実装は付録に示すプログラム全文を参照されたい。

次に、指揮デバイスにおける加速度センサの制御処理について説明する。指揮デバイスは、加速度センサを用いてユーザが指揮棒を振ったことを検知し、演奏情報の更新を行う。センサ値の急激な変化を読み取ることで動作を判定し、誤検知防止機能を設けることで安定した動作を実現している。処理の流れは、差分算出と判定、多重検知の防止、状態の反転、事後処理の 4 段階に大別される。まず差分算出と判定では、現在の合成加速度 `currentAccVal` と前回値 `lastAccVal` との差の絶対値を求める。この差分が検知閾値 `accThreshold` を超過し、かつ前回検知から `shakeInterval` 以上の時間が経過している場合に、指揮棒を振ったと判定する。多重検知の防止では、一度振る動作を検知すると `lastDetectTime` を現在時刻に更新し、以降 `shakeInterval` が経過するまでは加速度の変化を無視することで、一連の動作による複数回の誤検知を防止する。状態の反転では、振る動作を検知するたびに `isStarted` の bool 値を反転させる。事後処理として、次回の判定に備えて `currentAccVal` の値を `lastAccVal` へ代入し、前回のサンプリング時刻を更新する。また、振る動作の確定時には、可変抵抗器側で設定値の変更を検知したフラグ `bpmFrag` または `volFrag` (2.1.3 項で説明する) が有効になっている場合、`SyncManager` クラスを介してその情報の送信指示を行う。さらに、I2C 通信の異常を検知した場合には自動復旧を行う機能を備えている。加速度センサの制御で使用する変数の仕様を表 5 に示す。

表 5 加速度センサの制御で使用する変数の仕様一覧

変数名	型	説明
currentAccVal	float	加速度センサから算出した合成加速度の値を格納
lastAccVal	float	変化量算出のための直前の加速度センサ値を格納
accThreshold	const float	振ったと判断するための変化量の閾値を定義
lastDetectTime	unsigned long	前回振ったことを検知した時間を ms 単位で格納
shakeInterval	const unsigned long	連続検知を防止するための待機時間を定義
lastSampleTime	unsigned long	前回センサ値をサンプリングした時間を ms 単位で格納
ACC_SAMPLE_INTERVAL	const unsigned long	加速度センサ値をサンプリングする周期を定義
isStarted	bool	楽器の演奏状態を保持 (true: 演奏中/false: 停止中)
ADXL345_ADDR	const uint8_t	加速度センサ (ADXL345) の I2C アドレスを定義
REG_POWER_CTL	const uint8_t	加速度センサの電源管理レジスタのアドレスを定義
REG_DATA_FORMAT	const uint8_t	測定範囲設定用レジスタのアドレスを定義
REG_DATAX0	const uint8_t	測定値が格納される先頭レジスタのアドレスを定義
lastRecoverMs	unsigned long	I2C 復旧処理の連続実行を防ぐための最終復旧時刻を保持

続いて、加速度センサ制御 (AccManager クラス) で使用する関数の仕様を表 6 に示す。

表 6 加速度センサ制御 (AccManager クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
setupADXL345	void	なし	ADXL345 の電源投入と測定レンジの設定を行い、初期の加速度値を lastAccVal に格納する初期化関数
readAccMagnitude	float	なし	ADXL345 から X/Y/Z 軸の生の値を取得し、2 乗和の平方根を返す。I2C 通信異常を検知した場合は直前の加速度値を返す
recoverI2C	void	なし	I2C バスのロックアップ異常を検出した場合に、SCL 線のクロッキングによるバスクリアと STOP 条件の生成、Wire およびセンサの再初期化によって通信を復旧する
update	void	SyncManager& sync, PotManager& pot	ACC_SAMPLE_INTERVAL ごとに加速度値を取得し、差分算出・多重検知防止判定・状態の反転・事後処理までの一連の振る動作検知処理を行う。loop 内で毎回呼び出す
updateShakedInfo	void	SyncManager& sync, PotManager& pot	振る動作の確定時に呼び出され、PotManager の bpmFrag / volFrag が立っている場合に SyncManager 経由で BPM・音量の段階番号を送信する

以下では、表 6 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、setupADXL345 関数について説明する。この関数は、Wire.begin による I2C 通信の開始後、ADXL345 の POWER_CTL レジスタに Measure ビットを書き込むことでセンサを測定モードへ移行させ、DATA_FORMAT レジスタを初期値のまま設定する。setupADXL345 関数をコード 4 に示す。

コード 4 AccManager::setupADXL345 関数

```

1 void AccManager::setupADXL345() {
2     Wire.begin();
3
4     Wire.beginTransmission(ADXL345_ADDR);
5     Wire.write(REG_POWER_CTL);

```



```

6 Wire.write(0x08); // Measure bit ON
7 Wire.endTransmission();
8
9 Wire.beginTransaction(ADXL345_ADDR);
10 Wire.write(REG_DATA_FORMAT);
11 Wire.write(0x00);
12 Wire.endTransmission();
13
14 lastAccVal = readAccMagnitude();
15 }

```

次に、加速度の実測値を取得する readAccMagnitude 関数について説明する。この関数は、DATA_X0 レジスタから連続する 6 バイト (X, Y, Z 軸それぞれ 2 バイトの符号付き整数) を requestFrom で読み出し、3 軸のベクトル合成値 (合成加速度) を平方根の計算により求めて返り値とする。readAccMagnitude 関数をコード 5 に示す。

コード 5 AccManager::readAccMagnitude 関数

```

1 float AccManager::readAccMagnitude() {
2   Wire.beginTransaction(ADXL345_ADDR);
3   Wire.write(REG_DATA_X0);
4   if (Wire.endTransmission(false) != 0) {
5     recoverI2C();
6     return lastAccVal;
7   }
8
9   uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
10  if (n < 6) {
11    recoverI2C();
12    return lastAccVal;
13  }
14
15  int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8));
16  int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8));
17  int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8));
18
19  return sqrtf((float)rawX*rawX + (float)rawY*rawY + (float)rawZ*rawZ);
20 }

```

ここで、Wire.endTransmission(false) がエラーを返した場合、または requestFrom で 6 バイト読み出せなかった場合には、I2C バスがロックアップ (スレーブが SDA 線を握ったまま停止する現象) していると判断し、recoverI2C 関数を呼び出したうえで前回値 lastAccVal を返すことで、誤ったシェイク検知 (ニセ shake) を防いでいる。続いて、recoverI2C 関数について説明する。この関数は、SCL 線を 9 回手動でトグルさせて SDA 線の握りを解放したうえで STOP 条件を生成し、Wire を再初期化して ADXL345 を再設定することでバスを復旧させる。lastRecoverMs は、この復旧処理およびログ出力が短時間に連続して実行されることを防ぐスロットル用の時刻管理に用いられる。recoverI2C 関数をコード 6 に示す。

コード 6 AccManager::recoverI2C 関数

```
1 void AccManager::recoverI2C() {
2   unsigned long now = millis();
3   if (now - lastRecoverMs >= 1000) { // ログのスロットル
4     lastRecoverMs = now;
5   }
6
7   Wire.end();
8
9   // バスクリア: SCLを9クロック叩いて握られたSDAを解放
10  pinMode(SDA, INPUT_PULLUP);
11  pinMode(SCL, OUTPUT);
12  for (int i = 0; i < 9; i++) {
13    digitalWrite(SCL, LOW); delayMicroseconds(5);
14    digitalWrite(SCL, HIGH); delayMicroseconds(5);
15  }
16  // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
17  pinMode(SDA, OUTPUT);
18  digitalWrite(SDA, LOW); delayMicroseconds(5);
19  digitalWrite(SCL, HIGH); delayMicroseconds(5);
20  digitalWrite(SDA, HIGH); delayMicroseconds(5);
21
22  // Wire再初期化&センサ再設定
23  Wire.begin();
24  Wire.beginTransmission(ADXL345_ADDR);
25  Wire.write(REG_POWER_CTL); Wire.write(0x08);
26  Wire.endTransmission();
27  Wire.beginTransmission(ADXL345_ADDR);
28  Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
29  Wire.endTransmission();
30 }
```

次に、update 関数について説明する。この関数は loop 内で毎回呼び出され、ACC_SAMPLE_INTERVAL (20 ms) ごとに加速度の実測値を取得し、前回値との差 diff を求める。diff が検知閾値 accThreshold を超過し、かつ前回検知から shakeInterval (750 ms, 多重検知防止用) が経過している場合にのみ、振り動作として検知し updateShakedInfo 関数を呼び出す。update 関数をコード 7 に示す。

コード 7 AccManager::update 関数

```
1 void AccManager::update(SyncManager& sync, PotManager& pot) {
2   unsigned long now = millis();
3   if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
4     lastSampleTime = now;
5     currentAccVal = readAccMagnitude();
6     float diff = fabsf(currentAccVal - lastAccVal);
7
8     bool overThreshold = (diff > accThreshold);
```

```

9      bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
10
11      if (overThreshold && intervalPassed) {
12          lastDetectTime = now;
13          isStarted = !isStarted;
14          updateShakedInfo(sync, pot);
15      }
16      lastAccVal = currentAccVal;
17  }
18  }

```

最後に、updateShakedInfo 関数について説明する。この関数は、振り動作の検知をトリガとして、PotManager が保持する bpmFrag・volFrag フラグ（可変抵抗器の値が変化した場合に立つフラグ。2.1.3 項で説明する）を確認し、フラグが立っている項目についてのみ SyncManager 経由で BPM または音量の変更情報を送信する。すなわち、実際に無線送信が発生するのは、可変抵抗器を操作した後にデバイスを振った場合に限られる。updateShakedInfo 関数をコード 8 に示す。

コード 8 AccManager::updateShakedInfo 関数

```

1 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
2     if (pot.bpmFrag) {
3         sync.sendBPM(pot.getBpmStep());
4         pot.bpmFrag = false;
5     }
6     if (pot.volFrag) {
7         sync.sendVolume(pot.getVolStep());
8         pot.volFrag = false;
9     }
10 }

```

最後に、指揮デバイスにおける可変抵抗器を用いた BPM・音量読み取り機能について説明する。指揮デバイスでは、演奏の要素である BPM と音量を、ユーザが操作できるように可変抵抗器を用いて 5 段階で調整する。アナログ入力値の特性を考慮したデータ処理を行うことで、安定した制御環境を構築している。アルゴリズムは、サンプリングと移動平均化、範囲分割、パラメータの抽出と出力の 3 段階に分けられる。サンプリングと移動平均化では、SAMPLE_INTERVAL ごとに BPM および音量変更用の可変抵抗器の電圧値を、配列 bpmSamples・volSamples に sampleSize 分だけ読み取り保存する。このとき、配列の各要素は書き込み位置 (bpmReadIndex・volReadIndex) が指す最も古い値から順に上書きされ、上書きの直前に該当要素の値を合計 bpmTotal・volTotal からあらかじめ減算し、新たに読み取った値を加算することで、配列全体を読み直すことなく常に最新の合計値を保持する（移動合計）。配列が一巡し全要素が新しい値に更新された時点 (bpmBufferFull・volBufferFull が true になった時点) 以降、この合計値を sampleSize で除算した移動平均値を averageBpmVal・averageVolVal に格納する。範囲分割では、移動平均化された値を STEP_BOUNDARIES[] で定義した 4 つの閾値と比較し、1～5 の段階番号を currentBpmStep・currentVolStep に格納する。パラメータの抽出と出力では、算

出した段階が直前の段階 (prevBpmStep・prevVolStep) から変化した場合にのみ、公開フラグ bpmFrag・volFrag を立てる。外部からは getBpmStep()・getVolStep() を介して現在の段階を取得し、前述の加速度センサ制御における棒振り動作の確定時にこれらのフラグを参照して各楽器機への送信を行う。なお、音量用可変抵抗器は配線の都合上、回した方向とセンサ値の増減が逆になるため、getVolStep() は 6-currentVolStep を返すことで表示上の段階を反転させる。BPM・音量読み取り機能で使用する変数の仕様を表 7 に示す。

表 7: BPM・音量読み取り機能で使用する変数の仕様一覧

変数名	型	説明
PIN_BPM	const int	BPM 変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
PIN_VOL	const int	音量変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
SAMPLE_INTERVAL	const unsigned long	可変抵抗器の値をサンプリングする周期を定義
sampleSize	const int	移動平均算出のためのサンプル数を定義
STEP_BOUNDARIES[]	const int	段階判定に用いる 4 つの閾値を保持する配列
lastSampleTime	unsigned long	前回サンプリングを行った時間を格納
bpmSamples[]	int	BPM 用のサンプル値を保持するための配列
volSamples[]	int	音量用のサンプル値を保持するための配列
bpmReadIndex	int	bpmSamples[] 内での現在の書き込み位置を管理するインデックス
volReadIndex	int	volSamples[] 内での現在の書き込み位置を管理するインデックス
bpmBufferFull	bool	bpmSamples[] が一巡し、移動平均の算出が有効になったかを保持
volBufferFull	bool	volSamples[] が一巡し、移動平均の算出が有効になったかを保持
bpmTotal	long	bpmSamples[] 内の累積合計を格納
volTotal	long	volSamples[] 内の累積合計を格納

次ページに続く

表 7 続き

変数名	型	説明
averageBpmVal	float	算出された BPM 用のアナログ入力の移動平均値を格納
averageVolVal	float	算出された音量用のアナログ入力の移動平均値を格納
currentBpmStep	uint8_t	算出された BPM の 5 段階のレベル (1～5) を格納
currentVolStep	uint8_t	算出された音量の 5 段階のレベル (1～5) を格納
prevBpmStep	int	段階の変化検知のための直前の BPM 段階を格納
prevVolStep	int	段階の変化検知のための直前の音量段階を格納
bpmFrag	bool	BPM 段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用
volFrag	bool	音量段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用

続いて、可変抵抗器読み取り機能（PotManager クラス）で使用する関数の仕様を表 8 に示す。

表 8 可変抵抗器読み取り機能 (PotManager クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
update	void	なし	SAMPLE_INTERVAL ご と に BPM・音量それぞれの可変抵抗器の値をサンプリングし、移動合計・移動平均・段階算出を行う。loop 内で毎回呼び出す
calcStep	int	float val	0～1023 の 移 動 平 均 値 を STEP_BOUNDARIES[] と 比 較 し、1～5 のいずれかの段階を返す
bpmUpdatePotentiometers	bool	なし	BPM 用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
volUpdatePotentiometers	bool	なし	音量用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
getBpmStep	uint8_t	なし	現 在 の BPM 段 階 (currentBpmStep) を 外 部に返すゲッター
getVolStep	uint8_t	なし	現在の音量段階を外部に返すゲッター。配線特性による値の反転を補正し、6-currentVolStep を返す

以下では、表 8 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、update 関数について説明する。この関数は loop 内で毎回呼び出され、SAMPLE_INTERVAL (20 μ s) ごとに bpmUpdatePotentiometers 関数と volUpdatePotentiometers 関数を呼び出す。update 関数をコード 9 に示す。

コード 9 PotManager::update 関数

```

1 void PotManager::update() {
2     unsigned long now = micros();
3     if (now - lastSampleTime >= SAMPLE_INTERVAL) {
4         lastSampleTime = now;
5         if (bpmUpdatePotentiometers()) bpmFrag = true;

```

```

6   if (volUpdatePotentiometers()) volFrag = true;
7   }
8 }

```

次に、bpmUpdatePotentiometers 関数について説明する。この関数は、リングバッファ（配列 bpmSamples, 添字 bpmReadIndex）を用いた移動平均処理により、analogRead で取得したアナログ値のノイズを平滑化する。具体的には、配列の最も古いサンプルを合計値 bpmTotal から減算し、新たに取得した値を加算したうえで書き込み位置を進める。sampleSize（10 サンプル）分のバッファが一巡した時点から、合計値をサンプル数で割った移動平均 averageBpmVal を算出し、calcStep 関数で段階へ変換したうえで、前回段階から変化していれば true を返す。bpmUpdatePotentiometers 関数をコード 10 に示す。

コード 10 PotManager::bpmUpdatePotentiometers 関数

```

1 bool PotManager::bpmUpdatePotentiometers() {
2     bpmTotal -= bpmSamples[bpmReadIndex];
3     bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
4     bpmTotal += bpmSamples[bpmReadIndex];
5     bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
6     if (bpmReadIndex == 0) bpmBufferFull = true;
7     if (!bpmBufferFull) return false;
8
9     averageBpmVal = (float)bpmTotal / sampleSize;
10    currentBpmStep = calcStep(averageBpmVal);
11
12    if (currentBpmStep != prevBpmStep) {
13        prevBpmStep = currentBpmStep;
14        return true;
15    }
16    return false;
17 }

```

続いて、calcStep 関数について説明する。この関数は、bpmUpdatePotentiometers 関数および volUpdatePotentiometers 関数から呼び出され、得られた移動平均値を 1～5 の 5 段階に離散化する。STEP_BOUNDARIES 配列（300, 500, 650, 818）に対する大小比較によって段階を決定しており、境界値の間隔を均等にしていない理由は、可変抵抗器の実測特性に合わせて各段階の操作しやすさを調整しているためである。calcStep 関数をコード 11 に示す。

コード 11 PotManager::calcStep 関数

```

1 int PotManager::calcStep(float val) {
2     if (val <= STEP_BOUNDARIES[0]) return 1;
3     else if (val <= STEP_BOUNDARIES[1]) return 2;
4     else if (val <= STEP_BOUNDARIES[2]) return 3;
5     else if (val <= STEP_BOUNDARIES[3]) return 4;
6     else return 5;
7 }

```

次に、volUpdatePotentiometers 関数について説明する。この関数は bpmUpdatePotentiometers

関数と同様の移動平均処理を行うが、対象のピンおよび内部変数が音量用 (PIN_VOL, volSamples 等) に置き換わっている点のみが異なり、処理の骨格は共通している。volUpdatePotentiometers 関数をコード 12 に示す。

コード 12 PotManager::volUpdatePotentiometers 関数

```
1 bool PotManager::volUpdatePotentiometers() {
2     volTotal -= volSamples[volReadIndex];
3     volSamples[volReadIndex] = analogRead(PIN_VOL);
4     volTotal += volSamples[volReadIndex];
5     volReadIndex = (volReadIndex + 1) % sampleSize;
6     if (volReadIndex == 0) volBufferFull = true;
7     if (!volBufferFull) return false;
8
9     averageVolVal = (float)volTotal / sampleSize;
10    currentVolStep = calcStep(averageVolVal);
11
12    if (currentVolStep != prevVolStep) {
13        prevVolStep = currentVolStep;
14        return true;
15    }
16    return false;
17 }
```

最後に、getBpmStep 関数および getVolStep 関数について説明する。いずれも PotManager.h 内でインライン定義されたゲッター関数であり、外部 (AccManager クラス) から現在の段階番号を取得するために用いられる。getVolStep 関数では、可変抵抗器を回転させる向きと、音量として直感的に理解しやすい段階の増減方向を一致させるため、6 - currentVolStep という変換を行っている点が getBpmStep 関数と異なる。両関数をコード 13 に示す。

コード 13 PotManager::getBpmStep 関数・getVolStep 関数

```
1 uint8_t getBpmStep() const { return currentBpmStep; }
2 uint8_t getVolStep() const { return 6 - currentVolStep; }
```

なお、本節で説明した SyncManager クラスによって送信された演奏開始・終了、BPM、音量の各情報が、中継機デバイス側でどのように受信・反映されるかについては 2.2.3 項 (中継機デバイスの内部設計) で説明する。

2.2 中継機デバイス

2.2.1 概要

次に、中継機デバイスについて説明する。中継機デバイスの構成図を図 9 に示す。本デバイスは、指揮デバイスからの情報を受け取り中継機デバイスの回転を作動させ、現在の BPM を表示するデバイスであり、主に以下の 2 つの機能を有する。

第一の機能は、上記で述べた BPM に合わせて回転し楽曲の BPM を制御する機能である。デバイ

スに設置してあるレーザー受光のテンポで BPM を管理し、楽曲のテンポを視覚的にも理解できるようにする。

第二の機能は、Arduino Uno R4 WiFi に付属している LEDMatrix に BPM を表示する機能である。ここに指揮デバイスから受け取った BPM を表示し、視覚的に現在流れている楽曲の BPM を理解できるようにする。ここからは、外部設計と内部設計について説明する。

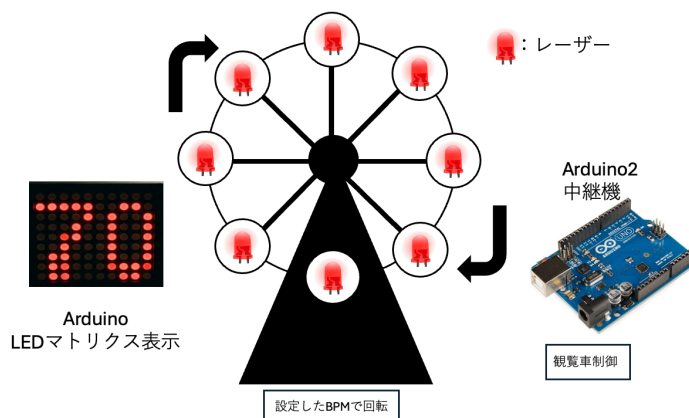


図 9 中継機デバイスの構成図

2.2.2 外部設計

本節では中継機デバイスの外部設計について説明する。まず、本デバイスを構成する部分について表 9 に示す。この内、個別で購入する必要があるものはハイパワーギヤーボックス、ブレッドボード用 DC ジャック DIP 化キット、ボタン電池、ボタン電池ホルダー、QI ケーブル、有極コンデンサーである。

また、中継機の回路図を図 10 に示す。モータードライバーに Arduino D5, D6 ピンを接続し、PWM 制御を行えるようにする。モータードライバーへの電力供給は当初、外部電源として 9V のアルカリ電池を検討していたが、Arduino の 5V からの電力供給と DC ジャックからの AC アダプター 5V1A による外部給電により、アルカリ電池が不要であることが実装時に確認されたため除外した。また、モータードライバー及び DC ジャックはピンヘッダーのはんだ付けを行う。加えて、ギアボックスは $0.01\mu\text{F}$ のコンデンサーと直接はんだ付けを行う。ギアボックスと並列に接続された $0.01\mu\text{F}$ のコンデンサは、モータのブラシ接点で発生する電気ノイズを吸収し、制御信号や周辺回路への影響を低減する役割を果たしている。

次に、観覧車の設計について説明する。作成するために、3D プリンタを用いて作成する。実際に設計に使用するモデルデータを図 11 から図 16 に示している。観覧車の直径は約 16cm とし、ボタン電池付きレーザーを 45 度間隔で配置する。そして、観覧車を支える左右の支柱および土台を設計する。観覧車本体は、以下のパーツに分割して設計を行う。

表 9 中継機デバイスを構成する部品

機材	品番	個数 [個]
Arduino Uno R4 WiFi	-	1
ルーター	-	1
ブレッドボード	-	1
赤色ドットレーザーモジュール	FU650AD5-C6	8
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4
ボタン電池	CR2477	4
スライドスイッチ	SS-12D00-G5	4
モータードライバーモジュール	AE-DRV8835-S	1
ハイパワーギヤーボックス HE	72003	1
ジャンパーワイヤー	-	適宜
ブレッドボード用 2.1mm 標準 DC ジャック DIP 化キット	AE-DC-POWER-JACK- DIP	1
超小型スイッチング AC アダプター 5 V 1 A	-	1
QI ケーブル 2P-1Px2	311-262	1
抵抗器 10 Ω	-	1
コンデンサー 0.01 μF	-	1
コンデンサー 0.047 μF	-	1
有極性コンデンサー 100 μF	-	1

- 土台 観覧車全体を支える部分であり、内部を空洞化させる。また、支柱を土台に差し込み、固定する役割も持つ。実際に設計する部品は図 11 の通りである。
- 支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 12 の通りである。
- 回転軸 ギアボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるよう、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 13 の通りである。
- 回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は図 14, 15 の通りである。
- 受光部 レーザーの延長線上に設置する照度センサーの受光部である。回転リングから平行な

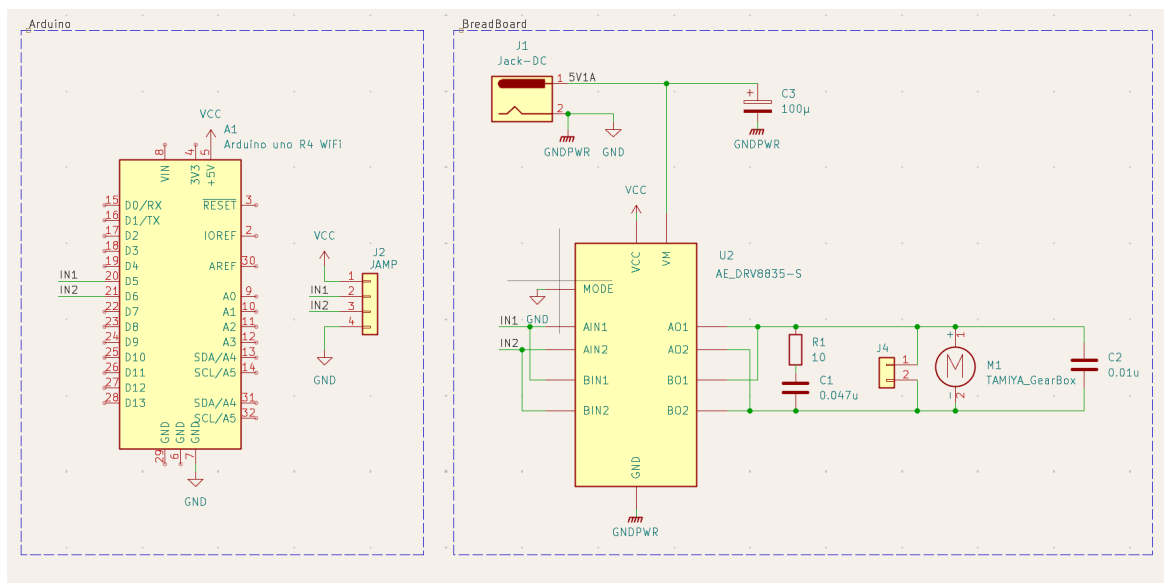


図 10 観覧車型中継機デバイス回路図

位置に円盤を設置し，中央にギアボックスモータを設置する台を設ける．90 度おきに円盤にくぼみを作り，センサーを設置できるようにした．受光部の円盤によりレーザー光が後ろに漏れないという設計となっている．実際に設計する部品は図 16 の通りである．

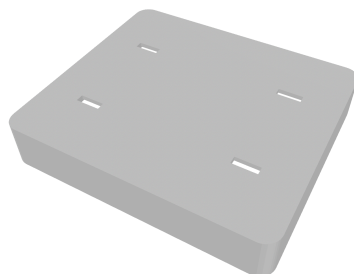


図 11 設計に使用するモデルデータ (土台部)

図～～～に実際に作成した中継機デバイスを示す．

2.2.3 内部設計

本節では中継機デバイスの内部設計について説明する．主に回転速度同期機構（モータ制御）と LEDMatrix 表示機構のプログラムについて説明する．

次に，中継機デバイスの制御プログラムのファイル構成を示す．表 10 の通り，各ファイルは機能ごとに分割されている．

FerrisWheel.ino の loop 関数は，Udp.parsePacket() でパケットの到着を監視し，届いていれば uint8_t packetBuffer[10] として 2 バイト分を読み取る．先頭バイト packetBuffer[0] を

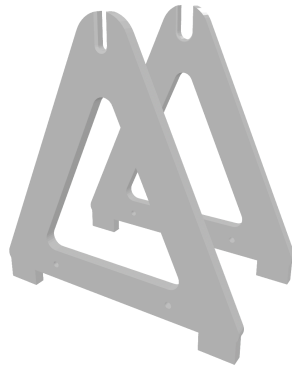


図 12 設計に使用するモデルデータ (支柱部)



図 13 設計に使用するモデルデータ (回転軸部)

ヘッダ文字として判定し、ヘッダが'S' (演奏開始) の場合は `motor.startRamp()` と `displayBPM()` を、'B' (BPM 変更) の場合は `motor.changeBPM()` と `displayBPM()` を、'E' (演奏終了) の場合は `motor.stopRamp()` と `clearDisplay()` を、2 バイト目のペイロード (BPM 段階番号) を引数として呼び出す構成となっている。loop 関数をコード 14 に示す。

コード 14 FerrisWheel.ino の loop 関数

```

1 void loop() {
2     uint8_t packetSize = Udp.parsePacket();
3     if (packetSize <= 0) return;
4     if (packetSize != 2) {
5         while (Udp.available()) Udp.read();
6         return;
7     }
8
9     Udp.read(packetBuffer, sizeof(packetBuffer));
10    char header = packetBuffer[0];
11    uint8_t payload = packetBuffer[1];
12
13    switch (header) {
14        case 'S':
15            running = true;
16            motor.startRamp(payload);
17            displayBPM(payload);
18            break;
19        case 'B':
20            if (!running) break;

```

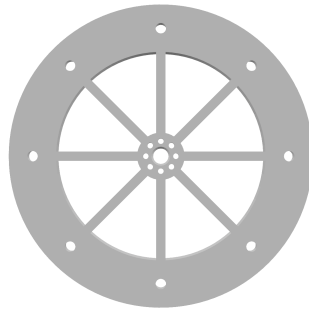


図 14 設計に使用するモデルデータ (回転リング部 1)

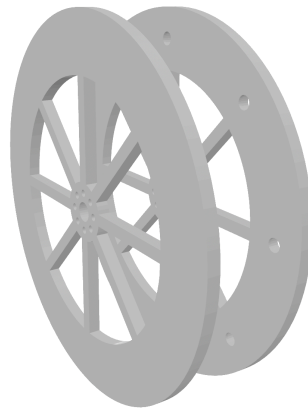


図 15 設計に使用するモデルデータ (回転リング部 2)

```

21         motor.changeBPM(payload);
22         displayBPM(payload);
23         break;
24     case 'E':
25         running = false;
26         motor.stopRamp();
27         clearDisplay();
28         break;
29     }
30 }
```

続いて、Main (FerrisWheel.ino) で使用する変数の仕様を表 11 に示す。

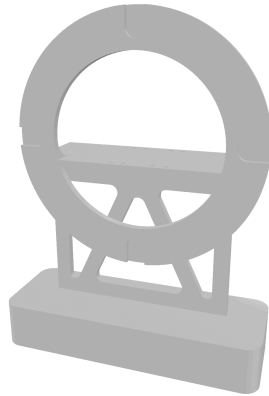


図 16 設計に使用するモデルデータ (受光部)

表 10 中継機デバイスのファイル構成

ファイル名	概要
FerrisWheel.ino	WiFi 接続・UDP 受信の管理, 受信ヘッダに応じたモータ制御・表示処理の呼び出し
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理
Display.h	定数定義, 関数宣言, 外部参照
MotorControl.cpp	PWM 出力によるモータ回転制御, 回転速度テーブルの生成
MotorControl.h	モータ制御用の定数・変数定義, 関数宣言

表 11 Main (FerrisWheel.ino) で使用する変数の仕様一覧

変数名	型	説明
SSID	const char[]	Wi-Fi のネットワーク名 (hackathon001-WPA2) を定義
PASS	const char[]	その Wi-Fi のパスワード Wi-Fi パスワードを定義
localIP	IPAddress	中継機デバイス自身の静的 IP アドレスを定義
gateway	IPAddress	静的 IP 設定に使用するデフォルトゲートウェイのアドレスを定義
subnet	IPAddress	静的 IP 設定に使用するサブネットマスクを定義
LOCAL_PORT	const uint16_t	UDP 受信に利用する自身のローカルポート番号を定義
packetBuffer	uint8_t[10]	UDP 通信で受信したパケットのデータを一時的に格納するバッファ配列
running	bool	演奏中かどうかを判定し, 演奏前 (待機中) の予期せぬモータ回転を防ぐための状態管理フラグ

続いて、Main (FerrisWheel.ino) で使用する関数の仕様を表 12 に示す。

表 12 Main (FerrisWheel.ino) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
loop	void	なし	UDP パケットの受信確認を行い、ヘッダが'S'であれば motor.startRamp() と displayBPM() を、'B'であれば motor.changeBPM() と displayBPM() を、'E'であれば motor.stopRamp() と clearDisplay() を呼び出す。演奏前 (running=false) に'B'を受信した場合は無視する。

ここからは、モータの制御、LEDMatrix での BPM 表示、回転速度実測プログラム (LaserIntervalTest) の順に、使用する変数・関数の仕様と処理内容について説明する。

まず、中継機デバイスにおけるモータの制御処理について説明する。本デバイスのギアボックスモータの回転速度を制御するために、Arduino の PWM(パルス幅変調) 機能を用いてモータの平均印加電圧を制御する。PWM とは、デジタル出力ピンから出力される矩形波の HIGH 状態と LOW 状態の時間比率(デューティ比)を変化させることで、擬似的にアナログ電圧を生成する制御方式である。Arduino のデジタル出力ピンは 5V の HIGH か 0V の LOW のいずれかしか出力できないが、HIGH と LOW を高速に切り替えてデューティ比を調整することで、負荷に対して見かけ上の連続的な電圧を印加することが可能となる。

本デバイスは、Arduino の D5 ピンをモータドライバモジュールに接続し、PWM 制御を行う構成としている。Arduino の analogWrite 関数は、引数として 0 から 255 の整数値を受け取り、この値に応じたデューティ比の矩形波を出力する。このとき引数が 0 のときデューティ比は常時 LOW であり、255 のときデューティ比は常時 HIGH となる。中間値では、引数に比例したデューティ比の矩形波が生成される。この関係を式で表すと、供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (1)$$

このとき、モータの巻線が持つインダクタンスとギアボックスの機械的慣性が電氣的・機械的なローパスフィルタとして機能するため、モータはパルスの平均値に応じた一定速度で滑らかに回転する。

次に、ルックアップテーブルによる速度制御について説明する。一般的な DC モータの回転速度は供給電圧に比例する性質があるが [11]、実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い。そのため、当初検討していた「4 拍で 90 度 (1 小節が 90 度) 回転する」といった幾何

学的な理論式による制御ではなく、より確実な速度追従を実現するため、幾何学的な縛りを廃止して実測値に基づくルックアップテーブル方式を採用する。

本デバイスでは、表 13 に示す離散的な 5 段階の BPM 設定値と、あらかじめ実測により求めた PWM 出力値を、コード内の配列 bpmTable・pwmTable として対応させ、受信した段階番号に応じた pwmValue を即座に選択・出力する構成とする。表 13 に示す実測周期は、後述する回転速度実測プログラム（LaserIntervalTest）を用いて計測した値に基づいて決定している。

表 13 BPM に対する実測周期と出力 PWM 値			
段階	目標 BPM	実測周期 [ms]	出力 PWM 値
1	60	230	26
2	90	180	28
3	120	140	30
4	150	110	32
5	180	80	37

続いて、モータ制御（MotorControl クラス）で使用する変数・定数の仕様を表 14 に示す。

表 14 モータ制御（MotorControl クラス）で使用する変数・定数の仕様一覧		
変数名	型	説明
PIN_MOTOR_PWM	const uint8_t	モータ制御用の出力ピンを定義
bpmTable[5]	const uint8_t	5 つの離散値 BPM を格納
pwmTable[5]	const uint8_t	5 つの離散値 PWM 出力値を格納
VCC	const float	Arduino への供給電圧 V_{cc} を定義
currentBPM	uint8_t	現在演奏中の BPM データを格納
RPM	float	BPM から計算された目標回転数を格納
omega	float	目標角速度 ω を格納
pwmValue	uint8_t	analogWrite 用の出力値を格納
V_average	float	平均電圧 $V_{average}$ を格納
rpmTable[5]	float	bpmTable に対応する目標回転数を格納する配列

続いて、モータ制御（MotorControl クラス）で使用する関数の仕様を表 15 に示す。

表 15 モータ制御 (MotorControl クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
begin	void	なし	setup 内で一度だけ呼び出される関数であり、モータピンの初期化を行う
createRpmTable	void	なし	bpmTable の各値から目標回転数を算出し、rpmTable を作成する
startRamp	void	uint8_t stepNumber	演奏開始時の加速を制御する。段階番号から求めた目標の pwmValue まで、PWM 出力値を 5 ずつ段階的に上げていくことで滑らかに目標速度へ到達させる
stopRamp	void	なし	演奏終了時の減速を制御する。現在の pwmValue から 0 まで、PWM 出力値を 5 ずつ段階的に下げていくことで滑らかに停止させる
changeBPM	void	uint8_t stepNumber	演奏中の BPM 変更を行う。startRamp とは異なり、段階的な加減速を行わず目標の pwmValue を即座に出力する
stepToNumber	void	uint8_t stepNumber	受信した段階番号 (1~5) から、bpmTable・pwmTable・rpmTable を参照して currentBPM・pwmValue・RPM を求める
updateStatus	void	なし	RPM から角速度 omega を、pwmValue から平均電圧 V_average を算出する

以下では、表 15 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、begin 関数について説明する。この関数は、setup 内で一度だけ呼び出され、モータドライバーへ接続された PIN_MOTOR_PWM を出力ピンとして初期化したうえで、出力を 0 にリセットする。begin 関数をコード 15 に示す。

コード 15 MotorControl::begin 関数

```

1 void MotorControl::begin() {
2     pinMode(PIN_MOTOR_PWM, OUTPUT);
3     analogWrite(PIN_MOTOR_PWM, 0);
4 }

```

次に、createRpmTable 関数について説明する。この関数も setup 内で一度だけ呼び出され、bpmTable の各要素を 16 で除算することで rpmTable を生成する。createRpmTable 関数をコー

ド 16 に示す。

コード 16 MotorControl::createRpmTable 関数

```
1 void MotorControl::createRpmTable() {
2     for (uint8_t i = 0; i < 5; i++) {
3         rpmTable[i] = bpmTable[i] / 16.0f;
4     }
5 }
```

次に、startRamp 関数について説明する。この関数は、まず stepToNumber 関数によって目標の pwmValue を求めたうえで、出力値を 0 から pwmValue まで 5 刻みで増加させながら 40 ms ずつ待機することで、モータを滑らかに加速させる。startRamp 関数をコード 17 に示す。

コード 17 MotorControl::startRamp 関数

```
1 void MotorControl::startRamp(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     for (int i = 0; i <= pwmValue; i += 5) {
4         analogWrite(PIN_MOTOR_PWM, i);
5         delay(40);
6     }
7     analogWrite(PIN_MOTOR_PWM, pwmValue);
8 }
```

次に、stopRamp 関数について説明する。この関数は startRamp 関数と対になる処理であり、現在の pwmValue から 0 まで 5 刻みで減少させながら 40 ms ずつ待機することで、モータを滑らかに減速・停止させる。stopRamp 関数をコード 18 に示す。

コード 18 MotorControl::stopRamp 関数

```
1 void MotorControl::stopRamp() {
2     for (int i = pwmValue; i >= 0; i -= 5) {
3         analogWrite(PIN_MOTOR_PWM, i);
4         delay(40);
5     }
6     analogWrite(PIN_MOTOR_PWM, 0);
7 }
```

次に、changeBPM 関数について説明する。この関数は演奏中に BPM の段階が変更された際に呼び出され、stepToNumber 関数と updateStatus 関数によって新しい段階の値を求めたうえで、startRamp 関数のような段階的な加減速を行わず、目標の pwmValue を即座に出力する。演奏を止めずにテンポを追従させるため、段階的な加減速は行わない仕様としている。changeBPM 関数をコード 19 に示す。

コード 19 MotorControl::changeBPM 関数

```
1 void MotorControl::changeBPM(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     updateStatus();
4     analogWrite(PIN_MOTOR_PWM, pwmValue);
5 }
```

最後に、stepToNumber 関数と updateStatus 関数について説明する。stepToNumber 関数は、受信した段階番号(1～5)が範囲内であることを確認したうえで、bpmTable・pwmTable・rpmTable の該当インデックスから currentBPM・pwmValue・RPM をそれぞれ求める。updateStatus 関数は、この RPM および pwmValue を基に、(1) 式に相当する計算によって omega と V_average を算出する。両関数をコード 20 に示す。

コード 20 MotorControl::stepToNumber 関数・updateStatus 関数

```

1 void MotorControl::stepToNumber(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     currentBPM = bpmTable[stepNumber - 1];
4     pwmValue    = pwmTable[stepNumber - 1];
5     RPM         = rpmTable[stepNumber - 1];
6 }
7
8 void MotorControl::updateStatus() {
9     omega      = 2.0 * PI * RPM / 60.0f;
10    V_average = VCC * (pwmValue / 256.0f);
11 }

```

次に、中継機デバイスにおける LEDMatrix 表示処理について説明する。今回のシステムでは表示用のマトリクスとして、Arduino Uno R4 WiFi 付属のマトリクスを使用する。このマトリクスを用いて、FerrisWheel.ino の loop 関数から受信した BPM 段階番号を数値変換し、独自定義したフォント配列を用いて表示を行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 のドットパターンをコード 21 に示す。

コード 21 font3x5 配列における数字 0 のドットパターン定義

```

1 {1,1,1,
2  1,0,1,
3  1,0,1,
4  1,0,1,
5  1,1,1}

```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3 × 5 のフォント配列を用いてフレームバッファに書き込むことで行う。続いて、LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様を表 16 に示す。

表 16 LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様一覧

変数名	型	説明
BPM_STEP_1~5	#define	BPM5 段階 (60,90,120,150,180) の各値を定義
FONT_WIDTH	#define	1 文字の横ドット数 (3) を定義
FONT_HEIGHT	#define	1 文字の縦ドット数 (5) を定義
MATRIX_ROWS	#define	LED マトリクスの行数 (8) を定義
MATRIX_COLS	#define	LED マトリクスの列数 (12) を定義
font3x5[10][15]	const uint8_t	独自定義した 0~9 の 3 × 5 ドットパターンフォント配列
matrix	ArduinoLEDMatrix	LED マトリクスのインスタンス
bpmTable[5]	static const int	段階番号 (1~5) から実際の BPM 値へ変換するための対応表
frameBuffer[8][12]	static uint8_t	描画用の共有フレームバッファ
currentBPM	static int	現在表示中の BPM 値を保持する内部状態変数. clearDisplay 呼び出し時に初期値 (-1) へリセットされる

続いて, LEDMatrix 表示 (Display モジュール) で使用する関数の仕様を表 17 に示す.

表 17 LEDMatrix 表示 (Display モジュール) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
displaySetup	void	なし	LED マトリクスの初期化を行う
displayBPM	void	uint8_t stepNumber	受信した BPM 段階番号を LED マトリクスに数値表示する
drawDigit	void	int digit, int x_offset	1 桁の数字をフレームバッファの指定位置に描画する
clearDisplay	void	なし	LED マトリクスを全消灯する

LEDMatrix プログラムのクラス図は以下の図 17 の通りである. WiFi 接続・UDP 受信および各処理の呼び出しを行う FerrisWheel.ino, LED マトリクスへの描画処理を行う Display.cpp, および関数宣言やフォント定義を行う Display.h で構成される. 各モジュール間の関係について, FerrisWheel.ino から Display.cpp への関係は, 表示処理を利用する依存関係である. また, Display.cpp から Display.h への関係は, 関数宣言およびフォント定義を参照する依存関係である.

以下では, 表 17 に示した順に, 各関数の処理内容を実際のコードとともに説明する. はじめに, displaySetup 関数について説明する. この関数は, matrix.begin() を呼び出した後, 消灯フレー

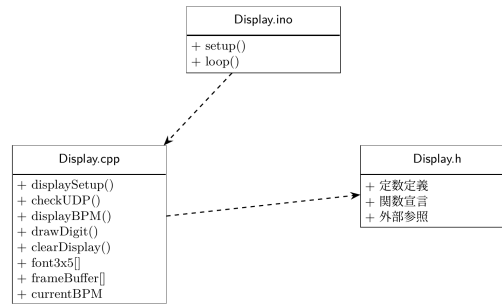


図 17 LEDMatrix のクラス図

ムを一度描画してから 200 ms 待機する。これは、`begin()` 直後の最初の `loadFrame()` は反映されない場合があるための対策である。`displaySetup` 関数をコード 22 に示す。

コード 22 `displaySetup` 関数

```

1 void displaySetup() {
2     matrix.begin();
3     uint32_t warmUp[3] = {0, 0, 0};
4     matrix.loadFrame(warmUp);
5     delay(200);
6 }
  
```

次に、`displayBPM` 関数について説明する。この関数は、`loop` 関数から受け取った BPM 段階番号 (1~5) を `bpmTable[]` で参照して実際の BPM 値に変換したうえで各桁ごとに分割し、`drawDigit()` を用いてフレームバッファに描画する。そして、`uint32_t[3]` 形式にビットパックした後、`matrix.loadFrame()` で描画を行う。`displayBPM` 関数をコード 23 に示す。

コード 23 `displayBPM` 関数

```

1 void displayBPM(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     uint8_t bpm = bpmTable[stepNumber - 1];
4
5     memset(frameBuffer, 0, sizeof(frameBuffer));
6
7     int digits[3];
8     int numDigits;
9     if (bpm >= 100) {
10         numDigits = 3;
11         digits[0] = bpm / 100;
12         digits[1] = (bpm / 10) % 10;
13         digits[2] = bpm % 10;
14     } else if (bpm >= 10) {
15         numDigits = 2;
16         digits[0] = bpm / 10;
17         digits[1] = bpm % 10;
18     } else {
19         numDigits = 1;
  
```

```

20     digits[0] = bpm;
21 }
22
23 int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
24 int startX     = (MATRIX_COLS - totalWidth) / 2;
25
26 for (int i = 0; i < numDigits; i++) {
27     drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
28 }
29
30 uint32_t bitmap[3] = {0, 0, 0};
31 for (int row = 0; row < MATRIX_ROWS; row++) {
32     for (int col = 0; col < MATRIX_COLS; col++) {
33         if (frameBuffer[row][col]) {
34             int n = row * MATRIX_COLS + col;
35             bitmap[n / 32] |= (1UL << (31 - (n % 32)));
36         }
37     }
38 }
39 matrix.loadFrame(bitmap);
40 }

```

次に、drawDigit 関数について説明する。この関数は、引数の数字が 0 から 9 の範囲内であることを確認した後、縦方向の中央寄せオフセットを算出し、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取ってフレームバッファの指定位置に書き込む。drawDigit 関数をコード 24 に示す。

コード 24 drawDigit 関数

```

1 void drawDigit(int digit, int x_offset) {
2     if (digit < 0 || digit > 9) return;
3     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
4     for (int y = 0; y < FONT_HEIGHT; y++) {
5         for (int x = 0; x < FONT_WIDTH; x++) {
6             int col = x_offset + x;
7             int row = y_offset + y;
8             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
9                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
10            }
11        }
12    }
13 }

```

最後に、clearDisplay 関数について説明する。この関数は、BPM 値を初期値 (-1) にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。clearDisplay 関数をコード 25 に示す。

コード 25 clearDisplay 関数

```

1 void clearDisplay() {

```

```

2   currentBPM = -1;
3   uint32_t blank[3] = {0, 0, 0};
4   matrix.loadFrame(blank);
5 }

```

最後に、中継機デバイスのモータ回転速度を実測するために用いたプログラムである `LaserIntervalTest` について説明する。このプログラムは中継機デバイス本体には搭載せず、開発段階で観覧車に設置したレーザーと CdS セルを用いて、各 PWM 出力値における実際の回転周期を計測するために使用したものであり、表 13 に示した実測周期の値はこのプログラムによって得られている。本プログラムは、CdS セルの出力値を監視し、レーザー光の通過（パルス）を検知するたびに、直前のパルス検知からの経過時間（interval）をシリアルモニタへ出力する。周囲光量やレーザーの取り付け誤差による検知レベルのばらつきに対応するため、固定閾値ではなく、起動直後のキャリブレーションと直近のピーク値履歴に基づいて検知しきい値を継続的に調整する適応しきい値方式を採用している。`LaserIntervalTest` で使用する主な定数の仕様を表 18 に示す。

表 18 `LaserIntervalTest` で使用する主な定数の仕様一覧

定数名	型	説明
CDS_PIN	const int	CdS セルの出力を読み取るアナログピン番号を定義
CDS_ON_THRESHOLD CDS_OFF_THRESHOLD	/ const int	起動直後にのみ使用する仮の ON/OFF しきい値を定義
CDS_MIN_DELTA	const int	暗い状態との差がこの値未満であればレーザーとして扱わない閾値を定義
CDS_PEAK_HISTORY	const uint8_t	しきい値の再計算に用いる直近ピーク値の保持個数を定義
CDS_ON_RATIO / CDS_OFF_RATIO	const uint8_t	baseline からピーク値までの何%を ON/OFF しきい値とするかを定義
CDS_THRESHOLD_STEP_LIMIT	const int	1 回の再計算でしきい値が変化できる上限幅を定義
CDS_STARTUP_CALIBRATION_MS	const unsigned long	起動直後の誤検出を防止するキャリブレーション時間を定義
DEBOUNCE_MS	const unsigned long	同一パルスの多重検知を防止するデバウンス時間を定義

続いて、`LaserIntervalTest` (`LaserDetector` クラス) で使用する主な関数の仕様を表 19 に示す。

表 19 LaserIntervalTest (LaserDetector クラス) で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
update	bool	int cdsValue, unsigned long nowMs, unsigned long &intervalOut	loop 内で毎回呼び出される公開関数. detectLight による検知およびデバウンス処理を経て, 新規パルスを検知した場合に true を返し, 前回検知からの経過時間 intervalOut を書き込む
detectLight	bool	int v, unsigned long nowMs	baseline の追従, 起動直後のキャリブレーション, ヒステリシスを持った ON/OFF 判定によってレーザーパルスの立ち上がりを検知する内部関数
registerPeak	void	int peak	検知したパルスのピーク値を履歴に登録し, その平均値から ON/OFF しきい値を再計算する内部関数
updateFallbackThresholds	void	なし	baseline の値から ON/OFF しきい値の暫定値を算出する内部関数
limitStep	int	int current, int target, int limit	しきい値の 1 回あたりの変化量を limit 以内に制限する内部関数

以下では, 表 19 に示した主要な関数のうち, 公開インタフェースである update 関数と,

検知処理の中心である `detectLight` 関数について、実際のコードとともに説明する。なお、`registerPeak` 関数・`updateFallbackThresholds` 関数・`limitStep` 関数の詳細な実装は付録に示すプログラム全文を参照されたい。

はじめに、`setup` 関数と `loop` 関数について説明する。`setup` 関数ではシリアル通信を開始するのみであり、`loop` 関数では CdS セルの値を毎回取得し、`LaserDetector` の `update` 関数へ渡している。新しいパルスが検知された場合は、得られた `interval[ms]` をシリアルモニタへ出力する。`setup` 関数・`loop` 関数をコード 26 に示す。

コード 26 `LaserIntervalTest.ino` の `setup` 関数・`loop` 関数

```
1 void setup() {
2   Serial.begin(115200);
3   Serial.println("---レーザー間隔計測_Setup_Started_---");
4 }
5
6 void loop() {
7   unsigned long now = millis();
8   int cdsValue = analogRead(CDS_PIN);
9   unsigned long interval = 0;
10
11   if (detector.update(cdsValue, now, interval)) {
12     Serial.print("[interval]_");
13     Serial.print(interval);
14     Serial.println("_ms");
15   }
16 }
```

次に、`LaserDetector` クラスの `update` 関数について説明する。この関数は、`detectLight` 関数によってパルスの立ち上がりが検知され、かつ前回検知から `DEBOUNCE_MS` (200 ms) 以上が経過している場合にのみ、`true` を返して `interval` (前回検知時刻との差) を書き込む。`update` 関数をコード 27 に示す。

コード 27 `LaserDetector::update` 関数

```
1 bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
2   if (calibrationStartMs == 0) calibrationStartMs = nowMs;
3   if (!detectLight(cdsValue, nowMs)) return false;
4   if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
5
6   intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
7   bool hasInterval = (lastDetectTime != 0);
8   lastDetectTime = nowMs;
9   return hasInterval;
10 }
```

最後に、`detectLight` 関数について説明する。この関数は、暗状態の推定値である `baseline` を継続的に追従させながら、値が `baseline+CDS_MIN_DELTA` 相当の `onThreshold` を上回った瞬間をパルスの立ち上がりとして検知する状態機械である。起動直後は `startupCalibrated` フラ

グが false の間, CDS_STARTUP_CALIBRATION_MS (1 秒) をかけて最も暗い値を `baseline` の初期値として探索し, 誤検知を防止する. また, キャリブレーション完了時点でレーザーが既に照射状態であった場合は, `waitingForRelease` フラグによって一度非照射状態に戻るまで検知を保留する. パルスの立ち下がり (照射終了) を検知すると `registerPeak` 関数を呼び出し, しきい値を実測値に基づいて更新する. `detectLight` 関数をコード 28 に示す.

コード 28 `LaserDetector::detectLight` 関数

```
1 bool detectLight(int v, unsigned long nowMs) {
2     if (!baselineReady) {
3         baseline = v;
4         startupMin = v;
5         baselineReady = true;
6         updateFallbackThresholds();
7     }
8
9     if (!startupCalibrated) {
10        if (v < startupMin) startupMin = v;
11        baseline = startupMin;
12        updateFallbackThresholds();
13        if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
14        startupCalibrated = true;
15        waitingForRelease = (v >= onThreshold);
16        releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
17                           : offThreshold;
18        return false;
19    }
20
21    if (waitingForRelease) {
22        if (v <= releaseThreshold) {
23            baseline = v;
24            updateFallbackThresholds();
25            waitingForRelease = false;
26        }
27        return false;
28    }
29
30    if (!laserOn) {
31        if (v < baseline) baseline = v;
32        else baseline = (baseline * 31 + v) / 32;
33        if (!calibrated) updateFallbackThresholds();
34    }
35
36    if (!laserOn && v >= onThreshold) {
37        laserOn = true;
38        pulsePeak = v;
39        return true;
40    }
```

```

40
41     if (laserOn) {
42         if (v > pulsePeak) pulsePeak = v;
43         if (v <= offThreshold) {
44             laserOn = false;
45             registerPeak(pulsePeak);
46         }
47     }
48     return false;
49 }

```

プログラム全文は付録に示す。

なお、本節で説明した FerrisWheel.ino の UDP 受信処理は、指揮デバイスの SyncManager クラスから送信された演奏開始・終了、BPM、音量の各情報を受信する側の処理であり、送信側の詳細は 2.1.3 項（指揮デバイスの内部設計）を参照されたい。また、中継機デバイスから楽器デバイスへの同期は、回転する観覧車に設置されたレーザーの通過光を各楽器デバイスの CdS セルで検知することで実現しているが、この受光により各楽器を演奏させる楽器デバイス側の Arduino プログラムについては、本報告書の内部設計としては未文書化である。

2.3 楽器デバイス

2.3.1 内部設計

本節では、楽器デバイスにおける音色表現を担うソフトウェア（Processing）の内部設計について説明する。楽器デバイスは、Arduino から受信した発音指示をもとに、Processing 上で波形を合成して音を鳴らす構成となっている。合成方式は、あらかじめ実在の楽器音を FFT 解析して得た周波数・振幅データ（ルックアップテーブル、以下 LUT）をもとに多数の正弦波を重ね合わせる加算合成を基本とし、これに息や弦の摩擦音を模したノイズ成分を加えることで音色を再現している。

続いて、Processing プログラムのファイル構成を示す。表 20 の通り、各ファイルは機能ごとに分割されている。

表 20 Processing プログラムのファイル構成

ファイル名	概要
SoundProcessingCode.pde	setup/draw/serialEvent による全体制御, シリアル通信の受信処理, デバッグ用のキー入力処理
Utils.pde	LUT データの読み込みと保持, ピッチ・ベロシティに応じた倍音プロファイルの生成を行う DataUtils クラス
InstrumentPlayer.pde	発音命令を各楽器クラスへ委譲する InstrumentPlayer クラス
NoteSampler.pde	Arduino 未接続でも単体動作確認できる自動演奏機能, および内蔵の試奏用楽譜データ
AddEffect.pde	残響音 (リバーブ) エフェクトを付加する AddReverb クラス
Flute.pde	フルートの音色合成を行う FluteInstrument クラス
Strings.pde	ストリングス (弦楽器群) の音色合成を行う StringsInstrument クラス
Piano.pde	ピアノの音色合成を行う PianoInstrument クラス
Drum.pde	キック・スネア・ハイハットの音色合成を行う KickInstrument・SnareInstrument・HiHatInstrument クラス
Horn.pde	ホルンの音色合成を行う HornInstrument クラス
Trumpet.pde	トランペットの音色合成を行う TrumpetInstrument クラス

各楽器クラスは, Minim ライブラリの提供する Instrument インタフェースを実装しており, コンストラクタで音色を構築したうえで, noteOn 関数で発音を, noteOff 関数で消音を行う共通の枠組みに従っている. 発音命令は InstrumentPlayer クラスを介して各楽器固有のクラスへ委譲され, 波形生成に必要な LUT データは DataUtils クラスに集約されている. 最終的な音声信号は, 楽器の種類に応じて AddReverb による残響エフェクトを通したうえで AudioOutput へ出力される. プログラム全文は付録に示す.

ここからは, シリアル通信による Arduino との連携, LUT データの管理 (DataUtils), 発音命令の委譲 (InstrumentPlayer), 残響エフェクト (AddEffect), デバッグ用自動演奏 (NoteSampler), 各楽器クラスによる音色合成の順に, 使用する変数・関数の仕様と処理内容について説明する.

まず, Arduino とのシリアル通信および発音ディスパッチについて説明する. SoundProcessingCode.pde の setup 関数では, Minim ライブラリの初期化, 残響用バス (longOut・shortOut) の生成, InstrumentPlayer および DataUtils の初期化, そしてシリアルポートの初期化を行う. 楽器デバイスは 1 台の PC につき 1 種類の楽器を担当する構成となっており, 定数 arduinoNumber によって, この Processing インスタンスがどの楽器として動作するかを切り替える. Main (SoundProcessingCode.pde) で使用する主な変数の仕様を表 21 に示す.

表 21 Main (SoundProcessingCode.pde) で使用する主な変数の仕様一覧

変数名	型	説明
myPort	Serial	Arduino とのシリアル通信ポート
musicBPM	int	中継機から受信した現在の BPM 値を保持（発音長さの変換に使用）
currentMasterVolume	float	指揮デバイスから受信したマスターボリューム (0.2~1.0) を保持
arduinoNumber	final byte	この Processing インスタンスが担当する楽器を指定 (1: フルート, 2: ストリングス, 3: ピアノ系, 4: ドラム)
isDebugMode	boolean	キーボード操作によるデバッグ発音を許可するかのフラグ
longOut / shortOut	Summer	残響エフェクト用の合成バス. ストリングス・フルート等の持続音は longOut へ, ドラム等の打撃音は shortOut へパッチされる
player	InstrumentPlayer	発音命令を委譲する InstrumentPlayer のインスタンス
utils	DataUtils	LUT データを保持・提供する DataUtils のインスタンス

続いて, Main (SoundProcessingCode.pde) で使用する主な関数の仕様を表 22 に示す.

表 22 Main (SoundProcessingCode.pde) で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Minim の初期化, 残響用バスの生成, InstrumentPlayer・DataUtils の初期化, シリアルポートの初期化を行う
draw	void	なし	オシロスコープおよび FFT スペクトルアナライザの描画を毎フレーム行う
serialEvent	void	Serial p	Arduino から改行区切りで届いたシリアルデータを解析し, ヘッダ ('N'/'B'/'V') に応じて発音・BPM 変更・音量変更の各処理を呼び出す
Play / keyPressed	void	なし	isDebugMode が true の場合にのみ有効な, キーボード操作による単体試奏・デバッグ用の関数

以下では、表 22 に示した関数のうち、実際の演奏に用いられる `serialEvent` 関数について、実際のコードとともに説明する。この関数は、Arduino から届いた 1 行のデータをカンマで分割し、先頭要素をヘッダとして判定する。ヘッダが 'N'（発音指示）の場合、5 個 1 組（音程・長さ・開始ベロシティ・終了ベロシティ・種別）のデータを順に読み取り、duration には BPM に応じた時間換算（ $60000 \div (\text{BPM} \times 24)$ ）を施したうえで、arduinoNumber に応じた楽器の Play 関数を呼び出す。serialEvent 関数の発音指示部分をコード 29 に示す。

コード 29 serialEvent 関数（発音指示 'N' の処理部分）

```

1  case "N" :
2      if (listLength % 5 != 0) break;
3      for (int i = 5; i <= listLength; i += 5) {
4          float pitch, duration, startVel, endVel, type;
5          try {
6              pitch    = float(list[i-4]);
7              duration = float(list[i-3]) * (2.5f / musicBPM); // ÷60000(BPM*24)
8              startVel = float(list[i-2]);
9              endVel   = float(list[i-1]);
10             type     = int(list[i]);
11         } catch (NumberFormatException e) {
12             continue;
13         }
14
15         switch (arduinoNumber) {
16             case 1 :
17                 player.PlayFlute(pitch, duration, startVel, endVel, currentMasterVolume);
18                 break;
19             case 2 :
20                 player.PlayStrings(pitch, duration, startVel, endVel, currentMasterVolume);
21                 break;
22             case 3 :
23                 if (type == 0) player.PlayPiano(pitch, duration, startVel,
24                     currentMasterVolume);
25                 else if (type == 1) player.PlayTrumpet(pitch, duration, startVel, endVel,
26                     currentMasterVolume);
27                 else if (type == 2) player.PlayHorn(pitch, duration, startVel, endVel,
28                     currentMasterVolume);
29                 break;
30             case 4 :
31                 int a = int(list[i-4]) % 12;
32                 if (a == 0) player.PlayKick(currentMasterVolume, startVel);
33                 else if (a == 3) player.PlaySnare(currentMasterVolume, startVel, endVel);
34                 else if (a == 4) player.PlayHiHat(currentMasterVolume, startVel);
35                 break;
36             default: break;
37         }
38     }
39     break;

```

ここで、種別 type による分岐は、ハッカソン第 11 回時点でトランペット (TrumpetInstrument) とホルン (HornInstrument) を追加したことに伴い、楽譜データ上で発音する楽器の種類を判別するために新設されたものである。また、ヘッダが'B' (BPM 変更) の場合は musicBPM を 60~180 の範囲に制限して更新し、ヘッダが'V' (マスターボリューム変更) の場合は currentMasterVolume を 0.2~1.0 の範囲に写像して更新する。

次に、楽器デバイスにおける LUT データの管理 (DataUtils クラス) について説明する。本デバイスでは、アイオワ大学等が公開する実楽器の音声データを FFT 解析し、各音程・各強弱 (pp/mf/ff) ごとの「周波数と振幅のペア」を JSON 形式の LUT として事前に用意している。DataUtils クラスは、この LUT の読み込みと、発音時のピッチ・ベロシティに応じた倍音プロファイルの動的な取得を担う。DataUtils クラスで使用する主な変数の仕様を表 23 に示す。

表 23 DataUtils クラスで使用する主な変数の仕様一覧

変数名	型	説明
note	float[][]	NoteSampler が参照する試奏用楽譜データ (2 次元 LUT)
fluteFFLUT / MFLUT / PPLUT 等	float[][][]	各楽器・各強弱ごとの 3 次元 LUT (音程インデックス × ピーク数 × [周波数, 振幅])。フルート・バイオリン・ビオラ・チェロ・コントラバス・ピアノ・ホルン・トランペットの各強弱分を保持
pianoDecayLUT 等	float[][]	ピアノ専用の減衰時間・うなり速度・うなり深さの 2 次元 LUT (強弱ごとに 3 種類 × 3 強弱の計 9 個)
kickLUT / snareLUT / hihatLUT	float[][]	ドラム各パートの [周波数, 強度, 減衰時間] を格納する 2 次元 LUT

続いて、DataUtils クラスで使用する主な関数の仕様を表 24 に示す。

表 24 DataUtils クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
Load2LUT	float[][]	String fileName	JSON ファイルを読み込み, float[][] の 2 次元 LUT へ変換する
Load3LUT	float[][][]	String fileName	JSON ファイルを読み込み, float[][][] の 3 次元 LUT ([周波数, 振幅] のペア配列) へ変換する
GetProfileForPitch	float[][]	float pitch, float[] lut, int baseMidiNote	指定ピッチに最も近い実測データを LUT から探索し, 元データとの半音差に応じて周波数を比例スケリング (ピッチスケリング) したうえで返す
GetDynamicProfile	float[][]	float pitch, float velocity, String type, int baseMidiNote, float basePower	pp/mf/ff の 3 つのプロファイルをベロシティに応じて選択・合成し, 振幅 0.05 未満のピークを除去したうえで, 目標音量に基づくゲイン正規化を行った最終的な [周波数, 音量] 配列を返す
GetLUT	float[][]	String type	文字列で指定した種類の LUT (ピアノの減衰特性やドラムの LUT など) を返す統合ゲッター

以下では, 表 24 に示した関数のうち, 本デバイスの音色再現の核となる GetProfileForPitch 関数と GetDynamicProfile 関数について, 実際のコードとともに説明する。はじめに,

GetProfileForPitch 関数について説明する。この関数は、要求されたピッチに最も近い有効なサンプルインデックスを LUT から探索し、そのサンプルが本来持つ基準音程 (sourceMidi) と要求ピッチとの半音差から求めた倍率 (pitchShiftFactor) を、実測された周波数構造にそのまま乗算することで、インハーモニシティ (倍音の非整数性) のデコボコを保持したまま滑らかにピッチをスケールリングする。GetProfileForPitch 関数の要部をコード 30 に示す。

コード 30 DataUtils::GetProfileForPitch 関数 (要部)

```

1 float[][] GetProfileForPitch(float pitch, float[][] lut, int baseMidiNote) {
2     float originalPitch = pitch;
3     float maxMidiNote = baseMidiNote + lut.length - 1;
4     float searchPitch = constrain(pitch, baseMidiNote, maxMidiNote);
5
6     int targetIndex = round(searchPitch) - baseMidiNote;
7     targetIndex = constrain(targetIndex, 0, lut.length - 1);
8
9     // targetIndexがnull (データ欠損) の場合は、最も近い有効なインデックスを探索する
10    int finalIndex = targetIndex;
11    // (探索処理は省略。詳細は付録のプログラム全文を参照)
12
13    float[][] sourceProfile = lut[finalIndex];
14    int numPeaks = sourceProfile.length;
15    float[][] result = new float[numPeaks][2];
16
17    // 流元サンプルの基準MIDIノート番号を逆算
18    float sourceMidi = finalIndex + baseMidiNote;
19    // 要求ピッチと流元との半音差から、周波数の倍率を算出
20    float pitchShiftFactor = pow(2.0f, (originalPitch - sourceMidi) / 12.0f);
21
22    for (int h = 0; h < numPeaks; h++) {
23        result[h][0] = sourceProfile[h][0] * pitchShiftFactor; // 実測周波数構造をそのままスケールリング
24        result[h][1] = sourceProfile[h][1]; // 振幅はそのままコピー
25    }
26    return result;
27 }

```

次に、GetDynamicProfile 関数について説明する。この関数は、まず pp/mf/ff の 3 つの強弱プロファイルを GetProfileForPitch 関数で取得したうえで、ベロシティを 0~1 に正規化し、あらかじめ定めた 3 つの閾値 (32/80/112 を 127 で除した値) と比較して、隣接する 2 つの強弱のうちベロシティに近い方のプロファイル形状を採用する。続いて、振幅 0.05 未満の微小なピークを除去したうえで、採用した各ピークの振幅の 2 乗和 (エネルギー) に対する目標音量の比からゲインを算出し、全ピークへ一括で乗算することで最終的な音量を決定する。GetDynamicProfile 関数の要部をコード 31 に示す。

コード 31 DataUtils::GetDynamicProfile 関数 (要部)

```

1 float[][] GetDynamicProfile(float pitch, float velocity, String type, int

```

```

    baseMidiNote, float basePower) {
2  float[][] profilePP, profileMF, profileFF;
3  // switch(type)によりPP/MF/FFの各LUTからGetProfileForPitchで取得（詳細略）
4
5  float normVel = constrain(velocity, 0, 127) / 127.0f;
6  float thresholdPP = 32.0f / 127.0f;
7  float thresholdMF = 80.0f / 127.0f;
8  float thresholdFF = 112.0f / 127.0f;
9
10 float[][] baseProfile;
11 float t = 0;
12 if (normVel <= thresholdPP) {
13     baseProfile = profilePP;
14 } else if (normVel <= thresholdMF) {
15     t = map(normVel, thresholdPP, thresholdMF, 0.0f, 1.0f);
16     baseProfile = (t < 0.5f) ? profilePP : profileMF;
17 } else if (normVel <= thresholdFF) {
18     t = map(normVel, thresholdMF, thresholdFF, 0.0f, 1.0f);
19     baseProfile = (t < 0.5f) ? profileMF : profileFF;
20 } else {
21     baseProfile = profileFF;
22 }
23
24 int numPeaks = baseProfile.length;
25 float ampThreshold = 0.05f;
26 int validPeakCount = 0;
27 float energy = 0;
28 float[][] finalProfile = new float[numPeaks][2]; // 実際は有効数分で確保
29
30 for (int h = 0; h < numPeaks; h++) {
31     if (baseProfile[h][1] < ampThreshold) continue; // 微小ピークは除去
32     finalProfile[validPeakCount][0] = baseProfile[h][0];
33     finalProfile[validPeakCount][1] = baseProfile[h][1];
34     energy += baseProfile[h][1] * baseProfile[h][1];
35     validPeakCount++;
36 }
37
38 float targetVolume = basePower * pow(normVel, 0.9f);
39 float gain = targetVolume / sqrt(energy + 1e-9f);
40 for (int i = 0; i < validPeakCount; i++) {
41     finalProfile[i][1] *= gain; // エネルギー正規化ゲインを適用
42 }
43 return finalProfile;
44 }

```

次に、発音命令の委譲を行う `InstrumentPlayer` クラスについて説明する。このクラスは、`AudioOutput` への参照を保持し、各 `PlayXxx` 関数が呼び出されるたびに、対応する楽器クラスの

インスタンスをその場で生成して `out.playNote` 関数へ渡すという単純な委譲処理のみを行う。
`InstrumentPlayer` クラスで使用する主な関数の仕様を表 25 に示す。

表 25 `InstrumentPlayer` クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
<code>PlayFlute</code>	<code>/ void</code>	<code>float pitch, float duration,</code>	音程・長さ・開始ベロシティ・終了ベロシティ・マスターボリュームを引数に取り、対応する <code>Instrument</code> のインスタンスを <code>out.playNote</code> へ渡す
<code>PlayStrings</code>		<code>float startVel, float endVel,</code>	
<code>/ PlayHorn</code>	<code>/</code>	<code>float masterVol</code>	
<code>PlayTrumpet</code>			
<code>PlayPiano</code>	<code>void</code>	<code>float pitch, float velocity, float masterVol</code>	音程・ベロシティ・マスターボリュームを引数に取り、 <code>PianoInstrument</code> を再生する（開始・終了ベロシティの区別はない）
<code>PlayKick</code>	<code>/ void</code>	<code>float masterVol, float velocity</code>	マスターボリューム・ベロシティ等を引数に取り、対応するドラムパートの <code>Instrument</code> を再生する
<code>PlaySnare</code>	<code>/</code>		
<code>PlayHiHat</code>			

`InstrumentPlayer` クラスの `PlayFlute` 関数をコード 32 に示す。他の `PlayXxx` 関数も、生成する `Instrument` クラスが異なるのみで、同様の構造となっている。

コード 32 `InstrumentPlayer::PlayFlute` 関数

```

1 void PlayFlute(float midiNote, float duration, float startVel, float endVel, float
  masterVol) {
2   out.playNote(0.0f, duration,
3     new FluteInstrument(midiNote, duration, startVel, endVel, masterVol));
4 }

```

次に、残響エフェクトを付加する `AddEffect.pde` の `AddReverb` クラスについて説明する。このクラスは `Minim` ライブラリの `UGen` を継承しており、講義で扱われたたたみ込み積分（コンボリューション）を、指数関数的に減衰するランダムノイズを疑似的なインパルス応答として用いることで実装している。左右のチャンネルの記憶バッファ（`historyBufferL`・`historyBufferR`）を独立させることで、ステレオ音場に対応している。`AddReverb` クラスの心臓部である `uGenerate` 関数をコード 33 に示す。

コード 33 `AddReverb::uGenerate` 関数

```

1 protected void uGenerate(float[] channels) {
2   float[] inputs = audio.getLastValues();
3   float inL = inputs[0];
4   float inR = (inputs.length > 1) ? inputs[1] : inL;

```

```

5
6 historyBufferL[bufferIndex] = inL;
7 historyBufferR[bufferIndex] = inR;
8
9 float outL = 0;
10 float outR = 0;
11 for (int j = 0; j < kernelSize; j++) {
12     int idx = (bufferIndex - j + kernelSize) % kernelSize;
13     outL += historyBufferL[idx] * impulseResponse[j];
14     outR += historyBufferR[idx] * impulseResponse[j];
15 }
16
17 bufferIndex = (bufferIndex + 1) % kernelSize;
18
19 if (channels.length == 1) {
20     channels[0] = inL + outL;
21 } else if (channels.length >= 2) {
22     channels[0] = inL + outL;
23     channels[1] = inR + outR;
24 }
25 }

```

setup 関数内では、持続音向けの残響バス longOut（カーネルサイズ 15000, レベル 0.20）と、打撃音向けの残響バス shortOut（カーネルサイズ 2000, レベル 0.25）の 2 種類の AddReverb が生成されており、各楽器クラスは noteOn 関数内で masterEnv をこのいずれかへパッチすることで、音色に適した残響を選択している。

次に、デバッグ用の自動演奏機能を提供する NoteSampler.pde について説明する。本機能は、指揮デバイスや中継機デバイスが未接続の状態でも PC 単体で音色の検証を行えるようにするためのものであり、Processing の標準描画ループ（draw）に依存すると波形描画の負荷でタイミングがずれるため、音楽専用のスレッド（audioThread）を用いて高精度にタイミングを管理している。NoteSampler.pde で使用する主な変数の仕様を表 26 に示す。

表 26 NoteSampler.pde で使用する主な変数の仕様一覧

変数名	型	説明
tick	volatile int	現在の演奏位置 (24 分音符単位の tick 数)
interval	volatile long	1 tick あたりのナノ秒単位の時間間隔
noteIndex	volatile int	Note 配列内で次に処理する音符のインデックス
returnTick[][]	volatile int[][]	ループ再生のためのジャンプ情報 (トリガー tick, ジャンプ先 tick, ジャンプ先 noteIndex) を格納する配列
isPlaying	volatile boolean	自動演奏中かどうかを示すフラグ
audioThread	Thread	高精度なタイミング管理を行う音楽専用スレッド
Note[][]	int[][]	試奏用に内蔵された楽譜データ (tick, 長さ, 音程, 開始ベロシティ, 終了ベロシティ, 種別)

続いて, NoteSampler.pde で使用する主な関数の仕様を表 27 に示す.

表 27 NoteSampler.pde で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
GetInterval	long	int playBPM	指定 BPM から 1 tick あたりのナノ秒間隔を算出する ($60000000000 \div (\text{BPM} \times 24)$)
SoundPlay	void	なし	音楽専用スレッドから毎 tick 呼び出され, returnTick によるループ判定を行った後, 現在の tick に該当する音符を Note 配列から取得して timbre に応じた PlayXxx 関数を呼び出す

SoundPlay 関数をコード 34 に示す.

コード 34 SoundPlay 関数

```

1 void SoundPlay() {
2   if (tick == returnTick[returnIndex][0]) {
3     tick = returnTick[returnIndex][1];
4     noteIndex = returnTick[returnIndex][2];
5     returnIndex++;
6     if (returnIndex == returnTick.length) returnIndex = 0;
7   }
8 }

```

```

9  if (noteIndex < Note.length) {
10     float durationTuning = interval / 1000000000.0f;
11     while (Note[noteIndex][0] == tick) {
12         float duration = float(Note[noteIndex][1]) * durationTuning;
13         float pitch = float(Note[noteIndex][2]);
14         float startVelocity = float(Note[noteIndex][3]);
15         float endVelocity = float(Note[noteIndex][4]);
16         float type = (Note[noteIndex].length == 6) ? float(Note[noteIndex][5]) : 0;
17         switch (timbre) {
18             case "Flute": case "Flute2":
19                 player.PlayFlute(pitch, duration, startVelocity, endVelocity, 1.0f);
20                 break;
21             // 以下、Strings/Piano/Drum/Horn/Trumpetも同様に分岐（詳細は付録参照）
22         }
23         noteIndex ++;
24         if (noteIndex == Note.length) break;
25     }
26 }
27 }

```

最後に、各楽器クラスによる音色合成について説明する。いずれの楽器クラスも、コンストラクタ内で DataUtils から取得した倍音プロファイルをもとに正弦波（Oscil）を加算合成し、ADSR（Attack, Decay, Sustain, Release）エンベロープおよび Line による時間変化を付加したうえでミキサー（Summer）へ結線し、noteOn 関数でエンベロープと Line を一斉に起動する、という共通の設計に従っている。以下では、各楽器クラスに特有の合成技法について、実際のコードとともに説明する。

フルート（FluteInstrument） まず、フルートの音色合成について説明する。FluteInstrument クラスは、DataUtils から取得した倍音プロファイルをもとに正弦波を加算合成する層と、息が管に吹き込まれる音（Chiff）を再現するノイズ層の2層構造で構成される。加算合成の各倍音には、globalBreathSpeed という1つの共通の乱数速度で駆動されるビブラート（FM 変調）と、倍音ごとに個別の深さを持つ音量の呼吸的揺らぎ（AM 変調）が付加されている。FluteInstrument クラスで使用する主な変数の仕様を表 28 に示す。

表 28 FluteInstrument クラスで使用する主な変数の仕様一覧

変数名	型	説明
numHarmonics	byte	実測データから動的に決定される倍音 (Oscil) の数
masterEnv	ADSR	音全体の発音・消音を制御するエンベロープ
harmonicEnvs[]	ADSR[]	各倍音の立ち上がりを個別に制御するエンベロープ配列
ampLines[]	Line[]	開始ベロシティと終了ベロシティの間で各倍音の音量を滑らかに変化させる Line 配列
wobbleLines[] wobbleFactors[]	/ Line[] / float[]	倍音ごとの音量揺らぎ (AM 変調) の深さをフェードさせる Line 配列と、その揺らぎ比率
breathEnv	ADSR	息のノイズ (Chiff) 成分のエンベロープ

続いて、FluteInstrument クラスで使用する主な関数の仕様を表 29 に示す。

表 29 FluteInstrument クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
FluteInstrument (コンストラクタ)	なし	float midiNote, float duration, float startVel, float endVel, float masterVol	DataUtils から取得した倍音プロファイルをもとに、加算合成層と息ノイズ層を構築する
noteOn	void	float dur	masterEnv・harmonicEnvs を起動し、ampLines・wobbleLines を開始値から終了値へフェードさせ、masterEnv を longOut へパッチする
noteOff	void	なし	masterEnv および harmonicEnvs を消音し、リリース完了後に longOut からアンパッチする

以下では、コンストラクタと noteOn 関数について、実際のコードとともに説明する。まず、加算合成部分について説明する。各倍音について Oscil を生成し、位相をランダムに散らしたうえで、ビブラート用の Oscil (freqController) を周波数へ、音量揺らぎ用の Oscil (ampLFO) を振幅へそれぞれパッチしている。加算合成部分をコード 35 に示す。

コード 35 FluteInstrument コンストラクタ (加算合成部分)

```

1  for (int i = 0; i < numHarmonics; i++) {
2      float targetFreq = startProfile[i][0];
3      Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
4      osc.setPhase(random(1.0f)); // アタック音の機械的な均一さを防ぐ
5
6      float baseAmp = startProfile[i][1] * volFactor;
7      startAmps[i] = baseAmp * startGainFactor;
8      endAmps[i] = baseAmp * endGainFactor;
9
10     // ビブラート (FM変調)
11     float vibDepthHz = targetFreq * 0.0015f;
12     Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz/10, Waves.SINE);
13     freqController.offset.setLastValue(targetFreq);
14     freqController.patch(osc.frequency);
15
16     // 音量揺らぎ (AM変調) : ウナリ自体をフェードさせる
17     wobbleFactors[i] = wobbleDepths[i];
18     wobbleLines[i] = new Line();
19     Oscil ampLFO = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE);
20     ampLFO.setPhase(random(1.0f));
21     wobbleLines[i].patch(ampLFO.amplitude);
22     ampLFO.patch(osc.amplitude);
23
24     ampLines[i] = new Line();
25     ampLines[i].patch(osc.amplitude);
26
27     float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
28     harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
29     osc.patch(harmonicEnvs[i]);
30     harmonicEnvs[i].patch(mixer);
31 }

```

次に、息のノイズ (Chiff) 部分について説明する。ホワイトノイズをバンドパスフィルター (MoogFilter) に通すことで、息が管に吹き込まれる瞬間の「シュッ」という音のみを抽出している。息ノイズ部分をコード 36 に示す。

コード 36 FluteInstrument コンストラクタ (息ノイズ部分)

```

1  float startNorm = constrain(startVel, 0, 127) / 127.0f;
2  float attackNoiseVol = masterVol * 0.008f * pow(startNorm, 0.9f);
3  Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
4
5  MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);

```

```

6 breathFilter.type = MoogFilter.Type.BP;
7
8 breathEnv = new ADSR(1.0f, 0.001f, 0.2f, 0.0f, 0.0f);
9 breathNoise.patch(breathFilter);
10 breathFilter.patch(breathEnv);
11 breathEnv.patch(mixer);

```

最後に、noteOn 関数について説明する。この関数は、masterEnv と harmonicEnvs を起動したうえで、ampLines（ベース音量）と wobbleLines（揺らぎの深さ）を、音符の長さに 0.5 秒の余裕を加えた safeDur にわたって開始値から終了値へフェードさせる。noteOn 関数をコード 37 に示す。

コード 37 FluteInstrument::noteOn 関数

```

1 void noteOn(float dur) {
2     masterEnv.noteOn();
3     float safeDur = dur + 0.5f;
4
5     for(int i = 0; i < numHarmonics; i++) {
6         harmonicEnvs[i].noteOn();
7         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
8         if (wobbleLines[i] != null) {
9             float factor = wobbleFactors[i];
10            wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
11        }
12    }
13    breathEnv.noteOn();
14    masterEnv.patch(longOut);
15 }

```

ストリングス (StringsInstrument) 次に、ストリングスの音色合成について説明する。StringsInstrument クラスは、単一の楽器データではなく、コントラバス・チェロ・ビオラ・バイオリンの 4 種類の LUT を音程に応じてクロスフェード（混合）させることで、低音から高音まで滑らかに音色が変化する弦楽器群を表現している点が最大の特徴である。音程が低いほどコントラバスの比率が高く、音程が上がるにつれてチェロ・ビオラ・バイオリンへと比率が滑らかに移行する。StringsInstrument クラスで使用する主な変数・関数の仕様を表 30 に示す。

表 30 StringsInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
maxTotalOscils (変数)	4 楽器分の混合に対応するため多めに確保した Oscil の最大本数 (40)
totalActiveOscils (変数)	実際に生成された Oscil の本数
createInstrumentOscillators (関数)	4 の楽器 (コントラバス等) の LUT プロファイルを、振幅の大きい順に上限本数まで選別したうえで Oscil 群として生成し、ミキサーへ接続する
noteOn / noteOff (関数)	生成された totalActiveOscils 本の Oscil に対して、音量・ビブラート・音量揺らぎのフェードを一括で起動・終了する

まず、楽器ブレンド比率の算出について説明する。音程 (midiNote) の値に応じて、隣接する 2 楽器間の比率を map 関数で線形補間する。ブレンド比率の算出部分をコード 38 に示す。

コード 38 StringsInstrument コンストラクタ (楽器ブレンド比率の算出)

```

1  float baseBassRatio = 0.0f, baseCelloRatio = 0.0f, baseViolaRatio = 0.0f,
    baseViolinRatio = 0.0f;
2
3  if (midiNote <= 36) {
4      baseBassRatio = 1.0f;
5  } else if (midiNote <= 48) {
6      baseBassRatio = map(midiNote, 36, 48, 1.0f, 0.0f);
7      baseCelloRatio = 1.0f - baseBassRatio;
8  } else if (midiNote <= 60) {
9      baseCelloRatio = map(midiNote, 48, 60, 1.0f, 0.0f);
10     baseViolaRatio = 1.0f - baseCelloRatio;
11 } else if (midiNote <= 72) {
12     baseViolaRatio = map(midiNote, 60, 72, 1.0f, 0.0f);
13     baseViolinRatio = 1.0f - baseViolaRatio;
14 } else {
15     baseViolinRatio = 1.0f;
16 }
```

次に、createInstrumentOscillators 関数について説明する。この関数は、指定された楽器の LUT プロファイルを振幅の大きい順にソートしたうえで上限本数 (maxPeaks) まで選別し、周波数の低い順に並べ直してから Oscil として生成する。これにより、計算負荷を抑えつつ音色に対する寄与の大きい成分を優先的に採用している。createInstrumentOscillators 関数の要部をコード 39 に示す。

コード 39 StringsInstrument::createInstrumentOscillators 関数 (要部)

```

1  private void createInstrumentOscillators(final float[][] profile, float blendRatio,
    int maxPeaks,
2      float volFactor, float globalVibSpeed, float fmDepthCents, Summer mixer,
```

```

3   float startGainFactor, float endGainFactor) {
4   if (profile == null || profile.length == 0) return;
5
6   // 振幅の大きい順にインデックスをソートし、上位limitPeaks個を抽出（詳細は付録参照）
7   // その後、周波数の低い順に並べ直す
8
9   for (int h = 0; h < limitPeaks; h++) {
10    if (totalActiveOscils >= maxTotalOscils) break;
11
12    float targetFreq = filteredProfile[h][0];
13    float rawAmp      = filteredProfile[h][1];
14    if (targetFreq > 14000.0f) continue;
15
16    // 7000Hz~17000Hzにかけて振幅を滑らかに減衰させる高域フィルター
17    if (targetFreq > 7000.0f) {
18        float filterFactor = map(targetFreq, 7000.0f, 17000.0f, 1.0f, 0.0f);
19        rawAmp *= constrain(filterFactor, 0.0f, 1.0f);
20    }
21
22    float baseAmp = rawAmp * volFactor * blendRatio;
23    int idx = totalActiveOscils;
24    startAmps[idx] = baseAmp * startGainFactor;
25    endAmps[idx]   = baseAmp * endGainFactor;
26
27    oscils[idx] = new Oscil(targetFreq, 0.0f, Waves.SINE);
28    oscils[idx].setPhase(random(1.0f));
29    oscils[idx].patch(mixer);
30    totalActiveOscils++;
31 }
32 }

```

ピアノ (PianoInstrument) 次に、ピアノの音色合成について説明する。PianoInstrument クラスは、1つの音に対して2本の弦をわずかにデチューン（周波数をずらす）して重ねる「デュアル・ストリング構造」により、時間が経つにつれて自然に生じる音のうなり（Beat）を再現している点が特徴である。また、実測データ（LUT）が存在しない倍音・音域については、ピアノの物理特性の傾向を数式化した近似モデルで補完するハイブリッド方式を採用している。PianoInstrument クラスで使用する主な変数・関数の仕様を表 31 に示す。

表 31 PianoInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
masterEnv (変数)	ダンパー（離鍵時のミュート）の挙動を表す全体エンベロープ
hammerEnv (変数)	ハンマーが弦を叩く瞬間のノイズ用エンベロープ
damps (変数)	芯の弦・共鳴弦それぞれの Damp（減衰）オブジェクトを保持する ArrayList
PianoInstrument (コンストラクタ)	ベロシティ層の決定、倍音ごとのデュアル・ストリング構築、ハンマーノイズの生成を行う
noteOn / noteOff (関数)	masterEnv・hammerEnv および全 damps 要素を一斉に起動・終了する

以下では、デュアル・ストリング構造の構築部分について説明する。各倍音について、アタックの芯となるレイヤー A (coreOsc) と、時間差で立ち上がる共鳴弦のレイヤー B (resOsc) の 2 層を生成する。レイヤー B はレイヤー A よりわずかに周波数をずらし (wobbleRate 分のデチューン)、buildUpTime 秒かけてゆっくり音量を上げることで、芯の音と干渉して自然なうなりを生む。デュアル・ストリング構造の構築部分をコード 40 に示す。

コード 40 PianoInstrument コンストラクタ (デュアル・ストリング構造)

```

1 // レイヤー A: アタックの芯 (Dry弦)
2 Oscil coreOsc = new Oscil(targetFreq, amp, Waves.SINE);
3 coreOsc.setPhase(random(0.0f, 0.1f));
4 Damp coreDamp = new Damp(0.001f, decayTime);
5 coreOsc.patch(coreDamp).patch(mixer);
6 damps.add(coreDamp);
7
8 // レイヤー B: 時間差で来る響き (共鳴弦)
9 float reverbAmp = amp * wobbleDepth;
10 if (reverbAmp > 0.0001f && wobbleRate > 0.0f) {
11     // 周波数をわずかにズラす (Detune) ことで、芯の弦と干渉して自然な「うなり(Beat)」を生む
12     Oscil resOsc = new Oscil(targetFreq + wobbleRate, reverbAmp, Waves.SINE);
13     Damp resDamp = new Damp(buildUpTime, decayTime);
14     resOsc.patch(resDamp).patch(mixer);
15     damps.add(resDamp);
16 }

```

続いて、ハンマーの打撃ノイズについて説明する。音程の高低に応じてフィルターのカットオフを変化させることで、高音の鍵盤ほど硬い「カチッ」としたノイズになるよう調整している。ハンマーノイズ部分をコード 41 に示す。

コード 41 PianoInstrument コンストラクタ (ハンマーノイズ部分)

```

1 float hammerVol = masterVol * 0.003f * normVel;
2 Noise hammerNoise = new Noise(hammerVol, Noise.Tint.WHITE);

```

```

3
4 float hammerCutoff = map(midiNote, 21, 108, 600f, 4000f);
5 MoogFilter hammerFilter = new MoogFilter(hammerCutoff, 0.1f);
6 hammerFilter.type = MoogFilter.Type.LP;
7
8 hammerEnv = new ADSR(1.0f, 0.001f, 0.02f, 0.0f, 0.01f);
9 hammerNoise.patch(hammerFilter).patch(hammerEnv);

```

ドラム (KickInstrument・SnareInstrument・HiHatInstrument) 次に、ドラムの音色合成について説明する。ドラムはフルート等の旋律楽器とは異なり、Instrument クラスをキック・スネア・ハイハットの3種類（および過去に試作された SnareInstrumentOld・HiHatInstrumentOld の旧版、未使用の CymbalInstrument）に分けて実装している。いずれも短時間で減衰する打撃音であるため、全体の出力は shortOut バスへパッチされる。3種の主要な変数・関数の仕様を表32に示す。

表 32 ドラム関連クラスで使用する主な変数・関数の仕様一覧

名称	説明
KickInstrument (コンストラクタ)	kickLUT の各成分について、衝撃の瞬間に高い周波数から本来の周波数へ急降下するピッチスweep (Line) を適用した Oscil を最大 50 本生成する
SnareInstrument (コンストラクタ)	胴鳴り (Body)、倍音 (Overtone)、スナッピー (ノイズ) の3パートを個別の Oscil / Noise とフィルターで構築する
HiHatInstrument (コンストラクタ)	実測ピークに基づく複数の金属共鳴用 Oscil と、高域ノイズによる打撃摩擦音を合成する
noteOn / noteOff (各クラス共通)	各パートのエンベロープを一斉に起動・終了し、masterEnv を shortOut へパッチ／アンパッチする

以下では、キックのピッチスweepと、スネアの胴鳴りパートについて、実際のコードとともに説明する。まず、KickInstrument のピッチスweepについて説明する。各周波数成分について、Line を用いて 0.01 秒かけて「本来の周波数の 1.5 倍」から「本来の周波数」へ直線的に下げること、太鼓の膜が叩かれた瞬間にピッチが下がっていく物理的な挙動を再現している。該当部分をコード 42 に示す。

コード 42 KickInstrument コンストラクタ (ピッチスweep部分)

```

1 for (int i = 0; i < numComponents; i++) {
2   if (kickLUT[i] == null || kickLUT[i].length < 3) continue;
3
4   float freq = kickLUT[i][0] * random(0.95f, 1.05f);
5   float mag = kickLUT[i][1];
6   float decayTime = kickLUT[i][2];
7   float amp = (baseAmp * mag) / numComponents;
8

```

```

9   tones[i] = new Oscil(freq, amp, Waves.SINE);
10  harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime*0.5f, 0.0f, 0.1f);
11
12  // ピッチスイープ：1.5倍の高さから本来の周波数へ急降下させる
13  pitchSweeps[i] = new Line(0.01f, freq * 1.5f, freq);
14  pitchSweeps[i].patch(tones[i].frequency);
15
16  tones[i].patch(harmonicEnvs[i]);
17  harmonicEnvs[i].patch(mixer);
18 }

```

次に、SnareInstrument の胴鳴りパートについて説明する。発音の瞬間に約 3 倍の周波数から本来の胴鳴り周波数（172.3 Hz）へ急降下させるピッチエンベロープにより、スネアドラム特有のアタック感を表現している。該当部分をコード 43 に示す。

コード 43 SnareInstrument コンストラクタ（胴鳴りパート）

```

1  float bodyFreq      = 172.3f * random(0.99f, 1.01f); // 実測データの最強ピーク
2  float bodyPitchMod  = 3.0f; // アタック時は約516Hzから172.3Hzへ急降下
3  float bodySweepTime = 0.025f;
4  float bodyDecay     = 0.07f;
5  float bodyVolRatio  = 0.63f * sinSet;
6
7  bodyOsc = new Oscil(bodyFreq, baseAmp * bodyVolRatio, Waves.SINE);
8  bodyPitchEnv = new Line(bodySweepTime, bodyFreq * bodyPitchMod, bodyFreq);
9  bodyPitchEnv.patch(bodyOsc.frequency);
10 bodyAmpEnv = new ADSR(1.0f, 0.001f, bodyDecay, 0.0f, 0.05f);
11 bodyOsc.patch(bodyAmpEnv).patch(mixer);

```

ホルンとトランペット（HornInstrument・TrumpetInstrument） 最後に、ホルンおよびトランペットの音色合成について説明する。両クラスは、ハッカソン第 11 回時点で追加された金管楽器であり、事前課題の報告の通り、既存の FluteInstrument クラスの加算合成・ビブラート・息ノイズの構造をそのまま流用したうえで、KickInstrument で用いていたアタック時のピッチエンベロープ機能を新たに付加して作成されている。HornInstrument と TrumpetInstrument は、基準 MIDI ノートや各種パラメータの数値が異なるのみで、クラス構造自体は同一である。両クラスで使用する主な変数・関数の仕様を表 33 に示す。

表 33 HornInstrument・TrumpetInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
pitchEnvLines[] (変数)	アタック時のピッチ急降下を制御する Line 配列 (Flute 由来の構造に新規追加)
targetFreqs[] (変数)	各倍音の本来の周波数を保持する配列, noteOn 時のピッチエンベロープ計算に使用
HornInstrument TrumpetInstrument (コンストラクタ)	/ Flute 同様の加算合成・ビブラート・息ノイズ構造に加え, 各倍音の pitchEnvLines を Oscil の周波数 LFO の offset へパッチする
noteOn (関数)	masterEnv・harmonicEnvs の起動と ampLines・wobbleLines のフェードに加え, pitchEnvLines を「本来周波数の (1+pitchMod) 倍」から「本来周波数」へ sweepTime 秒でフェードさせる

以下では, 両クラスに共通するアタック時のピッチエンベロープ機能について, HornInstrument の noteOn 関数を例に説明する. 発音直後, 各倍音の周波数を本来値より一時的に高く (ホルンでは pitchMod=0.05, すなわち 5%増しから) 設定し, sweepTime 秒 (ホルンでは 0.12 秒) かけて本来の周波数へ直線的に戻すことで, 金管楽器特有の「音の頭が一瞬上ずってから定まる」挙動を再現している. 該当部分をコード 44 に示す. なお, トランペットでは同じ構造に対して pitchMod=0.03, sweepTime=0.07 秒という異なる数値が用いられている.

コード 44 HornInstrument::noteOn 関数 (ピッチエンベロープ部分)

```

1 float sweepTime = 0.12f; // 変化にかかる時間
2 float pitchMod = 0.05f; // 本来の周波数に対する上乗せ割合
3
4 for(int i = 0; i < numHarmonics; i++) {
5     harmonicEnvs[i].noteOn();
6     ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
7     if (wobbleLines[i] != null) {
8         float factor = wobbleFactors[i];
9         wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
10    }
11
12    // アタック時のピッチエンベロープ (周波数の急降下)
13    if (pitchEnvLines[i] != null) {
14        float baseFreq = targetFreqs[i];
15        float startFreq = baseFreq * (1.0f + pitchMod); // 本来の周波数にpitchMod分を上乗
16        // せした値からスタート
17        pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq); // baseFreqへ一直線に
18        // 下げる
19    }
20 }

```


プログラム全文は付録に示す。

楽器デバイスの同期方式 (douki_saigo) 次に、各楽器デバイスの同期方式を担う Arduino プログラムについて説明する。楽器デバイスは、指揮デバイスからの演奏開始指示を受けた後、中継機デバイスの回転に伴うレーザー光を CdS セルで検知することでおおまかな BPM を推定し、さらに楽器機同士の UDP 通信によって発音タイミングを細かく補正することで、複数台の Arduino 間での高精度な同期を実現している。この同期方式では、ドラムを担当する Arduino を基準時刻源 (マスター) とし、ピアノ・ストリングス・フルートを担当する 3 台の Arduino をスレーブとするマスタースレーブ方式が採用されている。

続いて、同期プログラムのファイル構成を示す。表 34 の通り、各ファイルは担当する楽器ごとに用意されている。

表 34 同期プログラム (douki_saigo) のファイル構成

ファイル名	概要
drum_0706.ino	ドラム担当機のプログラム。同期のマスターとして動作し、全楽器機の発音タイミングを管理する
flute_0706.ino	フルート担当機のプログラム。同期のスレーブとして動作する
piano_0706.ino	ピアノ担当機のプログラム。同期のスレーブとして動作する
strings_0706.ino	ストリングス担当機のプログラム。同期のスレーブとして動作する

flute_0706.ino・piano_0706.ino・strings_0706.ino の 3 ファイルは、IP アドレス・楽器 ID・CdS 検知しきい値・埋め込みの楽譜データが異なるのみで、通信・同期ロジックの構造は完全に同一である。そのため、以下ではスレーブ側の代表例として flute_0706.ino を用いて説明し、マスター側は drum_0706.ino を用いて説明する。

次に、各 Arduino 間の同期に用いられる通信ヘッダの一覧を示す。表 35 の通り、指揮デバイスからの一斉指示に加え、マスター・スレーブ間で複数種類のパケットがやり取りされる。

表 35 楽器機間の同期通信で使用するヘッダの一覧

ヘッダ	送信元 → 送信先	概要
'S'	指揮デバイス → 全楽器機	演奏開始指示, 受信した各機は状態を初期化する
'E'	指揮デバイス → 全楽器機	演奏終了指示, 受信した各機は状態を停止状態に戻す
'V'	指揮デバイス → 全楽器機	音量段階の通知
'J'	スレーブ → マスター	同期への参加予告
'M'	スレーブ → マスター	レーザー検知による BPM 提案
'N'	マスター → 各スレーブ	多数決により決定した確定 BPM のブロードキャスト
'T'	スレーブ → マスター	自身の globalTick の報告 (diff 算出依頼)
'R'	マスター → スレーブ	T 受信時点での diff (Tick 差分) の返信
'O'	マスター → スレーブ	演奏開始タイミング (startGlobalTick) の指示

続いて、レーザー光の検知処理について説明する。各 Arduino は、中継機デバイスの回転周期を CdS セルで検知するために、2.2.3 項で説明した LaserDetector クラスと同一の適応しきい値方式のロジックを実装した SyncManager クラスを備えている。検知したパルス間隔から求めた実測 BPM は、マスターでは自身の BPM 推定値として、スレーブでは'M'通信による BPM 提案として用いられる。検知ロジックの詳細な実装は 2.2.3 項および付録に示すプログラム全文を参照されたい。

次に、ドラム機（マスター）における同期状態管理について説明する。マスターは、state という状態変数によって管理される 5 段階の状態遷移に沿って動作する。各状態の内容を表 36 に示す。

表 36 ドラム機（マスター）の状態遷移一覧

state	内容
0	BPM 安定待ち。レーザー検知による BPM 推定値が MIN_STABLE_COUNT 回連続で一致するまで待機する
1	楽器と同期中。参加を予告したスレーブ全機について、'T' 受信時に返す diff が閾値以内に収まるまで待機する
2	同期完了・演奏開始前。各スレーブへの'O'通信の送信タイミングを監視しつつ、ドラム自身の演奏開始タイミングを待つ
3	演奏中。elapsedPlayTick を進めながらドラム自身の楽譜を再生する
4	システム停止中。'S' 通信を受信するまで何も行わない

続いて、ドラム機（マスター）で使用する主な変数の仕様を表 37 に示す。

表 37 ドラム機（マスター）で使用する主な変数の仕様一覧

変数名	型	説明
state	int	現在の同期状態（0～4）を保持
globalTick	uint32_t	マスターを基準とする全体共通の Tick カウンタ
currentSongId	int	現在演奏中の楽譜 ID を保持. 'S' 受信のたびにローテーションする
elapsedPlayTick	int32_t	state2 移行後に進行する演奏開始からの経過 Tick
isDrumPlaying	bool	ドラム自身が演奏中かどうかを保持
isInstJoin[]	bool[4]	各楽器が同期に参加する予告（'J' 受信）をしたかを保持
isInstConnected[]	bool[4]	各楽器が'T' 通信により実際に接続済みであるかを保持
lastReceivedDiff[]	int32_t[4]	各楽器から'T' 受信時に算出した直近の diff を保持
receivedBpmLevels[]	uint8_t[4]	自身および各スレーブから提案された BPM 段階を集約する配列
currentBpmLevel	uint8_t	多数決により確定した現在の BPM 段階
oSentCount[]	uint8_t[10]	各演奏開始イベントに対する'O' 通信の送信済み回数（最大 3 回）
startSequence[] [] []	int	曲 ID ごとの各楽器の演奏開始 Tick と楽器 ID を定義する配列

続いて、ドラム機（マスター）で使用する主な関数の仕様を表 38 に示す。

表 38 ドラム機（マスター）で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Wi-Fi 接続と UDP 通信の初期化を行う
loop	void	なし	UDP 受信処理と state に応じたステートマシンの実行を毎回行う
checkSyncReady	bool	なし	参加予告のあった全スレーブが接続済みかつ diff が閾値以内であれば true を返す
processOCommands	void	なし	各演奏開始イベントについて、タイミングが近づいたスレーブへ'O' 通信を送信し、ドラム自身の演奏開始も管理する
sendOPacket_Single	void	uint8_t instId, uint8_t songId, uint32_t startGlobalTick	指定したスレーブへ'O' 通信（演奏開始タイミング指示）を送信する
sendNPacket	void	uint8_t bpm	確定 BPM を全スレーブへ'N' 通信でブロードキャストする
sendRPacket	void	uint8_t instId, int32_t diff	'T' 受信元のスレーブへ diff を'R' 通信で返信する
updateTick	void	なし	tickIval ごとに globalTick を進め、48Tick ごとの多数決 BPM 決定や自身の楽譜再生を行う
setBpm	void	uint8_t bpm	確定 BPM から tickIval を再計算する
determineSystemBpmLevel	uint8_t	⁶⁴ なし	参加中の全楽器の BPM 提案から中央値を求め、システム全体の BPM 段階を決定する

以下では、マスターの同期制御における中核部分について、実際のコードとともに説明する。まず、'T' 通信（スレーブからの Tick 報告）の受信処理について説明する。この処理は、スレーブの globalTick とマスター自身の globalTick との差である diff を算出し、'R' 通信で即座に返信する。該当部分をコード 45 に示す。

コード 45 drum_0706.ino の loop 関数 ('T' 通信受信部分)

```
1 case 'T':
2   if (pktSize >= 6) {
3     uint32_t slaveTick =
4       ((uint32_t)packetBuffer[1] << 24) |
5       ((uint32_t)packetBuffer[2] << 16) |
6       ((uint32_t)packetBuffer[3] << 8) |
7       (uint32_t)packetBuffer[4];
8     uint8_t id = packetBuffer[5];
9
10    if (id < MAX_INSTRUMENTS && id != drumInstId) {
11      isInstConnected[id] = true;
12      if (isInstJoin[id] != true) isInstJoin[id] = true;
13      int32_t diff = (int32_t)(globalTick - slaveTick);
14      lastReceivedDiff[id] = diff;
15
16      sendRPacket(id, diff);
17    }
18  }
19  break;
```

次に、同期完了判定を行う checkSyncReady 関数について説明する。この関数は、参加予告 ('J' 受信) のあった全スレーブについて、接続済みであり、かつ直近の diff が tickIval から算出したしきい値以内であるかを確認し、1 台でも条件を満たさなければ false を返す。checkSyncReady 関数をコード 46 に示す。

コード 46 checkSyncReady 関数

```
1 bool checkSyncReady() {
2   bool expectedToJoin = false;
3
4   for (uint8_t i = 1; i < MAX_INSTRUMENTS; i++) {
5     if (isInstJoin[i]) {
6       expectedToJoin = true; // 参加予定の楽器が少なくとも1台いる
7       float SYNC_READY_THRESHOLD = 50 / tickIval;
8
9       // 参加予定だが未接続、または同期のズレが大きい場合は「まだ準備未完了」
10      if (!isInstConnected[i] || abs(lastReceivedDiff[i]) > SYNC_READY_THRESHOLD) {
11        return false;
12      }
13    }
14  }
15  return expectedToJoin;
```

16 }

最後に、各楽器の演奏開始タイミングを管理する `processOCommands` 関数について説明する。この関数は、`startSequence` に定義された各演奏開始イベントについて、該当楽器がドラム自身であれば `elapsedPlayTick` が開始 Tick に達した時点で演奏を開始し、スレーブであれば開始 Tick の `O_ADVANCE_TICKS` (96Tick) 前から3段階に分けて'O' 通信を送信することで、パケットロスに備えた冗長性を確保している。該当部分をコード 47 に示す。

コード 47 `processOCommands` 関数 (要部)

```
1  if (instId == drumInstId) {
2      if (elapsedPlayTick >= evTick) {
3          if (!isDrumPlaying) {
4              isDrumPlaying = true;
5              curTick = songStartParams[currentSongId].startTick;
6              scoreIdx = songStartParams[currentSongId].startNoteIdx;
7              state = 3;
8          }
9          oSentCount[i] = 3;
10     }
11 } else {
12     if (isInstJoin[instId] && isInstConnected[instId]) {
13         uint32_t startGlobalTick = (uint32_t)((int32_t)globalTick + ticksUntilStart);
14         if (ticksUntilStart <= O_ADVANCE_TICKS && oSentCount[i] == 0) {
15             sendOPacket_Single(instId, currentSongId, startGlobalTick);
16             oSentCount[i] = 1;
17         } else if (ticksUntilStart <= O_ADVANCE_TICKS * 2 / 3 && oSentCount[i] == 1) {
18             sendOPacket_Single(instId, currentSongId, startGlobalTick);
19             oSentCount[i] = 2;
20         } else if (ticksUntilStart <= O_ADVANCE_TICKS / 3 && oSentCount[i] == 2) {
21             sendOPacket_Single(instId, currentSongId, startGlobalTick);
22             oSentCount[i] = 3;
23         }
24     }
25 }
```

次に、楽器機 (スレーブ) における同期処理について説明する。スレーブは、`state` が 0 (BPM 安定待ち)・1 (マスターと同期中)・2 (演奏中)・4 (停止中) の 4 状態で動作し、'R' 通信で受け取った `diff` をもとに自身の `tickIval` を補正することでマスターとの同期を保つ。続いて、楽器機 (スレーブ) で使用する主な変数の仕様を表 39 に示す。

表 39 楽器機（スレーブ）で使用する主な変数の仕様一覧

変数名	型	説明
baseTickIval	uint32_t	確定 BPM から算出した基本の 1Tick あたりの時間 [μs]
tickIval	uint32_t	同期補正を反映した実際に使用する 1Tick あたりの時間 [μs]
ticksToCorrect	int32_t	補正処理が残っている Tick 数
lastDiff	int32_t	直近に受信した diff. BPM 変化時の補正再計算に使用
drumNotStarted	bool	マスターがまだ globalTick の加算を開始していないかを保持
waitingForStart	bool	'O' 通信を受信し、演奏開始タイミングを待機中かどうかを保持
startGlobalTick	uint32_t	マスターから指示された演奏開始 globalTick

続いて、楽器機（スレーブ）で使用する主な関数の仕様を表 40 に示す。

表 40 楽器機（スレーブ）で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Wi-Fi 接続と UDP 通信の初期化を行う
loop	void	なし	UDP 受信処理 ('R'・'O'・'N'・'S'・'E'・'V') と state に応じたステートマシンの実行を毎回行う
applyDiffCorrection	void	int32_t diff	受信した diff の大きさに応じて tickIval を加減速し、マスターとの同期を図る
setBpm	void	uint8_t bpm	確定 BPM から baseTickIval を再計算する
sendTPacket	void	なし	自身の globalTick を 'T' 通信でマスターへ送信し、diff の算出を依頼する
sendJPacket	void	なし	同期への参加予告を 'J' 通信でマスターへ送信する
sendMPacket	void	uint8_t bpm	レーザー検知による BPM 提案を 'M' 通信でマスターへ送信する
updateTick	void	なし	tickIval ごとに globalTick を進め、補正カウントダウンや自身の楽譜再生を行う
sendPacket	void	なし	curTick に到達した音符を Note 配列から取得し、自身の Processing ヘシリアル送信する

以下では、スレーブの同期補正における中核部分について、実際のコードとともに説明する。まず、'R' 通信（マスターからの diff 返信）の受信処理について説明する。この処理は、パケットの往復時間から通信遅延を見積もって diff を補正したうえで、マスターがまだ動作していない場合の特別処理を経て、通常時は applyDiffCorrection 関数を呼び出す。該当部分をコード 48 に示す。

コード 48 flute_0706.ino の loop 関数 ('R' 通信受信部分・要部)

```

1 case 'R':
2   if (state == 0 || state == 4) break;
3   if (pktSize >= 5) {
4     int32_t rawDiff =
5       ((int32_t)(int8_t)packetBuffer[1] << 24) |
6       ((int32_t)packetBuffer[2] << 16) |
7       ((int32_t)packetBuffer[3] << 8) |
8       (int32_t)packetBuffer[4];
9
10    uint32_t rtt = micros() - lastTPacketMicroTime;
11    uint32_t latencyMicros = rtt / 2; // 片道遅延
12    int32_t latencyTicks = (baseTickIval > 0)
13      ? (int32_t)((latencyMicros + (baseTickIval / 2)) / baseTickIval) : 0;
14    int32_t diff = rawDiff - latencyTicks; // 通信遅延を差し引いた真のズレ
15
16    lastDiff = diff;
17    if (state >= 1) {
18      if (ticksToCorrect == 0 || abs(diff) >= 24) {
19        applyDiffCorrection(diff);
20      }
21    }
22  }
23  break;

```

次に、applyDiffCorrection 関数について説明する。この関数は、diff の大きさに応じて補正の強さを 4 段階に切り替える。ズレが小さい場合は不感帯として何もせず、大幅なズレの場合は tickIval を 2 倍または 2/3 倍にして急速に補正し、中程度のズレの場合は次の 48Tick で吸収するように比率補正を行う。applyDiffCorrection 関数をコード 49 に示す。

コード 49 applyDiffCorrection 関数

```

1 void applyDiffCorrection(int32_t diff) {
2   if (baseTickIval == 0) return;
3   float SYNC_READY_THRESHOLD = 50 / baseTickIval;
4
5   if (abs(diff) <= SYNC_READY_THRESHOLD) {
6     // 不感帯：ズレが小さければ何もしない
7     tickIval = baseTickIval;
8     ticksToCorrect = 0;
9   } else if (diff <= -24) {
10    // 大幅に早い → ×2 で -diff Tick間遅らせる
11    tickIval = baseTickIval * 2;
12    ticksToCorrect = -diff;
13  } else if (diff >= 24) {
14    // 大幅に遅い → ×2/3 で ×diff3 Tick間速める
15    tickIval = baseTickIval * 2 / 3;
16    ticksToCorrect = diff * 3;
17  } else {

```

```

18 // 小さいズレ → 次の48Tickでdiffを吸収する比率補正
19 tickIval = (uint32_t)((48UL * baseTickIval) / (48 + diff));
20 ticksToCorrect = 48 + diff;
21 }
22 }

```

最後に、'O' 通信（マスターからの演奏開始指示）の受信処理について説明する。この処理は、指示された startGlobalTick と自身の globalTick を比較し、まだ到達していなければ waitingForStart を立てて待機し、すでに超過していれば超過分だけ curTick をオフセットして即座に演奏へ合流する。なお、'O' 通信の受信および状態遷移に関する処理の詳細な実装は付録に示すプログラム全文を参照されたい。

プログラム全文は付録に示す。

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは、6 名で実施した。

3.2 報告者の担当範囲と技術的課題

本節では、担当したハードウェアの詳細設計と LEDMatrix での BPM 表示を行うプログラムについて説明する。

3.3 部品一覧

次に、楽器デバイスが受光するための部品について表 41 に示す。光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、観覧車デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するためそれぞれに Arduino を使用し、光センサおよびブレッドボードも 4 台用意する。

表 41 楽器デバイスに必要な機器

機材	品番	個数 [個]
Arduino UNO R4	-	4
CdS セル	GL5516	4
ブレッドボード	-	4
ジャンパーワイヤー	-	適宜

3.4 指揮デバイス

3.5 観覧車型中継機デバイス

本デバイスのモータ回転制御および LED マトリクス表示を行う制御プログラムの全文は付録 (FerrisWheel.ino, Display.h, Display.cpp, MotorControl.h, MotorControl.cpp) に示す。

3.6 レーザー光送信装置

レーザー光送信装置の詳細設計について説明する。図 18 について、回路図の通りにボタン電池とレーザーをはんだ付けする。これらを観覧車のリングに 45 度間隔に装着する。電源としては図 18 に示すようにボタン電池からの 3V 給電を行う。ここで、レーザーについて、データシートより定電流源回路を組み込んであるため、現在の設計であれば外部抵抗は必要とせず、素子を安全に利用することができる [?].

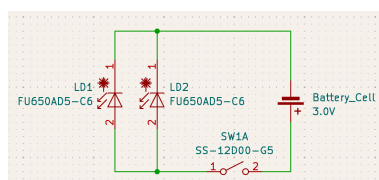


図 18 レーザー回路図

3.7 楽器デバイス受光部装置

楽器デバイスの受光部の回路図を図 19 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を Arduino で処理することで光を検知する仕様である。

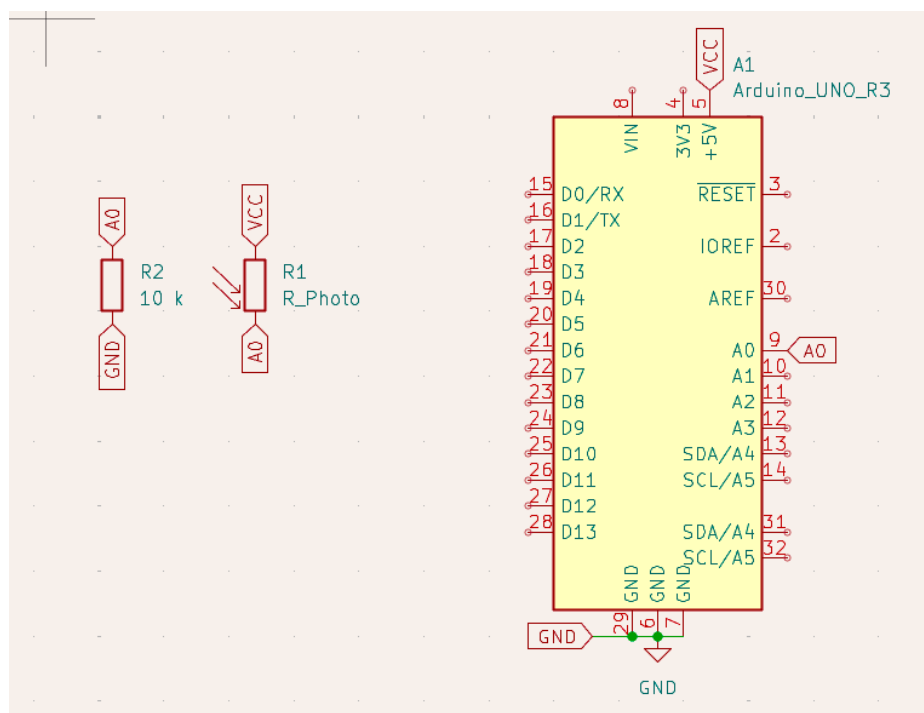


図 19 楽器デバイスの受光部の回路図

4 おわりに

これは $\text{T}_{\text{E}}\text{X}$ による報告書作成のテンプレートでした.

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場(～2031年)、市場規模(パフォーマンス、同期、デジタル収益)・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] 谷口由貴, “ストーリーミングサービスが生み出した新たな音楽消費サイクル,” 博報堂 The Hakuhodo, <https://www.hakuhodo.co.jp/magazine/61505/>, 2019/9/12, 2026/7/7 参照.
- [5] Martin Kreuzer, Melanie Wald-Fuhrmann, Christian Weining, Martin Tröndle, and Hauke Egermann, “Western Classical Music Concerts Are More Immersive, Intellectually Stimulating, and Social, When Experienced Live Rather Than in a Digital Stream: An Ecologically Valid Concert Study on Different Modes of Liveness,” *Music & Science*, vol.8, 2025.
- [6] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” *PLOS ONE*, vol.13, no.2, p.e0191846, 2018.
- [7] Martin, R., and Nielsen, N. Enacting Musical Aesthetics: The Embodied Experience of Live Music. *Music & Science*, 7, 2024
- [8] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [9] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.7, 2026, p.382-385, 2021
- [10] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.
- [11] Control.com, “DC Motor Speed Control,” *Lessons In Electric Circuits*, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, (参照 2026-05-04).

A 役割分担とスケジュール

このテンプレートでは、 $\text{T}_\text{E}\text{X}$ でガントチャートを描いてみました。

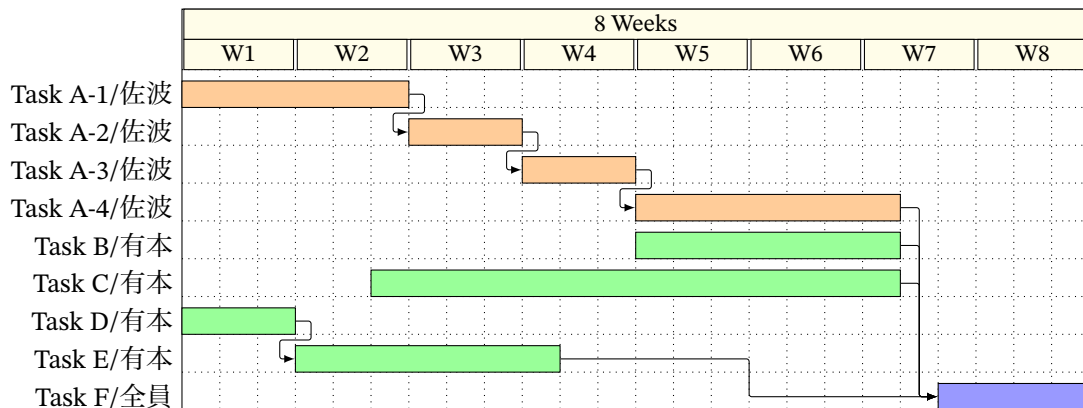


図 A.20 計画時のスケジュール

B 指揮デバイスのプログラムソース

本節では、指揮デバイスの制御プログラムの全文を示す。ファイル構成は 2.1.3 節の表 2 の通りである。

B.1 Controller_simultaneous.ino

コード 50 Controller_simultaneous.ino

```

1  #include "SyncManager.h"
2  #include "PotManager.h"
3  #include "AccManager.h"
4
5  #include <Wire.h>
6  #include <math.h>
7
8  // Wi-Fi接続情報
9  const char SSID[] = "hackathon001-WPA2";
10 const char PASS[] = "hackathon001";
11
12 const uint8_t input_PIN = 3; // スイッチの状態を読み取るピン
13 bool switch_status;
14 bool before_switch_status = 0;
15 bool isButtonPlaying = false;
16
17 SyncManager sync;
18 PotManager pot;
19 AccManager acc;
20
21 void setup() {
22     Serial.begin(115200);
23     while (!Serial) {}

```

```

24
25     Serial.println("---_Setup_Started_---");
26     pinMode(input_PIN, INPUT);
27
28     Serial.println("Attempting_to_connect_to_WiFi...");
29     sync.begin(SSID, PASS);
30
31     Serial.println("\nWiFi_Connected_successfully!");
32     Serial.println("Commands:_button_->_Start/End");
33
34     acc.setupADXL345();
35
36     before_switch_status = digitalRead(input_PIN);
37 }
38
39 void loop() {
40     button();
41     pot.update();
42     acc.update(sync, pot);
43 }
44
45 void button() {
46     switch_status = digitalRead(input_PIN);
47     if (switch_status == 0 && before_switch_status == 1) {
48         if (isButtonPlaying) {
49             Serial.println("Button:_[END]_sending_UDP...");
50             sync.sendEnd();
51             isButtonPlaying = false;
52         } else {
53             Serial.println("Button:_[START]_sending_UDP_to_all_simultaneously...");
54             uint8_t step = pot.getBpmStep();
55             sync.sendStart(step);
56             isButtonPlaying = true;
57         }
58         delay(300); // チャタリング防止
59     }
60     before_switch_status = switch_status;
61 }

```

B.2 AccManager.h

コード 51 AccManager.h

```

1 // 加速度センサー管理クラス
2 #ifndef ACC_MANAGER_H
3 #define ACC_MANAGER_H
4
5 #include <Arduino.h>

```

```

6  #include <Wire.h>
7  #include "SyncManager.h"
8  #include "PotManager.h"
9
10 class AccManager {
11 public:
12     AccManager();
13     void setupADXL345();
14     // loop内で毎ループ呼ぶ
15     void update(SyncManager& sync, PotManager& pot);
16
17     void updateShakedInfo(SyncManager& sync, PotManager& pot);
18
19 private:
20     float readAccMagnitude();
21     void recoverI2C();    // I2Cバスロックアップからの復旧（バスクリア+再初期化）
22
23     const uint8_t ADXL345_ADDR = 0x53; // SDO/ALT ADDRESS ピンがGNDの場合
24     const uint8_t REG_POWER_CTL = 0x2D; // 加速度センサの電源管理をするスイッチ
25     const uint8_t REG_DATA_FORMAT = 0x31; // 測定する範囲を設定するスイッチ
26     const uint8_t REG_DATAX0 = 0x32; // 測定した値が書き込まれる場所
27
28     const float accThreshold = 110.0f; // 検知閾値 必要に応じて調整
29     const unsigned long shakeInterval = 750; // 多重検知防止インターバル [ms]
30     const unsigned long ACC_SAMPLE_INTERVAL = 20;
31
32     // 内部変数
33     float currentAccVal = 0.0f; // 現在の加速度センサ値
34     float lastAccVal = 0.0f; // 前回の加速度センサ値
35     unsigned long lastDetectTime = 0; // 前回検知時刻 [ms]
36     unsigned long lastSampleTime = 0;
37     unsigned long lastRecoverMs = 0; // 復旧ログのスロットル用
38
39     bool isStarted = false; // 振られた判定のフラグ
40 };
41
42 #endif

```

B.3 AccManager.cpp

コード 52 AccManager.cpp

```

1  #include "AccManager.h"
2
3  AccManager::AccManager(){
4      // コンストラクタ（初期化処理なし）
5  }
6

```



```

7 void AccManager::setupADXL345() {
8     Wire.begin();
9
10    Wire.beginTransmission(ADXL345_ADDR);
11    Wire.write(REG_POWER_CTL);
12    Wire.write(0x08); // Measure bit ON
13    Wire.endTransmission();
14
15    Wire.beginTransmission(ADXL345_ADDR);
16    Wire.write(REG_DATA_FORMAT);
17    Wire.write(0x00);
18    Wire.endTransmission();
19
20    lastAccVal = readAccMagnitude();
21
22    Serial.println("GY-291_初期化完了");
23    Serial.print("閾値:");
24    Serial.print(accThreshold);
25    Serial.print("インターバル:");
26    Serial.print(shakeInterval);
27    Serial.println("_ms");
28 }
29
30 // ★loopから引っ越してきた時間管理とシェイク判定
31 void AccManager::update(SyncManager& sync, PotManager& pot) {
32     unsigned long now = millis();
33
34     if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
35         lastSampleTime = now;
36
37         currentAccVal = readAccMagnitude();
38         float diff = fabsf(currentAccVal - lastAccVal);
39
40         bool overThreshold = (diff > accThreshold);
41         bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
42
43         // 500msごとにdiff値を表示 (閾値チューニング用)
44         static unsigned long lastDebugTime = 0;
45         if (now - lastDebugTime >= 500) {
46             lastDebugTime = now;
47             //Serial.print("[ACC] diff="); Serial.print(diff, 1);
48             //Serial.print(" / 閾値="); Serial.println(accThreshold, 1);
49         }
50
51         if (overThreshold && intervalPassed) {
52             lastDetectTime = now;
53             isStarted = !isStarted;
54

```

```

55     Serial.print("[SHAKE検知]_diff=");
56     Serial.print(diff, 4);
57     Serial.println(isStarted ? "_START送信" : "_END送信");
58
59     updateShakedInfo(sync, pot);
60 }
61
62     lastAccVal = currentAccVal;
63 }
64 }
65
66 float AccManager::readAccMagnitude() {
67     Wire.beginTransaction(ADXL345_ADDR);
68     Wire.write(REG_DATA_X0);
69     // endTransmission!=0 はNACK/バス異常。リピーテッドスタート前に検出する。
70     if (Wire.endTransmission(false) != 0) {
71         recoverI2C();
72         return lastAccVal;    // 直前値を返してニセshakeを防ぐ
73     }
74
75     uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
76     if (n < 6) {                // 6バイト読めない=ロックアップ等
77         recoverI2C();
78         return lastAccVal;
79     }
80
81     int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8)));
82     int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8)));
83     int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8)));
84
85     float fx = (float)rawX;
86     float fy = (float)rawY;
87     float fz = (float)rawZ;
88
89     return sqrtf(fx * fx + fy * fy + fz * fz);
90 }
91
92 // I2Cバスのロックアップから復旧する。
93 // スレーブがSDAを握ったまま固まった場合、SCLを手で9回叩いて解放し、
94 // STOP条件を作ってから Wire を張り直し、ADXL345 を再初期化する。
95 void AccManager::recoverI2C() {
96     unsigned long now = millis();
97     if (now - lastRecoverMs >= 1000) {    // ログのスロットル（毎20msの連発を防ぐ）
98         lastRecoverMs = now;
99         Serial.println("[ACC]_I2C異常検出→_バス復旧を実行");
100     }
101
102     Wire.end();

```

```

103
104 // バスクリア: SCLを9クロック叩いて握られたSDAを解放
105 pinMode(SDA, INPUT_PULLUP);
106 pinMode(SCL, OUTPUT);
107 for (int i = 0; i < 9; i++) {
108     digitalWrite(SCL, LOW); delayMicroseconds(5);
109     digitalWrite(SCL, HIGH); delayMicroseconds(5);
110 }
111 // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
112 pinMode(SDA, OUTPUT);
113 digitalWrite(SDA, LOW); delayMicroseconds(5);
114 digitalWrite(SCL, HIGH); delayMicroseconds(5);
115 digitalWrite(SDA, HIGH); delayMicroseconds(5);
116
117 // Wire再初期化&センサ再設定 (setupADXL345の通信部と同じ)
118 Wire.begin();
119 Wire.beginTransmission(ADXL345_ADDR);
120 Wire.write(REG_POWER_CTL); Wire.write(0x08);
121 Wire.endTransmission();
122 Wire.beginTransmission(ADXL345_ADDR);
123 Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
124 Wire.endTransmission();
125 }
126
127 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
128     if (pot.bpmFrag) {
129         sync.sendBPM(pot.getBpmStep());
130         Serial.print("[SHAKE]_BPM送信_step="); Serial.println(pot.getBpmStep());
131         pot.bpmFrag = false;
132     }
133     if (pot.volFrag) {
134         sync.sendVolume(pot.getVolStep());
135         Serial.print("[SHAKE]_VOL送信_step="); Serial.println(pot.getVolStep());
136         pot.volFrag = false;
137     }
138 }

```

B.4 PotManager.h

コード 53 PotManager.h

```

1 // 可変抵抗器
2 #ifndef POT_MANAGER_H
3 #define POT_MANAGER_H
4
5 #include <Arduino.h>
6 #include "SyncManager.h"
7

```

```

8  class PotManager {
9  public:
10     // コンストラクタ（ポート番号などを初期化する）
11     PotManager();
12     //loop内で毎ループ呼ぶ
13     void update();
14     //現在の段階や変化を知るための窓口（ゲッター）
15     uint8_t getBpmStep() const { return currentBpmStep; }
16     uint8_t getVolStep() const { return 6 - currentVolStep; }
17
18     bool bpmFrag = false;
19     bool volFrag = false;
20
21 private:
22     int calcStep(float val);
23     bool bpmUpdatePotentiometers();
24     bool volUpdatePotentiometers();
25
26     // ピン定義
27     const int PIN_BPM = A0; // BPM用可変抵抗器
28     const int PIN_VOL = A1; // 音量用可変抵抗器
29     const unsigned long SAMPLE_INTERVAL = 20; // センサを読みとる周期
30     const int sampleSize = 10; // 移動平均のサンプル数
31     const int STEP_BOUNDARIES[4] = {300, 500, 650, 818};
32
33     // 内部変数
34     unsigned long lastSampleTime = 0; // 前回サンプリング時刻
35     int bpmSamples[10] = {0}; // BPMサンプル配列
36     long bpmTotal = 0; // サンプル合計値
37     float averageBpmVal = 0.0f; // BPM移動平均値
38
39     int volSamples[10] = {0}; // 音量サンプル配列
40     long volTotal = 0; // サンプル合計値
41     float averageVolVal = 0.0f; // 音量移動平均値
42
43     int bpmReadIndex = 0;
44     int volReadIndex = 0;
45     bool bpmBufferFull = false;
46     bool volBufferFull = false;
47
48     uint8_t currentBpmStep = 0; // BPM段階（1～5）
49     uint8_t currentVolStep = 0; // 音量段階（1～5）
50     int prevBpmStep = 0;
51     int prevVolStep = 0;
52
53 };
54
55 #endif

```

B.5 PotManager.cpp

コード 54 PotManager.cpp

```
1 #include "PotManager.h"
2
3 PotManager::PotManager(){
4     // コンストラクタ (初期化处理なし)
5 }
6
7 // ★loopから引っ越してきた時間管理
8 void PotManager::update() {
9     unsigned long now = micros();
10    if (now - lastSampleTime >= SAMPLE_INTERVAL) {
11        lastSampleTime = now;
12        if (bpmUpdatePotentiometers()) bpmFrag = true;
13        if (volUpdatePotentiometers()) volFrag = true;
14    }
15 }
16
17 int PotManager::calcStep(float val) {
18     if (val <= STEP_BOUNDARIES[0]) return 1;
19     else if (val <= STEP_BOUNDARIES[1]) return 2;
20     else if (val <= STEP_BOUNDARIES[2]) return 3;
21     else if (val <= STEP_BOUNDARIES[3]) return 4;
22     else return 5;
23 }
24
25 bool PotManager::bpmUpdatePotentiometers() {
26     bpmTotal -= bpmSamples[bpmReadIndex];
27
28     bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
29
30     bpmTotal += bpmSamples[bpmReadIndex];
31
32     bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
33
34     if (bpmReadIndex == 0) bpmBufferFull = true;
35
36     if (!bpmBufferFull) return false;
37
38     averageBpmVal = (float)bpmTotal / sampleSize;
39
40     currentBpmStep = calcStep(averageBpmVal);
41
42     if (currentBpmStep != prevBpmStep) {
43         Serial.println("-----_BPM/パラメータ更新_-----");
```

```

44
45     Serial.print("[BPM]_平均センサ値="); Serial.print(averageBpmVal, 1);
46     Serial.print("_段階=");          Serial.print(currentBpmStep);
47
48     prevBpmStep = currentBpmStep;
49     return true;
50 }
51 return false;
52 }
53
54 bool PotManager::volUpdatePotentiometers() {
55     volTotal -= volSamples[volReadIndex];
56
57     volSamples[volReadIndex] = analogRead(PIN_VOL);
58
59     volTotal += volSamples[volReadIndex];
60
61     volReadIndex = (volReadIndex + 1) % sampleSize;
62
63     if (volReadIndex == 0) volBufferFull = true;
64
65     if (!volBufferFull) return false;
66
67     averageVolVal = (float)volTotal / sampleSize;
68
69     currentVolStep = calcStep(averageVolVal);
70
71     if(currentVolStep != prevVolStep){
72         Serial.println("-----_VOLパラメータ更新_-----");
73
74         Serial.print("[VOL]_平均センサ値="); Serial.print(averageVolVal, 1);
75         Serial.print("_段階=");          Serial.print(6 - currentVolStep);
76
77         prevVolStep = currentVolStep;
78         return true;
79     }
80     return false;
81 }

```

B.6 SyncManager.h

コード 55 SyncManager.h

```

1 #ifndef SYNC_MANAGER_H
2 #define SYNC_MANAGER_H
3
4 #include <WiFiS3.h>
5 #include <WiFiUdp.h>

```

```

6
7 //楽器の個別IP (ユニキャスト用)
8 extern IPAddress fluteIP;
9 extern IPAddress pianoIP;
10 extern IPAddress stringsIP;
11 extern IPAddress drumIP;
12
13 //観覧車の個別IP (ユニキャスト用)
14 extern IPAddress ferrisWheelIP;
15
16 //チャンネル (マルチキャスト用)
17 extern IPAddress ferrisWheelGroup; // 観覧車だけが聴くチャンネル (現在未使用)
18 extern IPAddress instrumentGroup; // 楽器全員が聴くチャンネル
19
20 class SyncManager {
21 public:
22     // コンストラクタ (ポート番号などを初期化する)
23     SyncManager();
24     // Wi-FiとUDPを準備する関数
25     void begin(const char ssid[], const char pass[]);
26     // 演奏開始合図('S')を全機へ同時に送信する自作関数
27     void sendStart(uint8_t stepNumber);
28     // BPM情報を送信する関数 (引数の型を追加)
29     void sendBPM(uint8_t stepNumber);
30     // 音量情報を送信する関数 (引数の型を追加)
31     void sendVolume(uint8_t stepNumber);
32     // 演奏終了合図('E')を全機へ同時に送信する関数
33     void sendEnd();
34
35 private:
36     // 複数ターゲットヘラウンドロビンで3回冗長送信を行う共通処理。
37     // ターゲットごとに3連射してから次のターゲットへ進む方式だと、後方の
38     // ターゲットほど到達が遅れて機器間に無視できない時間差が生まれるため、
39     // 1周ごとに全ターゲットへ送ってから次の周に進む方式にしている。
40     void packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress targets
41         [], uint8_t targetCount);
42 };
43 #endif

```

B.7 SyncManager.cpp

コード 56 SyncManager.cpp

```

1 #include "SyncManager.h"
2
3 const uint16_t TARGET_PORT = 5005;
4 const uint16_t LOCAL_PORT = 8080;

```

```

5  WiFiUDP Udp;
6
7  IPAddress fluteIP(192, 168, 11, 14);
8  IPAddress pianoIP(192, 168, 11, 12);
9  IPAddress stringsIP(192, 168, 11, 13);
10 IPAddress drumIP(192, 168, 11, 11);
11 IPAddress ferrisWheelIP(192, 168, 11, 10); // 観覧車ユニキャスト
12 IPAddress ferrisWheelGroup(239, 255, 0, 1); // 未使用（マルチキャスト不安定のため）
13 IPAddress instrumentGroup(239, 255, 0, 2);
14
15 SyncManager::SyncManager() {
16     // コンストラクタ（今回は初期化処理なし）（インスタンスが作られた瞬間に、自動的に
17     // 一番最初に1回だけ実行される特殊な関数）
18 }
19 // Wi-FiおよびUDPの初期化フェーズ
20 void SyncManager::begin(const char ssid[], const char pass[]) {
21     WiFi.begin(ssid, pass); // Wi-Fi接続処理を開始
22     // 接続状態が「WL_CONNECTED（接続完了）」になるまでループ待機
23     while (WiFi.status() != WL_CONNECTED) {
24     }
25     Udp.begin(LOCAL_PORT); // 自身のポートを開放し、UDP通信を有効化
26 }
27 // 演奏開始合図（'S'）を全機へラウンドロビンで同時送信
28 void SyncManager::sendStart(uint8_t stepNumber) {
29     // 設計書通りの2バイト固定長ペイロード（第1バイト:ヘッダ'S', 第2バイト:ダミー
30     // 0x00）
31     uint8_t startPacket[2] = {'S', stepNumber};
32     // ドラムはisInstJoinを'S'受信時にリセットするため、他機より早く
33     // 届けたい。ただしラウンドロビン送信なら全ターゲットへの到達は
34     // ほぼ同時になるため、この並び自体はもう重要ではない。
35     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
36     packetRoundRobin(startPacket, 2, targets, 5);
37 }
38 // BPM情報の送信（引数で段階番号を受け取る）
39 // 'B' は観覧車のみが受信するため、観覧車へユニキャスト送信
40 void SyncManager::sendBPM(uint8_t stepNumber){
41     uint8_t bpmPacket[2] = {'B', stepNumber};
42     IPAddress targets[] = {ferrisWheelIP};
43     packetRoundRobin(bpmPacket, 2, targets, 1);
44 }
45 // 音量情報の送信（引数で段階番号を受け取る）
46 // 'V' は全楽器機が受信するため、各楽器機へユニキャスト送信
47 void SyncManager::sendVolume(uint8_t stepNumber){
48     uint8_t volumePacket[2] = {'V', (uint8_t)(6 - stepNumber)};
49     IPAddress targets[] = {fluteIP, pianoIP, stringsIP, drumIP};
50     packetRoundRobin(volumePacket, 2, targets, 4);
51 }
52 // 演奏終了合図（'E'）を全機へラウンドロビンで同時送信

```



```

51 void SyncManager::sendEnd(){
52     uint8_t endPacket[2] = {'E', 0x00};
53     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
54     packetRoundRobin(endPacket, 2, targets, 5);
55 }
56
57 // 複数ターゲットへのラウンドロビン3回冗長送信（パケットロス対策）
58 // 1周目で全ターゲットに1回ずつ送り、20ms空けてから2周目、3周目と繰り返す。
59 // ターゲットごとに3連射してから次へ進む方式だと、後方のターゲットほど
60 // 到達が遅れて機器間で無視できない時間差が生まれるため、この方式にしている。
61 void SyncManager::packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress
    targets[], uint8_t targetCount){
62     for (int round = 0; round < 3; round++) {
63         for (uint8_t t = 0; t < targetCount; t++) {
64             Udp.beginPacket(targets[t], TARGET_PORT); // 送信先IPとポートをセット
65             Udp.write(packet, b);
66             Udp.endPacket(); // パケットをパースして実際に送信
67         }
68         // 最後の周（3周目）以外は、20msのインターバル（遅延）を挟む
69         if (round < 2) {
70             delay(20);
71         }
72     }
73 }

```

C 中継機デバイス（Display）のプログラムソース

本節では、中継機デバイスのモータ回転制御およびLEDマトリクス表示を行う制御プログラムの全文を示す。

C.1 FerrisWheel.ino

コード 57 FerrisWheel.ino

```

1  #include "MotorControl.h"
2  #include "Display.h"
3  #include <WiFiS3.h>
4  #include <WiFiUdp.h>
5
6  const char SSID[] = "hackathon001-WPA2";
7  const char PASS[] = "hackathon001";
8
9  IPAddress localIP(192, 168, 11, 10);
10 IPAddress gateway(192, 168, 11, 1);
11 IPAddress subnet(255, 255, 255, 0);
12
13 const uint16_t LOCAL_PORT = 5005;

```

```

14
15 MotorControl motor;
16
17 WiFiUDP Udp;
18 uint8_t packetBuffer[10];
19
20 // 演奏中フラグ。'S'でtrue / 'E'でfalse。
21 // 'B'(BPM変更)は演奏中のみ受け付け、開始前の不意の'B'でモーターが回り出すのを防ぐ。
22 bool running = false;
23
24 void setup() {
25     Serial.begin(115200);
26     delay(2000);
27
28     Serial.println("---_FerrisWheel_Setup_Start_---");
29
30     motor.begin();
31     motor.createRpmTable();
32
33     WiFi.config(localIP, gateway, subnet);
34     WiFi.begin(SSID, PASS);
35     Serial.print("Connecting_to_WiFi...");
36     while (WiFi.status() != WL_CONNECTED) {
37         delay(500);
38         Serial.print(".");
39     }
40     Serial.println("\nWiFi_Connected!");
41     Serial.print("IP:_"); Serial.println(WiFi.localIP());
42
43     Udp.begin(LOCAL_PORT);
44     Serial.print("Listening_on_Unicast_Port_"); Serial.println(LOCAL_PORT);
45
46     displaySetup();
47 }
48
49 void loop() {
50     uint8_t packetSize = Udp.parsePacket();
51     if (packetSize <= 0) return;
52
53     if (packetSize != 2) {
54         Serial.print("想定外のパケットサイズ:_"); Serial.println(packetSize);
55         while (Udp.available()) Udp.read();
56         return;
57     }
58
59     Udp.read(packetBuffer, sizeof(packetBuffer));
60     char header = packetBuffer[0];
61     uint8_t payload = packetBuffer[1];

```

```

62
63     switch (header) {
64         case 'S':
65             Serial.print("演奏開始_('S')_payload="); Serial.println(payload);
66             // payload に BPM 段階が入っているのでそれで起動（元コードの global 変数
               参照バグを修正）
67             running = true;
68             motor.startRamp(payload);
69             displayBPM(payload);
70             // 冗長送信の残りパケットを捨てる
71             while (Udp.parsePacket() > 0) {
72                 while (Udp.available()) Udp.read();
73             }
74             break;
75
76         case 'B':
77             if (!running) {
78                 Serial.println("演奏前の_'B'_を無視");
79                 break;
80             }
81             Serial.print("BPM変更_('B')_段階="); Serial.println(payload);
82             motor.changeBPM(payload);
83             displayBPM(payload);
84             break;
85
86         case 'E':
87             Serial.println("演奏終了_('E')");
88             running = false;
89             motor.stopRamp();
90             clearDisplay();
91             break;
92
93         default:
94             Serial.print("想定外のヘッダ:_"); Serial.println(header);
95             break;
96     }
97 }

```

C.2 Display.h

コード 58 Display.h

```

1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h"
6

```

```

7  #define BPM_STEP_1  60
8  #define BPM_STEP_2  90
9  #define BPM_STEP_3  120
10 #define BPM_STEP_4  150
11 #define BPM_STEP_5  180
12
13 #define FONT_WIDTH   3
14 #define FONT_HEIGHT  5
15 #define MATRIX_ROWS  8
16 #define MATRIX_COLS  12
17
18 void displaySetup();
19 void displayBPM(uint8_t stepNumber);
20 void clearDisplay();
21 void drawDigit(int digit, int x_offset);
22
23 extern const uint8_t font3x5[10][15];
24 extern ArduinoLEDMatrix matrix;
25
26 #endif

```

C.3 Display.cpp

コード 59 Display.cpp

```

1  #include "Display.h"
2  #include "MotorControl.h"
3  #include "Arduino_LED_Matrix.h"
4
5  const uint8_t font3x5[10][15] = {
6      // 0
7      {1,1,1, 1,0,1, 1,0,1, 1,0,1, 1,1,1},
8      // 1
9      {0,1,0, 1,1,0, 0,1,0, 0,1,0, 1,1,1},
10     // 2
11     {1,1,1, 0,0,1, 1,1,1, 1,0,0, 1,1,1},
12     // 3
13     {1,1,1, 0,0,1, 0,1,1, 0,0,1, 1,1,1},
14     // 4
15     {1,0,1, 1,0,1, 1,1,1, 0,0,1, 0,0,1},
16     // 5
17     {1,1,1, 1,0,0, 1,1,1, 0,0,1, 1,1,1},
18     // 6
19     {1,1,1, 1,0,0, 1,1,1, 1,0,1, 1,1,1},
20     // 7
21     {1,1,1, 0,0,1, 0,0,1, 0,0,1, 0,0,1},
22     // 8
23     {1,1,1, 1,0,1, 1,1,1, 1,0,1, 1,1,1},

```

```

24 // 9
25 {1,1,1, 1,0,1, 1,1,1, 0,0,1, 1,1,1}
26 };
27
28 static const int bpmTable[5] = {
29     BPM_STEP_1, BPM_STEP_2, BPM_STEP_3, BPM_STEP_4, BPM_STEP_5
30 };
31
32 ArduinoLEDMatrix matrix;
33
34 void displaySetup() {
35     matrix.begin();
36     uint32_t warmUp[3] = {0, 0, 0};
37     matrix.loadFrame(warmUp);
38     delay(200);
39 }
40
41 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
42 static int currentBPM = -1;
43
44 void displayBPM(uint8_t stepNumber) {
45     if (stepNumber < 1 || stepNumber > 5) return;
46     uint8_t bpm = bpmTable[stepNumber - 1];
47
48     memset(frameBuffer, 0, sizeof(frameBuffer));
49
50     int digits[3];
51     int numDigits;
52     if (bpm >= 100) {
53         numDigits = 3;
54         digits[0] = bpm / 100;
55         digits[1] = (bpm / 10) % 10;
56         digits[2] = bpm % 10;
57     } else if (bpm >= 10) {
58         numDigits = 2;
59         digits[0] = bpm / 10;
60         digits[1] = bpm % 10;
61     } else {
62         numDigits = 1;
63         digits[0] = bpm;
64     }
65
66     int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
67     int startX = (MATRIX_COLS - totalWidth) / 2;
68
69     for (int i = 0; i < numDigits; i++) {
70         drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
71     }

```

```

72
73     uint32_t bitmap[3] = {0, 0, 0};
74     for (int row = 0; row < MATRIX_ROWS; row++) {
75         for (int col = 0; col < MATRIX_COLS; col++) {
76             if (frameBuffer[row][col]) {
77                 int n = row * MATRIX_COLS + col;
78                 bitmap[n / 32] |= (1UL << (31 - (n % 32)));
79             }
80         }
81     }
82     matrix.loadFrame(bitmap);
83 }
84
85 void drawDigit(int digit, int x_offset) {
86     if (digit < 0 || digit > 9) return;
87     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
88     for (int y = 0; y < FONT_HEIGHT; y++) {
89         for (int x = 0; x < FONT_WIDTH; x++) {
90             int col = x_offset + x;
91             int row = y_offset + y;
92             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
93                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
94             }
95         }
96     }
97 }
98
99 void clearDisplay() {
100     currentBPM = -1;
101     uint32_t blank[3] = {0, 0, 0};
102     matrix.loadFrame(blank);
103 }

```

C.4 MotorControl.h

コード 60 MotorControl.h

```

1  #ifndef MOTOR_CONTROL_H
2  #define MOTOR_CONTROL_H
3
4  #include <Arduino.h>
5
6  class MotorControl {
7  public:
8      MotorControl();
9      void begin();
10     void createRpmTable();
11     void startRamp(uint8_t stepNumber);

```

```

12     void changeBPM(uint8_t stepNumber);
13     void stopRamp();
14
15 private:
16     void stepToNumber(uint8_t stepNumber);
17     void updateStatus();
18
19     const uint8_t PIN_MOTOR_PWM = 5;
20     const uint8_t bpmTable[5] = {60, 90, 120, 150, 180};
21     const float VCC = 5.0;
22     const uint8_t pwmTable[5] = {26, 28, 30, 32, 37}; // 仮の数値
23
24     uint8_t currentBPM = 0;
25     float RPM = 0;
26     float omega = 0;
27     uint8_t pwmValue = 0;
28     float V_average = 0;
29     float rpmTable[5] = {};
30 };
31
32 #endif

```

C.5 MotorControl.cpp

コード 61 MotorControl.cpp

```

1  #include "MotorControl.h"
2
3  MotorControl::MotorControl() {}
4
5  void MotorControl::begin() {
6      pinMode(PIN_MOTOR_PWM, OUTPUT);
7      analogWrite(PIN_MOTOR_PWM, 0);
8  }
9
10 void MotorControl::createRpmTable() {
11     for (uint8_t i = 0; i < 5; i++) {
12         rpmTable[i] = bpmTable[i] / 16.0f;
13     }
14     Serial.println("[Motor]_rpmTable_初期化完了");
15 }
16
17 void MotorControl::startRamp(uint8_t stepNumber) {
18     stepToNumber(stepNumber);
19     Serial.println("[Motor]_Start_Ramp_Up...");
20     for (int i = 0; i <= pwmValue; i += 5) {
21         analogWrite(PIN_MOTOR_PWM, i);
22         delay(40);

```

```

23     }
24     analogWrite(PIN_MOTOR_PWM, pwmValue);
25     Serial.println("[Motor]_Target_speed_reached.");
26 }
27
28 void MotorControl::stopRamp() {
29     Serial.println("[Motor]_Start_Ramp_Down...");
30     for (int i = pwmValue; i >= 0; i -= 5) {
31         analogWrite(PIN_MOTOR_PWM, i);
32         delay(40);
33     }
34     analogWrite(PIN_MOTOR_PWM, 0);
35     Serial.println("[Motor]_Stopped.");
36 }
37
38 void MotorControl::changeBPM(uint8_t stepNumber) {
39     stepToNumber(stepNumber);
40     updateStatus();
41     analogWrite(PIN_MOTOR_PWM, pwmValue);
42     Serial.print("[Motor]_Speed_Changed._PWM:_"); Serial.println(pwmValue);
43 }
44
45 void MotorControl::stepToNumber(uint8_t stepNumber) {
46     if (stepNumber < 1 || stepNumber > 5) return;
47     currentBPM = bpmTable[stepNumber - 1];
48     pwmValue    = pwmTable[stepNumber - 1];
49     RPM         = rpmTable[stepNumber - 1];
50 }
51
52 void MotorControl::updateStatus() {
53     omega      = 2.0 * PI * RPM / 60.0f;
54     V_average = VCC * (pwmValue / 256.0f);
55 }

```

D 回転速度実測プログラム (LaserIntervalTest) のプログラムソース

本節では、中継機デバイスの回転速度実測に用いた LaserIntervalTest.ino の全文を示す。

D.1 LaserIntervalTest.ino

コード 62 LaserIntervalTest.ino

```

1 // CdSセルでのレーザー検知間隔を計測するスケッチ
2 // 連続する2回のパルス検知の時間差(interval)をシリアルモニタに表示し続ける。
3 //
4 // 検知ロジックは HCK_inst_0701 の drum_0701.ino / notDrum_0701.ino で使っている
5 // SyncManager::detectLight() をそのまま流用 (起動時キャリブレーション+

```



```

6 // 直近ピーク履歴による適応しきい値）。BPM段階への変換は行わず、生のinterval[ms]
7 // だけを出す。
8
9 const int CDS_PIN = A0;
10
11 const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
12 const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
13 const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
14 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
15 const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
16 const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
17 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
18 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
19 const unsigned long DEBOUNCE_MS = 200;
20
21 class LaserDetector {
22 public:
23     LaserDetector()
24         : lastDetectTime(0), laserOn(false),
25           baseline(0), baselineReady(false),
26           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
27           pulsePeak(0), peakIndex(0), peakCount(0),
28           calibrated(false), startupCalibrated(false), waitingForRelease(false),
29           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
30         clearPeakHistory();
31     }
32
33     // 新しいパルスを検知したら true を返す。interval[ms] を out に書く（前回検知が無ければ0）。
34     bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
35         if (calibrationStartMs == 0) calibrationStartMs = nowMs;
36         if (!detectLight(cdsValue, nowMs)) return false;
37         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
38
39         intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
40         bool hasInterval = (lastDetectTime != 0);
41         lastDetectTime = nowMs;
42         return hasInterval;
43     }
44
45 private:
46     unsigned long lastDetectTime;
47     bool laserOn;
48     int baseline;
49     bool baselineReady;
50     int onThreshold;
51     int offThreshold;
52     int pulsePeak;

```

```

53     int                peakHistory[CDS_PEAK_HISTORY];
54     uint8_t            peakIndex;
55     uint8_t            peakCount;
56     bool               calibrated;
57     bool               startupCalibrated;
58     bool               waitingForRelease;
59     unsigned long      calibrationStartMs;
60     int                startupMin;
61     int                releaseThreshold;
62
63     bool detectLight(int v, unsigned long nowMs) {
64         if (!baselineReady) {
65             baseline = v;
66             startupMin = v;
67             baselineReady = true;
68             updateFallbackThresholds();
69         }
70
71         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
72         if (!startupCalibrated) {
73             if (v < startupMin) startupMin = v;
74             baseline = startupMin;
75             updateFallbackThresholds();
76             if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
77             startupCalibrated = true;
78             waitingForRelease = (v >= onThreshold);
79             releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
                : offThreshold;
80             return false;
81         }
82
83         if (waitingForRelease) {
84             if (v <= releaseThreshold) {
85                 baseline = v;
86                 updateFallbackThresholds();
87                 waitingForRelease = false;
88             }
89             return false;
90         }
91
92         if (!laserOn) {
93             if (v < baseline) baseline = v;
94             else baseline = (baseline * 31 + v) / 32;
95             if (!calibrated) updateFallbackThresholds();
96         }
97
98         if (!laserOn && v >= onThreshold) {
99             laserOn = true;

```

```

100     pulsePeak = v;
101     return true;
102 }
103
104 if (laserOn) {
105     if (v > pulsePeak) pulsePeak = v;
106     if (v <= offThreshold) {
107         laserOn = false;
108         registerPeak(pulsePeak);
109     }
110 }
111 return false;
112 }
113
114 void registerPeak(int peak) {
115     if (peak - baseline < CDS_MIN_DELTA) return;
116
117     peakHistory[peakIndex] = peak;
118     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
119     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
120     if (peakCount < 3) return;
121
122     long sum = 0;
123     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
124     int peakAvg = sum / peakCount;
125     int amplitude = peakAvg - baseline;
126     if (amplitude < CDS_MIN_DELTA) return;
127
128     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
129     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);
130
131     if (!calibrated) {
132         onThreshold = targetOn;
133         offThreshold = targetOff;
134         calibrated = true;
135     } else {
136         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
137         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
138     }
139     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
140 }
141
142 void updateFallbackThresholds() {
143     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
144     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5));
145 }

```

```

145     }
146
147     int limitStep(int current, int target, int limit) {
148         if (target > current + limit) return current + limit;
149         if (target < current - limit) return current - limit;
150         return target;
151     }
152
153     void clearPeakHistory() {
154         for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
155     }
156 };
157
158 LaserDetector detector;
159
160 void setup() {
161     Serial.begin(115200);
162     Serial.println("---レーザー間隔計測_Setup_Started_---");
163 }
164
165 void loop() {
166     unsigned long now = millis();
167     int cdsValue = analogRead(CDS_PIN);
168     unsigned long interval = 0;
169
170     if (detector.update(cdsValue, now, interval)) {
171         Serial.print("[interval]_");
172         Serial.print(interval);
173         Serial.println("_ms");
174     }
175 }

```

E 楽器デバイス（Processing）のプログラムソース

本節では、楽器デバイスにおける音色表現を担う Processing プログラムの全文を示す。

E.1 SoundProcessingCode.pde

コード 63 SoundProcessingCode.pde

```

1 import ddf.minim.*;
2 import ddf.minim.analysis.*;
3 import ddf.minim.effects.*;
4 //import ddf.minim.signals.*;
5 //import ddf.minim.spi.*;
6 import ddf.minim.ugens.*;
7

```

```

8 //import ddf.minim.*;
9 //import ddf.minim.ugens.*;
10 //import ddf.minim.analysis.*;
11
12 //import ddf.minim.effects.*; // 追加
13
14 import processing.serial.*;
15 Serial myPort;
16
17 Minim minim;
18 AudioOutput out;
19 FFT fft;
20
21 int musicBPM = 120;
22
23 String timbre = "Flute";
24
25 byte soundOctave = 60;
26 byte soundDoReMi = 9;
27
28 byte soundStartVelocity = 96;
29 byte soundEndVelocity = 96;
30
31
32
33
34
35 float currentMasterVolume = 1.0;
36
37 // 0:なし 1:フルート 2:ストリングス 3:ピアノ 4:ドラム
38 final byte arduinoNumber = 3;
39
40 boolean isDebugMode = true;
41
42
43
44 // 反響音用
45 Summer longOut;
46 Summer shortOut;
47
48
49 InstrumentPlayer player;
50 DataUtils utils;
51
52
53
54
55

```

```

56
57
58
59 void setup()
60 {
61     size(1024, 850);
62     minim = new Minim(this);
63     out = minim.getLineOut(Minim.MONO, 1024);
64
65     // --- 1. エフェクト専用のバス（部屋）を3つだけ作る ---
66     // 1. スtrings専用のバス（通り道）を生成 (UGen)
67     /*longOut = new Summer();
68     // 2. 新しいUGen形式の反響音エフェクトを生成 (UGen)
69     SimpleConvolutionUGen longReverb = new SimpleConvolutionUGen(20000, 0.17f);
70     // 3. 【超重要】すべてを .patch() で数珠繋ぎ（シグナルチェーン）にする
71     longOut.patch(longReverb); // ミキサーの音を、反響音エフェクトへ流す
72     longReverb.patch(out); */ // 反響音エフェクトの音を、全体の出口へ流す
73
74     longOut = new Summer();
75     AddReverb longReverb = new AddReverb(15000, 0.20f);
76     longOut.patch(longReverb);
77     longReverb.patch(out);
78
79     shortOut = new Summer();
80     AddReverb shortReverb = new AddReverb(2000, 0.25f);
81     shortOut.patch(shortReverb);
82     shortReverb.patch(out);
83
84
85     player = new InstrumentPlayer(out); // thisとoutを渡す
86
87     // ★ここに追記：過去2000サンプル（約45ミリ秒分）を参照する反響音を適用
88     //out.patch(new SimpleConvolutionEffect(15000));
89
90     //out.setTempo (musicBPM);
91     //currentWaveform = Waves.SINE;
92
93     utils = new DataUtils();
94     // 1. float型の二次元配列を取得
95     //float[][] Note_f = utils.GetLUT("Note");
96     //// 2. Note_fと同じ大きさで、int型の二次元配列「Note」を作成
97     //Note = new int[Note_f.length][Note_f[0].length];
98     //// 3. 2重ループで1要素ずつint型に変換して代入
99     //for (int i = 0; i < Note_f.length; i++) {
100     //    for (int j = 0; j < Note_f[i].length; j++) {
101     //        Note[i][j] = int(Note_f[i][j]); // 小数点以下は切り捨て
102     //    }
103     //}

```

```

104
105 out.setGain(-6.0f);
106
107 fft = new FFT(out.bufferSize(), out.sampleRate());
108
109
110
111 audioThread = new Thread(new Runnable() {
112     public void run() {
113         while (true) {
114             if (isPlaying && isDebugMode) {
115                 long getTime = System.nanoTime();
116                 while (getTime - lastTime >= interval) {
117                     SoundPlay();
118                     lastTime += interval;
119                     tick++;
120                 }
121             }
122
123
124             // CPUの暴走（負荷100%）を防ぐため、1ミリ秒だけ休憩させる
125             // これにより、毎秒1000回の頻度で正確にタイマーがチェックされます
126             //try {
127             //    Thread.sleep(interval / 1000000000 *2);
128             //    //Thread.sleep(1);
129             //} catch (InterruptedException e) {
130             //    break; // エラーが起きたらループを抜ける
131             //}
132
133
134             // ☒ 休憩（sleep）を廃止し、CPUに「超高速空回り中」と伝えるだけにする
135             //Thread.onSpinWait();
136
137
138
139             else {
140                 // 停止中はCPUを休ませる
141                 try { Thread.sleep(100); } catch (InterruptedException e) {}
142             }
143
144             // ☒ sleep(1) を廃止し、これに置き換える
145             if (isPlaying) {
146                 Thread.onSpinWait(); // CPUに「今は待機中だよ」と伝えて効率化する（Java 9以
147                                     降）
148             }
149         }
150     });

```

```

151 audioThread.start(); // スレッドをスタート！
152 //fpsCheckStartTime = System.nanoTime();
153
154
155 // シリアル通信開始処理
156 println("==_利用可能なシリアルポート_==");
157 String portList[] = Serial.list();
158 println(portList); // ポート一覧を確認
159 myPort = new Serial(this, portList[2], 115200); // ポートは適切に変更
160 myPort.bufferUntil('\n'); // 開業まで溜めてから
161 println("受信待機中・・・");
162 println("=====");
163
164 }
165
166
167
168 void draw()
169 {
170     background(0);
171     //if (!(isPlaying && isDebugMode)) {
172     // --- オシロスコープの描画 ---
173     stroke(255);
174     for (int i=0; i < out.bufferSize() - 1; i++) {
175         float x1 = map(i, 0, out.bufferSize(), 0, width);
176         float x2 = map(i+1, 0, out.bufferSize(), 0, width);
177         line(x1, 150 - out.left.get(i)*250, x2, 150 - out.left.get(i+1)*250);
178     }
179
180     // --- スペクトルアナライザの描画 ---
181     fft.forward(out.mix);
182     stroke(255, 0, 255);
183
184     for(int i = 0; i < fft.specSize() - 1; i++) {
185         // FFTのインデックスが何Hzに相当するか取得
186         float f1 = fft.indexToFreq(i);
187         float f2 = fft.indexToFreq(i+1);
188
189         // 表示範囲(20~20000Hz)から外れる場合は線を描画しない
190         if (f2 < 20 || f1 > 20000) continue;
191
192         float amp1 = fft.getBand(i);
193         float amp2 = fft.getBand(i+1);
194
195         float y1 = 800 - amp1 * 4;
196         float y2 = 800 - amp2 * 4;
197
198         // 専用関数を使ってX座標を計算

```



```

199     float x1 = FreqToX(f1);
200     float x2 = FreqToX(f2);
201
202     line(x1, y1, x2, y2);
203 }
204
205 // --- 周波数の目盛りとラベルの描画 (X軸) ---
206 fill(255);
207 textSize(12);
208 textAlign(CENTER, TOP);
209
210 // 添付画像通りの目盛りとラベルの配列
211 float[] labelFreqsX = {20, 30, 40, 50, 60, 80, 100, 200, 300, 400, 500,
212                        800, 1000, 2000, 3000, 4000, 6000, 8000, 10000, 20000};
213 String[] labelStrsX = {"20", "30", "40", "50", "60", "80", "100", "200", "300", "
214                        400", "500",
215                        "800", "1k", "2k", "3k", "4k", "6k", "8k", "10k", "20k"};
216
217 for (int i = 0; i < labelFreqsX.length; i++) {
218     float x = FreqToX(labelFreqsX[i]);
219
220     stroke(100); // 縦線 (薄いグレー)
221     line(x, 300, x, 820);
222
223     fill(200);
224     text(labelStrsX[i], x, 830);
225 }
226
227 // --- 振幅の目盛りとラベルの描画 (Y軸) ---
228 textAlign(RIGHT, CENTER); // テキストを右揃えにして数値を綺麗に並べる
229
230 // 振幅(amp)を0から120まで、20刻みで横線を描画します (上限を160から120に変更)
231 for (int amp = 20; amp <= 120; amp += 20) {
232     // スペクトルアナライザの描画と同じ計算式を使用
233     float y = 800 - amp * 4;
234
235     // オシロスコープの描画領域 (Y=300より上) に被らない場合のみ描画
236     if (y >= 320) {
237         stroke(50); // 横線 (暗いグレー)
238         line(40, y, width, y); // 数値ラベルと被らないようにX=40から開始
239
240         fill(200);
241         text(amp, 35, y); // 左端に振幅の数値を描画
242     }
243 }
244
245 // 操作説明
246 //fill(200);

```

```

246 //textAlign(LEFT, TOP);
247 //text("Press 'p' to play song. Press '1'-'6' to change waveform.", 20, 20);
248 //}
249
250
251 /*
252 if (isPlaying && isDebugMode) {
253     long getTime = System.nanoTime();
254     // 現在の時間と、前回記録した時間の差分を計算
255     // if ではなく while にすることで、遅れた分（数tick分）をこの1フレーム内で一気に
        処理します
256     while (getTime - lastTime >= interval) {
257         tick++;
258         SoundPlay();
259         lastTime += interval; // すっきりした累積補正の式
260     }
261 }
262 */
263 /*
264 long a = System.nanoTime();
265 fps ++;
266 if (fpsCheckStartTime + 1000000000L <= a) {
267     fpsCheckStartTime += 1000000000L;
268     println(fps);
269     fps = 0;
270 }
271 */
272 }
273 //long fpsCheckStartTime;
274 //int fps = 0;
275
276 // --- 周波数を対数スケールのx座標に変換する関数 ---
277 float FreqToX(float f) {
278     float minFreq = 20.0f;
279     float maxFreq = 20000.0f;
280
281     // 範囲外の値を制限する
282     if (f < minFreq) f = minFreq;
283     if (f > maxFreq) f = maxFreq;
284
285     // log() を使って対数スケールにマッピングする
286     return map(log(f), log(minFreq), log(maxFreq), 0, width);
287 };
288
289
290
291 //音符数と遅延の関係を調査したときに使用
292 //int noteChecker = 0;

```

```

293
294 void serialEvent(Serial p) {
295     //println("受信");
296     // 改行まで文字列を読み込む
297     String inString = p.readStringUntil('\n');
298
299     // 何も読み込めなかった場合は処理停止
300     if (inString == null) return;
301
302     // 末尾の改行コードや余分な空白を削除
303     inString = trim(inString);
304
305     // カンマで分割して配列にする
306     String[] list = split(inString, ',');
307
308     // データが空の場合は処理停止
309     if (list.length <= 1) return;
310
311     // ヘッダとデータ数
312     String header = list[0];
313     int listLength = list.length - 1;
314
315     // ヘッダによる分岐
316     switch (header) {
317
318         // 発音指示
319         case "N" :
320             if (listLength % 5 != 0) break;
321             for (int i = 5; i <= listLength; i += 5) {
322
323                 float pitch, duration, startVel, endVel, type;
324                 try {
325                     pitch = float(list[i-4]);
326                     duration = float(list[i-3]) * (2.5f / musicBPM); // ÷60000(BPM*24)
327                     startVel = float(list[i-2]);
328                     endVel = float(list[i-1]);
329                     type = int(list[i]);
330                     println("N受信(" + list[i-4] + ",_" + list[i-3] + ",_" + list[i-2] + ",_" +
331                             list[i-1] + ",_" + list[i] + ")");
332                 } catch (NumberFormatException e) {
333                     continue;
334                 }
335                 // 数値の場合のみ処理続行
336
337                 /*if (pitch == 72) {
338                     if (noteChecker == 0) noteChecker -= millis();
339                     else {noteChecker += millis(); println("所要時間: " + noteChecker);
340                             noteChecker = 0;}
341                 }
342             }

```

```

339     */
340
341     switch (arduinoNumber) {
342     case 1 :
343         player.PlayFlute(pitch, duration, startVel, endVel, currentMasterVolume);
344         break;
345     case 2 :
346         player.PlayStrings(pitch, duration, startVel, endVel, currentMasterVolume
347                             );
348         break;
349     case 3 :
350         if (type == 0) player.PlayPiano(pitch, duration, startVel,
351                                         currentMasterVolume);
352         else if (type == 1) player.PlayTrumpet(pitch, duration, startVel, endVel,
353                                                 currentMasterVolume);
354         else if (type == 2) player.PlayHorn(pitch, duration, startVel, endVel,
355                                             currentMasterVolume);
356         break;
357     case 4 :
358         int a = int(list[i-4]) % 12;
359         if (a == 0) player.PlayKick(currentMasterVolume, startVel);
360         else if (a == 3) player.PlaySnare(currentMasterVolume, startVel, endVel);
361         else if (a == 4) player.PlayHiHat(currentMasterVolume, startVel);
362         break;
363     default: break;
364     }
365 }
366 break;
367
368 // BPM変更指示 (60~180)
369 case "B" :
370     if (listLength != 1) break;
371     int getBPM;
372     try {
373         getBPM = int(list[1]);
374         println("B受信_(" + list[1] + ")");
375     } catch (NumberFormatException e) {
376         break;
377     }
378 // 数値の場合のみ処理続行
379 int minimumBPM = 60;
380 int maximumBPM = 180;
381 musicBPM = constrain(getBPM, minimumBPM, maximumBPM);
382 break;
383
384 // マスターボリューム変更指示 (0~255)
385 case "V" :
386     if (listLength != 1) break;

```

```

383     float getVolume;
384     try {
385         getVolume = float(list[1]);
386         println("V受信_(" + list[1] + ")");
387     } catch (NumberFormatException e) {
388         break;
389     }
390     // 数値の場合のみ処理続行
391     float minimumVolume = 10.0f;
392     float maximumVolume = 50.0f;
393     currentMasterVolume = map( constrain( getVolume, minimumVolume, maximumVolume),
394                               minimumVolume, maximumVolume, 0.2, 1.0);
394     break;
395
396     default: break;
397 }
398 }
399
400
401
402
403
404
405
406
407 void Play()
408 {
409     switch(timbre){
410         case "Flute":
411             player.PlayFlute(soundOctave + soundDoReMi, 2.0f, soundStartVelocity,
412                             soundEndVelocity, 1.0f);
412             break;
413         //case "Flure2":
414         // player.PlayFlute(soundOctave + soundDoReMi + 128, 4.0f, soundStartVelocity,
415         //                 soundEndVelocity, 1.0f);
416         // break;
417         case "Strings":
418             player.PlayStrings(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
419                               soundEndVelocity, 1.0f);
418             break;
419         case "Drum":
420             if (soundDoReMi == 0) {
421                 player.PlayKick(1.0f, soundStartVelocity);
422             } else if (soundDoReMi == 1) {
423                 player.PlaySnareOld(1.0f, soundStartVelocity);
424             } else if (soundDoReMi == 2) {
425                 player.PlayHiHatOld(1.0f, soundStartVelocity);
426             } else if (soundDoReMi == 3) {

```

```

427     player.PlaySnare(1.0f, soundStartVelocity, soundEndVelocity);
428 } else if (soundDoReMi == 4) {
429     player.PlayHiHat(1.0f, soundStartVelocity);
430 } //else if (soundDoReMi == 5) {
431 //     player.PlayCymbal(1.0f, soundStartVelocity);
432 //}
433 break;
434 case "Piano":
435     //int a = millis();
436     player.PlayPiano(soundOctave + soundDoReMi, 10.0f, soundStartVelocity, 1.0f);
437     //a = millis()-a;
438     //println("經過時間: " + a);
439     break;
440 case "Horn":
441     player.PlayHorn(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
442         soundEndVelocity, 1.0f);
443     break;
444 case "Trumpet":
445     player.PlayTrumpet(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
446         soundEndVelocity, 1.0f);
447     break;
448 }
449 }
450
451 void keyPressed() {
452     if (isDebugMode == false) return;
453     switch (key)
454     {
455     case '1':
456         soundOctave = 24;
457         break;
458     case '2':
459         soundOctave = 36;
460         break;
461     case '3':
462         soundOctave = 48;
463         break;
464     case '4':
465         soundOctave = 60;
466         break;
467     case '5':
468         soundOctave = 72;
469         break;
470     case '6':
471         soundOctave = 84;
472         break;

```

```

473 case '7':
474     soundOctave = 96;
475     break;
476
477 case '0':
478     soundStartVelocity = 127;
479     break;
480 case '-':
481     soundStartVelocity = 112;
482     break;
483 case 'o':
484     soundStartVelocity = 96;
485     break;
486 case 'p':
487     soundStartVelocity = 80;
488     break;
489 case 'l':
490     soundStartVelocity = 64;
491     break;
492 case ';':
493     soundStartVelocity = 48;
494     break;
495 case ',':
496     soundStartVelocity = 32;
497     break;
498 case '.':
499     soundStartVelocity = 16;
500     break;
501 case '^':
502     soundEndVelocity = 127;
503     break;
504 case '¥':
505     soundEndVelocity = 112;
506     break;
507 case '@':
508     soundEndVelocity = 96;
509     break;
510 case '[':
511     soundEndVelocity = 80;
512     break;
513 case ':':
514     soundEndVelocity = 64;
515     break;
516 case ']':
517     soundEndVelocity = 48;
518     break;
519 case '/':
520     soundEndVelocity = 32;

```

```
521     break;
522 case '_':
523     soundEndVelocity = 16;
524     break;
525
526 case 'a':
527     soundDoReMi = 0;
528     Play();
529     break;
530 case 'w':
531     soundDoReMi = 1;
532     Play();
533     break;
534 case 's':
535     soundDoReMi = 2;
536     Play();
537     break;
538 case 'e':
539     soundDoReMi = 3;
540     Play();
541     break;
542 case 'd':
543     soundDoReMi = 4;
544     Play();
545     break;
546 case 'f':
547     soundDoReMi = 5;
548     Play();
549     break;
550 case 't':
551     soundDoReMi = 6;
552     Play();
553     break;
554 case 'g':
555     soundDoReMi = 7;
556     Play();
557     break;
558 case 'y':
559     soundDoReMi = 8;
560     Play();
561     break;
562 case 'h':
563     soundDoReMi = 9;
564     Play();
565     break;
566 case 'u':
567     soundDoReMi = 10;
568     Play();
```



```

569     break;
570 case 'j':
571     soundDoReMi = 11;
572     Play();
573     break;
574
575 case 'z':
576 case 'Z':
577     timbre = "Flute";
578     break;
579 //case 'x':
580 //case 'X':
581 //    timbre = "Flure2";
582 //    break;
583 case 'c':
584 case 'C':
585     timbre = "Strings";
586     break;
587 case 'v':
588 case 'V':
589     timbre = "Drum";
590     break;
591 case 'b':
592 case 'B':
593     timbre = "Piano";
594     break;
595 case 'n':
596 case 'N':
597     timbre = "Horn";
598     //Oscil sawOsc = new Oscil(37, 0.1, Waves.SQUARE);
599     //sawOsc.patch(new HighPassSP(500f, out.sampleRate())).patch(out);
600     //sawOsc = new Oscil(199, 0.01, Waves.SQUARE);
601     //sawOsc.patch(new HighPassSP(1000f, out.sampleRate())).patch(out);
602     //sawOsc = new Oscil(347, 0.01, Waves.SAW);
603     //sawOsc.patch(new HighPassSP(1800f, out.sampleRate())).patch(out);
604     //sawOsc = new Oscil(433, 0.01, Waves.SQUARE);
605     //sawOsc.patch(new HighPassSP(2200f, out.sampleRate())).patch(out);
606     //sawOsc = new Oscil(859, 0.01, Waves.SAW);
607     //sawOsc.patch(new HighPassSP(5000f, out.sampleRate())).patch(out);
608     //sawOsc = new Oscil(2100, 0.01, Waves.SAW);
609     //sawOsc.patch(new HighPassSP(9000f, out.sampleRate())).patch(out);
610     //sawOsc = new Oscil(71, 0.01, Waves.SQUARE);
611     //sawOsc.patch(new HighPassSP(600f, out.sampleRate())).patch(out);
612     //sawOsc = new Oscil(47, 0.01, Waves.SAW);
613     //sawOsc.patch(new HighPassSP(550f, out.sampleRate())).patch(out);
614     //sawOsc = new Oscil(13, 0.01, Waves.SAW);
615     //sawOsc.patch(new HighPassSP(300f, out.sampleRate())).patch(out);
616     break;

```

```

617 case 'm':
618 case 'M':
619     timbre = "Trumpet";
620     break;
621
622
623
624
625
626 case 'A' : // 酒場でブギウギ
627     tick = 1536;
628     returnTick = new int[][] {{2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728,
        48}};
629     returnIndex = 0;
630     noteIndex = 29;
631     interval = GetInterval( 139 );
632     lastTime = System.nanoTime();
633     isPlaying = true;
634     break;
635
636 case 'S' : // かえるのうた (オクターブ5)
637     tick = 0;
638     returnTick = new int[][] {{1536, 0, 0}};
639     returnIndex = 0;
640     noteIndex = 0;
641     interval = GetInterval( 120*2 );
642     lastTime = System.nanoTime();
643     isPlaying = true;
644     break;
645
646 case 'D' : // 王宮のメヌエット (強弱なし)
647     tick = 4608;
648     returnTick = new int[][] {{5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856,
        856}, {6216, 6456, 993}, {6816, 4608, 651}, {5760, 6816, 1046}, {8832,
        4608, 651}};
649     returnIndex = 0;
650     noteIndex = 651;
651     interval = GetInterval( 122 );
652     lastTime = System.nanoTime();
653     isPlaying = true;
654     break;
655
656 case 'F' : // トルコ行進曲
657     tick = 8856;
658     returnTick = new int[][] {{9240, 8856, 1399}, {10008, 9240, 1499}, {10392,
        10008, 1669}, {10776, 10392, 1783}, {11544, 10776, 1895}, {11544, 10008,
        1669}, {10392, 10008, 1669}, {10392, 8856, 1399}, {9240, 8856, 1399},
        {10008, 11544, 2111}, {11928, 11544, 2111}, {11904, 11952, 2225}, {13632,

```

```

        8856, 1399}};
659     returnIndex = 0;
660     noteIndex = 1399;
661     interval = GetInterval( 120 );
662     lastTime = System.nanoTime();
663     isPlaying = true;
664     break;
665
666     case 'G' : // 序曲_中盤（製作中用）
667         tick = 96;
668         returnTick = new int[][] {{1608, 72, 0}};
669         returnIndex = 0;
670         noteIndex = 0;
671         interval = GetInterval( 120 );
672         lastTime = System.nanoTime();
673         isPlaying = true;
674         break;
675
676     case 'H' : // 序曲_終盤なし
677         tick = 13632;
678         //returnTick = new int[][] {{16488, 14952, 2899}}; // ピアノ用
679         returnTick = new int[][] {{16488, 14952, 3053}}; // スtringス用
680         //returnTick = new int[][] {{16488, 14952, 2985}}; // フルート用
681         returnIndex = 0;
682         noteIndex = 2638;
683         interval = GetInterval( 120 );
684         lastTime = System.nanoTime();
685         isPlaying = true;
686         break;
687
688     case 'J' : // 序曲_ドラム用
689         tick = 768;
690         returnTick = new int[][] {{3624, 2088, 637}};
691         returnIndex = 0;
692         noteIndex = 92;
693         interval = GetInterval( 120 );
694         lastTime = System.nanoTime();
695         isPlaying = true;
696         break;
697
698     case 'Q' : // 演奏停止
699         isPlaying = false;
700         break;
701
702     default:
703         break;
704 }
705 }

```

E.2 Utils.pde

コード 64 Utils.pde

```
1 class DataUtils {
2   private float[][] note;
3
4   private float[][][] fluteFFLUT, fluteMFLUT, flutePPLUT;
5
6   private float[][][] violinFFLUT, violinMFLUT, violinPPLUT;
7   private float[][][] violaFFLUT, violaMFLUT, violaPPLUT;
8   private float[][][] celloFFLUT, celloMFLUT, celloPPLUT;
9   private float[][][] bassFFLUT, bassMFLUT, bassPPLUT;
10
11   private float[][][] pianoFFLUT, pianoMFLUT, pianoPPLUT;
12   private float[][] pianoDecayFFLUT, pianoWobbleRateFFLUT, pianoWobbleDepthFFLUT;
13   private float[][] pianoDecayMFLUT, pianoWobbleRateMFLUT, pianoWobbleDepthMFLUT;
14   private float[][] pianoDecayPPLUT, pianoWobbleRatePPLUT, pianoWobbleDepthPPLUT;
15
16   private float[][] kickLUT, snareLUT, hihatLUT;
17
18
19
20   private float[][][] hornFFLUT, hornMFLUT, hornPPLUT;
21   private float[][][] trumpetFFLUT, trumpetMFLUT, trumpetPPLUT;
22
23
24   DataUtils() {
25     //println("Loading JSON data...");
26
27     // JSONファイルからデータをロードする
28
29     //note = Load2LUT("序曲_フルート冒頭_processing.json");
30     //note = Load2LUT("序曲_フルート中盤_processing.json");
31     //note = Load2LUT("序曲_ピアノ冒頭_processing.json");
32     //note = Load2LUT("序曲_ピアノ中盤_processing.json");
33     //note = Load2LUT("序曲_ストリングス冒頭_processing.json");
34     //note = Load2LUT("序曲_ストリングス中盤_processing.json");
35     //note = Load2LUT("序曲_ドラム冒頭_processing.json");
36     //note = Load2LUT("序曲_ドラム中盤_processing.json");
37
38     //note = Load2LUT("かえるのうた8小節ver「酒場でブギウギ(強弱付き)」 「王宮のメヌ
      エット(強弱なし)」 「トルコ行進曲」_processing.json");
39     //note = Load2LUT("共通部分0706_processing.json");
40     //note = Load2LUT("ドラム0706_1_processing.json");
41     //note = Load2LUT("ピアノ0706_1_processing.json");
42     note = Load2LUT("ストリングス0706_1_processing.json");
```

```

43 //note = Load2LUT("フルート0706_1_processing.json");
44
45
46 fluteFFLUT = Load3LUT("fluteFFLUT.json");
47 fluteMFLUT = Load3LUT("fluteMFLUT.json");
48 flutePPLUT = Load3LUT("flutePPLUT.json");
49
50 violinFFLUT = Load3LUT("violinFFLUT.json");
51 violinMFLUT = Load3LUT("violinMFLUT.json");
52 violinPPLUT = Load3LUT("violinPPLUT.json");
53
54 violaFFLUT = Load3LUT("violaFFLUT.json");
55 violaMFLUT = Load3LUT("violaMFLUT.json");
56 violaPPLUT = Load3LUT("violaPPLUT.json");
57
58 celloFFLUT = Load3LUT("celloFFLUT.json");
59 celloMFLUT = Load3LUT("celloMFLUT.json");
60 celloPPLUT = Load3LUT("celloPPLUT.json");
61
62 bassFFLUT = Load3LUT("bassFFLUT.json");
63 bassMFLUT = Load3LUT("bassMFLUT.json");
64 bassPPLUT = Load3LUT("bassPPLUT.json");
65
66 pianoFFLUT = Load3LUT("pianoFFLUT.json");
67 pianoMFLUT = Load3LUT("pianoMFLUT.json");
68 pianoPPLUT = Load3LUT("pianoPPLUT.json");
69 pianoDecayFFLUT = Load2LUT("pianoDecaysFF.json");
70 pianoDecayMFLUT = Load2LUT("pianoDecaysMF.json");
71 pianoDecayPPLUT = Load2LUT("pianoDecaysPP.json");
72 pianoWobbleRateFFLUT = Load2LUT("pianoWobbleRatesFF.json");
73 pianoWobbleRateMFLUT = Load2LUT("pianoWobbleRatesMF.json");
74 pianoWobbleRatePPLUT = Load2LUT("pianoWobbleRatesPP.json");
75 pianoWobbleDepthFFLUT = Load2LUT("pianoWobbleDepthsFF.json");
76 pianoWobbleDepthMFLUT = Load2LUT("pianoWobbleDepthsMF.json");
77 pianoWobbleDepthPPLUT = Load2LUT("pianoWobbleDepthsPP.json");
78
79 // ドラムのLUTをロード
80 kickLUT = Load2LUT("kickLUT.json");
81 snareLUT = Load2LUT("snareLUT.json");
82 hihatLUT = Load2LUT("hihatLUT.json");
83
84
85
86
87
88
89
90

```

```

91     hornFFLUT = Load3LUT("hornFFLUT.json");
92     hornMFLUT = Load3LUT("hornMFLUT.json");
93     hornPPLUT = Load3LUT("hornPPLUT.json");
94
95     trumpetFFLUT = Load3LUT("trumpetFFLUT.json");
96     trumpetMFLUT = Load3LUT("trumpetMFLUT.json");
97     trumpetPPLUT = Load3LUT("trumpetPPLUT.json");
98
99     //println("Data loaded successfully!");
100 }
101
102
103 // JSONファイルを読み込んで float[][] 配列に変換する便利関数
104 float[][] Load2LUT(String fileName) {
105     JSONArray jsonArray = loadJSONArray(fileName);
106     float[][] lut = new float[jsonArray.size()][];
107
108     for (int i = 0; i < jsonArray.size(); i++) {
109         if (jsonArray.isNull(i)) {
110             lut[i] = null; // 今までの仕様通り、存在しない音程は null にする
111             continue;
112         }
113         JSONArray row = jsonArray.getJSONArray(i);
114         if (row.size() > 0) {
115             lut[i] = new float[row.size()];
116             for (int j = 0; j < row.size(); j++) {
117                 lut[i][j] = row.getFloat(j);
118             }
119         } else {
120             lut[i] = null; // データが欠損している部分は null にする（今までの仕様通り）
121         }
122     }
123     return lut;
124 }
125
126
127 // JSONファイルを読み込んで float[][][] の3次元可変長配列に変換する関数
128 float[][][] Load3LUT(String fileName) {
129     JSONArray jsonArray = loadJSONArray(fileName);
130     // 第1次元（音程の数）を確定
131     float[][][] lut = new float[jsonArray.size()][][];
132
133     for (int i = 0; i < jsonArray.size(); i++) {
134         // JSON上で null になっている、または要素が欠損している場合の処理
135         if (jsonArray.isNull(i)) {
136             lut[i] = null; // 今までの仕様通り、存在しない音程は null にする
137             continue;
138         }

```

```

139
140     JSONArray peaksArray = jsonArray.getJSONArray(i);
141     int numPeaks = peaksArray.size();
142
143     if (numPeaks > 0) {
144         // 第2次元（その音程が持つピークの数）を動的に確定
145         lut[i] = new float[numPeaks][2];
146
147         for (int j = 0; j < numPeaks; j++) {
148             JSONArray peakPair = peaksArray.getJSONArray(j);
149
150             // [周波数, 振幅] のペア（要素数2）を確実に取得
151             if (peakPair.size() == 2) {
152                 lut[i][j][0] = peakPair.getFloat(0); // 周波数 (Hz)
153                 lut[i][j][1] = peakPair.getFloat(1); // 相対振幅 (0.0 ~ 1.0)
154             }
155         }
156     } else {
157         lut[i] = null; // ピーク数が0個の場合もデータなしとして null を代入
158     }
159 }
160 return lut;
161 }
162
163
164
165 // ① 汎用版：特定のLUTから、指定したピッチの倍音レシピを安全に取り出す関数
166 /*
167 float[] GetProfileForPitch(float pitch, float[][] lut, int numHarmonics, int
    baseMidiNote) {
168
169     float[] result = new float[numHarmonics];
170
171     // LUTの要素数から自動的に最高音を算出して安全にクランプする
172     float maxMidiNote = baseMidiNote + lut.length - 1;
173     float clampedPitch = constrain(pitch, baseMidiNote, maxMidiNote);
174
175     int baseIndexLower = floor(clampedPitch) - baseMidiNote;
176     int baseIndexUpper = ceil(clampedPitch) - baseMidiNote;
177
178     // 下に向かって一番近い有効データを探す
179     int lowerIndex = baseIndexLower;
180     while (lowerIndex >= 0 && lut[lowerIndex] == null) { lowerIndex--; }
181
182     // 上に向かって一番近い有効データを探す
183     int upperIndex = baseIndexUpper;
184     while (upperIndex < lut.length && lut[upperIndex] == null) { upperIndex++; }

```

```

185
186
187 // =====
188 // 限界値のセーフティーネット
189 // =====
190 // 上に有効データが一つも無かったら、下のデータを採用する（最高音の延長）
191 if (upperIndex >= lut.length) upperIndex = lowerIndex;
192
193 // 下に有効データが一つも無かったら、上のデータを採用する（最低音の延長）
194 if (lowerIndex < 0) lowerIndex = upperIndex;
195
196 if (lowerIndex < 0 && upperIndex >= lut.length) {
197     // どちらにもデータが無い場合のフォールバック（基音のみ1.0）
198     float[] fallback = new float[numHarmonics];
199     fallback[0] = 1.0f;
200     return fallback;
201 }
202
203 float lowerMidi = lowerIndex + baseMidiNote;
204 float upperMidi = upperIndex + baseMidiNote;
205
206 float[] lowerProfile = lut[lowerIndex];
207 float[] upperProfile = lut[upperIndex];
208
209 float t = 0;
210 if (upperMidi > lowerMidi) {
211     t = (clampedPitch - lowerMidi) / (upperMidi - lowerMidi);
212 }
213
214 for (int i = 0; i < numHarmonics; i++) {
215     result[i] = lerp(lowerProfile[i], upperProfile[i], t);
216 }
217
218 return result;
219 }
220 */
221
222 /*
223 // ① 刷新版：特定の3次元LUTから、指定したピッチに「最も近い有効データ」を生のまま
    取り出す関数
224 // 戻り値：float[ピーク数][2] -> [h][0]:周波数(Hz), [h][1]:振幅
225 float[][] GetProfileForPitch(float pitch, float[][][] lut, int baseMidiNote) {
226
227     // LUTの要素数から自動的に最高音を算出して安全にクランプする
228     float maxMidiNote = baseMidiNote + lut.length - 1;
229     float clampedPitch = constrain(pitch, baseMidiNote, maxMidiNote);
230
231     // 四捨五入(round)で、最もピッチに近いサンプルのインデックスを狙う

```



```

232 int targetIndex = round(clampedPitch) - baseMidiNote;
233 targetIndex = constrain(targetIndex, 0, lut.length - 1);
234
235 // もし狙ったインデックスがnull（データ欠損）の場合、最も近い「有効なデータ」を探
    索する
236 int finalIndex = targetIndex;
237 if (lut[finalIndex] == null) {
238     int lowerIndex = finalIndex;
239     while (lowerIndex >= 0 && lut[lowerIndex] == null) { lowerIndex--; }
240
241     int upperIndex = finalIndex;
242     while (upperIndex < lut.length && lut[upperIndex] == null) { upperIndex++; }
243
244     // 上下のうち、存在する方、あるいはより近い方を採用する
245     if (lowerIndex >= 0 && upperIndex < lut.length) {
246         if ((finalIndex - lowerIndex) <= (upperIndex - finalIndex)) {
247             finalIndex = lowerIndex;
248         } else {
249             finalIndex = upperIndex;
250         }
251     } else if (lowerIndex >= 0) {
252         finalIndex = lowerIndex;
253     } else if (upperIndex < lut.length) {
254         finalIndex = upperIndex;
255     } else {
256         // 万が一、LUT全体が空っぽだった場合の完全フォールバック（MIDIピッチから計算
            した基音のみ生成）
257         float f0 = 440.0f * pow(2.0f, (clampedPitch - 69.0f) / 12.0f);
258         float[][] fallback = new float[1][2];
259         fallback[0][0] = f0; // 周波数
260         fallback[0][1] = 1.0f; // 振幅
261         return fallback;
262     }
263 }
264
265 // 見つかった有効なサンプルのデータをディープコピーして返す（参照渡しによるデータ
    破壊を防ぐため）
266 float[][] sourceProfile = lut[finalIndex];
267 int numPeaks = sourceProfile.length;
268 float[][] result = new float[numPeaks][2];
269
270 for (int h = 0; h < numPeaks; h++) {
271     result[h][0] = sourceProfile[h][0]; // 周波数 (Hz)
272     result[h][1] = sourceProfile[h][1]; // 振幅
273 }
274
275 return result;
276 }

```

```

277 */
278
279
280 // ① 確定修正版：特定の3次元LUTから、指定したピッチに「最も近い有効データ」を取り
    出し、正確なピッチヘシフトして返す関数
281 // 戻り値：float[ピーク数][2] -> [h][0]:周波数(Hz), [h][1]:振幅
282 float[][] GetProfileForPitch(float pitch, float[][][] lut, int baseMidiNote) {
283
284     // 【バグ解決の核心1】ユーザーが要求した本来のピッチ（小数ビブラートや範囲外のC3/
        A7など）を完全に生のまま保持
285     float originalPitch = pitch;
286
287     // LUTの要素数から自動的に最高音を算出し、インデックス探索用のみ安全にクランプす
        る
288     float maxMidiNote = baseMidiNote + lut.length - 1;
289     float searchPitch = constrain(pitch, baseMidiNote, maxMidiNote);
290
291     // 四捨五入(round)で、最もピッチに近いサンプルのインデックスを狙う
292     int targetIndex = round(searchPitch) - baseMidiNote;
293     targetIndex = constrain(targetIndex, 0, lut.length - 1);
294
295     // もし狙ったインデックスがnull（データ欠損）の場合、最も近い「有効なデータ」を探
        索する
296     int finalIndex = targetIndex;
297     println(finalIndex);
298     if (lut[finalIndex] == null) {
299         int lowerIndex = finalIndex;
300         while (lowerIndex >= 0 && lut[lowerIndex] == null) { lowerIndex--; }
301
302         int upperIndex = finalIndex;
303         while (upperIndex < lut.length && lut[upperIndex] == null) { upperIndex++; }
304
305         // 上下のうち、存在する方、あるいは要求された元のピッチ（searchPitch）により近
            い方を採用する
306         if (lowerIndex >= 0 && upperIndex < lut.length) {
307             if (abs(searchPitch - (lowerIndex + baseMidiNote)) <= abs((upperIndex +
                baseMidiNote) - searchPitch)) {
308                 finalIndex = lowerIndex;
309             } else {
310                 finalIndex = upperIndex;
311             }
312         } else if (lowerIndex >= 0) {
313             finalIndex = lowerIndex;
314         } else if (upperIndex < lut.length) {
315             finalIndex = upperIndex;
316         } else {
317             // 万が一、LUT全体が空っぽだった場合の完全フォールバック（MIDIピッチから計算
                した基音のみ生成）

```

```

318     float f0 = 440.0f * pow(2.0f, (originalPitch - 69.0f) / 12.0f);
319     float[][] fallback = new float[1][2];
320     fallback[0][0] = f0;    // 周波数
321     fallback[0][1] = 1.0f;  // 振幅
322     return fallback;
323 }
324 }
325 println(finalIndex);
326
327 // 見つかったサンプルのデータを取得
328 float[][] sourceProfile = lut[finalIndex];
329 int numPeaks = sourceProfile.length;
330 float[][] result = new float[numPeaks][2];
331
332 // =====
333 // 【バグ解決の核心2：流用元ピッチからの絶対距離によるシフト計算】
334 // =====
335 // 流用元のサンプルが本来持っている「正しい基準MIDIノート番号」を逆算
336 float sourceMidi = finalIndex + baseMidiNote;
337
338 // 制限のない本来の要求ピッチ（originalPitch）と、流用元（sourceMidi）のガチの差
339 // 分（半音単位）を計算。
340 // これにより、範囲外のC3を弾いた時（例：48 - 59 = -11半音）でも正確なスケーリン
341 // グ係数が算出されます！
342 float pitchShiftFactor = pow(2.0f, (originalPitch - sourceMidi) / 12.0f);
343 // =====
344
345 for (int h = 0; h < numPeaks; h++) {
346     // 元のデータが持っている「実測の生の周波数構造」に対して、計算した倍率を掛け算
347     // する
348     // これにより、インハーモニシティのデコボコを100%維持したまま、滑らかにピッチが
349     // スケーリングされます
350     result[h][0] = sourceProfile[h][0] * pitchShiftFactor;
351     result[h][1] = sourceProfile[h][1]; // 振幅はそのままコピー
352 }
353
354 return result;
355 }
356
357 // ② 汎用版：ピッチとベロシティの2軸から、最終的な倍音レシビを生成する関数
358 /*
359 float[] GetDynamicProfile(float pitch, float velocity, String type, int
360     numHarmonics, int baseMidiNote, float basePower) {
361     float[] profile = new float[numHarmonics];

```

```

361
362 // 3つのLUTから、現在のピッチに応じた倍音構成をそれぞれ取得
363 float[] profileFF, profilePP, profileMF;
364
365 switch (type)
366 {
367     case "Flute":
368         profilePP = GetProfileForPitch(pitch, flutePPLUT, numHarmonics, baseMidiNote);
369         profileMF = GetProfileForPitch(pitch, fluteMFLUT, numHarmonics, baseMidiNote);
370         profileFF = GetProfileForPitch(pitch, fluteFFLUT, numHarmonics, baseMidiNote);
371         break;
372     case "Violin":
373         profilePP = GetProfileForPitch(pitch, violinPPLUT, numHarmonics, baseMidiNote);
374         profileMF = GetProfileForPitch(pitch, violinMFLUT, numHarmonics, baseMidiNote);
375         profileFF = GetProfileForPitch(pitch, violinFFLUT, numHarmonics, baseMidiNote);
376         break;
377     case "Viola":
378         profilePP = GetProfileForPitch(pitch, violaPPLUT, numHarmonics, baseMidiNote);
379         profileMF = GetProfileForPitch(pitch, violaMFLUT, numHarmonics, baseMidiNote);
380         profileFF = GetProfileForPitch(pitch, violaFFLUT, numHarmonics, baseMidiNote);
381         break;
382     case "Cello":
383         profilePP = GetProfileForPitch(pitch, celloPPLUT, numHarmonics, baseMidiNote);
384         profileMF = GetProfileForPitch(pitch, celloMFLUT, numHarmonics, baseMidiNote);
385         profileFF = GetProfileForPitch(pitch, celloFFLUT, numHarmonics, baseMidiNote);
386         break;
387     case "Bass":
388         profilePP = GetProfileForPitch(pitch, bassPPLUT, numHarmonics, baseMidiNote);
389         profileMF = GetProfileForPitch(pitch, bassMFLUT, numHarmonics, baseMidiNote);
390         profileFF = GetProfileForPitch(pitch, bassFFLUT, numHarmonics, baseMidiNote);
391         break;
392     case "Piano":
393         profilePP = GetProfileForPitch(pitch, pianoPPLUT, numHarmonics, baseMidiNote);
394         profileMF = GetProfileForPitch(pitch, pianoMFLUT, numHarmonics, baseMidiNote);
395         profileFF = GetProfileForPitch(pitch, pianoFFLUT, numHarmonics, baseMidiNote);
396         break;
397     default:
398         println("Error: LUT type '" + type + "'が見つかりません!! ピアノを適用しまし
399             た。");
400         profilePP = GetProfileForPitch(pitch, pianoPPLUT, numHarmonics, baseMidiNote);
401         profileMF = GetProfileForPitch(pitch, pianoMFLUT, numHarmonics, baseMidiNote);
402         profileFF = GetProfileForPitch(pitch, pianoFFLUT, numHarmonics, baseMidiNote);
403         break;
404     }
405
406 // ペロシティ(0~127)を0.0~1.0に正規化
407 float normVel = constrain(velocity, 0, 127) / 127.0f;

```

```

408 // ユーザー定義のベロシティ閾値
409 float threshPP = 32.0f / 127.0f;
410 float threshMF = 80.0f / 127.0f;
411 float threshFF = 112.0f / 127.0f;
412
413 for (int i = 0; i < numHarmonics; i++) {
414     if (normVel <= threshPP) {
415         // 0~32 (無音~pp) の間は、波形の形はppのまま固定 (音量だけが小さくなる)
416         profile[i] = profilePP[i];
417
418     } else if (normVel <= threshMF) {
419         // 32~80 (pp~mf) の間を補間
420         float t = map(normVel, threshPP, threshMF, 0.0f, 1.0f);
421         profile[i] = lerp(profilePP[i], profileMF[i], t);
422
423     } else if (normVel <= threshFF) {
424         // 80~112 (mf~ff) の間を補間
425         float t = map(normVel, threshMF, threshFF, 0.0f, 1.0f);
426         profile[i] = lerp(profileMF[i], profileFF[i], t);
427
428     } else {
429         // 112~127 (ff~fff) の間は、波形の形はffのまま固定 (音量は最大へ向かう)
430         profile[i] = profileFF[i];
431     }
432 }
433
434 // 合計値に対する割合 (シェア) による音量算出
435 float totalSum = 0;
436 for (int i = 0; i < numHarmonics; i++) {
437     totalSum += profile[i];
438 }
439
440 // ベロシティに応じた「目標の全体音量 (パイの大きさ)」を決定する
441 // 音量(0.0)の場合はここでtargetVolumeが0になり、完全に無音になります
442 float targetVolume = basePower * pow(normVel, 0.7f);
443
444 // 各倍音の割合(シェア)を計算し、目標音量を掛け合わせる
445 for (int i = 0; i < numHarmonics; i++) {
446     // profile[i] / totalSum が「割合(0.0~1.0)」。そこにtargetVolumeを掛ける。
447     // +0.0001f は、ゼロ除算 (0で割ってエラーになること) を防ぐためのおまじないで
        す。
448     profile[i] = (profile[i] / (totalSum + 0.0001f)) * targetVolume;
449 }
450
451 return profile;
452 }
453 */
454

```

```

455
456 // ② 刷新版：ピッチとベロシティの2軸から、最終的な「周波数と音量のペア」の2次元配
    列を生成する関数
457 // 戻り値：float[ピーク数][2] -> [h][0]:周波数(Hz), [h][1]:最終音量
458 float[][] GetDynamicProfile(float pitch, float velocity, String type, int
    baseMidiNote, float basePower) {

459
460 // 3つの3次元LUTから、現在のピッチに最も近い実測スペクトルをそれぞれ取得
461 float[][] profilePP, profileMF, profileFF;
462
463 switch (type) {
464     case "Flute":
465         profilePP = GetProfileForPitch(pitch, flutePPLUT, baseMidiNote);
466         profileMF = GetProfileForPitch(pitch, fluteMFLUT, baseMidiNote);
467         profileFF = GetProfileForPitch(pitch, fluteFFLUT, baseMidiNote);
468         break;
469     case "Violin":
470         profilePP = GetProfileForPitch(pitch, violinPPLUT, baseMidiNote);
471         profileMF = GetProfileForPitch(pitch, violinMFLUT, baseMidiNote);
472         profileFF = GetProfileForPitch(pitch, violinFFLUT, baseMidiNote);
473         break;
474     case "Viola":
475         profilePP = GetProfileForPitch(pitch, violaPPLUT, baseMidiNote);
476         profileMF = GetProfileForPitch(pitch, violaMFLUT, baseMidiNote);
477         profileFF = GetProfileForPitch(pitch, violaFFLUT, baseMidiNote);
478         break;
479     case "Cello":
480         profilePP = GetProfileForPitch(pitch, celloPPLUT, baseMidiNote);
481         profileMF = GetProfileForPitch(pitch, celloMFLUT, baseMidiNote);
482         profileFF = GetProfileForPitch(pitch, celloFFLUT, baseMidiNote);
483         break;
484     case "Bass":
485         profilePP = GetProfileForPitch(pitch, bassPPLUT, baseMidiNote);
486         profileMF = GetProfileForPitch(pitch, bassMFLUT, baseMidiNote);
487         profileFF = GetProfileForPitch(pitch, bassFFLUT, baseMidiNote);
488         break;
489     case "Piano":
490         profilePP = GetProfileForPitch(pitch, pianoPPLUT, baseMidiNote);
491         profileMF = GetProfileForPitch(pitch, pianoMFLUT, baseMidiNote);
492         profileFF = GetProfileForPitch(pitch, pianoFFLUT, baseMidiNote);
493         break;
494     case "Horn":
495         profilePP = GetProfileForPitch(pitch, hornPPLUT, baseMidiNote);
496         profileMF = GetProfileForPitch(pitch, hornMFLUT, baseMidiNote);
497         profileFF = GetProfileForPitch(pitch, hornFFLUT, baseMidiNote);
498         break;
499     case "Trumpet":

```

```

500     profilePP = GetProfileForPitch(pitch, trumpetPPLUT, baseMidiNote);
501     profileMF = GetProfileForPitch(pitch, trumpetMFLUT, baseMidiNote);
502     profileFF = GetProfileForPitch(pitch, trumpetFFLUT, baseMidiNote);
503     break;
504 default:
505     println("Error: _LUT_type_' + type + "'が見つかりません!!_ピアノを適用しまし
        た。");
506     profilePP = GetProfileForPitch(pitch, pianoPPLUT, baseMidiNote);
507     profileMF = GetProfileForPitch(pitch, pianoMFLUT, baseMidiNote);
508     profileFF = GetProfileForPitch(pitch, pianoFFLUT, baseMidiNote);
509     break;
510 }
511
512 // ペロシティ(0~127)を0.0~1.0に正規化
513 float normVel = constrain(velocity, 0, 127) / 127.0f;
514
515 // ユーザー定義のペロシティ閾値
516 float thresholdPP = 32.0f / 127.0f;
517 float thresholdMF = 80.0f / 127.0f;
518 float thresholdFF = 112.0f / 127.0f;
519
520 // 現在のペロシティ層に応じて、ベースとなる波形プロファイル（周波数・振幅の骨組
        み）を決定
521 float[][] baseProfile;
522 float t = 0;
523 //boolean needInterpolation = false;
524 //float[][] lowerProfile = null;
525 //float[][] upperProfile = null;
526
527 if (normVel <= thresholdPP) {
528     baseProfile = profilePP;
529 } else if (normVel <= thresholdMF) {
530     // pp ~ mf の間：構造に近い方のスペクトル形状を選択（あるいは、より高度に処理
        するためのフラグ設定）
531     t = map(normVel, thresholdPP, thresholdMF, 0.0f, 1.0f);
532     baseProfile = (t < 0.5f) ? profilePP : profileMF;
533 } else if (normVel <= thresholdFF) {
534     // mf ~ ff の間
535     t = map(normVel, thresholdMF, thresholdFF, 0.0f, 1.0f);
536     baseProfile = (t < 0.5f) ? profileMF : profileFF;
537 } else {
538     baseProfile = profileFF;
539 }
540
541 //if (normVel >= thresholdFF) baseProfile = profileFF;
542 //else if (normVel >= thresholdMF) baseProfile = profileMF;
543 //else baseProfile = profilePP;
544

```

```

545 // 決定した形状ベース (baseProfile) を元に、最終出力用の2次元配列を確保
546 int numPeaks = baseProfile.length;
547 int validCount = 0;
548 float ampThreshold = 0.05f;
549 // 1. まずは有効なデータの数のカウントするだけ (上限は変更しない)
550 for (int h = 0; h < numPeaks; h++) {
551     if (baseProfile[h][1] >= ampThreshold) validCount++;
552 }
553 // 2. 判明した有効な数で、ぴったりサイズの配列を初期化
554 float[][] finalProfile = new float[validCount][2];
555
556 // 周波数は選択したベースサンプルの値を100%そのまま継承 (濁りを防ぐ)
557 //float totalSum = 0;
558 //for (int h = 0; h < numPeaks; h++) {
559 //    if(baseProfile[h][1] < 0.05f) continue;
560 //    finalProfile[h][0] = baseProfile[h][0]; // 周波数 (Hz) 固定
561 //    finalProfile[h][1] = baseProfile[h][1]; // 振幅の初期値
562 //    //totalSum += finalProfile[h][1];
563 //}
564
565
566
567 //// ペロシティに応じた「目標の全体音量 (エネルギーの総量)」の計算
568 //float targetVolume = basePower * pow(normVel, 0.9f);
569
570 //// 全体のエネルギー配分 (シェア) を計算し、各ピークに最終音量を割り当てる
571 //**for (int h = 0; h < numPeaks; h++) {
572 //    //finalProfile[h][1] = (finalProfile[h][1] / (totalSum + 0.0001f)) *
573 //        targetVolume;
574 //    finalProfile[h][1] = finalProfile[h][1] * targetVolume;
575 //}*/
576
577 //float energy = 0;
578 //for(int h = 0; h < numPeaks; h++){
579 //    energy += baseProfile[h][1] * baseProfile[h][1];
580 //}
581 //float gain = targetVolume / sqrt(energy + 1e-9f);
582
583 //for (int h = 0; h < numPeaks; h++) {
584 //    //float freq = baseProfile[h][0]; // 周波数 (Hz)
585
586 //    // --- 低音ブーストの計算 ---
587 //    // 例: 200Hz以下を緩やかに持ち上げるカーブ (ローシェルフ風)
588 //    float bassBoost = 1.0f;
589 //    if (freq < 400.0f) {
590 //        // 400Hz未満なら、低い周波数ほど最大1.5倍までゲインを上げる
591 //        // (400 - freq) / 400 で 0.0~1.0 のブレンド率を作り、0.5fを掛ける
592 //        bassBoost += ( (400.0f - freq) / 400.0f ) * 0.1f;

```



```

592 // }*/
593
594 // // 最終的なゲインを適用
595 // finalProfile[h][1] = baseProfile[h][1] * gain;
596 // //finalProfile[h][1] *= bassBoost;
597 //}
598
599 // 有効なピークの数のカウントするための変数
600 int validPeakCount = 0;
601 float energy = 0;
602
603 for (int h = 0; h < numPeaks; h++) {
604     // 0.05未満のデータは完全に無視してスキップ
605     if(baseProfile[h][1] < ampThreshold) {
606         continue;
607     }
608
609     // 有効なデータだけを finalProfile の前から順に詰める
610     finalProfile[validPeakCount][0] = baseProfile[h][0]; // 周波数 (Hz) 固定
611     finalProfile[validPeakCount][1] = baseProfile[h][1]; // 振幅の初期値
612
613     // 除外されなかった「有効なピークだけ」でエネルギーを計算
614     energy += baseProfile[h][1] * baseProfile[h][1];
615
616     // 次の保存先ヘインデックスを進める
617     validPeakCount++;
618 }
619
620 // ベロシティに応じた「目標の全体音量（エネルギーの総量）」の計算
621 float targetVolume = basePower * pow(normVel, 0.9f);
622
623 // ゲインの算出
624 float gain = targetVolume / sqrt(energy + 1e-9f);
625
626 // 最終的なゲインを適用（※元の numPeaks ではなく、validPeakCount まで回す！）
627 for (int i = 0; i < validPeakCount; i++) {
628     // すでに finalProfile に格納されている振幅に対してゲインを掛け算する
629     finalProfile[i][1] *= gain;
630 }
631
632 return finalProfile;
633 }
634
635
636
637
638
639 float[][] GetLUT(String type) {

```

```

640     switch (type) {
641         case "Note":
642             return note;
643
644         case "PianoDecayFF":
645             return pianoDecayFFLUT;
646         case "PianoWobbleRateFF":
647             return pianoWobbleRateFFLUT;
648         case "PianoWobbleDepthFF":
649             return pianoWobbleDepthFFLUT;
650
651         case "PianoDecayMF":
652             return pianoDecayMFLUT;
653         case "PianoWobbleRateMF":
654             return pianoWobbleRateMFLUT;
655         case "PianoWobbleDepthMF":
656             return pianoWobbleDepthMFLUT;
657
658         case "PianoDecayPP":
659             return pianoDecayPPLUT;
660         case "PianoWobbleRatePP":
661             return pianoWobbleRatePPLUT;
662         case "PianoWobbleDepthPP":
663             return pianoWobbleDepthPPLUT;
664
665         case "Kick":
666             return kickLUT;
667         case "Snare":
668             return snareLUT;
669         case "HiHat":
670             return hihatLUT;
671
672         default:
673             println("Error:_LUT_type_" + type + "'が見つかりません!!");
674             return null; // 存在しない名前が指定された場合は null を返す
675     }
676 }
677 };

```

E.3 InstrumentPlayer.pde

コード 65 InstrumentPlayer.pde

```

1  class InstrumentPlayer {
2      AudioOutput out;
3
4      InstrumentPlayer(AudioOutput out) {
5          this.out = out; // Mainから渡された出力を保持する

```

```

6   }
7
8   void PlayFlute(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
9       out.playNote(0.0f, duration,
10          new FluteInstrument(midiNote, duration, startVel, endVel, masterVol));
11   }
12
13   void PlayStrings(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
14       out.playNote(0.0f, duration,
15          new StringsInstrument(midiNote, duration, startVel, endVel, masterVol));
16   }
17
18   void PlayPiano(float midiNote, float duration, float velocity, float masterVol) {
19       out.playNote(0.0f, duration,
20          new PianoInstrument(midiNote, velocity, masterVol));
21   }
22
23   // キックを鳴らす関数
24   void PlayKick(float masterVol, float velocity) {
25       // out.playNote(開始時間, 鳴らす長さ, 音色クラス);
26       // ドラムはエンベロープ側で長さを制御するため、第2引数のdurationは適当な値(1.0な
        ど)でOKです。
27       //out.playNote(0, 0.1, new KickInstrument(masterVol, velocity));
28       out.playNote(0, 0.1, new KickInstrument(masterVol, velocity));
29   }
30
31   // スネアを鳴らす関数
32   void PlaySnareOld(float masterVol, float velocity) {
33       out.playNote(0, 0.5, new SnareInstrumentOld(masterVol, velocity));
34   }
35
36   // スネアver2を鳴らす関数
37   void PlaySnare(float masterVol, float velocity, float sinAmp) {
38       out.playNote(0, 0.3, new SnareInstrument(masterVol, velocity, sinAmp));
39   }
40
41   // クローズハイハットを鳴らす関数
42   void PlayHiHatOld(float masterVol, float velocity) {
43       out.playNote(0, 0.2, new HiHatInstrumentOld(masterVol, velocity));
44   }
45
46   // クローズハイハットver2を鳴らす関数
47   void PlayHiHat(float masterVol, float velocity) {
48       out.playNote(0, 0.15, new HiHatInstrument(masterVol, velocity));
49   }
50

```

```

51
52
53
54
55
56
57 void PlayHorn(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
58     out.playNote(0.0f, duration,
59         new HornInstrument(midiNote, duration, startVel, endVel, masterVol));
60 }
61 void PlayTrumpet(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
62     out.playNote(0.0f, duration,
63         new TrumpetInstrument(midiNote, duration, startVel, endVel, masterVol));
64 }
65
66 //void PlayCymbal(float masterVol, float velocity) {
67 //    out.playNote(0, 0.2, new CymbalInstrument(masterVol, velocity));
68 //}
69 }

```

E.4 NoteSampler.pde

コード 66 NoteSampler.pde

```

1 // 既存の変数に volatile をつける
2 volatile int tick;
3 volatile long interval;
4 volatile long lastTime = 0;
5 volatile int[][] returnTick; // {トリガーtick, ジャンプ先tick, ジャンプ先noteIndex}
6 volatile int returnIndex;
7 volatile int noteIndex;
8 volatile boolean isPlaying = false;
9
10 // 音楽専用スレッド用の変数を追加
11 Thread audioThread;
12
13
14
15
16 long GetInterval (int playBPM) {
17     return 60000000000L / (playBPM * 24);
18 }
19
20
21
22 void SoundPlay() {

```

```

23  if (tick == returnTick[returnIndex][0]) {
24      tick = returnTick[returnIndex][1];
25      noteIndex = returnTick[returnIndex][2];
26      returnIndex ++;
27      if (returnIndex == returnTick.length) returnIndex = 0;
28  }
29
30  if (noteIndex < Note.length) {
31      //long a = System.nanoTime();
32      float durationTuning = interval / 1000000000.0f;
33      while (Note[noteIndex][0] == tick) {
34          float duration = float(Note[noteIndex][1]) * durationTuning;
35          float pitch = float(Note[noteIndex][2]);
36          float startVelocity = float(Note[noteIndex][3]);
37          float endVelocity = float(Note[noteIndex][4]);          //startVelocity=127;
38          endVelocity=127;
39          float type = (Note[noteIndex].length == 6) ? float(Note[noteIndex][5]) : 0;
40          switch (timbre) {
41              case "Flute": case "Flute2":
42                  player.PlayFlute(pitch, duration, startVelocity, endVelocity, 1.0f);
43                  break;
44              case "Strings":
45                  player.PlayStrings(pitch, duration, startVelocity, endVelocity, 1.0f);
46                  break;
47              case "Piano":
48                  if (type == 0) player.PlayPiano(pitch, duration, startVelocity, 1.0f);
49                  else if (type == 1) player.PlayTrumpet(pitch, duration, startVelocity,
50                      endVelocity, 1.0f);
51                  else if (type == 2) player.PlayHorn(pitch, duration, startVelocity,
52                      endVelocity, 1.0f);
53                  break;
54              case "Drum":
55                  int a = int(pitch) % 12;
56                  if (a == 0) player.PlayKick(1.0f, startVelocity);
57                  //else if (a == 1) player.PlaySnareOld(1.0f, startVelocity);
58                  //else if (a == 2) player.PlayHiHatOld(1.0f, startVelocity);
59                  else if (a == 3) player.PlaySnare(1.0f, startVelocity, endVelocity);
60                  else if (a == 4) player.PlayHiHat(1.0f, startVelocity);
61                  break;
62              case "Horn":
63                  player.PlayHorn(pitch, duration, startVelocity, endVelocity, 1.0f);
64                  break;
65              case "Trumpet":
66                  player.PlayTrumpet(pitch, duration, startVelocity, endVelocity, 1.0f);
67                  break;
68              default:
69                  break;
70          }
71      }
72  }

```

```

68     noteIndex ++;
69     if (noteIndex == Note.length) break;
70 }
71 //println(System.nanoTime() - a);
72 }
73 }
74
75
76
77
78
79 int[][] Note = {
80 // ドラム用楽譜 (setupでの上書きをコメント文にすれば演奏可能)
81
82
83 // 【1小節目】
84 {0, 96, 72, 96, 96, 0}, // [0] C5
85
86 // 【3小節目】
87 {192, 96, 74, 96, 96, 0}, // [1] D5
88
89 // 【5小節目】
90 {384, 96, 76, 96, 96, 0}, // [2] E5
91
92 // 【7小節目】
93 {576, 96, 77, 96, 96, 0}, // [3] F5
94
95 // 【9小節目】
96 {768, 96, 79, 96, 96, 0}, // [4] G5
97
98 // 【11小節目】
99 {960, 96, 81, 96, 96, 0}, // [5] A5
100
101 // 【13小節目】
102 {1152, 96, 83, 96, 96, 0}, // [6] B5
103
104 };
105
106
107
108
109
110 // int[][] Note;

```

E.5 AddEffect.pde

コード 67 AddEffect.pde

```

1  /*import ddf.minim.AudioEffect;
2
3  // 講義の「たたみ込み積分」をそのまま再現した自作エフェクトクラス
4  class SimpleConvolutionEffect implements AudioEffect {
5      float[] impulseResponse; //  $h(\tau)$  に相当するインパルス応答データ
6      float[] historyBuffer;    // 過去の音を記憶しておくバッファ
7      int bufferIndex = 0;
8      int kernelSize;          // ★過去の音をどれくらい参照するか（サンプル数）
9
10     SimpleConvolutionEffect(int size, float levrl) {
11         this.kernelSize = size;
12         impulseResponse = new float[kernelSize];
13         historyBuffer = new float[kernelSize];
14
15         // 簡易的なインパルス応答を自動生成（指数関数的に減衰するノイズ = 擬似的な部屋の響き）
16         for (int i = 0; i < kernelSize; i++) {
17             float decay = exp(-3.8f * i / kernelSize); // 過去にいくほど音が小さくなる重み
18             impulseResponse[i] = random(-1.0f, 1.0f) * decay * levrl; // 響きの強さを調整
19         }
20     }
21
22     // モノラル信号（現在のシステム）にリアルタイムでたたみ込み演算を行う関数
23     void process(float[] signal) {
24         for (int i = 0; i < signal.length; i++) {
25             float input = signal[i];
26
27             // 1. 最新の入力を過去の記憶バッファに格納
28             historyBuffer[bufferIndex] = input;
29
30             // 2. たたみ込み積分の計算（スライドの数式の再現）
31             float output = 0;
32             for (int j = 0; j < kernelSize; j++) {
33                 // 過去へ遡るインデックスを計算
34                 int idx = (bufferIndex - j + kernelSize) % kernelSize;
35                 output += historyBuffer[idx] * impulseResponse[j];
36             }
37
38             // 3. 元のクリーンな音に、生成された反響音を混ぜて出力
39             signal[i] = input + output;
40
41             // 4. 記憶バッファの書き込み位置を1つ進める
42             bufferIndex = (bufferIndex + 1) % kernelSize;
43         }
44     }
45
46     // ステレオ用（Minimの仕様上必要ですが、中身はモノラル処理を両チャンネルに適用）
47     void process(float[] left, float[] right) {

```

```

48     process(left);
49     process(right);
50 }
51 }
52
53 */
54
55
56
57
58
59 /*
60 // Minimの最新推奨規格「UGen」に従って生まれ変わった反響音クラス
61 class SimpleConvolutionUGen extends UGen {
62     // 前のミキサー (stringsBus) から流れてくる音声信号を受け取るための「ポート (入り口)」
63     public UGenInput audio;
64
65     float[] impulseResponse; //  $h(\tau)$  に相当するインパルス応答データ
66     float[] historyBuffer;    // 過去の音を記憶しておくバッファ
67     int bufferSize = 0;
68     int kernelSize;          // 過去の音をどれくらい参照するか (サンプル数)
69
70     SimpleConvolutionUGen(int size, float level) {
71         // 入力ポートを初期化
72         audio = new UGenInput(InputType.AUDIO);
73
74         this.kernelSize = size;
75         impulseResponse = new float[kernelSize];
76         historyBuffer = new float[kernelSize];
77
78         // 簡易的なインパルス応答を自動生成 (指数関数的に減衰する部屋の響き)
79         for (int i = 0; i < kernelSize; i++) {
80             float decay = exp(-3.8f * i / kernelSize);
81             impulseResponse[i] = random(-1.0f, 1.0f) * decay * level;
82         }
83     }
84
85     // UGenの心臓部: Minimが音声を1サンプル処理するたびに自動で呼び出される関数
86     protected void uGenerate(float[] channels) {
87         // 1. 直前のミキサー (stringsBus) から流れてきた「現在の入力音」を1サンプル取得
88         float input = audio.getLastValue();
89
90         // 2. 最新の入力を過去の記憶バッファに格納
91         historyBuffer[bufferIndex] = input;
92
93         // 3. たたみ込み積分の計算 (講義スライドの数式の完全再現)
94         float output = 0;

```



```

95     for (int j = 0; j < kernelSize; j++) {
96         int idx = (bufferIndex - j + kernelSize) % kernelSize;
97         output += historyBuffer[idx] * impulseResponse[j];
98     }
99
100    // 4. 記憶バッファの書き込み位置を1つ進める
101    bufferIndex = (bufferIndex + 1) % kernelSize;
102
103    // 5. 元の音に反響音を足し合わせる
104    float finalOutput = input + output;
105
106    // 6. 出力先へ音を流し込む（ループにすることでモノラル・ステレオどちらにも自動対
107        応！）
108    for (int i = 0; i < channels.length; i++) {
109        channels[i] = finalOutput;
110    }
111 }*/
112
113
114
115
116
117
118 // Minimの最新推奨規格「UGen」に従って生まれ変わった反響音クラス（完全ステレオ対応
119 // 版）
120 class AddReverb extends UGen {
121     public UGenInput audio;
122
123     float[] impulseResponse;
124
125     // ★改造ポイント1：記憶バッファを左右の耳で完全に独立させる
126     float[] historyBufferL;
127     float[] historyBufferR;
128
129     // UGenの uGenerate は1サンプルフレーム（左右同時）ごとに呼ばれるため、時間は共通
130     // （1つ）でOK
131
132     int bufferIndex = 0;
133     int kernelSize;
134
135     AddReverb(int size, float level) {
136         audio = new UGenInput(InputType.AUDIO);
137         this.kernelSize = size;
138
139         impulseResponse = new float[kernelSize];
140         historyBufferL = new float[kernelSize];
141         historyBufferR = new float[kernelSize];
142     }
143
144     void process() {
145         float input = audio.get();
146         float outputL = 0;
147         float outputR = 0;
148
149         for (int j = 0; j < kernelSize; j++) {
150             int idxL = (bufferIndex - j + kernelSize) % kernelSize;
151             int idxR = (bufferIndex - j + kernelSize) % kernelSize;
152             outputL += historyBufferL[idxL] * impulseResponse[j];
153             outputR += historyBufferR[idxR] * impulseResponse[j];
154         }
155
156         bufferIndex = (bufferIndex + 1) % kernelSize;
157
158         float finalOutputL = input + outputL;
159         float finalOutputR = input + outputR;
160
161         audio.set(finalOutputL, finalOutputR);
162     }
163 }

```

```

140 // インパルス応答の生成
141 for (int i = 0; i < kernelSize; i++) {
142     float decay = exp(-3.8f * i / kernelSize);
143     impulseResponse[i] = random(-1.0f, 1.0f) * decay * level;
144 }
145 }
146
147 // UGenの心臓部
148 protected void uGenerate(float[] channels) {
149     // ★改造ポイント2：入力音を「配列」として取得し、左右の音を別々に取り出す
150     float[] inputs = audio.getLastValues();
151     float inL = inputs[0];
152     // もし入力がモノラル（1ch）だった場合は、右耳用にも左と同じ音を入れるセーフティ
        ガード
153     float inR = (inputs.length > 1) ? inputs[1] : inL;
154
155     // 最新の入力を左右それぞれの記憶バッファに格納
156     historyBufferL[bufferIndex] = inL;
157     historyBufferR[bufferIndex] = inR;
158
159     // ★改造ポイント3：たたみ込み積分の計算を、左右独立して行う
160     float outL = 0;
161     float outR = 0;
162     for (int j = 0; j < kernelSize; j++) {
163         int idx = (bufferIndex - j + kernelSize) % kernelSize;
164         outL += historyBufferL[idx] * impulseResponse[j];
165         outR += historyBufferR[idx] * impulseResponse[j];
166     }
167
168     // 記憶バッファの書き込み位置を1つ進める（左右共通の時間が進む）
169     bufferIndex = (bufferIndex + 1) % kernelSize;
170
171     // ★改造ポイント4：出力先（channels）がモノラルかステレオかで安全に音を振り分け
        る
172     if (channels.length == 1) {
173         // 出力先がモノラルの場合
174         channels[0] = inL + outL;
175     } else if (channels.length >= 2) {
176         // 出力先がステレオの場合
177         channels[0] = inL + outL; // 左チャンネルの出力
178         channels[1] = inR + outR; // 右チャンネルの出力
179     }
180 }
181 }

```

E.6 Flute.pde

コード 68 Flute.pde

```

1 // =====
2 // トリル専用 LF0ウェーブテーブル（台形波）の生成と保持
3 // =====
4 Wavetable trillWave = CreateTrapezoidWave();
5
6 // 台形波（角の丸いパルス波）を生成して返す関数
7 Wavetable CreateTrapezoidWave() {
8     int waveSize = 1024; // 波形の解像度
9     float[] waveData = new float[waveSize];
10
11     // 傾斜（スロープ）の幅を設定。
12     // 全体の何%を「移動時間」に使うか。例えば 0.15 なら 15% がスロープ、85%が平らな部
        分になる。
13     float slopeRatio = 0.5f;
14     // この数値を 0.05（より矩形波に近い、素早い指の動き）や 0.3（もったりとしたゆっく
        りな指の動き）に変更することで
15     // トリルの「ニュアンス（奏者の癖）」までプログラミングできるようになる。
16
17     for (int i = 0; i < waveSize; i++) {
18         float phase = (float)i / waveSize; // 1周期を -1.0 ~ +1.0 の間で描く
19
20         if (phase < 0.5f) { // 前半（高い音に向かう部分）
21             float localPhase = phase / 0.5f; // 0.0 ~ 1.0 に正規化
22             if (localPhase < slopeRatio) {
23                 // 立ち上がりのスロープ（-1.0 から +1.0 へ滑らかに移動）
24                 // smoothstep関数(3x^2 - 2x^3)を使って、S字カーブを描いてより滑らかにする
25                 float t = localPhase / slopeRatio;
26                 waveData[i] = -1.0f + 2.0f * (3*t*t - 2*t*t*t); // smoothstep
27             } else {
28                 // 平らな部分（高い音を維持）
29                 waveData[i] = 1.0f;
30             }
31
32         } else {
33             // 後半（低い音に戻る部分）
34             float localPhase = (phase - 0.5f) / 0.5f; // 0.0 ~ 1.0 に正規化
35             if (localPhase < slopeRatio) {
36                 // 立ち下がりスロープ（+1.0 から -1.0 へ滑らかに移動）
37                 float t = localPhase / slopeRatio;
38                 waveData[i] = 1.0f - 2.0f * (3*t*t - 2*t*t*t); // smoothstep
39             } else {
40                 // 平らな部分（低い音を維持）
41                 waveData[i] = -1.0f;
42             }
43         }
44     }

```

```

45
46     return new Wavetable(waveData);
47 }
48
49
50
51
52
53
54
55
56
57
58
59
60 class FluteInstrument implements Instrument {
61     byte numHarmonics/* = 15*/;
62
63     ADSR masterEnv;
64     ADSR[] harmonicEnvs/* = new ADSR[numHarmonics]*/;
65
66     Line[] wobbleLines;    // 6/12 【追加】 倍音ウナリ (AM) 用のフェードライン
67     float[] wobbleFactors; // 6/12 【追加】 倍音ごとの固有ウナリ比率の保持用
68
69     Line[] ampLines/* = new Line[numHarmonics]*/;
70     float[] startAmps/* = new float[numHarmonics]*/;
71     float[] endAmps/* = new float[numHarmonics]*/;
72
73     ADSR breathEnv;
74
75
76     // =====
77     // 【新規追加】 持続する空気の渦と管の共鳴 (サブハーモニクス層)
78     // =====
79     ADSR sustainEnv;
80
81     // 【追加】 持続ノイズの音量変化 (クレッシェンド/デクレッシェンド) 用
82     Line sustainAmpLine;
83     Line sustainWobbleLine; // 6/12 【追加】 持続ノイズのウナリ用のフェードライン
84     float startSustainVol;
85     float endSustainVol;
86
87
88
89     FluteInstrument(float midiNote, float duration, float startVel, float endVel, float
        masterVol) {
90         Summer mixer = new Summer();
91

```

```

92 //float masterAttackTime = 0.2f - constrain(startVel, 0, 127) / 127.0f * 0.15f;
93 float masterAttackTime = 0.18f;
94 masterAttackTime = min(masterAttackTime, duration - 0.1f);
95 if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
96
97 // 【修正】 masterEnv の一番最後の引数（リリース時間）を 0.3f から 0.4f に延ばす。
98 // 内部の音が 0.3秒 かけて完全に消え去るのを待ってから、安全にアンパッチさせるた
    めの余裕（バッファ）です。
99 // 【修正】 masterEnv のリリース時間を、希望のフェードアウト時間（例：0.35秒）に設
    定
100 masterEnv = new ADSR(1.0f, masterAttackTime, 0.1f, 0.9f, 0.2f);
101
102 mixer.patch(masterEnv);
103
104 // =====
105 // 【新規】 トリル判定と実音の計算
106 // =====
107 boolean isTrill = midiNote >= 128;
108 float actualMidiNote = isTrill ? midiNote - 128 : midiNote;
109
110 // 実音の周波数と、トリル先（全音上=+2）の周波数
111 //float f0 = Frequency.ofMidiNote(actualMidiNote).asHz();
112 //float f0_trill = Frequency.ofMidiNote(actualMidiNote + 1).asHz(); // 半音トリル
    にしたい場合は +1 にする

113
114 // LUTの倍音プロファイル
115 // 10倍音、最低音59、LUT配列、フルートのパワー(1.4f)
116 //float[][] startProfile = utils.GetDynamicProfile( actualMidiNote, startVel, "
    Flute", 59, 1.3f);
117 //float[][] endProfile = utils.GetDynamicProfile( actualMidiNote, endVel, "
    Flute", 59, 1.3f);

118
119 //float volFactor = masterVol * 0.11f;
120
121
122 //numHarmonics = (byte)startProfile.length;
123 // 【重要：NPE解決策】 実際のデータ数に合わせて、動的に配列のメモリを確保する！
124 //harmonicEnvs = new ADSR[numHarmonics];
125 //ampLines = new Line[numHarmonics];
126 //startAmps = new float[numHarmonics];
127 //endAmps = new float[numHarmonics];
128
129
130 // 変更6/12
131 // 1. 倍音レシピを安全に取り出すための基準ベロシティ（0除算・ロード不全の防止）
132 float profileVel = (startVel > 0) ? startVel : endVel;

```

```

133     if (profileVel <= 0) profileVel = 64.0f; // どちらも0の場合のセーフティ
134
135     // 2. 開始・終了のゲインファクターをフルートの特性 (0.7乗カーブ) に基づいて独立計
        算
136     float startGainFactor = pow(startVel / profileVel, 0.9f);
137     float endGainFactor   = pow(endVel / profileVel, 0.9f);
138
139     // 安全にロードされた基準プロファイルを取得
140     float[][] startProfile = utils.GetDynamicProfile(actualMidiNote, profileVel, "
        Flute", 59, 1.3f);

141
142     float volFactor = masterVol * 0.022f;
143
144     numHarmonics = (byte)startProfile.length;
145
146     harmonicEnvs = new ADSR[numHarmonics];
147     ampLines     = new Line[numHarmonics];
148     wobbleLines  = new Line[numHarmonics]; // 初期化を追加
149     wobbleFactors = new float[numHarmonics]; // 初期化を追加
150     startAmps    = new float[numHarmonics];
151     endAmps      = new float[numHarmonics];
152
153
154
155
156     //float fundamentalFreq = 440.0 * pow(2.0, (actualMidiNote - 69) / 12.0);
157     byte baseWaveNum = 0;
158     for (byte i = 1; i < numHarmonics; i++) {
159         if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
160     }
161
162
163     // =====
164     // 【新規】倍音ごとの自然な振幅揺らぎ (変動率ベース)
165     // =====
166     // Pythonで解析した変動率(%)を参考に、基音の振幅に対して何倍の揺れ±()を許容するか
        を設定。
167     // 実測データの変動率(%)から数式 [ Variation / 500 ] によって導出された、
168     // 倍音ごとの自然な振幅揺らぎ (AM変調) の深度設定。
169     float[] wobbleDepths = {0.1250f, 0.1616f, 0.2078f, 0.2340f, 0.2114f, 0.2742f,
        0.3716f, 0.2946f, 0.7420f, 0.8796f,
170         0.7850f, 0.5070f, 0.6242f, 0.7654f, 0.6702f, 0.3898f,
        0.8850f, 0.5942f, 0.3152f, 0.5030f};

171
172     // =====
173     // 【新規】この音符全体の「呼吸のスピード」を決定

```

```

174 // forループの外で1つだけ定義し、全倍音で共有することで相殺を防ぐ
175 // =====
176 float globalBreathSpeed = random(2.5f, 3.5f); // 生楽器の自然なビブラート周期
177
178
179
180
181
182 // 1. 加算合成パート（7つのサイン波）
183 for (int i = 0; i < numHarmonics; i++) {
184     /*
185     int h = i + 1;
186     // あなたがPythonで導き出した厳密なインハーモニシティ公式を適用
187     float B = 0.0005f; // インハーモニシティ係数
188     float inharmonicity = sqrt(1.0f + B * h * h);
189     float targetFreq = f0 * h * inharmonicity;
190     */
191     float targetFreq = startProfile[i][0];
192
193     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
194
195     // 【追加】初期位相を 0.0 ~ 1.0 (0度~360度) の間で完全にランダムに散らす
196     // これにより「カチッ」とした機械的なアタック音が消え、音割れ（ピーク）も防げる
197     osc.setPhase(random(1.0f));
198
199
200     //startAmps[i] = startProfile[i][1] * volFactor;
201     //endAmps[i] = endProfile[i] * volFactor;
202     // 聴覚特性（0.7乗）に完全同期させた、開始時から終了時への音量変化比率を算出
203     //float velRatio = pow(endVel / (startVel + 0.0001f), 0.7f);
204     // 【確定修正】すでにvolFactorが掛けられたstartAmpsに対して、正しいべき乗比率を
205     // 乗算する
206     //endAmps[i] = startAmps[i] * pow(endVel / (startVel + 0.0001f), 0.7f);
207
208     // 6/12追加
209     // 独立した開始・終了音量を厳密に算出
210     float baseAmp = startProfile[i][1] * volFactor;
211     startAmps[i] = baseAmp * startGainFactor;
212     endAmps[i] = baseAmp * endGainFactor;
213
214     // =====
215     // 【修正】LF0をトリルとビブラートで分岐
216     // =====
217     //if (isTrill) {
218     // // トリル時の上限周波数
219     // //float trillFreq = f0_trill * (i+1) * inharmonicity;
220     // float trillFreq = targetFreq * pow(2.0f, 1.0f/12.0f);

```

```

221
222 // // 矩形波LF0の中心を2つの音の真ん中に置き、振幅を「差の半分」にする
223 // float centerFreq = (targetFreq + trillFreq) / 2.0f;
224 // float ampFreq = abs(trillFreq - targetFreq) / 2.0f;
225
226 // // 7.5Hzのスピードで生成したグローバル変数 trillWave を使って瞬間的に音を切り替える
227 // Oscil freqController = new Oscil(6.5f, ampFreq, trillWave);
228 // freqController.offset.setLastValue(centerFreq);
229
230 // // トリル時のみ、オシレーターの周波数にLF0を接続する
231 // freqController.patch(osc.frequency);
232
233 // wobbleLines[i] = null; // トリル時はAM不要のため安全ガード
234 // wobbleFactors[i] = 0.0f;
235
236 //} else {
237 // 通常のビブラート（サイン波）
238
239 // =====
240 // ① 極小のピッチ揺れ（FM変調）の復活
241 // 以前の0.0132fを「0.003f」に激減させ、音痴にならない程度の「空気の揺らぎ」
// を作る
242 // =====
243 float vibDepthHz = targetFreq * 0.0015f;
244 Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz/10, Waves.SINE
);
245 freqController.offset.setLastValue(targetFreq);
246 freqController.patch(osc.frequency);
247
248 // =====
249 // ② 呼吸に同期した音量揺らぎ（AM変調）
250 // =====
251 // この倍音の基準音量に対して、配列で設定した割合だけ揺らす
252 //float wobbleAmp = startAmps[i] * wobbleDepths[i];
253 //float wobbleAmp = startAmps[i] * random(0.1f, 0.3f);
254
255 // 1.5Hz ～ 3.5Hz の間で、倍音ごとにランダムなスピードで揺らす
256 // 【修正】ランダムではなく、統一された globalBreathSpeed を使う！
257 //Oscil ampLF0 = new Oscil(globalBreathSpeed, wobbleAmp, Waves.SINE);
258
259 // 位相（スタート位置）は少しだけバラけさせると、音が立体的になります
260 //ampLF0.setPhase(random(1.0f));
261
262 // ampLF0を osc.amplitude にパッチ(追加)する。
263 // Minimでは複数のUGenを同じ入力にパッチすると「加算(Sum)」されるため、
264 // ampLines の基準値に対して、ampLF0 が ±wobbleAmp の揺れを足し引きしてくれます。

```



```

265         //ampLF0.patch(osc.amplitude);
266
267         // --- 音量揺らぎ (AM変調) のフェード構造化 ---
268         wobbleFactors[i] = random(0.1f, 0.2f);
269         wobbleFactors[i] = wobbleDepths[i];
270         wobbleLines[i] = new Line();
271         Oscil ampLF0 = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE); // 初期振幅は0
272         ampLF0.setPhase(random(1.0f));
273
274         wobbleLines[i].patch(ampLF0.amplitude); // 新設LineをLF0の振幅へ接続
275         ampLF0.patch(osc.amplitude);
276     //}
277
278     ampLines[i] = new Line();
279     ampLines[i].patch(osc.amplitude);
280
281     float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
282
283     // 【修正】内部のサイン波のリリースを 0.3f から 10.0f (10秒) にして、急激な角を
284     // 作らせない
285     harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
286     osc.patch(harmonicEnvs[i]);
287     harmonicEnvs[i].patch(mixer);
288 }
289
290
291 // =====
292 // 2. 息のノイズ (Chiff) パートの実測値アップデート
293 // =====
294 // 0.0~1.0 のベロシティ割合を計算 (ノイズの音量調整用)
295 float startNorm = constrain(startVel, 0, 127) / 127.0f;
296
297 // 中低音の破裂音 (ドスツという音) になるため、少しだけ音量を上げます (0.01f ->
298 // 0.02f)
299 float attackNoiseVol = masterVol * 0.008f * pow(startNorm, 0.9f);
300 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
301
302 // 【修正】ハイパス(HP)からバンドパス(BP)に変更。
303 // 3500Hz付近の「シュツ」という音だけを残し、耳障りな高音を削る。
304 // 【修正】実測データに基づき、3500Hzから 581.4Hz に変更！
305 // 473Hzの山も一緒に拾うため、レゾナンスを 0.3 から 0.15 に下げて帯域を広げる
306 MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);
307 breathFilter.type = MoogFilter.Type.BP;
308
309 breathEnv = new ADSR(1.0f, 0.001f, 0.2f, 0.0f, 0.0f);
310 breathNoise.patch(breathFilter);

```

```

311 breathFilter.patch(breathEnv);
312 breathEnv.patch(mixer);
313
314
315 /*
316 // =====
317 // 3. 【新規追加】持続する空気の渦と管の共鳴（サブハーモニクス層）
318 // =====
319 float endNorm = constrain(endVel, 0, 127) / 127.0f; // 【追加】終了時のベロシティ
    割合

320
321 // 【修正】開始時と終了時のノイズ音量を計算して保持する
322 startSustainVol = masterVol * 0.005f * pow(startNorm, 0.9f);
323 endSustainVol   = masterVol * 0.005f * pow(endNorm, 0.9f);
324
325 // ピンクノイズで低音の「フオォ」という空気感を出す
326 // 【修正】Noiseの初期音量は0.0fにし、音量制御はLineに任せる
327 Noise sustainNoise = new Noise(0.0f, Noise.Tint.PINK);
328
329 // 【追加】持続ノイズのボリュームつまみにLineを接続
330 sustainAmpLine = new Line();
331 sustainAmpLine.patch(sustainNoise.amplitude);
332
333 // =====
334 // 【新規】持続ノイズ自体にも「息の乱れ」を適用する
335 // サイン波の第1倍音と同等のスピードと深さでノイズを揺らす
336 // =====
337 //float noiseWobbleAmp = startSustainVol * 0.20f; // 20%揺らす
338 //Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), noiseWobbleAmp, Waves.SINE)
    ;
339 //sustainWobble.setPhase(random(1.0f));
340 //sustainWobble.patch(sustainNoise.amplitude); // Lineの音量にLFOの揺れを加算
341 // =====
342
343 // 6/12変更
344 // --- 持続ノイズの息の乱れも完全にフェードさせる ---
345 sustainWobbleLine = new Line();
346 Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), 0.0f, Waves.SINE); // 初期振
    幅は0
347 sustainWobble.setPhase(random(1.0f));
348
349 sustainWobbleLine.patch(sustainWobble.amplitude); // 新設LineをノイズLFOの振幅へ
    接続
350 sustainWobble.patch(sustainNoise.amplitude); // LFO出力をノイズ振幅へ（加
    算）

```

```

352 // ローパスフィルターで高音を削りつつ、2500Hz付近に共鳴(レゾナンス0.5)の山を作る
353 MoogFilter sustainFilter = new MoogFilter(2500f, 0.5f);
354 sustainFilter.type = MoogFilter.Type.LP;
355
356 // 音が鳴っている間ずっと持続するエンベロープ（サイン波のアタックに合わせる）
357 // 【修正】 持続ノイズのリリースも 0.3f から 10.0f（10秒）にする
358 sustainEnv = new ADSR(1.0f, 0.058f, 0.0f, 1.0f, 10.0f);
359
360 sustainNoise.patch(sustainFilter);
361 sustainFilter.patch(sustainEnv);
362 sustainEnv.patch(mixer);
363 */
364 }
365
366
367 /*
368 void noteOn(float dur) {
369     masterEnv.noteOn();
370
371     // =====
372     // 【修正】 Lineの寿命に、リリース時間（0.4秒）分の猶予を足す！
373     // これにより、減衰中もLineが生き残り、オシレーターの音量が突然0.0fにリセットされ
374     // るのを防ぐ
375     // =====
376     // 【微調整】 masterEnvのリリース時間(0.35f)より少し長ければOK
377     float safeDur = dur + 0.5f;
378
379     for(int i=0; i<numHarmonics; i++) {
380         harmonicEnvs[i].noteOn();
381         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
382     }
383
384     breathEnv.noteOn();
385     sustainEnv.noteOn(); // 【追加】 持続ノイズをオン
386     sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol); // 【追加】 発音
387     と同時に、持続ノイズの音量変化をスタートさせる
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

397 // 1. 各倍音のベース音量をフェード
398 ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
399
400 // 2. 通常ビブラート時のみ、ウナリ (AM) の深さも比例フェードさせる
401 if (wobbleLines[i] != null) {
402     float factor = wobbleFactors[i];
403     wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
404 }
405 }
406
407 breathEnv.noteOn();
408 //sustainEnv.noteOn();
409
410 // 3. 持続ノイズの全体音量と、そのウナリの深さを同時に追従フェード
411 //sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol);
412 //sustainWobbleLine.activate(safeDur, startSustainVol * 0.20f, endSustainVol *
    0.20f);
413
414 masterEnv.patch(longOut);
415 }
416
417
418
419 void noteOff() {
420     masterEnv.noteOff();
421     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
422
423     // 【削除】 breathEnv は既にな音なので noteOff を呼ばない (空撃ちグリッチ防止)
424     // breathEnv.noteOff();
425
426     //sustainEnv.noteOff(); // 【追加】 持続ノイズをオフ
427     masterEnv.unpatchAfterRelease(longOut);
428 }
429 }

```

E.7 Strings.pde

コード 69 Strings.pde

```

1 import java.util.Arrays;
2 import java.util.Comparator;
3
4 // =====
5 // 究極のストリングス・インストゥルメント (ビブラート数式モデル実装版)
6 // =====
7 class StringsInstrument implements Instrument {
8     //byte numHarmonics = 30;

```

```

9 //Summer mixer;
10 ADSR masterEnv;
11 //ADSR[] harmonicEnvs = new ADSR[numHarmonics];
12
13 /*
14 Line[] ampLines = new Line[numHarmonics];
15 float[] startAmps = new float[numHarmonics];
16 float[] endAmps = new float[numHarmonics];
17
18 // ビブラートの深さをコントロールするためのLine
19 Line[] vibDepthLines = new Line[numHarmonics];
20 float[] targetVibDepths = new float[numHarmonics]; // 各倍音の最終的な揺れ幅(Hz)を
    保存
21 */
22
23 // クラス上部のメンバー変数宣言部分の修正
24 byte maxTotalOscils = 40; // クロスフェード時に2つの楽器が混ざるため、多めに確保
25 Oscil[] oscils = new Oscil[maxTotalOscils];
26 Line[] ampLines = new Line[maxTotalOscils];
27 Line[] vibDepthLines = new Line[maxTotalOscils];
28
29 float[] startAmps = new float[maxTotalOscils];
30 float[] endAmps = new float[maxTotalOscils];
31 float[] targetVibDepths = new float[maxTotalOscils];
32
33 byte totalActiveOscils = 0; // 今回の音で実際に生成されたオシレーターの総数
34
35 Line[] wobbleLines = new Line[maxTotalOscils]; // 【新規追加】ウナリのフェード用ラ
    イン 6/12
36
37 //Noise biteNoise;
38 //MoogFilter biteFilter;
39 //ADSR biteEnv;
40 //ADSR hissEnv;
41
42 //Noise hissNoise;
43 //MoogFilter hissFilter;
44 //Line hissAmpLine;
45 //Oscil hissWobble;
46 //float startHissVol, endHissVol;
47
48 StringsInstrument(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
49     Summer mixer = new Summer();
50
51     // =====
52     // 6/13 【新設計】音符の長さに応じた動的アタックエンベロープ

```

```

53 // =====
54 float attackTime = 0.3f - constrain(startVel, 0, 127) / 127.0f * 0.25f;
55 attackTime = min(attackTime, duration - 0.1f);
56 if (attackTime < 0.035f) attackTime = 0.035f;
57
58
59 masterEnv = new ADSR(1.0f, attackTime, 0.5f, 0.9f, 0.12f);
60 mixer.patch(masterEnv);
61
62 //float f0 = Frequency.ofMidiNote(midiNote).asHz();
63 float volFactor = masterVol * 0.020f;
64
65 //float startNorm = constrain(startVel, 0, 127) / 127.0f;
66 //float endNorm = constrain(endVel, 0, 127) / 127.0f;
67
68
69 // =====
70 // 【データ分析から導いた数式①】ビブラートスピード
71 // 低音(G3=55)はゆっくり、高音(E7=100)は速く。さらに人間らしいランダムな揺らぎを
    付加。
72 // =====
73 float baseVibSpeed = map(midiNote, 48, 100, 5.2f, 6.5f);
74 float globalVibSpeed = baseVibSpeed + random(-0.2f, 0.2f);
75
76 // =====
77 // 【データ分析から導いた数式②】ビブラートの深さ (Cents)
78 // 低音は浅く±(10cents)、高音は深く±(25cents)揺れる。異常な暴走はここでカットさ
    れる。
79 // =====
80 float fmDepthCents = map(midiNote, 48, 100, 10.0f, 20.0f) * random(0.9f, 1.0f) *
    0.2f;
81
82
83 /*
84 // =====
85 // 【クロスフェード処理】オーケストラ・ストリングスの4Wayブレンド
86 // =====
87
88 // コントラバス：最低音 C1 (MIDI 24)
89 float[][] bassStart = utils.GetDynamicProfile(midiNote, startVel, "Bass", 24,
    0.9f);
90 float[][] bassEnd = utils.GetDynamicProfile(midiNote, endVel, "Bass", 24, 0.9
    f);
91
92 // チェロ：最低音 C2 (MIDI 36)

```

```

93     float[][] celloStart = utils.GetDynamicProfile(midiNote, startVel, "Cello", 36,
94           0.9f);
95
96     // ビオラ：最低音 C3 (MIDI 48)
97     float[][] violaStart = utils.GetDynamicProfile(midiNote, startVel, "Viola", 48,
98           0.9f);
99
100    // バイオリン：最低音 G3 (MIDI 55)
101    float[][] violinStart = utils.GetDynamicProfile(midiNote, startVel, "Violin", 55,
102          0.9f);
103
104    float[] startProfile = new float[numHarmonics];
105    float[] endProfile = new float[numHarmonics];
106
107    // -----
108    // ベースとなるブレンド割合の計算（合計が必ず1.0になるように分配）
109    // -----
110    float baseBassRatio = 0.0f;
111    float baseCelloRatio = 0.0f;
112    float baseViolaRatio = 0.0f;
113    float baseViolinRatio = 0.0f;
114
115    if (midiNote <= 36) {
116        // C2(36) 以下はコントラバスの独壇場
117        baseBassRatio = 1.0f;
118    } else if (midiNote <= 48) {
119        // C2(36) ~ C3(48)：コントラバスからチェロへ滑らかにクロスフェード
120        baseBassRatio = map(midiNote, 36, 48, 1.0f, 0.0f);
121        baseCelloRatio = 1.0f - baseBassRatio;
122    } else if (midiNote <= 60) {
123        // C3(48) ~ C4(60)：チェロからビオラへ滑らかにクロスフェード
124        baseCelloRatio = map(midiNote, 48, 60, 1.0f, 0.0f);
125        baseViolaRatio = 1.0f - baseCelloRatio;
126    } else if (midiNote <= 72) {
127        // C4(60) ~ C5(d)：ビオラからバイオリンへ滑らかにクロスフェード
128        baseViolaRatio = map(midiNote, 60, 72, 1.0f, 0.0f);
129        baseViolinRatio = 1.0f - baseViolaRatio;
130    } else {
131        // C5(72) 以上はバイオリンの独壇場

```

```

132     baseViolinRatio = 1.0f;
133 }
134
135 float bassBlendStart = baseBassRatio, bassBlendEnd = baseBassRatio;
136 float celloBlendStart = baseCelloRatio, celloBlendEnd = baseCelloRatio;
137 float violaBlendStart = baseViolaRatio, violaBlendEnd = baseViolaRatio;
138 float violinBlendStart = baseViolinRatio, violinBlendEnd = baseViolinRatio;
139
140 // -----
141 // 【特例パッチ】 D6(86) と D#6/Eb6(87) の波形破綻への対応
142 // mf(80)まではバイオリン本来の音色。そこからff(112)に向かってビオラを混ぜる
143 // -----
144 if (midiNote == 86 || midiNote == 87) {
145     float threshMF = 80.0f / 127.0f;
146     float threshFF = 112.0f / 127.0f;
147
148     if (startNorm > threshMF) {
149         float clampedStart = constrain(startNorm, threshMF, threshFF);
150         violinBlendStart = map(clampedStart, threshMF, threshFF, 1.0f, 0.3f);
151         violaBlendStart = 1.0f - violinBlendStart;
152     }
153     if (endNorm > threshMF) {
154         float clampedEnd = constrain(endNorm, threshMF, threshFF);
155         violinBlendEnd = map(clampedEnd, threshMF, threshFF, 1.0f, 0.3f);
156         violaBlendEnd = 1.0f - violinBlendEnd;
157     }
158
159     bassBlendStart = 0.0f; bassBlendEnd = 0.0f;
160     celloBlendStart = 0.0f; celloBlendEnd = 0.0f;
161 }
162 // -----
163
164 // 4つの楽器のプロファイルを、計算した割合で混ぜ合わせる
165 for (int i = 0; i < numHarmonics; i++) {
166     startProfile[i] = (bassStart[i] * bassBlendStart) + (celloStart[i] *
167         celloBlendStart) + (violaStart[i] * violaBlendStart) + (violinStart[i] *
168         violinBlendStart);
169     endProfile[i] = (bassEnd[i] * bassBlendEnd) + (celloEnd[i] *
170         celloBlendEnd) + (violaEnd[i] * violaBlendEnd) + (violinEnd[i] *
171         violinBlendEnd);
172 }
173 // =====
174 // 【インハーモニシティ係数(B)の動的計算セクション】
175 // =====

```



```

176
177 // 1. 各楽器の基準B係数（解析結果の中央値から設定）
178 float bBassBase = 0.000070f;
179 float bCelloBase = 0.000065f;
180 float bViolaBase = 0.000055f;
181 float bViolinBase = 0.000052f;
182
183 // 2. 各楽器の基準となる開放弦周波数（正規化用）
184 float fRefBass = 32.7f; // C1相当
185 float fRefCello = 65.4f; // C2相当
186 float fRefViola = 130.8f; // C3相当
187 float fRefViolin = 196.0f; // G3相当
188
189 // 3. 現在の楽器ブレンド状態に合わせた「基準B」と「基準周波数」を算出
190 float blendedBBase = (bBassBase * baseBassRatio) + (bCelloBase * baseCelloRatio)
191                      + (bViolaBase * baseViolaRatio) + (bViolinBase *
192                      baseViolinRatio);
193
194 float blendedFRef = (fRefBass * baseBassRatio) + (fRefCello * baseCelloRatio)
195                  + (fRefViola * baseViolaRatio) + (fRefViolin * baseViolinRatio
196                  );
197
198 // 4. 【核心】ピッチの2乗に比例してBを増幅させる（ハイポジション効果の再現）
199 // あなたの観察通り、C5(523Hz)において約0.00006になるようスケーリングされます
200 float B = blendedBBase * pow(f0 / blendedFRef, 2.0f);
201
202 // 異常値ガード（物理的に破綻しない範囲に制限）
203 B = constrain(B, 0.00001f, 0.0005f);
204
205 // =====
206
207 for (int i = 0; i < numHarmonics; i++) {
208     int h = i + 1;
209     // 動的に計算された B を適用
210     float inharmonicity = sqrt(1.0f + B * h * h);
211     float targetFreq = f0 * h * inharmonicity;
212
213     // 【改善】アンチエイリアス（20000Hz超えの倍音は、生成処理自体をストップしてCPU
214     // を節約）
215     if (targetFreq > 22050.0f) {
216         continue; // ここでループを抜け、これ以上高い倍音は作らない
217     }
218
219     float finalStartAmp = startProfile[i] * volFactor;

```

```

219     float finalEndAmp = endProfile[i] * volFactor;
220
221     // アンチエイリアス (20000Hz超えの強制ミュート)
222     //if (targetFreq > 20000.0f) {
223     //    finalStartAmp = 0.0f;
224     //    finalEndAmp = 0.0f;
225     //}
226
227     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
228     osc.setPhase(0.0f); // スtringスの核 (エッジ)
229
230     // =====
231     // ピッチ揺れ (Delayed FM変調) の準備
232     // セント(Cents)を周波数(Hz)の揺れ幅に変換して配列に保存しておく
233     // =====
234     float vibDepthRatio = pow(2.0f, fmDepthCents / 1200.0f) - 1.0f;
235     targetVibDepths[i] = targetFreq * vibDepthRatio;
236
237     // 揺れの深さをコントロールするLFO (最初は振幅0でスタート)
238     Oscil freqController = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
239     freqController.offset.setLastValue(targetFreq);
240     freqController.patch(osc.frequency);
241
242     // ビブラートの深さを0から目標値へ変化させるLine
243     vibDepthLines[i] = new Line();
244     vibDepthLines[i].patch(freqController.amplitude);
245
246     // =====
247     // 音量揺らぎ (AM変調) の適用
248     // 高次倍音ほど激しく揺れる (15%~60%)
249     // =====
250     startAmps[i] = finalStartAmp;
251     endAmps[i] = finalEndAmp;
252
253     float wobbleDepth = map(i, 0, numHarmonics - 1, 0.15f, 0.60f);
254     float wobbleAmp = startAmps[i] * wobbleDepth;
255
256     Oscil ampLFO = new Oscil(globalVibSpeed, wobbleAmp, Waves.SINE);
257     ampLFO.setPhase(random(1.0f));
258     ampLFO.patch(osc.amplitude);
259
260     ampLines[i] = new Line();
261     ampLines[i].patch(osc.amplitude);
262
263     //harmonicEnvs[i] = new ADSR(1.0f, 0.2f, 0.0f, 1.0f, 0.3f);
264     //osc.patch(harmonicEnvs[i]);
265     //harmonicEnvs[i].patch(mixer);
266

```

```

267 // 【修正1】 harmonicEnvsを削除し、純粹にミキサーへ送る
268 osc.patch(mixer);
269 }
270 */
271
272 // =====
273 // 【クロスフェード処理】 オーケストラ・ストリングスの4Wayブレンド
274 // =====
275 // 1. 各楽器のブレンド割合の計算 (合計が必ず1.0になるように分配)
276 float baseBassRatio = 0.0f;
277 float baseCelloRatio = 0.0f;
278 float baseViolaRatio = 0.0f;
279 float baseViolinRatio = 0.0f;
280
281 if (midiNote <= 36) {
282     baseBassRatio = 1.0f;
283 } else if (midiNote <= 48) {
284     baseBassRatio = map(midiNote, 36, 48, 1.0f, 0.0f);
285     baseCelloRatio = 1.0f - baseBassRatio;
286 } else if (midiNote <= 60) {
287     baseCelloRatio = map(midiNote, 48, 60, 1.0f, 0.0f);
288     baseViolaRatio = 1.0f - baseCelloRatio;
289 } else if (midiNote <= 72) {
290     baseViolaRatio = map(midiNote, 60, 72, 1.0f, 0.0f);
291     baseViolinRatio = 1.0f - baseViolaRatio;
292 } else {
293     baseViolinRatio = 1.0f;
294 }
295
296 //float bassBlend = baseBassRatio;
297 //float celloBlend = baseCelloRatio;
298 //float violaBlend = baseViolaRatio;
299 //float violinBlend = baseViolinRatio;
300
301 float bassBlend = 0.0f;
302 float celloBlend = 0.0f;
303 float violaBlend = 0.0f;
304 float violinBlend = 1.0f;
305
306 /*
307 // 【特例パッチ】 D6(86) と D#6/Eb6(87) の波形破綻への対応
308 if (midiNote == 86 || midiNote == 87) {
309     float threshMF = 80.0f / 127.0f;
310     float threshFF = 112.0f / 127.0f;
311     if (startNorm > threshMF) {
312         violinBlend = map(constrain(startNorm, threshMF, threshFF), threshMF,
313             threshFF, 1.0f, 0.0f);
314         violaBlend = 1.0f - violinBlend;
315     }
316 }

```

```

314     }
315     bassBlend = 0.0f; celloBlend = 0.0f;
316 }
317 */
318
319
320 /*
321 // -----
322 // 2. 新設計：各楽器のプロファイルを「必要な分だけ」動的にオシレーター化して配置
323 // -----
324 totalActiveOscils = 0; // カウンターリセット
325
326 // ループ処理を共通化するための構造化（内部用の一時クラスや処理の連続実行）
327 // 比率が 0.0 より大きい楽器だけを狙い撃ちしてオシレーターを生成する（安全装置・
    負荷対策）

```

```

328
329 if (bassBlend > 0.0f) {
330     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Bass", 24, 0.9f
    );
331     createInstrumentOscillators(pStart, bassBlend, volFactor, globalVibSpeed,
    fmDepthCents, mixer);
332 }
333 if (celloBlend > 0.0f) {
334     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Cello", 36, 0.9
    f);
335     createInstrumentOscillators(pStart, celloBlend, volFactor, globalVibSpeed,
    fmDepthCents, mixer);
336 }
337 if (violaBlend > 0.0f) {
338     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Viola", 48, 0.9
    f);
339     createInstrumentOscillators(pStart, violaBlend, volFactor, globalVibSpeed,
    fmDepthCents, mixer);
340 }
341 if (violinBlend > 0.0f) {
342     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Violin", 55,
    0.9f);
343     createInstrumentOscillators(pStart, violinBlend, volFactor, globalVibSpeed,
    fmDepthCents, mixer);
344 }
345 */
346 /*
347 // -----
348 // 2. 改修版：各楽器のプロファイルを「動的な上限数」でオシレーター化
349 // -----
350 // --- コンストラクタ下部：各楽器呼び出しの直前に追加 ---
351 //float velRatio = pow(endVel / (startVel + 0.0001f), 0.7f);

```

```

352 float velRatio = pow((endVel + 0.0001f) / (startVel + 0.0001f), 0.9f);
353
354 totalActiveOscils = 0; // カウンターリセット
355
356 if (bassBlend > 0.0f) {
357     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Bass", 24, 0.9f
358 );
359     // ユーザー様提案のルール：1.0未満（ブレンド時）なら20、単一なら30を上限とする
360     int maxPeaks = (bassBlend < 1.0f) ? 20 : 30;
361     createInstrumentOscillators(pStart, bassBlend, maxPeaks, volFactor,
362         globalVibSpeed, fmDepthCents, mixer, velRatio);
363 }
364
365 if (celloBlend > 0.0f) {
366     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Cello", 36, 0.9
367 f);
368     int maxPeaks = (celloBlend < 1.0f) ? 20 : 30;
369     createInstrumentOscillators(pStart, celloBlend, maxPeaks, volFactor,
370         globalVibSpeed, fmDepthCents, mixer, velRatio);
371 }
372
373 if (violaBlend > 0.0f) {
374     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Viola", 48, 0.9
375 f);
376     int maxPeaks = (violaBlend < 1.0f) ? 20 : 30;
377     createInstrumentOscillators(pStart, violaBlend, maxPeaks, volFactor,
378         globalVibSpeed, fmDepthCents, mixer, velRatio);
379 }
380
381 if (violinBlend > 0.0f) {
382     float[][] pStart = utils.GetDynamicProfile(midiNote, startVel, "Violin", 55,
383         0.9f);
384     int maxPeaks = (violinBlend < 1.0f) ? 20 : 30;
385     createInstrumentOscillators(pStart, violinBlend, maxPeaks, volFactor,
386         globalVibSpeed, fmDepthCents, mixer, velRatio);
387 }
388
389 /*
390 // 更新日6/12
391 // =====
392 // 【新設計】開始・終了ベロシティから音量変化を正しくスケーリング
393 // =====
394 // 1. 倍音レシビを安全に取り出すための基準ベロシティ（0除算・無音化の防止）
395 float profileVel = (startVel > 0) ? startVel : endVel;
396 if (profileVel <= 0) profileVel = 64.0f; // 両方0の場合の完全セーフティ
397
398 // 2. 開始時と終了時の実際の音量ゲインファクターを、0.9乗カーブに基づいて個別に計
399 算

```

```

391 float startGainFactor = pow(startVel / profileVel, 0.9f);
392 float endGainFactor   = pow(endVel / profileVel, 0.9f);
393
394 // -----
395 // 2. 各楽器のプロファイルを動的にオシレーター化して配置
396 // -----
397 totalActiveOscils = 0; // カウンターリセット
398
399 if (bassBlend > 0.0f) {
400     float[][] pStart = utils.GetDynamicProfile(midiNote, profileVel, "Bass", 24,
401         0.9f); // profileVelを渡す
402     int maxPeaks = (bassBlend < 1.0f) ? 20 : 30;
403     createInstrumentOscillators(pStart, bassBlend, maxPeaks, volFactor,
404         globalVibSpeed, fmDepthCents, mixer, startGainFactor, endGainFactor); //
405         ファクターを渡す
406 }
407
408 if (celloBlend > 0.0f) {
409     float[][] pStart = utils.GetDynamicProfile(midiNote, profileVel, "Cello", 36,
410         0.9f);
411     int maxPeaks = (celloBlend < 1.0f) ? 20 : 30;
412     createInstrumentOscillators(pStart, celloBlend, maxPeaks, volFactor,
413         globalVibSpeed, fmDepthCents, mixer, startGainFactor, endGainFactor);
414 }
415
416 if (violaBlend > 0.0f) {
417     float[][] pStart = utils.GetDynamicProfile(midiNote, profileVel, "Viola", 48,
418         0.9f);
419     int maxPeaks = (violaBlend < 1.0f) ? 20 : 30;
420     createInstrumentOscillators(pStart, violaBlend, maxPeaks, volFactor,
421         globalVibSpeed, fmDepthCents, mixer, startGainFactor, endGainFactor);
422 }
423
424 if (violinBlend > 0.0f) {
425     float[][] pStart = utils.GetDynamicProfile(midiNote, profileVel, "Violin", 55,
426         0.9f);
427     int maxPeaks = (violinBlend < 1.0f) ? 20 : 30;
428     createInstrumentOscillators(pStart, violinBlend, maxPeaks, volFactor,
429         globalVibSpeed, fmDepthCents, mixer, startGainFactor, endGainFactor);
430 }
431
432 /*
433 // =====
434 // 【データ分析から導いた数式③】 摩擦ノイズ(Bite)の動的パラメータ
435 // =====
436 // 1. 各楽器の基準値を設定 (Python解析データより)
437 float biteCutoffBass   = 1150f; float biteDecayBass   = 0.180f;

```

```

430 float biteCutoffCello = 1840f; float biteDecayCello = 0.093f;
431 float biteCutoffViola = 3280f; float biteDecayViola = 0.116f;
432 float biteCutoffViolin = 3370f; float biteDecayViolin = 0.035f;
433
434 // 2. 現在の音階（クロスフェード割合）に合わせてパラメータをブレンド
435 float blendedBiteCutoff = (biteCutoffBass * baseBassRatio) + (biteCutoffCello *
436     baseCelloRatio)
437     + (biteCutoffViola * baseViolaRatio) + (biteCutoffViolin *
438     baseViolinRatio);
439
440 float blendedBiteDecay = (biteDecayBass * baseBassRatio) + (biteDecayCello *
441     baseCelloRatio)
442     + (biteDecayViola * baseViolaRatio) + (biteDecayViolin *
443     baseViolinRatio);
444
445 // グラフの傾向（高音ほどノイズが短く・高くなる）を少し加味する微調整
446 // 基音(f0)が上がるにつれて、カットオフを少し上げ、ディケイを少し短くする
447 float pitchFactor = map(midiNote, 24, 84, 0.8f, 1.2f);
448 float finalBiteCutoff = constrain(blendedBiteCutoff * pitchFactor, 80f, 6000f);
449 float finalBiteDecay = constrain(blendedBiteDecay / pitchFactor, 0.01f, 0.3f);
450
451 // --- 摩擦ノイズ層（Bite）の生成 ---
452 // アタックの強さ（ベロシティ）に応じてノイズ音量を変える
453 float biteVol = masterVol * 0.001f * startNorm;
454
455 Noise biteNoise = new Noise(biteVol, Noise.Tint.WHITE);
456 MoogFilter biteFilter = new MoogFilter(finalBiteCutoff, 0.2f); // 動的カットオフ
457     を適用
458 biteFilter.type = MoogFilter.Type.LP;
459
460 // Attackは固定(超短く)、Decayに動的パラメータを適用
461 biteEnv = new ADSR(1.0f, 0.1f, finalBiteDecay, 0.0f, 0.1f);
462
463 biteNoise.patch(biteFilter);
464 biteFilter.patch(biteEnv);
465 // (biteEnvは直接outへ出すためmixerには繋がらない)
466 */
467
468 /* 【持続ノイズHiss削除 6/12】
469 // --- 共鳴ノイズ層（Hiss） ---
470 startHissVol = masterVol * 0.001f * startNorm;
471 endHissVol = masterVol * 0.001f * endNorm;
472
473 Noise hissNoise = new Noise(0.0f, Noise.Tint.PINK);

```

```

471     hissAmplLine = new Line();
472     hissAmplLine.patch(hissNoise.amplitude);
473
474     Oscil hissWobble = new Oscil(globalVibSpeed, startHissVol * 0.3f, Waves.SINE);
475     hissWobble.setPhase(random(1.0f));
476     hissWobble.patch(hissNoise.amplitude);
477
478     MoogFilter hissFilter = new MoogFilter(1500f, 0.15f);
479     hissFilter.type = MoogFilter.Type.HP;
480     hissEnv = new ADSR(1.0f, 0.05f, 0.0f, 1.0f, 0.1f);
481
482     hissNoise.patch(hissFilter);
483     hissFilter.patch(hissEnv);
484     hissEnv.patch(mixer);
485     */
486 }
487
488 /*
489 // 各楽器のピークデータをグローバルなオシレーター配列に安全に展開するヘルパー関数
490 private void createInstrumentOscillators(float[][] profile, float blendRatio, float
    volFactor, float globalVibSpeed, float fmDepthCents, Summer mixer) {
491     if (profile == null) return;
492
493     byte numPeaks = (byte)profile.length;
494     for (byte h = 0; h < numPeaks; h++) {
495         // 安全装置：配列の上限を超えないようにガード
496         if (totalActiveOscils >= MAX_TOTAL_OSCILS) break;
497
498         float targetFreq = profile[h][0]; // Pythonが実測した、非整数倍音が含まれる生の
            周波数
499         float rawAmp      = profile[h][1]; // 正規化された振幅
500
501         // 20000Hz超えの超音波は生成をスキップしてCPUを徹底節約（アンチエイリアス）
502         if (targetFreq > 22050.0f) continue;
503
504         // 【核心】元のスペクトル振幅に、全体のボリュームと「この楽器のブレンド比率」を
            掛け合わせる
505         float finalStartAmp = rawAmp * volFactor * blendRatio;
506         float finalEndAmp   = finalStartAmp * (endHissVol / (startHissVol + 0.0001f));
            // 代案2の音量比率の維持
507
508         int idx = totalActiveOscils;
509         startAmps[idx] = finalStartAmp;
510         endAmps[idx]   = finalEndAmp;
511
512         // オシレーターとエンベロープの生成
513         oscils[idx] = new Oscil(targetFreq, 0.0f, Waves.SINE);

```



```

514         oscils[idx].setPhase(0.0f);
515
516         // ビブラート (FM変調) の適用
517         float vibDepthRatio = pow(2.0f, fmDepthCents / 1200.0f) - 1.0f;
518         targetVibDepths[idx] = targetFreq * vibDepthRatio;
519
520         Oscil freqController = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
521         freqController.offset.setLastValue(targetFreq);
522         freqController.patch(oscils[idx].frequency);
523
524         vibDepthLines[idx] = new Line();
525         vibDepthLines[idx].patch(freqController.amplitude);
526
527         // 音量揺らぎ (AM変調) の適用 (高次倍音ほど激しく揺らす特性は継承)
528         float wobbleDepth = map(idx, 0, MAX_TOTAL_OSCILS - 1, 0.15f, 0.60f);
529         float wobbleAmp = startAmps[idx] * wobbleDepth;
530
531         Oscil ampLFO = new Oscil(globalVibSpeed, wobbleAmp, Waves.SINE);
532         ampLFO.setPhase(random(1.0f));
533         ampLFO.patch(oscils[idx].amplitude);
534
535         ampLines[idx] = new Line();
536         ampLines[idx].patch(oscils[idx].amplitude);
537
538         oscils[idx].patch(mixer);
539
540         totalActiveOscils++; // 生成に成功したら総数をインクリメント
541     }
542 }
543 */
544 /*
545 // 各楽器のピークデータを、振幅選別フィルタを通した上で安全にオシレーター展開するヘルパー関数
546 private void createInstrumentOscillators(final float[][] profile, float blendRatio,
547     int maxPeaks, float volFactor, float globalVibSpeed, float fmDepthCents,
548     Summer mixer, float velRatio) {
549     if (profile == null || profile.length == 0) return;
550
551     int rawLength = profile.length;
552     // 実際に採用する山の数を決定 (データ数と制限数の小さい方)
553     final int limitPeaks = min(maxPeaks, rawLength);
554
555     // -----
556     // 【振幅選別フィルタ】 振幅が大きい順にインデックスをソートする
557     // -----
558     Integer[] indices = new Integer[rawLength];
559     for (int i = 0; i < rawLength; i++) indices[i] = i;

```

```

559 Arrays.sort(indices, new Comparator<Integer>() {
560     public int compare(Integer a, Integer b) {
561         // 振幅[1]の降順（大きい順）で並び替え
562         return Float.compare(profile[b][1], profile[a][1]);
563     }
564 });
565
566 // 上位 N 個（limitPeaks分）だけを一度抽出し、一時配列に格納
567 float[][] filteredProfile = new float[limitPeaks][2];
568 for (int i = 0; i < limitPeaks; i++) {
569     filteredProfile[i][0] = profile[indices[i]][0]; // 周波数
570     filteredProfile[i][1] = profile[indices[i]][1]; // 振幅
571 }
572
573 // -----
574 // 【周波数再整列】Processing側（ビブラート計算など）の仕様に合わせ、周波数の低い
575 // 順に戻す
576 // -----
577 Arrays.sort(filteredProfile, new Comparator<float[]>() {
578     public int compare(float[] a, float[] b) {
579         // 周波数[0]の昇順（低い順）で並び替え
580         return Float.compare(a[0], b[0]);
581     }
582 });
583
584 // -----
585 // 3. 厳選されたピークデータのみをオシレーターとして生成・接続
586 // -----
587 for (int h = 0; h < limitPeaks; h++) {
588     if (totalActiveOscils >= MAX_TOTAL_OSCILS) break;
589
590     float targetFreq = filteredProfile[h][0]; // 周波数
591     float rawAmp      = filteredProfile[h][1]; // 厳選された強い振幅
592
593     if (targetFreq > 22050.0f) continue;
594
595     float finalStartAmp = rawAmp * volFactor * blendRatio;
596     float finalEndAmp   = finalStartAmp * velRatio;
597
598     int idx = totalActiveOscils;
599     startAmps[idx] = finalStartAmp;
600     endAmps[idx]   = finalEndAmp;
601
602     // オシレーターとビブラート（LFO）、AM変調の適用（既存の高品質ロジックをそのま
603     // ま継承）
604     oscils[idx] = new Oscil(targetFreq, 0.0f, Waves.SINE);
605     oscils[idx].setPhase(0.0f);

```

```

605     float vibDepthRatio = pow(2.0f, fmDepthCents / 1200.0f) - 1.0f;
606     targetVibDepths[idx] = targetFreq * vibDepthRatio;
607
608     Oscil freqController = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
609     freqController.offset.setLastValue(targetFreq);
610     freqController.patch(oscils[idx].frequency);
611
612     vibDepthLines[idx] = new Line();
613     vibDepthLines[idx].patch(freqController.amplitude);
614
615     float wobbleDepth = map(idx, 0, MAX_TOTAL_OSCILS - 1, 0.15f, 0.60f);
616     float wobbleAmp = startAmps[idx] * wobbleDepth;
617
618     Oscil ampLFO = new Oscil(globalVibSpeed, wobbleAmp, Waves.SINE);
619     ampLFO.setPhase(random(1.0f));
620     ampLFO.patch(oscils[idx].amplitude);
621
622     ampLines[idx] = new Line();
623     ampLines[idx].patch(oscils[idx].amplitude);
624
625     oscils[idx].patch(mixer);
626
627     totalActiveOscils++;
628 }
629 }
630 */
631 // 更新日6/12
632 private void createInstrumentOscillators(final float[][] profile, float blendRatio,
        int maxPeaks, float volFactor, float globalVibSpeed, float fmDepthCents,
        Summer mixer, float startGainFactor, float endGainFactor) {
633     if (profile == null || profile.length == 0) return;
634
635     int rawLength = profile.length;
636     final int limitPeaks = min(maxPeaks, rawLength);
637
638     Integer[] indices = new Integer[rawLength];
639     for (int i = 0; i < rawLength; i++) indices[i] = i;
640
641     Arrays.sort(indices, new Comparator<Integer>() {
642         public int compare(Integer a, Integer b) {
643             return Float.compare(profile[b][1], profile[a][1]);
644         }
645     });
646
647     float[][] filteredProfile = new float[limitPeaks][2];
648     for (int i = 0; i < limitPeaks; i++) {
649         filteredProfile[i][0] = profile[indices[i]][0];
650         filteredProfile[i][1] = profile[indices[i]][1];

```

```

651     }
652
653     Arrays.sort(filteredProfile, new Comparator<float[]>() {
654         public int compare(float[] a, float[] b) {
655             return Float.compare(a[0], b[0]);
656         }
657     });
658
659     for (int h = 0; h < limitPeaks; h++) {
660         if (totalActiveOscils >= maxTotalOscils) break;
661
662         float targetFreq = filteredProfile[h][0];
663         float rawAmp      = filteredProfile[h][1];
664
665         //if (targetFreq > 22050.0f) continue;
666         if (targetFreq > 14000.0f) continue;
667
668         // 6/13 【新設：数学的倍音フィルター】
669         // 8000Hzから18000Hzにかけて、その倍音の振幅（パワー）を滑らかに 100% から 0%
670         //   へ減衰させる
671         if (targetFreq > 7000.0f) {
672             float filterFactor = map(targetFreq, 7000.0f, 17000.0f, 1.0f, 0.0f);
673             rawAmp *= constrain(filterFactor, 0.0f, 1.0f); // 厳密に0~1の範囲に固定して
674             //   乗算
675         }
676
677         // 【確実なスケーリング】実際の開始音量と終了音量を個別に決定
678         float baseAmp = rawAmp * volFactor * blendRatio;
679         float finalStartAmp = baseAmp * startGainFactor;
680         float finalEndAmp   = baseAmp * endGainFactor;
681
682         int idx = totalActiveOscils;
683         startAmps[idx] = finalStartAmp;
684         endAmps[idx]   = finalEndAmp;
685
686         oscils[idx] = new Oscil(targetFreq, 0.0f, Waves.SINE);
687         oscils[idx].setPhase(random(1.0f));
688
689         // --- ビブラート（FM変調）の適用 ---
690         float vibDepthRatio = pow(2.0f, fmDepthCents / 1200.0f) - 1.0f;
691         targetVibDepths[idx] = targetFreq * vibDepthRatio;
692
693         Oscil freqController = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
694         freqController.offset.setLastValue(targetFreq);
695         //freqController.patch(freqController.amplitude); // ※元のコードママ
696         freqController.patch(oscils[idx].frequency);
697
698         vibDepthLines[idx] = new Line();

```

```

697     vibDepthLines[idx].patch(freqController.amplitude);
698
699     // --- 音量揺らぎ (AM変調) : ウナリ自体をフェードさせる革新的な設計 ---
700     wobbleLines[idx] = new Line();
701     Oscil ampLFO = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
702     ampLFO.setPhase(random(1.0f));
703
704     wobbleLines[idx].patch(ampLFO.amplitude); // 新設LineをLFOの振幅へパッチ！これ
        でウナリがフェードする
705     ampLFO.patch(oscils[idx].amplitude); // LFOの出力をオシレーターへ (加算)
706
707     // ベース音量の適用
708     ampLines[idx] = new Line();
709     ampLines[idx].patch(oscils[idx].amplitude); // ベース音量変化をオシレーターへ
        (加算)
710
711     oscils[idx].patch(mixer);
712     totalActiveOscils++;
713 }
714 }
715
716
717
718 /*
719 void noteOn(float dur) {
720     masterEnv.noteOn();
721     biteEnv.noteOn();
722     hissEnv.noteOn();
723
724     float safeDur = dur + 0.7f;
725     // ビブラートが最大になるまでの時間 (0.5秒、または音符の長さの短い方)
726     float vibDelayTime = min(0.5f, dur);
727
728
729     //for(int i = 0; i < numHarmonics; i++) {
730         // 【修正】 インスタンスが生成されていない (null) 場合は、それ以上の倍音はないの
            でループを終了する
731         // if (ampLines[i] == null || vibDepthLines[i] == null) {
732             // continue;
733         // }
734         //
735         //
736         // ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
737         // // 【実行】 0Hzから目標の揺れ幅(Hz)へ、vibDelayTimeかけて徐々にビブラートを深
            くする
738         // vibDepthLines[i].activate(vibDelayTime, 0.0f, targetVibDepths[i]);
739         // }

```

```

740
741 // noteOn 内のループ部分の修正
742 for(int i = 0; i < totalActiveOscils; i++) {
743     if (ampLines[i] == null || vibDepthLines[i] == null) {
744         continue;
745     }
746     ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
747     vibDepthLines[i].activate(vibDelayTime, 0.0f, targetVibDepths[i]);
748 }
749
750 hissAmplLine.activate(safeDur, startHissVol, endHissVol);
751
752 masterEnv.patch(out);
753 biteEnv.patch(out);
754 }
755 */
756
757 // 更新日6/12
758 void noteOn(float dur) {
759     masterEnv.noteOn();
760     //biteEnv.noteOn();
761     //hissEnv.noteOn();
762
763     float safeDur = dur + 0.7f;
764     float vibDelayTime = min(0.5f, dur);
765
766     for(int i = 0; i < totalActiveOscils; i++) {
767         if (ampLines[i] == null || vibDepthLines[i] == null || wobbleLines[i] == null)
768         {
769             continue;
770         }
771         // 1. ベース音量のフェード
772         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
773
774         // 2. ウナリ (AM変調) の揺れ幅フェード (ベース音量に比例してウナリの深さも綺麗
775         //     に消える)
776         float wobbleDepth = map(i, 0, maxTotalOscils - 1, 0.15f, 0.60f);
777         wobbleLines[i].activate(safeDur, startAmps[i] * wobbleDepth, endAmps[i] *
778             wobbleDepth);
779
780         // 3. ビブラート (FM変調) の変化
781         vibDepthLines[i].activate(vibDelayTime, 0.0f, targetVibDepths[i]);
782     }
783
784     //hissAmplLine.activate(safeDur, startHissVol, endHissVol);
785
786     masterEnv.patch(longOut);

```

```

784     //biteEnv.patch(longOut);
785 }
786
787 void noteOff() {
788     masterEnv.noteOff();
789     //for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
790     //biteEnv.noteOff();
791     //hissEnv.noteOff();
792     masterEnv.unpatchAfterRelease(longOut);
793     //biteEnv.unpatchAfterRelease(longOut);
794 }
795 }

```

E.8 Piano.pde

コード 70 Piano.pde

```

1
2 //float[][] pianoDecayLUT;
3 //float[][] pianoWobbleRateLUT;
4 //float[][] pianoWobbleDepthLUT;
5
6 // =====
7 // 究極のピアノ・インストゥルメント (デュアル・ストリング & ハイブリッドデータ駆動版)
8 // =====
9 class PianoInstrument implements Instrument {
10     // 分析データに合わせて最大80倍音まで生成
11     byte numHarmonics/* = 40*/;
12
13     ADSR masterEnv; // 全体の音の長さを管理し、離鍵時にミュートする (ダンパーフェルトの役割)
14     ADSR hammerEnv; // ハンマーが弦を叩く一瞬のノイズ用エンベロープ
15
16     // 個別の減衰 (Damp) を管理するリスト。
17     // 芯の弦と共鳴弦でDampの数変動するため、配列ではなくArrayListを使用します。
18     ArrayList<Damp> damp = new ArrayList<Damp>();
19
20     PianoInstrument(float midiNote, float velocity, float masterVol) {
21         Summer mixer = new Summer();
22
23         // ユーザー定義のベロシティ閾値
24         float thresholdPP = 32.0f / 127.0f;
25         float thresholdMF = 80.0f / 127.0f;
26         float thresholdFF = 112.0f / 127.0f;
27
28         // 現在のベロシティ層に応じて、ベースとなる波形プロファイル (周波数・振幅の骨組み) を決定
29         String baseVelocity;

```

```

30 float t = 0;
31 //boolean needInterpolation = false;
32 //float[][] lowerProfile = null;
33 //float[][] upperProfile = null;
34
35 if (velocity <= thresholdPP) {
36     baseVelocity = "PP";
37 } else if (velocity <= thresholdMF) {
38     // pp ~ mf の間：構造が近い方のスペクトル形状を選択（あるいは、より高度に処理
        するためのフラグ設定）
39     t = map(velocity, thresholdPP, thresholdMF, 0.0f, 1.0f);
40     baseVelocity = (t < 0.5f) ? "PP" : "MF";
41 } else if (velocity <= thresholdFF) {
42     // mf ~ ff の間
43     t = map(velocity, thresholdMF, thresholdFF, 0.0f, 1.0f);
44     baseVelocity = (t < 0.5f) ? "MF" : "FF";
45 } else {
46     baseVelocity = "FF";
47 }
48
49 float[][] pianoDecayLUT = utils.GetLUT("PianoDecay" + baseVelocity);
50 float[][] pianoWobbleRateLUT = utils.GetLUT("PianoWobbleRate" + baseVelocity);
51 float[][] pianoWobbleDepthLUT = utils.GetLUT("PianoWobbleDepth" + baseVelocity);
52
53 // =====
54 // 1. 全体のエンベロープ設定（ダンパーの挙動）
55 // =====
56 // 鍵盤を押している間は減衰を邪魔せず(Sustain=1.0)、離れた瞬間(noteOff)に0.15秒で
        ミュート
57 masterEnv = new ADSR(1.0f, 0.001f, 0.4f, 0.5f, 0.09f);
58 mixer.patch(masterEnv);
59
60 //float f0 = Frequency.ofMidiNote(midiNote).asHz();
61 float normVel = pow(constrain(velocity, 0, 127) / 127.0f, 0.9f);
62 float volFactor = masterVol * 0.022f;
63
64 //volFactor *= (0.5 + (midiNote/254.0f)); // 必要かどうか検討 --> 不要
65
66 // =====
67 // 2. LUTデータの取得とインハーモニシティ設定
68 // =====
69 // 強弱に応じた倍音構成（音色レシピ）を取得
70 float[][] profile = utils.GetDynamicProfile(midiNote, velocity, "Piano", 21, 0.9f
        );
71
72 // 【修正の核心1】実測ピークデータから、今回のループ回数を動的に確定させる！
73 numHarmonics = (byte)profile.length;

```



```

74 //if (numHarmonics > 30) numHarmonics = 30;
75
76 /*
77 // インハーモニシティ（弦の硬さによるピッチの上擦り）。高音ほどズレが大きくなる。
78 float B = 0.0001f * pow(f0 / 261.6f, 1.5f);
79 B = constrain(B, 0.00002f, 0.003f);
80 */
81
82 // 現在の音階のLUT配列用インデックス (A0(MIDI:21)を0とする)
83 int lutIndex = constrain(round(midiNote) - 21, 0, 87);
84
85 // Pythonで解析したJSONデータが欠損していないかチェック
86 // ※汎用メソッド LoadLUT で読み込んだ3つのグローバル配列を使用します
87 //boolean hasRawData = (pianoDecayLUT != null && pianoDecayLUT[lutIndex] != null)
88 ;
89 // 【修正の核心2】コンパイル警告を完全に消し去るための厳格な3軸構造チェック
90 /*boolean hasRawData = false;
91 if (pianoDecayLUT != null && lutIndex < pianoDecayLUT.length && pianoDecayLUT[
92     lutIndex] != null) {
93     if (pianoWobbleRateLUT != null && lutIndex < pianoWobbleRateLUT.length &&
94         pianoWobbleRateLUT[lutIndex] != null) {
95         if (pianoWobbleDepthLUT != null && lutIndex < pianoWobbleDepthLUT.length &&
96             pianoWobbleDepthLUT[lutIndex] != null) {
97             hasRawData = true;
98         }
99     }
100 }*/
101
102 // --- データ欠損時のバックアップ用（数式モデル）の事前計算 ---
103 // Pythonのグラフ解析から導き出した「ピアノの物理特性の傾向」を数式化
104 float mathBaseDecay = 29.0f * pow(0.9757f, midiNote);
105 if (midiNote < 40) mathBaseDecay = min(mathBaseDecay, 20.0f); // 低音
106 // 物理的限界（頭打ち）
107 if (midiNote >= 89) mathBaseDecay *= map(midiNote, 89, 108, 1.0f, 3.0f); // 超高
108 // 音のダンパーレス構造による伸び
109
110
111 float mathBaseWobbleRate = map(abs(midiNote - 66.0f), 0, 45, 0.5f, 4.5f); // うな
112 // りの速さ(MIDI66を底としたU字型)
113 float mathBaseDepth = map(midiNote, 21, 108, 0.03f, 0.15f); // うな
114 // りの深さ(高音ほど深い)
115
116
117 // リバース（共鳴）が最大音量になるまでの時間。低音はゆっくり、高音は速く立ち上がる。
118 float buildUpTime = map(midiNote, 21, 108, 3.0f, 0.5f);
119
120 // =====

```

```

111 // 3. 弦のモデリング (デュアル・ストリング構造)
112 // =====
113 for (int i = 0; i < numHarmonics; i++) {
114     float amp = profile[i][1] * volFactor;
115     if (amp <= 0.0001f) continue; // 音量が極端に小さい倍音はCPU負荷軽減のためスキップ
116     // 省エネ設計
117
118     /*
119     int h = i + 1;
120     float inharmonicity = sqrt(1.0f + B * h * h);
121     float targetFreq = f0 * h * inharmonicity; // 実際の周波数
122     */
123
124     float targetFreq = profile[i][0]; // 実測された上擦り済みの正確な周波数
125     if (targetFreq > 22050.0f) continue; // 人間の可聴域を超えたらスキップ
126
127     float decayTime, wobbleRate, wobbleDepth;
128
129     // 実測データ(JSON)があれば使い、なければ数式で補完する「ハイブリッド方式」
130     //if (hasRawData && i < pianoDecayLUT[lutIndex].length && pianoDecayLUT[
131         lutIndex][i] > 0.1f) {
132     // 実測データ(JSON)の適用と数式補完
133     if (pianoDecayLUT != null && pianoWobbleRateLUT != null && pianoWobbleDepthLUT
134         != null &&
135         pianoDecayLUT[lutIndex] != null &&
136         i < pianoDecayLUT[lutIndex].length) {
137         decayTime = pianoDecayLUT[lutIndex][i];
138         wobbleRate = pianoWobbleRateLUT[lutIndex][i];
139         wobbleDepth = pianoWobbleDepthLUT[lutIndex][i];
140
141         // もしデータが0、または異常値だった場合は数式モデルで安全に保護する
142         if (decayTime <= 0.1f) {
143             float harmonicDecayCurve = (i <= 14) ? map(i, 0, 14, 1.0f, 0.3f) : map(i,
144                 14, 80, 0.3f, 1.5f);
145             decayTime = max(mathBaseDecay * harmonicDecayCurve, 0.1f);
146         }
147     } else {
148         // データがない場合は、インデックス14を底とする減衰のU字カーブを適用
149         float harmonicDecayCurve = (i <= 14) ? map(i, 0, 14, 1.0f, 0.3f) : map(i, 14,
150             80, 0.3f, 1.5f);
151         decayTime = max(mathBaseDecay * harmonicDecayCurve, 0.1f);
152         wobbleRate = mathBaseWobbleRate * (1.0f + i * 0.02f);
153         wobbleDepth = mathBaseDepth * pow(0.9f, i);
154     }
155
156     decayTime *= 0.7;
157 }

```

```

154 // -----
155 // レイヤーA：アタックの芯（Dry弦）
156 // -----
157 Oscil coreOsc = new Oscil(targetFreq, amp, Waves.SINE);
158 // アタックのインパクトを最大化するため、位相(波のスタート位置)は0付近に固定
159 coreOsc.setPhase(random(0.0f, 0.1f));
160
161 // 0.001秒で素早く立ち上がり、それぞれの寿命(decayTime)で消えていく
162 Damp coreDamp = new Damp(0.001f, decayTime);
163 coreOsc.patch(coreDamp).patch(mixer);
164 damps.add(coreDamp);
165
166 // -----
167 // レイヤーB：時間差で来る響き（共鳴弦）
168 // -----
169 float reverbAmp = amp * wobbleDepth; // 共鳴の音量は、芯の音の数%~十数%程度に
    抑える

170
171 // うなりのスピード(wobbleRate)が設定されており、かつ音量が十分ある場合のみ共鳴
    弦を生成
172 if (reverbAmp > 0.0001f && wobbleRate > 0.0f) {
173     // 周波数をわずかにズラす（Detune）ことで、芯の弦と干渉して自然な「うなり(
        Beat)」を生む
174     Oscil resOsc = new Oscil(targetFreq + wobbleRate, reverbAmp, Waves.SINE);
175     //resOsc.setPhase(random(1.0f)); // 芯の弦と複雑に干渉させるため、位相は完全
        にランダム

176
177     // アタックタイムを buildUpTime に設定し、数秒かけてゆっくり音量を上げる
178     Damp resDamp = new Damp(buildUpTime, decayTime);
179     resOsc.patch(resDamp).patch(mixer);
180     damps.add(resDamp);
181 }
182 }
183
184 // =====
185 // 4. ハンマーの打撃ノイズ（アタック）
186 // =====
187 float hammerVol = masterVol * 0.003f * normVel;
188 Noise hammerNoise = new Noise(hammerVol, Noise.Tint.WHITE);
189
190 // 高音の鍵盤ほど、硬い「カチッ」としたノイズになるようにフィルターを調整
191 float hammerCutoff = map(midiNote, 21, 108, 600f, 4000f);
192 MoogFilter hammerFilter = new MoogFilter(hammerCutoff, 0.1f);
193 hammerFilter.type = MoogFilter.Type.LP;
194
195 // 0.03秒で消滅する、極めてタイトなエンベロープ（ミュートの影響を受けない）

```

```

196     hammerEnv = new ADSR(1.0f, 0.001f, 0.02f, 0.0f, 0.01f);
197
198     hammerNoise.patch(hammerFilter).patch(hammerEnv);
199 }
200
201 // =====
202 // 発音と停止のコントロール
203 // =====
204 void noteOn(float dur) {
205     masterEnv.noteOn();
206     hammerEnv.noteOn();
207
208     // 準備した芯の弦と共鳴弦のDamp（エンベロープ）を一斉にスタート
209     for (Damp d : damps) {
210         d.activate();
211     }
212
213     masterEnv.patch(longOut);
214     hammerEnv.patch(longOut); // ハンマーノイズは独立して出力
215 }
216
217 void noteOff() {
218     masterEnv.noteOff(); // ここでダンパーペダルが降りてミュート開始
219     hammerEnv.noteOff();
220     masterEnv.unpatchAfterRelease(longOut);
221     hammerEnv.unpatchAfterRelease(longOut);
222 }
223 }

```

E.9 Drum.pde

コード 71 Drum.pde

```

1 // ドラム用LUTの宣言 [インデックス][0:周波数, 1:強度, 2:衰退時間(Decay)]
2 //float[][] kickLUT, snareLUT, hihatLUT;
3
4
5
6
7
8
9
10
11
12
13
14 // =====
15 // 1. キック (Kick) のクラス

```

```

16 // =====
17 class KickInstrument implements Instrument {
18     //Summer mixer;
19     ADSR masterEnv;
20
21     //Oscil[] tones;
22     ADSR[] harmonicEnvs;
23     //Line[] pitchSweeps;
24
25     KickInstrument(float masterVol, float velocity) {
26         Summer mixer = new Summer();
27
28         float[][]kickLUT = utils.GetLUT("Kick");
29
30         // フルートと同じく、リリース時のバッファを持たせたマスターエンベロープ
31         masterEnv = new ADSR(1.0f, 0.001f, 0.0f, 1.0f, 0.1f);
32         mixer.patch(masterEnv);
33
34         //int numComponents = kickLUT.length;
35         int numComponents = 50;
36         Oscil[] tones = new Oscil[numComponents];
37         harmonicEnvs = new ADSR[numComponents];
38         Line[] pitchSweeps = new Line[numComponents];
39
40         //float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
41             25.0f;
42         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
43             14.0f;
44
45
46         for (int i = 0; i < numComponents; i++) {
47             if (kickLUT[i] == null || kickLUT[i].length < 3) continue;
48
49             float freq = kickLUT[i][0] * random(0.95f, 1.05f);
50             float mag = kickLUT[i][1];
51             float decayTime = kickLUT[i][2];
52             float amp = (baseAmp * mag) / numComponents;
53
54             tones[i] = new Oscil(freq, amp, Waves.SINE);
55             //tones[i].setPhase(random(1.0f)); // 初期位相を散らしてアタックのピーク割れを
56                 防ぐ
57
58
59             // 個別の成分のエンベロープ
60             //harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime * 0.05f, 0.0f, 0.1f);
61             harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime*0.5f, 0.0f, 0.1f);
62
63             // ピッチスイープ

```

```

59     pitchSweeps[i] = new Line(0.01f, freq * 1.5f, freq);
60     pitchSweeps[i].patch(tones[i].frequency);
61
62     // 結線: Oscil -> 個別ADSR -> Mixer
63     tones[i].patch(harmonicEnvs[i]);
64     harmonicEnvs[i].patch(mixer);
65 }
66 }
67
68 void noteOn(float duration) {
69     masterEnv.noteOn();
70     for (int i = 0; i < harmonicEnvs.length; i++) {
71         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOn();
72     }
73     // グローバルな out に直接パッチ！
74     masterEnv.patch(shortOut);
75 }
76
77 void noteOff() {
78     masterEnv.noteOff();
79     for (int i = 0; i < harmonicEnvs.length; i++) {
80         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOff();
81     }
82     // グローバルな out からアンパッチ！
83     masterEnv.unpatchAfterRelease(shortOut);
84 }
85 }
86
87 // =====
88 // 2. スネア (Snare) のクラス
89 // =====
90 class SnareInstrumentOld implements Instrument {
91     //Summer mixer;
92     ADSR masterEnv;
93
94     // LUT (胴鳴り・倍音) 用
95     //Oscil[] tones;
96     ADSR[] harmonicEnvs;
97     //Line[] pitchSweeps;
98
99     // ノイズ (スナッピー) 用
100    //Noise snareNoise;
101    //HighPassSP snareFilter;
102    ADSR noiseEnv;
103
104    SnareInstrumentOld(float masterVol, float velocity) {
105        Summer mixer = new Summer();
106        masterEnv = new ADSR(1.0f, 0.001f, 0.0f, 1.0f, 0.1f);

```

```

107 mixer.patch(masterEnv);
108
109 float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) * 4.0
    f;

110
111 float[][]snareLUT = utils.GetLUT("Snare");
112
113 // --- 1. 胴鳴りパート (LUTによる加算合成) ---
114 int numComponents = snareLUT.length;
115 Oscil[] tones = new Oscil[numComponents];
116 harmonicEnvs = new ADSR[numComponents];
117 Line[] pitchSweeps = new Line[numComponents];
118
119 for (int i = 0; i < numComponents; i++) {
120     if (snareLUT[i] == null || snareLUT[i].length < 3) continue;
121
122     float freq = snareLUT[i][0];
123     float mag = snareLUT[i][1];
124     float decayTime = snareLUT[i][2];
125     float amp = (baseAmp * mag) / numComponents;
126
127     tones[i] = new Oscil(freq, amp, Waves.SINE);
128     tones[i].setPhase(random(1.0f));
129
130     harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime * 0.7, 0.0f, 0.1f);
131
132     // スネア特有：低音（胴鳴り）のみピッチスイープ
133     if (freq < 1000) {
134         pitchSweeps[i] = new Line(0.05f, freq * 1.5f, freq);
135         pitchSweeps[i].patch(tones[i].frequency);
136     }
137
138     tones[i].patch(harmonicEnvs[i]);
139     harmonicEnvs[i].patch(mixer);
140 }
141
142 // --- 2. スナッピーパート（ノイズによる減算合成） ---
143 // 以前の解析で導き出した数値をそのまま採用
144 float noiseVol = baseAmp * 0.022f;
145 Noise snareNoise = new Noise(noiseVol, Noise.Tint.WHITE);
146 HighPassSP snareFilter = new HighPassSP(512, out.sampleRate()); // 7750Hz以上を通
    す
147 noiseEnv = new ADSR(1.0f, 0.003f, 0.15f, 0.0f, 0.1f); // ディケイ0.25秒
148
149 // ノイズをフィルターし、エンベロープを通してミキサーへ合流
150 snareNoise.patch(snareFilter).patch(noiseEnv).patch(mixer);
151 }

```

```

152
153 void noteOn(float duration) {
154     masterEnv.noteOn();
155     for (int i = 0; i < harmonicEnvs.length; i++) {
156         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOn();
157     }
158     noiseEnv.noteOn();
159     masterEnv.patch(shortOut);
160 }
161
162 void noteOff() {
163     masterEnv.noteOff();
164     for (int i = 0; i < harmonicEnvs.length; i++) {
165         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOff();
166     }
167     noiseEnv.noteOff();
168     masterEnv.unpatchAfterRelease(shortOut);
169 }
170 }
171
172 // =====
173 // 3. クローズハイハット (Closed Hi-Hat) のクラス
174 // =====
175 class HiHatInstrumentOld implements Instrument {
176     //Summer mixer;
177     ADSR masterEnv;
178
179     //Oscil[] tones;
180     ADSR[] harmonicEnvs;
181
182     //Noise hatNoise;
183     //HighPassSP hatFilter;
184     ADSR noiseEnv;
185
186     HiHatInstrumentOld(float masterVol, float velocity) {
187         Summer mixer = new Summer();
188         // ハイハットは余韻が非常に短いため、マスターのリリースも極短に
189         masterEnv = new ADSR(1.0f, 0.001f, 0.0f, 1.0f, 0.05f);
190         mixer.patch(masterEnv);
191
192         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) * 5.0
            f;
193
194         float[][] hihatLUT = utils.GetLUT("HiHat");
195
196         // --- 1. 金属共鳴パート (LUTによる加算合成) ---
197         int numComponents = hihatLUT.length;

```



```

198 Oscil[] tones = new Oscil[numComponents];
199 harmonicEnvs = new ADSR[numComponents];
200
201 for (int i = 0; i < numComponents; i++) {
202     if (hihatLUT[i] == null || hihatLUT[i].length < 3) continue;
203
204     float freq = hihatLUT[i][0];
205     float mag = hihatLUT[i][1];
206     float decayTime = hihatLUT[i][2];
207     float amp = (baseAmp * mag) / numComponents;
208
209     tones[i] = new Oscil(freq, amp, Waves.SINE);
210     tones[i].setPhase(random(1.0f));
211
212     harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime, 0.0f, 0.05f);
213
214     tones[i].patch(harmonicEnvs[i]);
215     harmonicEnvs[i].patch(mixer);
216 }
217
218 // --- 2. 打撃摩擦パート（ノイズによる減算合成） ---
219 float noiseVol = baseAmp * 0.042f;
220 Noise hatNoise = new Noise(noiseVol, Noise.Tint.WHITE);
221 HighPassSP hatFilter = new HighPassSP(10250, out.sampleRate()); // 10000Hz以上の
    超高音のみ
222 noiseEnv = new ADSR(1.0f, 0.001f, 0.06f, 0.0f, 0.05f); // 0.06秒でスパッと消える
223
224 hatNoise.patch(hatFilter).patch(noiseEnv).patch(mixer);
225 }
226
227 void noteOn(float duration) {
228     masterEnv.noteOn();
229     for (int i = 0; i < harmonicEnvs.length; i++) {
230         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOn();
231     }
232     noiseEnv.noteOn();
233     masterEnv.patch(shortOut);
234 }
235
236 void noteOff() {
237     masterEnv.noteOff();
238     for (int i = 0; i < harmonicEnvs.length; i++) {
239         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOff();
240     }
241     noiseEnv.noteOff();
242     masterEnv.unpatchAfterRelease(shortOut);
243 }
244 }

```

```

245
246
247
248
249
250
251
252
253
254 // =====
255 // 2. 新・スネア (Snare) のクラス (シンセサイズ方式)
256 // =====
257 class SnareInstrument implements Instrument {
258     ADSR masterEnv;
259
260     // 1. 胴鳴り (Body) パート用
261     Oscil bodyOsc;
262     Line  bodyPitchEnv;
263     ADSR  bodyAmpEnv;
264
265     // 2. 倍音 (Overtone) パート用
266     Oscil overtoneOsc;
267     ADSR  overtoneAmpEnv;
268
269     // 3. スナッピー (Noise) パート用
270     Noise      snappyNoise;
271     //HighPassSP snappyFilter;
272     ADSR      snappyAmpEnv;
273
274     SnareInstrument(float masterVol, float velocity, float sinAmp) {
275         Summer mixer = new Summer();
276
277         // 全体の音量調整
278         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
279             0.15f;
280         float sinSet = constrain(sinAmp, 0, 127) / 127.0f;
281
282         // =====
283         // パラメータ設定 (ここをいじって音色をチューニングします)
284         // =====
285
286         // --- 1. 胴鳴り (ベースとなる太さ) ---
287         float bodyFreq      = 172.3f * random(0.99f, 1.01f); // ★データ最上部の「172.3
288             Hz (強さ: 0.63)」をそのまま指定！
289         float bodyPitchMod = 3.0f; // アタック時は約516Hz付近から172.3Hzへ急降下させる
290         float bodySweepTime= 0.025f; // 一瞬でピッチを落とす
291         float bodyDecay    = 0.07f;

```

```

290 float bodyVolRatio = 0.63f * sinSet;    // 172.3Hzは強いエネルギーを持っているの
    で、音量比率を高めに

291
292 // --- 2. 倍音（カンカンしたヘッドの芯） ---
293 float overtoneFreq = 323.0f * random(0.98f, 1.02f); // ★データ2番目の「323.0 Hz
    (強さ: 0.55)」をそのまま指定！
294 float overtoneDecay = 0.048f; // 胴鳴り（172.3Hz）よりも早く減衰させて歯切れを良
    くする
295 float overtoneVolRatio = 0.55f * sinSet;
296
297 // --- 3. スナッピー（ホワイトノイズ） ---
298 // データを見ると 1184.3 Hz 付近から徐々に強さが上がり、2.5kHz～8kHzがエネルギー
    の塊になっています。
299 //float noiseFilterCutoff = 2000.0f; // 2kHzあたりから上のノイズをまるごと通す
300 float noiseDecay = 0.17f; // スナッピーの余韻
301 float noiseVolRatio = 3.5f;
302 float noiseHPCutoff = 2500.0f * random(0.9f, 1.1f);
303 float noiseLPCutoff1 = 2800.0f * random(0.9f, 1.1f);
304 float noiseLPCutoff2 = 4000.0f * random(0.9f, 1.1f);
305 float noiseLPCutoff3 = 7000.0f * random(0.9f, 1.1f);
306
307 // =====
308 // 各モジュールの構築とパッチング
309 // =====
310
311 // --- 1. 胴鳴りパートの構築 ---
312 bodyOsc = new Oscil(bodyFreq, baseAmp * bodyVolRatio, Waves.SINE);
313 // 叩いた瞬間に音程を急激に下げる（アタック感の演出）
314 bodyPitchEnv = new Line(bodySweepTime, bodyFreq * bodyPitchMod, bodyFreq);
315 bodyPitchEnv.patch(bodyOsc.frequency);
316 // 音量エンベロープ (Attack, Decay, Sustain, Release)
317 bodyAmpEnv = new ADSR(1.0f, 0.001f, bodyDecay, 0.0f, 0.05f);
318
319 bodyOsc.patch(bodyAmpEnv).patch(mixer);
320
321
322
323 // --- 2. 倍音パートの構築 ---
324 overtoneOsc = new Oscil(overtoneFreq, baseAmp * overtoneVolRatio, Waves.SINE);
325 overtoneAmpEnv = new ADSR(1.0f, 0.002f, overtoneDecay, 0.0f, 0.05f);
326
327 overtoneOsc.patch(overtoneAmpEnv).patch(mixer);
328
329
330
331 // --- 3. スナッピー（Noise）パートの構築 ---
332 snappyNoise = new Noise(baseAmp * noiseVolRatio, Noise.Tint.WHITE);

```

```

333
334 // 4,500Hzに向かってなだらかに立ち上げるためのハイパス
335 HighPassSP snappyHP = new HighPassSP(noiseHPCutoff, out.sampleRate());
336
337 // 5,500Hzから20,000Hzに向けてなだらかに落とすためのローパス
338 LowPassSP snappyLP1 = new LowPassSP(noiseLPCutoff1, out.sampleRate());
339 LowPassSP snappyLP2 = new LowPassSP(noiseLPCutoff2, out.sampleRate());
340 LowPassSP snappyLP3 = new LowPassSP(noiseLPCutoff3, out.sampleRate());
341
342 snappyAmpEnv = new ADSR(1.0f, 0.003f, noiseDecay, 0.0f, 0.05f);
343
344 // ノイズを2つのフィルターに順番に通して（挟み撃ちにして）エンベロープへ送る
345 snappyNoise.patch(snappyHP).patch(snappyLP1).patch(snappyLP2).patch(snappyLP3).
    patch(snappyAmpEnv).patch(mixer);

346
347
348
349 // --- 4. マスターエンベロープ ---
350 masterEnv = new ADSR(1.0f, 0.0001f, 0.0f, 1.0f, 0.01f);
351 mixer.patch(masterEnv);
352 }
353
354 void noteOn(float duration) {
355     masterEnv.noteOn();
356     bodyAmpEnv.noteOn();
357     overtoneAmpEnv.noteOn();
358     snappyAmpEnv.noteOn();
359
360     masterEnv.patch(shortOut);
361 }
362
363 void noteOff() {
364     masterEnv.noteOff();
365     bodyAmpEnv.noteOff();
366     overtoneAmpEnv.noteOff();
367     snappyAmpEnv.noteOff();
368
369     masterEnv.unpatchAfterRelease(shortOut);
370 }
371 }
372
373
374
375 // =====
376 // 3. 新・ハイハット (Hi-Hat) のクラス (シンセサイズ方式)
377 // =====
378 class HiHatInstrument implements Instrument {

```

```

379  ADSR masterEnv;
380
381  // 1. 金属共鳴 (Metallic Core) パート用
382  // グラフから厳選した主要な4つのピークを個別に発振・制御します
383  //Oscil toneOsc1, toneOsc2, toneOsc3, toneOsc4;
384  //ADSR toneEnv1, toneEnv2, toneEnv3, toneEnv4;
385
386  Oscil[] toneOscList;
387  ADSR[] toneEnvList;
388
389  // 2. 打撃摩擦 (Noise) パート用
390  Noise      hatNoise;
391  //HighPassSP hatHP;
392  ADSR      noiseEnv;
393
394  HiHatInstrument(float masterVol, float velocity) {
395      Summer mixer = new Summer();
396
397      // 全体の音量調整 (歪まないようにスネア等のバランスに合わせて調整してください)
398      float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
          0.04f;
399
400      // =====
401      // パラメータ設定 (ここをいじって音色をチューニングします!)
402      // =====
403
404      // --- 1. 金属共鳴 (チキチキ・カンカンした芯の成分) ---
405      // FFT解析データの上位トップ4の強いピークをそのままスタンドアロンで指定
406      /*
407      float toneFreq1      = 7149.0f; // ★最強のセンターピーク (強さ: 1.00)
408      float toneVolRatio1 = 1.00f;
409      float toneDecay1     = 0.109f; // 金属的なアタックのみを出すため、非常に短く
410
411      float toneFreq2      = 7644.3f; // ★2番目に強い高域ピーク (強さ: 0.62)
412      float toneVolRatio2 = 0.62f;
413      float toneDecay2     = 0.040f;
414
415      float toneFreq3      = 5964.7f; // ★3番目に強い中高域ピーク (強さ: 0.61)
416      float toneVolRatio3 = 0.61f;
417      float toneDecay3     = 0.060f;
418
419      float toneFreq4      = 13006.1f; // ★超高域の金属的な鋭さを補うピーク (強さ:
          0.52)
420      float toneVolRatio4 = 0.52f;
421      float toneDecay4     = 0.015f; // 超高域は一瞬で消え去るように
422      */
423

```

```

424 float[][] toneList = {
425     {21.53f, 0.0619f, 0.131f},
426     {7149.0f, 1.00f, 0.109f},
427     {7644.3f, 0.62f, 0.040f},
428     {5964.7f, 0.61f, 0.060f},
429     {13006.1f, 0.52f, 0.015f},
430     {5555.57f, 0.4585f, 0.118f},
431     {14610.28f, 0.1893f, 0.091f},
432     {4931.1f, 0.1834f, 0.109f}
433 };
434
435 // --- 2. 打撃摩擦（シャリシャリ・シュワシュワした高域ノイズ） ---
436 float noiseVolRatio = 6.00f * random(0.99f, 1.01f); // ノイズのブレンド比率。
    シャリシャリ感を強めるなら上げる
437 float noiseHPCutoff1 = 7000.0f * random(0.9f, 1.1f); // ★ハイパスのカットオフ
    (Hz)。
438                                     // 8k~10kHz以上にすると、ゴソゴソした中音域が消
    え、綺麗な「チッ」になります
439 float noiseHPCutoff2 = 4000.0f * random(0.9f, 1.1f);
440 float noiseLPCutoff1 = 5000.0f * random(0.9f, 1.1f);
441 float noiseLPCutoff2 = 30000.0f * random(0.9f, 1.1f);
442 //float noiseLPCutoff3 = 3000.0f;
443 float noiseDecay = 0.080f; // クローズドハイハットの「余韻の長さ（キレ）」を
    左右する最重要パラメータ
444
445 // =====
446 // 各モジュールの構築とパッチング
447 // =====
448
449 // --- 1. 金属共鳴パートの構築 ---
450 // 各正弦波の位相（Phase）をランダムにすることで、発音ごとの微妙な「なじみ」を生
    み出します
451 /*
452 toneOsc1 = new Oscil(toneFreq1, baseAmp * toneVolRatio1, Waves.SINE);
453 toneOsc1.setPhase(random(1.0f));
454 toneEnv1 = new ADSR(1.0f, 0.001f, toneDecay1, 0.0f, 0.01f);
455 toneOsc1.patch(toneEnv1).patch(mixer);
456
457 toneOsc2 = new Oscil(toneFreq2, baseAmp * toneVolRatio2, Waves.SINE);
458 toneOsc2.setPhase(random(1.0f));
459 toneEnv2 = new ADSR(1.0f, 0.001f, toneDecay2, 0.0f, 0.01f);
460 toneOsc2.patch(toneEnv2).patch(mixer);
461
462 toneOsc3 = new Oscil(toneFreq3, baseAmp * toneVolRatio3, Waves.SINE);
463 toneOsc3.setPhase(random(1.0f));
464 toneEnv3 = new ADSR(1.0f, 0.001f, toneDecay3, 0.0f, 0.01f);
465 toneOsc3.patch(toneEnv3).patch(mixer);

```

```

466
467 toneOsc4 = new Oscil(toneFreq4, baseAmp * toneVolRatio4, Waves.SINE);
468 toneOsc4.setPhase(random(1.0f));
469 toneEnv4 = new ADSR(1.0f, 0.001f, toneDecay4, 0.0f, 0.01f);
470 toneOsc4.patch(toneEnv4).patch(mixer);
471 */
472
473 toneOscList = new Oscil[toneList.length];
474 toneEnvList = new ADSR[toneList.length];
475
476 for (int i = 0; i < toneList.length; i++){
477     toneOscList[i] = new Oscil(toneList[i][0] * random(0.98f, 1.02f), baseAmp *
478         toneList[i][1], Waves.SINE);
479     toneOscList[i].setPhase(random(1.0f));
480     toneEnvList[i] = new ADSR(1.0f, 0.001f, toneList[i][2]*0.5, 0.0f, 0.01f);
481     toneOscList[i].patch(toneEnvList[i]).patch(mixer);
482 }
483
484 // --- 2. ノイズパートの構築 ---
485 hatNoise = new Noise(baseAmp * noiseVolRatio, Noise.Tint.WHITE);
486 HighPassSP hatHP1 = new HighPassSP(noiseHPCutoff1, out.sampleRate());
487 HighPassSP hatHP2 = new HighPassSP(noiseHPCutoff2, out.sampleRate());
488 LowPassSP hatLP1 = new LowPassSP(noiseLPCutoff1, out.sampleRate());
489 LowPassSP hatLP2 = new LowPassSP(noiseLPCutoff2, out.sampleRate());
490 //LowPassSP hatLP3 = new LowPassSP(noiseLPCutoff3, out.sampleRate());
491 noiseEnv = new ADSR(1.0f, 0.001f, noiseDecay, 0.0f, 0.02f);
492
493 // ホワイトノイズ -> ハイパスフィルター -> エンベロープ -> ミキサー
494 hatNoise.patch(hatHP1).patch(hatHP2).patch(hatLP1).patch(hatLP2).patch(noiseEnv).
495     patch(mixer);
496
497
498 // --- 3. 全体の結合とマスターエンベロープ ---
499 masterEnv = new ADSR(1.0f, 0.0001f, 0.0f, 1.0f, 0.02f); // リリースも極短に
500 mixer.patch(masterEnv);
501 }
502
503 void noteOn(float duration) {
504     masterEnv.noteOn();
505     //toneEnv1.noteOn();
506     //toneEnv2.noteOn();
507     //toneEnv3.noteOn();
508     //toneEnv4.noteOn();
509
510     for (int i = 0; i<toneEnvList.length; i++ ) toneEnvList[i].noteOn();
511
512     noiseEnv.noteOn();
513 }

```

```

511     masterEnv.patch(shortOut);
512 }
513
514 void noteOff() {
515     masterEnv.noteOff();
516     //toneEnv1.noteOff();
517     //toneEnv2.noteOff();
518     //toneEnv3.noteOff();
519     //toneEnv4.noteOff();
520
521     for (int i = 0; i < toneEnvList.length; i++) {if (toneEnvList[i] != null)
        toneEnvList[i].noteOff();}

522
523
524     noiseEnv.noteOff();
525
526     masterEnv.unpatchAfterRelease(shortOut);
527 }
528 }
529
530
531
532
533
534 //class SnareRoll implements Thread {
535 //    SnareRoll () {
536
537 //    }
538 //    void run() {
539
540 //    }
541 //}
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556

```



```

557  /*
558  // =====
559  // シンバル (Cymbal) のクラス
560  // =====
561  class CymbalInstrument implements Instrument {
562      ADSR masterEnv;
563      ADSR[] harmonicEnvs;
564      ADSR noiseEnv; // シャーンという余韻ノイズ用
565
566      CymbalInstrument(float masterVol, float velocity) {
567          Summer mixer = new Summer();
568
569          // JSONデータの取得 (ファイル名やキーは環境に合わせてください)
570          float[][] cymbalLUT = utils.GetLUT("Cymbal");
571
572          // 全体のマスターエンベロープ (余韻を長く残すためリリースは3.5秒)
573          masterEnv = new ADSR(1.0f, 0.005f, 0.0f, 1.0f, 3.5f);
574          mixer.patch(masterEnv);
575
576          int numComponents = cymbalLUT.length;
577          Oscil[] tones = new Oscil[numComponents];
578          harmonicEnvs = new ADSR[numComponents];
579
580          // 倍音成分が多く音が割れやすいため、ベース音量を少し控えめに設定
581          float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) * 1.0
              f;
582
583          // -----
584          // 1. 金属のコア層 (加算合成 + FM/AM変調)
585          // -----
586          for (int i = 0; i < numComponents; i++) {
587              if (cymbalLUT[i] == null || cymbalLUT[i].length < 3) continue;
588
589              float freq = cymbalLUT[i][0];
590              float mag = cymbalLUT[i][1];
591              float decayTime = cymbalLUT[i][2];
592
593              // 高音域ほど音量が大きくなりすぎないように微調整
594              float amp = (baseAmp * mag) / numComponents/3;
595
596              tones[i] = new Oscil(freq, amp, Waves.SINE);
597
598              // アタック時の位相が揃って「バチッ」と鳴るのを防ぐ
599              tones[i].setPhase(random(1.0f));
600
601              // --- ① 非整数倍音の干渉をシミュレートするFM変調 (ピッチ揺れ) ---
602              // フルーツよりも少し速く、ランダムな周期で揺らすことで金属の歪みを生む

```

```

603     float fmSpeed = random(3.0f, 15.0f);
604     float vibDepthHz = freq * random(0.002f, 0.008f);
605     Oscil freqController = new Oscil(fmSpeed, vibDepthHz, Waves.SINE);
606     freqController.offset.setLastValue(freq);
607     freqController.patch(tones[i].frequency);
608
609     // --- ② 複雑なビートを生み出すAM変調（音量揺らぎ） ---
610     // 倍音ごとに異なる周期で音量をウネウネさせる
611     float amSpeed = random(1.0f, 8.0f);
612     Oscil ampLFO = new Oscil(amSpeed, amp * random(0.3f, 0.7f), Waves.SINE);
613     ampLFO.offset.setLastValue(amp);
614     ampLFO.patch(tones[i].amplitude);
615
616     // --- 個別エンベロープ（LUTのdecayTimeを適用） ---
617     // decayTimeをそのままReleaseにも適用し、自然に消えさせる
618     harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime/5, 0.0f, decayTime);
619
620     tones[i].patch(harmonicEnvs[i]);
621     harmonicEnvs[i].patch(mixer);
622 }
623
624 // -----
625 // 2. 超高域の余韻層（ノイズ + ハイパスフィルター）
626 // -----
627 // 「バーン」の後の「シャーン」という細かく散る粒子感をホワイトノイズで補完
628 Noise cymbalNoise = new Noise(baseAmp * 0.10f, Noise.Tint.PINK);
629
630 // 5000Hz以下の重たい成分を削り、チリチリとした超高域だけを残す
631 HighPassSP hpf = new HighPassSP(5000f, out.sampleRate());
632
633 // ノイズ層はゆっくり立ち上がり、コア層よりも長く残るように設定
634 noiseEnv = new ADSR(1.0f, 0.02f, 1.5f, 0.3f, 3.5f);
635
636 cymbalNoise.patch(hpf).patch(noiseEnv).patch(mixer);
637 }
638
639 void noteOn(float duration) {
640     masterEnv.noteOn();
641     noiseEnv.noteOn();
642     for (int i = 0; i < harmonicEnvs.length; i++) {
643         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOn();
644     }
645     // 出力先（shortOut等）は環境に合わせて変更してください
646     masterEnv.patch(out);
647 }
648
649 void noteOff() {
650     masterEnv.noteOff();

```

```

651     noiseEnv.noteOff();
652     for (int i = 0; i < harmonicEnvs.length; i++) {
653         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOff();
654     }
655     masterEnv.unpatchAfterRelease(out);
656 }
657 }*/

```

E.10 Horn.pde

コード 72 Horn.pde

```

1  class HornInstrument implements Instrument {
2      byte numHarmonics/* = 15*/;
3
4      ADSR masterEnv;
5      ADSR[] harmonicEnvs/* = new ADSR[numHarmonics]*/;
6
7      Line[] wobbleLines;    // 6/12 【追加】 倍音ウナリ (AM) 用のフェードライン
8      float[] wobbleFactors; // 6/12 【追加】 倍音ごとの固有ウナリ比率の保持用
9
10     // =====
11     // 7/5 【追加1】 ピッチエンベロープ用の変数
12     // =====
13     Line[] pitchEnvLines; // アタック時のピッチ急降下用
14     float[] targetFreqs;  // 各倍音の本来の周波数 (noteOnで計算に使うため保持)
15
16     Line[] ampLines/* = new Line[numHarmonics]*/;
17     float[] startAmps/* = new float[numHarmonics]*/;
18     float[] endAmps/* = new float[numHarmonics]*/;
19
20     ADSR breathEnv;
21
22
23     // =====
24     // 【新規追加】 持続する空気の渦と管の共鳴 (サブハーモニクス層)
25     // =====
26     //ADSR sustainEnv;
27
28     // 【追加】 持続ノイズの音量変化 (クレッシェンド/デクレッシェンド) 用
29     //Line sustainAmpLine;
30     //Line sustainWobbleLine; // 6/12 【追加】 持続ノイズのウナリ用のフェードライン
31     //float startSustainVol;
32     //float endSustainVol;
33
34
35

```

```

36  HornInstrument(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
37      Summer mixer = new Summer();
38
39      //float masterAttackTime = 0.2f - constrain(startVel, 0, 127) / 127.0f * 0.19f;
40      float masterAttackTime = 0.1f;
41      masterAttackTime = min(masterAttackTime, duration - 0.1f);
42      if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
43
44      // 【修正】 masterEnv の一番最後の引数（リリース時間）を 0.3f から 0.4f に延ばす。
45      // 内部の音が 0.3秒 かけて完全に消え去るのを待ってから、安全にアンパッチさせるた
        めの余裕（バッファ）です。
46      // 【修正】 masterEnv のリリース時間を、希望のフェードアウト時間（例：0.35秒）に設
        定
47      masterEnv = new ADSR(1.0f, masterAttackTime, 0.0f, 1.0f, 0.13f);
48
49      mixer.patch(masterEnv);
50
51
52      // 変更6/12
53      // 1. 倍音レシビを安全に取り出すための基準ベロシティ（0除算・ロード不全の防止）
54      float profileVel = (startVel > 0) ? startVel : endVel;
55      if (profileVel <= 0) profileVel = 64.0f; // どちらも0の場合のセーフティ
56
57      // 2. 開始・終了のゲインファクターをフルートの特性（0.7乗カーブ）に基づいて独立計
        算
58      float startGainFactor = pow(startVel / profileVel, 0.9f);
59      float endGainFactor    = pow(endVel / profileVel, 0.9f);
60
61      // 安全にロードされた基準プロファイルを取得
62      float[][] startProfile = utils.GetDynamicProfile(midiNote, profileVel, "Horn",
        34, 1.0f);

63
64      float volFactor = masterVol * 0.017f;
65
66      numHarmonics = (byte)startProfile.length;
67
68      harmonicEnvs = new ADSR[numHarmonics];
69      ampLines     = new Line[numHarmonics];
70      wobbleLines  = new Line[numHarmonics]; // 初期化を追加
71      wobbleFactors = new float[numHarmonics]; // 初期化を追加
72      startAmps    = new float[numHarmonics];
73      endAmps      = new float[numHarmonics];
74
75      // =====
76      // 7/5【追加2】配列の初期化
77      // =====

```

```

78 pitchEnvLines = new Line[numHarmonics];
79 targetFreqs   = new float[numHarmonics];
80
81
82
83 //float fundamentalFreq = 440.0 * pow(2.0, (actualMidiNote - 69) / 12.0);
84 byte baseWaveNum = 0;
85 for (byte i = 1; i < numHarmonics; i++) {
86     if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
87 }
88
89
90 // =====
91 // 【新規】倍音ごとの自然な振幅揺らぎ（変動率ベース）
92 // =====
93 // Pythonで解析した変動率(%)を参考に、基音の振幅に対して何倍の揺れ±()を許容するか
   // を設定。
94 // 実測データの変動率(%)から数式 [ Variation / 500 ] によって導出された、
95 // 倍音ごとの自然な振幅揺らぎ（AM変調）の深度設定。
96 //float[] wobbleDepths = {0.1250f, 0.1616f, 0.2078f, 0.2340f, 0.2114f, 0.2742f,
   // 0.3716f, 0.2946f, 0.7420f, 0.8796f,
97 // 0.7850f, 0.5070f, 0.6242f, 0.7654f, 0.6702f, 0.3898f,
   // 0.8850f, 0.5942f, 0.3152f, 0.5030f};
98
99 // =====
100 // 【新規】この音符全体の「呼吸のスピード」を決定
101 // forループの外で1つだけ定義し、全倍音で共有することで相殺を防ぐ
102 // =====
103 float globalBreathSpeed = random(2.5f, 3.5f); // 生楽器の自然なビブラート周期
104
105
106
107
108
109 // 1. 加算合成パート（7つのサイン波）
110 for (int i = 0; i < numHarmonics; i++) {
111     /*
112     int h = i + 1;
113     // あなたがPythonで導き出した厳密なインハーモニシティ公式を適用
114     float B = 0.0005f; // インハーモニシティ係数
115     float inharmonicity = sqrt(1.0f + B * h * h);
116     float targetFreq = f0 * h * inharmonicity;
117     */
118     float targetFreq = startProfile[i][0];
119
120     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
121

```

```

122 // 【追加】初期位相を 0.0 ~ 1.0 (0度~360度) の間で完全にランダムに散らす
123 // これにより「カチッ」とした機械的なアタック音が消え、音割れ（ピーク）も防げる
124 osc.setPhase(random(1.0f));
125
126
127 //startAmps[i] = startProfile[i][1] * volFactor;
128 //endAmps[i] = endProfile[i] * volFactor;
129 // 聴覚特性 (0.7乗) に完全同期させた、開始時から終了時への音量変化比率を算出
130 //float velRatio = pow(endVel / (startVel + 0.0001f), 0.7f);
131 // 【確定修正】すでにvolFactorが掛けられたstartAmpsに対して、正しいべき乗比率を
    乗算する
132 //endAmps[i] = startAmps[i] * pow(endVel / (startVel + 0.0001f), 0.7f);
133
134 // 6/12追加
135 // 独立した開始・終了音量を厳密に算出
136 float baseAmp = startProfile[i][1] * volFactor;
137 startAmps[i] = baseAmp * startGainFactor;
138 endAmps[i] = baseAmp * endGainFactor;
139
140
141
142 // 通常のビブラート（サイン波）
143
144 // =====
145 // ① 極小のピッチ揺れ（FM変調）の復活
146 // 以前の0.0132fを「0.003f」に激減させ、音痴にならない程度の「空気の揺らぎ」
    を作る
147 // =====
148 float vibDepthHz = targetFreq * 0.001f;
149 Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz, Waves.SINE);
150 freqController.offset.setLastValue(targetFreq);
151
152
153 //freqController.patch(osc.frequency);
154
155 // =====
156 // 7/5 【追加3】 本来の周波数を保存し、Pitch用のLineを接続
157 // =====
158 targetFreqs[i] = targetFreq; // noteOnで使うために記憶しておく
159
160 //pitchEnvLines[i] = new Line();
161 //pitchEnvLines[i].patch(osc.frequency); // オシレーターの周波数に加算接続
162 // 【変更点】 7/5
163 // 以前あった freqController.offset.setLastValue(targetFreq); を削除します。
164 // 代わりに、Pitch用のLineをLF0の「offset（基準周波数）」に直接パッチします！
165 pitchEnvLines[i] = new Line();
166 pitchEnvLines[i].patch(freqController.offset);
167

```

```

168 // LF0(基準値がLineで動く)を、メインオシレーターの周波数にパッチ
169 freqController.patch(osc.frequency);
170
171 // =====
172 // ② 呼吸に同期した音量揺らぎ (AM変調)
173 // =====
174 // この倍音の基準音量に対して、配列で設定した割合だけ揺らす
175 //float wobbleAmp = startAmps[i] * wobbleDepths[i];
176 //float wobbleAmp = startAmps[i] * random(0.1f, 0.3f);
177
178 // 1.5Hz ~ 3.5Hz の間で、倍音ごとにランダムなスピードで揺らす
179 // 【修正】ランダムではなく、統一された globalBreathSpeed を使う！
180 //Oscil ampLF0 = new Oscil(globalBreathSpeed, wobbleAmp, Waves.SINE);
181
182 // 位相（スタート位置）は少しだけバラけさせると、音が立体的になります
183 //ampLF0.setPhase(random(1.0f));
184
185 // ampLF0を osc.amplitude にパッチ(追加)する。
186 // Minimでは複数のUGenを同じ入力にパッチすると「加算(Sum)」されるため、
187 // ampLines の基準値に対して、ampLF0 が ±wobbleAmp の揺れを足し引きしてくれま
188 //す。
189 //ampLF0.patch(osc.amplitude);
190
191 // --- 音量揺らぎ (AM変調) のフェード構造化 ---
192 wobbleFactors[i] = random(0.12f, 0.17f);
193 //wobbleFactors[i] = wobbleDepths[i];
194 wobbleLines[i] = new Line();
195 Oscil ampLF0 = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE); // 初期振幅は0
196 ampLF0.setPhase(random(1.0f));
197
198 wobbleLines[i].patch(ampLF0.amplitude); // 新設LineをLF0の振幅へ接続
199 ampLF0.patch(osc.amplitude);
200
201 ampLines[i] = new Line();
202 ampLines[i].patch(osc.amplitude);
203
204 float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
205
206 // 【修正】内部のサイン波のリリースを 0.3f から 10.0f (10秒) にして、急激な角を
207 // 作らせない
208 harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
209 osc.patch(harmonicEnvs[i]);
210 harmonicEnvs[i].patch(mixer);
211 }
212
213 // =====

```

```

214 // 2. 息のノイズ (Chiff) パートの実測値アップデート
215 // =====
216 // 0.0~1.0 のベロシティ割合を計算 (ノイズの音量調整用)
217 float startNorm = constrain(startVel, 0, 127) / 127.0f;
218
219 // 中低音の破裂音 (ドスッという音) になるため、少しだけ音量を上げます (0.01f ->
    0.02f)
220 float attackNoiseVol = masterVol * 0.003f * pow(startNorm, 0.9f);
221 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
222
223 // 【修正】ハイパス(HP)からバンドパス(BP)に変更。
224 // 3500Hz付近の「シュッ」という音だけを残し、耳障りな高音を削る。
225 // 【修正】実測データに基づき、3500Hzから 581.4Hz に変更！
226 // 473Hzの山も一緒に拾うため、レゾナンスを 0.3 から 0.15 に下げて帯域を広げる
227 MoogFilter breathFilter = new MoogFilter(381.4f, 0.15f);
228 breathFilter.type = MoogFilter.Type.BP;
229
230 breathEnv = new ADSR(1.0f, 0.01f, 0.08f, 0.0f, 0.0f);
231
232 breathNoise.patch(breathFilter);
233 breathFilter.patch(breathEnv);
234 breathEnv.patch(mixer);
235
236
237 /*
238 // =====
239 // 3. 【新規追加】持続する空気の渦と管の共鳴 (サブハーモニクス層)
240 // =====
241 float endNorm = constrain(endVel, 0, 127) / 127.0f; // 【追加】終了時のベロシティ
    割合
242
243 // 【修正】開始時と終了時のノイズ音量を計算して保持する
244 startSustainVol = masterVol * 0.005f * pow(startNorm, 0.9f);
245 endSustainVol = masterVol * 0.005f * pow(endNorm, 0.9f);
246
247 // ピンクノイズで低音の「フオォ」という空気感を出す
248 // 【修正】Noiseの初期音量は0.0fにし、音量制御はLineに任せる
249 Noise sustainNoise = new Noise(0.0f, Noise.Tint.PINK);
250
251 // 【追加】持続ノイズのボリュームつまみにLineを接続
252 sustainAmpLine = new Line();
253 sustainAmpLine.patch(sustainNoise.amplitude);
254
255 // =====
256 // 【新規】持続ノイズ自体にも「息の乱れ」を適用する
257 // サイン波の第1倍音と同等のスピードと深さでノイズを揺らす
258 // =====

```



```

259 //float noiseWobbleAmp = startSustainVol * 0.20f; // 20%揺らす
260 //Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), noiseWobbleAmp, Waves.SINE)
    ;
261 //sustainWobble.setPhase(random(1.0f));
262 //sustainWobble.patch(sustainNoise.amplitude); // Lineの音量にLFOの揺れを加算
263 // =====
264
265 // 6/12変更
266 // --- 持続ノイズの息の乱れも完全にフェードさせる ---
267 sustainWobbleLine = new Line();
268 Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), 0.0f, Waves.SINE); // 初期振
    幅は0
269 sustainWobble.setPhase(random(1.0f));
270
271 sustainWobbleLine.patch(sustainWobble.amplitude); // 新設LineをノイズLFOの振幅へ
    接続
272 sustainWobble.patch(sustainNoise.amplitude); // LFO出力をノイズ振幅へ（加
    算）

273
274 // ローパスフィルターで高音を削りつつ、2500Hz付近に共鳴(レゾナンス0.5)の山を作る
275 MoogFilter sustainFilter = new MoogFilter(2500f, 0.5f);
276 sustainFilter.type = MoogFilter.Type.LP;
277
278 // 音が鳴っている間ずっと持続するエンベロープ（サイン波のアタックに合わせる）
279 // 【修正】 持続ノイズのリリースも 0.3f から 10.0f（10秒）にする
280 sustainEnv = new ADSR(1.0f, 0.058f, 0.0f, 1.0f, 10.0f);
281
282 sustainNoise.patch(sustainFilter);
283 sustainFilter.patch(sustainEnv);
284 sustainEnv.patch(mixer);
285 */
286 }
287
288
289 /*
290 void noteOn(float dur) {
291     masterEnv.noteOn();
292
293     // =====
294     // 【修正】 Lineの寿命に、リリース時間（0.4秒）分の猶予を足す！
295     // これにより、減衰中もLineが生き残り、オシレーターの音量が突然0.0fにリセットされ
        るのを防ぐ
296     // =====
297     // 【微調整】 masterEnvのリリース時間(0.35f)より少し長ければOK
298     float safeDur = dur + 0.5f;
299
300     for(int i=0; i<numHarmonics; i++) {

```

```

301     harmonicEnvs[i].noteOn();
302     ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
303 }
304
305 breathEnv.noteOn();
306 sustainEnv.noteOn(); // 【追加】持続ノイズをオン
307 sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol); // 【追加】発音
    と同時に、持続ノイズの音量変化をスタートさせる

308
309     masterEnv.patch(out);
310 }*/
311 // 6/12変更
312 void noteOn(float dur) {
313     masterEnv.noteOn();
314     float safeDur = dur + 0.5f;
315
316     // 7/5 【追加4】トランペット/ホルン特有の「アタック時のピッチ降下」パラメータ
317     float sweepTime = 0.12f; // 変化にかかる時間 (0.04秒)
318     float pitchMod = 0.05f; // 高くなる割合 (0.03 = 本来の周波数の3%増しからスター
        ト)

319
320 for(int i = 0; i < numHarmonics; i++) {
321     harmonicEnvs[i].noteOn();
322
323     // 1. 各倍音のベース音量をフェード
324     ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
325
326     // 2. 通常ビブラート時のみ、ウナリ (AM) の深さも比例フェードさせる
327     if (wobbleLines[i] != null) {
328         float factor = wobbleFactors[i];
329         wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
330     }
331
332     // 3. 7/5 【新規追加】アタック時のピッチエンベロープ (周波数の急降下)
333     // 「本来の周波数の3%分上乗せした値」から「0 (上乗せなし)」へと一直線に下げる
334     //if (pitchEnvLines[i] != null) {
335     //    float baseFreq = targetFreqs[i];
336     //    pitchEnvLines[i].activate(sweepTime, baseFreq * pitchMod, 0.0f);
337     //}
338     // 3. 7/5 【新規追加】アタック時のピッチエンベロープ (周波数の急降下)
339     if (pitchEnvLines[i] != null) {
340         float baseFreq = targetFreqs[i];
341
342         // スタート地点の周波数: 本来の周波数に pitchMod(%) を上乗せした値
343         // 例: pitchMod が 0.03 なら 3%増し、2.0 なら 200%増し (3倍)
344         float startFreq = baseFreq * (1.0f + pitchMod);

```

```

345
346     // startFreq から baseFreq(本来の周波数) へ、一直線に下げる
347     pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq);
348 }
349 }
350
351 breathEnv.noteOn();
352 //sustainEnv.noteOn();
353
354 // 3. 持続ノイズの全体音量と、そのウナリの深さを同時に追従フェード
355 //sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol);
356 //sustainWobbleLine.activate(safeDur, startSustainVol * 0.20f, endSustainVol *
    0.20f);

357
358 masterEnv.patch(longOut);
359 }
360
361
362
363 void noteOff() {
364     masterEnv.noteOff();
365     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
366
367     // 【削除】 breathEnv は既にな音なので noteOff を呼ばない (空撃ちグリッチ防止)
368     // breathEnv.noteOff();
369
370     //sustainEnv.noteOff(); // 【追加】 持続ノイズをオフ
371     masterEnv.unpatchAfterRelease(longOut);
372 }
373 }

```

E.11 Trumpet.pde

コード 73 Trumpet.pde

```

1 class TrumpetInstrument implements Instrument {
2     byte numHarmonics/* = 15*/;
3
4     ADSR masterEnv;
5     ADSR[] harmonicEnvs/* = new ADSR[numHarmonics]*/;
6
7     Line[] wobbleLines;    // 6/12 【追加】 倍音ウナリ (AM) 用のフェードライン
8     float[] wobbleFactors; // 6/12 【追加】 倍音ごとの固有ウナリ比率の保持用
9
10    // =====
11    // 7/5 【追加1】 ピッチエンベロープ用の変数
12    // =====

```

```

13 Line[] pitchEnvLines; // アタック時のピッチ急降下用
14 float[] targetFreqs; // 各倍音の本来の周波数 (noteOnで計算に使うため保持)
15
16 Line[] ampLines/* = new Line[numHarmonics]*/*;
17 float[] startAmps/* = new float[numHarmonics]*/*;
18 float[] endAmps/* = new float[numHarmonics]*/*;
19
20 ADSR breathEnv;
21
22
23 // =====
24 // 【新規追加】 持続する空気の渦と管の共鳴 (サブハーモニクス層)
25 // =====
26 //ADSR sustainEnv;
27
28 // 【追加】 持続ノイズの音量変化 (クレッシェンド/デクレッシェンド) 用
29 //Line sustainAmpLine;
30 //Line sustainWobbleLine; // 6/12 【追加】 持続ノイズのウナリ用のフェードライン
31 //float startSustainVol;
32 //float endSustainVol;
33
34
35
36 TrumpetInstrument(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
37     Summer mixer = new Summer();
38
39     float masterAttackTime = 0.1f - constrain(startVel, 0, 127) / 127.0f * 0.8f;
40     masterAttackTime = min(masterAttackTime, duration - 0.05f);
41     if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
42
43     // 【修正】 masterEnv の一番最後の引数 (リリース時間) を 0.3f から 0.4f に延ばす。
44     // 内部の音が 0.3秒 かけて完全に消え去るのを待ってから、安全にアンパッチさせるた
        めの余裕 (バッファ) です。
45     // 【修正】 masterEnv のリリース時間を、希望のフェードアウト時間 (例: 0.35秒) に設
        定
46     masterEnv = new ADSR(1.0f, masterAttackTime, 0.0f, 1.0f, 0.17f);
47
48     mixer.patch(masterEnv);
49
50
51     // 変更6/12
52     // 1. 倍音レシビを安全に取り出すための基準ベロシティ (0除算・ロード不全の防止)
53     float profileVel = (startVel > 0) ? startVel : endVel;
54     if (profileVel <= 0) profileVel = 64.0f; // どちらも0の場合のセーフティ
55
56     // 2. 開始・終了のゲインファクターをフルートの特性 (0.7乗カーブ) に基づいて独立計
        算

```

```

57     float startGainFactor = pow(startVel / profileVel, 0.9f);
58     float endGainFactor   = pow(endVel / profileVel, 0.9f);
59
60     // 安全にロードされた基準プロファイルを取得
61     float[][] startProfile = utils.GetDynamicProfile(midiNote, profileVel, "Trumpet",
62                                                       52, 1.0f);
63
64
65     float volFactor = masterVol * 0.018f;
66
67     numHarmonics = (byte)startProfile.length;
68
69     harmonicEnvs = new ADSR[numHarmonics];
70     ampLines     = new Line[numHarmonics];
71     wobbleLines  = new Line[numHarmonics]; // 初期化を追加
72     wobbleFactors = new float[numHarmonics]; // 初期化を追加
73     startAmps    = new float[numHarmonics];
74     endAmps      = new float[numHarmonics];
75
76     // =====
77     // 7/5【追加2】配列の初期化
78     // =====
79     pitchEnvLines = new Line[numHarmonics];
80     targetFreqs   = new float[numHarmonics];
81
82
83     //float fundamentalFreq = 440.0 * pow(2.0, (actualMidiNote - 69) / 12.0);
84     byte baseWaveNum = 0;
85     for (byte i = 1; i < numHarmonics; i++) {
86         if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
87     }
88
89
90     // =====
91     // 【新規】倍音ごとの自然な振幅揺らぎ（変動率ベース）
92     // =====
93     // Pythonで解析した変動率(%)を参考に、基音の振幅に対して何倍の揺れ±()を許容するか
94     // を設定。
95     // 実測データの変動率(%)から数式 [ Variation / 500 ] によって導出された、
96     // 倍音ごとの自然な振幅揺らぎ（AM変調）の深度設定。
97     //float[] wobbleDepths = {0.1250f, 0.1616f, 0.2078f, 0.2340f, 0.2114f, 0.2742f,
98                             0.3716f, 0.2946f, 0.7420f, 0.8796f,
99                             0.7850f, 0.5070f, 0.6242f, 0.7654f, 0.6702f, 0.3898f,
100                             0.8850f, 0.5942f, 0.3152f, 0.5030f};

```

```

99 // =====
100 // 【新規】 この音符全体の「呼吸のスピード」を決定
101 // forループの外で1つだけ定義し、全倍音で共有することで相殺を防ぐ
102 // =====
103 float globalBreathSpeed = random(4.0f, 5.0f); // 生楽器の自然なビブラート周期
104
105
106
107
108
109 // 1. 加算合成パート（7つのサイン波）
110 for (int i = 0; i < numHarmonics; i++) {
111     /*
112     int h = i + 1;
113     // あなたがPythonで導き出した厳密なインハーモニシティ公式を適用
114     float B = 0.0005f; // インハーモニシティ係数
115     float inharmonicity = sqrt(1.0f + B * h * h);
116     float targetFreq = f0 * h * inharmonicity;
117     */
118     float targetFreq = startProfile[i][0];
119
120     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
121
122     // 【追加】 初期位相を 0.0 ~ 1.0 (0度~360度) の間で完全にランダムに散らす
123     // これにより「カチッ」とした機械的なアタック音が消え、音割れ（ピーク）も防げる
124     osc.setPhase(random(1.0f));
125
126
127
128
129     //startAmps[i] = startProfile[i][1] * volFactor;
130     //endAmps[i] = endProfile[i] * volFactor;
131     // 聴覚特性（0.7乗）に完全同期させた、開始時から終了時への音量変化比率を算出
132     //float velRatio = pow(endVel / (startVel + 0.0001f), 0.7f);
133     // 【確定修正】すでにvolFactorが掛けられたstartAmpsに対して、正しいべき乗比率を
134     // 乗算する
135     //endAmps[i] = startAmps[i] * pow(endVel / (startVel + 0.0001f), 0.7f);
136
137     // 6/12追加
138     // 独立した開始・終了音量を厳密に算出
139     float baseAmp = startProfile[i][1] * volFactor;
140     startAmps[i] = baseAmp * startGainFactor;
141     endAmps[i] = baseAmp * endGainFactor;
142
143
144     // 通常のビブラート（サイン波）
145

```

```

146 // =====
147 // ① 極小のピッチ揺れ (FM変調) の復活
148 // 以前の0.0132fを「0.003f」に激減させ、音痴にならない程度の「空気の揺らぎ」
// を作る
149 // =====
150 float vibDepthHz = targetFreq * 0.0007f;
151 Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz, Waves.SINE);

152
153
154
155 //freqController.offset.setLastValue(targetFreq);
156
157 // =====
158 // 7/5 【追加3】 本来の周波数を保存し、Pitch用のLineを接続
159 // =====
160 targetFreqs[i] = targetFreq; // noteOnで使うために記憶しておく
161
162 //pitchEnvLines[i] = new Line();
163 //pitchEnvLines[i].patch(osc.frequency); // オシレーターの周波数に加算接続
164 // 【変更点】 7/5
165 // 以前あった freqController.offset.setLastValue(targetFreq); を削除します。
166 // 代わりに、Pitch用のLineをLF0の「offset (基準周波数)」に直接パッチします！
167 pitchEnvLines[i] = new Line();
168 pitchEnvLines[i].patch(freqController.offset);
169
170 // LF0(基準値がLineで動く)を、メインオシレーターの周波数にパッチ
171 freqController.patch(osc.frequency);
172
173
174
175 // =====
176 // ② 呼吸に同期した音量揺らぎ (AM変調)
177 // =====
178 // この倍音の基準音量に対して、配列で設定した割合だけ揺らす
179 //float wobbleAmp = startAmps[i] * wobbleDepths[i];
180 //float wobbleAmp = startAmps[i] * random(0.1f, 0.3f);
181
182 // 1.5Hz ~ 3.5Hz の間で、倍音ごとにランダムなスピードで揺らす
183 // 【修正】 ランダムではなく、統一された globalBreathSpeed を使う！
184 //Oscil ampLF0 = new Oscil(globalBreathSpeed, wobbleAmp, Waves.SINE);
185
186 // 位相 (スタート位置) は少しだけバラけさせると、音が立体的になります
187 //ampLF0.setPhase(random(1.0f));
188
189 // ampLF0を osc.amplitude にパッチ(追加)する。
190 // Minimでは複数のUGenを同じ入力にパッチすると「加算(Sum)」されるため、

```

```

191 // ampLines の基準値に対して、ampLF0 が ±wobbleAmp の揺れを足し引きしてくれま
192 す。
193 //ampLF0.patch(osc.amplitude);
194
195 // --- 音量揺らぎ (AM変調) のフェード構造化 ---
196 wobbleFactors[i] = random(0.10f, 0.20f);
197 //wobbleFactors[i] = wobbleDepths[i];
198 wobbleLines[i] = new Line();
199 Oscil ampLF0 = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE); // 初期振幅は0
200 ampLF0.setPhase(random(1.0f));
201
202 wobbleLines[i].patch(ampLF0.amplitude); // 新設LineをLF0の振幅へ接続
203 ampLF0.patch(osc.amplitude);
204
205 ampLines[i] = new Line();
206 ampLines[i].patch(osc.amplitude);
207
208 float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
209
210 // 【修正】内部のサイン波のリリースを 0.3f から 10.0f (10秒) にして、急激な角を
211 作らない
212 harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
213 osc.patch(harmonicEnvs[i]);
214 harmonicEnvs[i].patch(mixer);
215 }
216
217 // =====
218 // 2. 息のノイズ (Chiff) パートの実測値アップデート
219 // =====
220 // 0.0~1.0 のベロシティ割合を計算 (ノイズの音量調整用)
221 float startNorm = constrain(startVel, 0, 127) / 127.0f;
222
223 // 中低音の破裂音 (ドスッという音) になるため、少しだけ音量を上げます (0.01f ->
224 0.02f)
225 float attackNoiseVol = masterVol * 0.01f * pow(startNorm, 0.9f);
226 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
227
228 // 【修正】ハイパス(HP)からバンドパス(BP)に変更。
229 // 3500Hz付近の「シュッ」という音だけを残し、耳障りな高音を削る。
230 // 【修正】実測データに基づき、3500Hzから 581.4Hz に変更！
231 // 473Hzの山も一緒に拾うため、レゾナンスを 0.3 から 0.15 に下げて帯域を広げる
232 MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);
233 breathFilter.type = MoogFilter.Type.BP;
234
235 breathEnv = new ADSR(1.0f, 0.01f, 0.08f, 0.0f, 0.0f);

```



```

236 breathNoise.patch(breathFilter);
237 breathFilter.patch(breathEnv);
238 breathEnv.patch(mixer);
239
240
241 /*
242 // =====
243 // 3. 【新規追加】持続する空気の渦と管の共鳴（サブハーモニクス層）
244 // =====
245 float endNorm = constrain(endVel, 0, 127) / 127.0f; // 【追加】終了時のベロシティ
    割合

246
247 // 【修正】開始時と終了時のノイズ音量を計算して保持する
248 startSustainVol = masterVol * 0.005f * pow(startNorm, 0.9f);
249 endSustainVol   = masterVol * 0.005f * pow(endNorm, 0.9f);
250
251 // ピンクノイズで低音の「フオォ」という空気感を出す
252 // 【修正】Noiseの初期音量は0.0fにし、音量制御はLineに任せる
253 Noise sustainNoise = new Noise(0.0f, Noise.Tint.PINK);
254
255 // 【追加】持続ノイズのボリュームつまみにLineを接続
256 sustainAmpLine = new Line();
257 sustainAmpLine.patch(sustainNoise.amplitude);
258
259 // =====
260 // 【新規】持続ノイズ自体にも「息の乱れ」を適用する
261 // サイン波の第1倍音と同等のスピードと深さでノイズを揺らす
262 // =====
263 //float noiseWobbleAmp = startSustainVol * 0.20f; // 20%揺らす
264 //Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), noiseWobbleAmp, Waves.SINE)
    ;
265 //sustainWobble.setPhase(random(1.0f));
266 //sustainWobble.patch(sustainNoise.amplitude); // Lineの音量にLF0の揺れを加算
267 // =====
268
269 // 6/12変更
270 // --- 持続ノイズの息の乱れも完全にフェードさせる ---
271 sustainWobbleLine = new Line();
272 Oscil sustainWobble = new Oscil(random(1.5f, 3.0f), 0.0f, Waves.SINE); // 初期振
    幅は0
273 sustainWobble.setPhase(random(1.0f));
274
275 sustainWobbleLine.patch(sustainWobble.amplitude); // 新設LineをノイズLF0の振幅へ
    接続
276 sustainWobble.patch(sustainNoise.amplitude); // LF0出力をノイズ振幅へ（加
    算）

```

```

277 // ローパスフィルターで高音を削りつつ、2500Hz付近に共鳴(レゾナンス0.5)の山を作る
278 MoogFilter sustainFilter = new MoogFilter(2500f, 0.5f);
279 sustainFilter.type = MoogFilter.Type.LP;
280
281 // 音が鳴っている間ずっと持続するエンベロープ (サイン波のアタックに合わせる)
282 // 【修正】 持続ノイズのリリースも 0.3f から 10.0f (10秒) にする
283 sustainEnv = new ADSR(1.0f, 0.058f, 0.0f, 1.0f, 10.0f);
284
285 sustainNoise.patch(sustainFilter);
286 sustainFilter.patch(sustainEnv);
287 sustainEnv.patch(mixer);
288 */
289 }
290
291 /*
292 void noteOn(float dur) {
293     masterEnv.noteOn();
294
295     // =====
296     // 【修正】 Lineの寿命に、リリース時間 (0.4秒) 分の猶予を足す！
297     // これにより、減衰中もLineが生き残り、オシレーターの音量が突然0.0fにリセットされ
298     // るのを防ぐ
299     // =====
300     // 【微調整】 masterEnvのリリース時間(0.35f)より少し長ければOK
301     float safeDur = dur + 0.5f;
302
303     for(int i=0; i<numHarmonics; i++) {
304         harmonicEnvs[i].noteOn();
305         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
306     }
307
308     breathEnv.noteOn();
309     sustainEnv.noteOn(); // 【追加】 持続ノイズをオン
310     sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol); // 【追加】 発音
311     と同時に、持続ノイズの音量変化をスタートさせる
312
313     masterEnv.patch(out);
314 }*/
315 // 6/12変更
316 void noteOn(float dur) {
317     masterEnv.noteOn();
318     float safeDur = dur + 0.5f;
319
320     // 7/5 【追加4】 トランペット/ホルン特有の「アタック時のピッチ降下」パラメータ
321     float sweepTime = 0.07f; // 変化にかかる時間 (0.04秒)

```

```

322     float pitchMod = 0.03f; // 高くなる割合 (0.03 = 本来の周波数の3%増しからスタート)
323
324     for(int i = 0; i < numHarmonics; i++) {
325         harmonicEnvs[i].noteOn();
326
327         // 1. 各倍音のベース音量をフェード
328         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
329
330         // 2. 通常ビブラート時のみ、ウナリ (AM) の深さも比例フェードさせる
331         if (wobbleLines[i] != null) {
332             float factor = wobbleFactors[i];
333             wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
334         }
335
336         // 3. 7/5 【新規追加】 アタック時のピッチエンベロープ (周波数の急降下)
337         // 「本来の周波数の3%分上乗せした値」から「0 (上乗せなし)」へと一直線に下げる
338         //if (pitchEnvLines[i] != null) {
339         //    float baseFreq = targetFreqs[i];
340         //    pitchEnvLines[i].activate(sweepTime, baseFreq * pitchMod, 0.0f);
341         //}
342         // 3. 7/5 【新規追加】 アタック時のピッチエンベロープ (周波数の急降下)
343         if (pitchEnvLines[i] != null) {
344             float baseFreq = targetFreqs[i];
345
346             // スタート地点の周波数: 本来の周波数に pitchMod(%) を上乗せした値
347             // 例: pitchMod が 0.03 なら 3%増し、2.0 なら 200%増し (3倍)
348             float startFreq = baseFreq * (1.0f + pitchMod);
349
350             // startFreq から baseFreq(本来の周波数) へ、一直線に下げる
351             pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq);
352         }
353     }
354
355     breathEnv.noteOn();
356     //sustainEnv.noteOn();
357
358     // 3. 持続ノイズの全体音量と、そのウナリの深さを同時に追従フェード
359     //sustainAmpLine.activate(safeDur, startSustainVol, endSustainVol);
360     //sustainWobbleLine.activate(safeDur, startSustainVol * 0.20f, endSustainVol *
361         0.20f);
362
363     masterEnv.patch(longOut);
364 }
365

```

```

366
367 void noteOff() {
368     masterEnv.noteOff();
369     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
370
371     // 【削除】 breathEnv は既にな音なので noteOff を呼ばない (空撃ちグリッチ防止)
372     // breathEnv.noteOff();
373
374     //sustainEnv.noteOff(); // 【追加】 持続ノイズをオフ
375     masterEnv.unpatchAfterRelease(longOut);
376 }
377 }

```

F 楽器デバイスの同期プログラム (douki_saigo) のプログラムソース

本節では、楽器デバイスの同期方式を担う Arduino プログラム (douki_saigo) の全文を示す。なお、埋め込みの楽譜データ (Note[] 配列) は、各小節ごとに同一形式のデータが多数繰り返されるのみであるため、冒頭 2 小節分のみを掲載し、以降は省略している。

F.1 drum_0706.ino

コード 74 drum_0706.ino (楽譜データ省略)

```

1  #include <WiFiS3.h>
2  #include <WiFiUdp.h>
3  #include <Arduino.h>
4
5  const char SSID[] = "hackathon001-WPA2";
6  const char PASS[] = "hackathon001";
7
8  IPAddress localIP(192, 168, 11, 11); // ドラム (マスター) のIPアドレス
9  const uint16_t LOCAL_PORT = 5005;
10
11  WiFiUDP Udp;
12  uint8_t packetBuffer[32];
13
14  const int CDS_PIN = A0;
15  const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
16  const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
17  const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
18  const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
19  const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
20  const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
21  const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
22  const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
23  const unsigned long DEBOUNCE_MS = 20;

```

```

24 const int BPM_LIST[] = { 60, 90, 120, 150, 180 };
25 // 段階1~5(BPM_LISTと対応)の実測の1周あたり時間[ms]。理論値(30000/BPM)では観覧車の
26 // 実際の回転速度と合わないことが判明したため、実測値でBPMを判定する。
27 const float ROTATION_PERIOD_LIST[] = { 330, 240, 190, 150, 100 };
28 const int MIN_STABLE_COUNT = 3;
29
30 struct NoteData {
31     uint16_t timing;
32     uint16_t duration;
33     uint8_t pitch;
34     uint8_t startVelocity;
35     uint8_t endVelocity;
36     uint8_t type;
37 };
38
39 // 楽譜データ
40 const NoteData Note[] = {
41
42     // 【1小節目】
43     {0, 1, 60, 80, 96, 0}, // [0] C4
44     {12, 1, 64, 96, 80, 0}, // [1] E4
45     {24, 1, 63, 80, 127, 0}, // [2] D#4
46     {24, 1, 60, 80, 96, 0}, // [3] C4
47     {36, 1, 64, 96, 80, 0}, // [4] E4
48     {48, 1, 60, 80, 96, 0}, // [5] C4
49     {60, 1, 64, 96, 80, 0}, // [6] E4
50     {72, 1, 63, 80, 127, 0}, // [7] D#4
51     {72, 1, 60, 80, 96, 0}, // [8] C4
52     {84, 1, 64, 96, 80, 0}, // [9] E4
53
54     // 【2小節目】
55     {96, 1, 60, 80, 96, 0}, // [10] C4
56     {108, 1, 64, 96, 80, 0}, // [11] E4
57     {120, 1, 63, 80, 127, 0}, // [12] D#4
58     {120, 1, 60, 80, 96, 0}, // [13] C4
59     {132, 1, 64, 96, 80, 0}, // [14] E4
60     {144, 1, 60, 80, 96, 0}, // [15] C4
61     {156, 1, 64, 96, 80, 0}, // [16] E4
62     {168, 1, 63, 80, 127, 0}, // [17] D#4
63     {168, 1, 60, 80, 96, 0}, // [18] C4
64     {180, 1, 64, 96, 80, 0}, // [19] E4
65     {186, 1, 64, 48, 80, 0}, // [20] E4
66
67     // ... (以下、同様の形式の楽譜データが続くため省略) ...
68 };
69
70 const uint16_t note_count = sizeof(Note) / sizeof(Note[0]);
71

```

```

72 // =====
73 // Tick・演奏管理変数
74 // =====
75 uint32_t globalTick    = 0; // 全体基準Tick
76 uint32_t curTick      = 0; // ドラム楽譜用Tick
77 uint32_t lastTick     = 0; // updateTick用タイムスタンプμ[s]
78 uint32_t tickIval     = 0; // 1Tickあたりμのs
79 uint16_t scoreIdx     = 0; // 次に発音する予定のデータのインデックス
80
81 uint8_t curBpm = 0;
82 uint8_t curVol = 50;
83
84 // =====
85 // 楽譜シーケンス定義
86 //   startSequence[曲ID][イベント][0=演奏開始elapsedPlayTick, 1=楽器ID]
87 //   楽器ID: 0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート   終端={-1,-1}
88 //
89 //   ★ 0通信送信タイミング = 演奏開始elapsedPlayTick - O_ADVANCE_TICKS
90 //   ★ state2移行後の最初のイベントがelapsedPlayTick=0 (ドラム以外) の場合、
91 //     そのイベントの0通信は elapsedPlayTick=0 では間に合わないため、
92 //     実際の演奏開始は O_ADVANCE_TICKS 分だけ後ろにずれる (後述)
93 // =====
94 const int32_t O_ADVANCE_TICKS = 96; // 0通信を演奏開始の何Tick前に送るか
95 const int     MAX_SONGS      = 5;
96 const int     MAX_EVENTS     = 10;
97 const int     drumInstId     = 0;
98
99 // ★実際の曲構成に合わせて書き換える★
100 // elapsedPlayTick=0 からドラム以外の楽器が始まる場合、
101 // その楽器のtriggerTickを 0 ではなく O_ADVANCE_TICKS(=96) 以上に設定するか、
102 // もしくはドラム自身のtriggerTickを0にして、他楽器を96以降にする
103 int startSequence[MAX_SONGS][MAX_EVENTS][2] = {
104     { // 曲ID: 0 かえるのうた
105         {0,    drumInstId}, // elapsed=0でドラム自身が演奏開始
106         {192,  1},          // elapsed=192でピアノ開始 (0通信はelapsed=96)
107         {384,  2},          // elapsed=384でストリングス (0通信はelapsed=288)
108         {576,  3},          // elapsed=576でフルート (0通信はelapsed=480)
109         {-1, -1}
110     },
111     { // 曲ID: 1 酒場でブギウギ
112         // {0,    drumInstId}, // elapsed=0でドラム自身が演奏開始
113         {0,    1},          // elapsed=192でピアノ開始 (0通信はelapsed=96)
114         // {0,    2},          // elapsed=384でストリングス (0通信はelapsed=288)
115         // {0,    3},          // elapsed=576でフルート (0通信はelapsed=480)
116         {-1, -1}
117     },
118     { // 曲ID: 2 王宮のメヌエット
119         // {0,    drumInstId}, // elapsed=0でドラム自身が演奏開始

```

```

120 // {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
121 {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
122 // {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
123 {-1, -1}
124 },
125 { // 曲ID: 3 トルコ行進曲
126 // {0, drumInstId}, // elapsed=0でドラム自身が演奏開始
127 {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
128 // {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
129 // {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
130 {-1, -1}
131 },
132 { // 曲ID: 4 序曲
133 {0, drumInstId}, // elapsed=0でドラム自身が演奏開始
134 {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
135 {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
136 {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
137 {-1, -1}
138 }
139 };
140
141 int currentSongId = 0;
142 // int currentEventIndex = 0;
143 int32_t elapsedPlayTick = 0; // state3移行後 (演奏開始後) の経過Tick
144
145 bool isDrumPlaying = false;
146
147 const uint8_t MAX_INSTRUMENTS = 4;
148 // 0通信は T受信に頼らず非同期で送るため、送信済みフラグを管理する
149 // bool oSentFlags[MAX_EVENTS] = {false}; // 各イベントの0通信送信済み
150 // 変更後：送信回数を記録する配列に変更
151 uint8_t oSentCount[MAX_EVENTS] = {0};
152 bool isInstConnected[MAX_INSTRUMENTS] = {false}; // 同期に参加してるかどうかv
153 bool isInstJoin[MAX_INSTRUMENTS] = {false}; // 演奏同期に参加する予告が来た
154 // か
155 int32_t lastReceivedDiff[MAX_INSTRUMENTS] = {0}; // 他の楽器からT通信で送られるdiffを
156 // 格納
157
158 // 値は1~5などを想定。0はデータ未取得扱いなどにする。
159 uint8_t receivedBpmLevels[MAX_INSTRUMENTS] = {0}; // 参加している楽器のBPM段階を集約
160 // する配列 (ドラム自身も含む)
161 uint8_t currentBpmLevel = 1; // 現在確定しているBPM段階 (前回値)
162 // =====
163 // state 1→2 sync閾値 (diff 6tick以内)

```

```

163 // =====
164 //int32_t SYNC_READY_THRESHOLD = 6; // tick
165 // const int32_t SYNC_STABLE_THRESHOLD = 10; // state1での「まあ良い」閾値（ログ用）

166
167 // =====
168 // SyncManager（レーザー検知・BPM安定判定）
169 // =====
170 class SyncManager {
171 public:
172     SyncManager()
173         : lastDetectTime(0), correctedBPM(0),
174           stableCount(0), prevCorrectedBPM(0), laserOn(false),
175           baseline(0), baselineReady(false),
176           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
177           pulsePeak(0), peakIndex(0), peakCount(0),
178           calibrated(false), startupCalibrated(false), waitingForRelease(false),
179           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
180         clearPeakHistory();
181     }
182
183     void reset() {
184         lastDetectTime = 0;
185         correctedBPM = 0;
186         stableCount = 0;
187         prevCorrectedBPM = 0;
188         laserOn = false;
189         baseline = 0;
190         baselineReady = false;
191         onThreshold = CDS_ON_THRESHOLD;
192         offThreshold = CDS_OFF_THRESHOLD;
193         pulsePeak = 0;
194         peakIndex = 0;
195         peakCount = 0;
196         calibrated = false;
197         startupCalibrated = false;
198         waitingForRelease = false;
199         calibrationStartMs = 0;
200         startupMin = 1023;
201         releaseThreshold = 0;
202         clearPeakHistory();
203     }
204
205     bool update(int cdsValue, unsigned long nowMs) {
206         if (calibrationStartMs == 0) calibrationStartMs = nowMs;
207         if (!detectLight(cdsValue, nowMs)) return false;
208         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
209         if (lastDetectTime != 0) {

```



```

210     unsigned long interval = nowMs - lastDetectTime;
211     int newBPM = intervalToBpm((float)interval);
212     if (newBPM == prevCorrectedBPM) stableCount++;
213     else stableCount = 1;
214     prevCorrectedBPM = newBPM;
215     correctedBPM = newBPM;
216 }
217 lastDetectTime = nowMs;
218 return true;
219 }
220
221 int getCorrectedBPM() const { return correctedBPM; }
222 int getStableCount() const { return stableCount; }
223
224 private:
225     unsigned long lastDetectTime;
226     int correctedBPM;
227     int stableCount;
228     int prevCorrectedBPM;
229     bool laserOn;
230     int baseline;
231     bool baselineReady;
232     int onThreshold;
233     int offThreshold;
234     int pulsePeak;
235     int peakHistory[CDS_PEAK_HISTORY];
236     uint8_t peakIndex;
237     uint8_t peakCount;
238     bool calibrated;
239     bool startupCalibrated;
240     bool waitingForRelease;
241     unsigned long calibrationStartMs;
242     int startupMin;
243     int releaseThreshold;
244
245     bool detectLight(int v, unsigned long nowMs) {
246         if (!baselineReady) {
247             baseline = v;
248             startupMin = v;
249             baselineReady = true;
250             updateFallbackThresholds();
251         }
252
253         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
254         if (!startupCalibrated) {
255             if (v < startupMin) startupMin = v;
256             baseline = startupMin;
257             updateFallbackThresholds();

```

```

258     if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
259     startupCalibrated = true;
260     waitingForRelease = (v >= onThreshold);
261     releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
        : offThreshold;
262     return false;
263 }
264
265 if (waitingForRelease) {
266     if (v <= releaseThreshold) {
267         baseline = v;
268         updateFallbackThresholds();
269         waitingForRelease = false;
270     }
271     return false;
272 }
273
274 if (!laserOn) {
275     if (v < baseline) baseline = v;
276     else baseline = (baseline * 31 + v) / 32;
277     if (!calibrated) updateFallbackThresholds();
278 }
279
280 if (!laserOn && v >= onThreshold) {
281     laserOn = true;
282     pulsePeak = v;
283     return true;
284 }
285
286 if (laserOn) {
287     if (v > pulsePeak) pulsePeak = v;
288     if (v <= offThreshold) {
289         laserOn = false;
290         registerPeak(pulsePeak);
291     }
292 }
293 return false;
294 }
295
296 void registerPeak(int peak) {
297     if (peak - baseline < CDS_MIN_DELTA) return;
298
299     peakHistory[peakIndex] = peak;
300     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
301     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
302     if (peakCount < 3) return;
303
304     long sum = 0;

```

```

305     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
306     int peakAvg = sum / peakCount;
307     int amplitude = peakAvg - baseline;
308     if (amplitude < CDS_MIN_DELTA) return;
309
310     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
311     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);

312
313     if (!calibrated) {
314         onThreshold = targetOn;
315         offThreshold = targetOff;
316         calibrated = true;
317     } else {
318         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
319         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
320     }
321     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
322 }
323
324 void updateFallbackThresholds() {
325     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
326     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
        );
327 }
328
329 int limitStep(int current, int target, int limit) {
330     if (target > current + limit) return current + limit;
331     if (target < current - limit) return current - limit;
332     return target;
333 }
334
335 void clearPeakHistory() {
336     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
337 }
338
339 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
340 int intervalToBpm(float intervalMs) {
341     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
342     for (int i = 1; i < 5; i++) {
343         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
344         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
345     }
346     return best;
347 }
348 };
349

```

```

350 SyncManager syncManager;
351
352 // =====
353 // state:
354 // 0 = 待機 (BPM安定待ち)
355 // 1 = BPM安定、楽器と同期中 (diff収束待ち)
356 // 2 = 同期完了・演奏開始前 (0通信準備 / state3への96tick待機)
357 // 3 = 演奏中 (elapsedPlayTick が進む、ドラムも演奏する場合は isDrumPlaying=true)
358 // 4 = システム停止
359 // =====
360 int state = 4;
361
362
363
364 // =====
365 // 楽譜遷移 (ジャンプ) 定義
366 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
    scoreIdx]
367 // 終端は {-1, -1, -1}
368 // =====
369 const int MAX_JUMPS = 20;
370 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
371     { // 曲ID: 0 かえるのうた
372         {768, 0, 0},
373         {-1, -1, -1}
374     },
375     {
376         {768, 0, 0},
377         {-1, -1, -1}
378     },
379     {
380         {768, 0, 0},
381         {-1, -1, -1}
382     },
383     {
384         {768, 0, 0},
385         {-1, -1, -1}
386     },
387     {
388         {3624, 2088, 637},
389         {-1, -1, -1}
390     }
391 };
392
393 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
394
395 // =====
396 // 曲ごとの初期設定データ

```

```

397 // =====
398 struct SongStartData {
399     uint32_t startTick;
400     uint16_t startNoteIdx;
401 };
402
403 // Processingの設定に合わせた初期値の定義
404 // インデックスがそのまま currentSongId に対応します
405 const SongStartData songStartParams[MAX_SONGS] = {
406     {0, 0}, // 曲ID: 0 (かえるのうた)
407     {0, 0}, // 曲ID: 1 (酒場でブギウギ風)
408     {0, 0}, // 曲ID: 2 (王宮のメヌエット)
409     {0, 0}, // 曲ID: 3 (トルコ行進曲)
410     {768, 92} // 曲ID: 4 (序曲)
411 };
412
413
414
415
416 // =====
417 // 関数宣言
418 // =====
419 void updateTick();
420 void setBpm(uint8_t bpm);
421 void sendVolumeValue(uint8_t vol);
422 void sendPacket();
423 void processOCommands();
424 void sendOPacket(uint8_t instId, uint8_t songId, uint32_t startGlobalTick);
425 bool checkSyncReady();
426 void sendNPacket(uint8_t bpm);
427 void sendRPacket(uint8_t instId, int32_t diff);
428
429 // =====
430 // IPアドレス (楽器IDからIPを引く)
431 // =====
432 IPAddress instIP(uint8_t instId) {
433     // ID=1→192.168.11.12, ID=2→.13, ID=3→.14
434     return IPAddress(192, 168, 11, 11 + instId);
435 }
436
437 // =====
438 // セットアップ
439 // =====
440 void setup() {
441     Serial.begin(115200);
442     WiFi.config(localIP);
443     WiFi.begin(SSID, PASS);
444     Serial.print("Connecting_WiFi...");

```

```

445 while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
446 Serial.println("\nWiFi_Connected!_IP:_ " + WiFi.localIP().toString());
447 Udp.begin(LOCAL_PORT);
448 Serial.println("state=0_BPM安定待ち");
449 }
450
451 // =====
452 // メインループ
453 // =====
454 void loop() {
455     unsigned long now = millis();
456     int cdsValue = analogRead(CDS_PIN);
457
458     // ---- UDP受信 ----
459     int pktSize = Udp.parsePacket();
460     while (pktSize > 0) {
461         if (pktSize >= 2) {
462             Udp.read(packetBuffer, sizeof(packetBuffer));
463             char header = (char)packetBuffer[0];
464
465             switch (header) {
466
467                 // -----
468                 // 'T': 楽器からの同期パケット
469                 // [0]='T' [1..4]=slaveTick(uint32 big-endian) [5]=instId
470                 //
471                 // → R通信 (diff返信) は T受信時に即送信
472                 // → O通信は R通信とは独立。processOCommands() が管理。
473                 // -----
474                 case 'T':
475                     if (pktSize >= 6) {
476                         uint32_t slaveTick =
477                             ((uint32_t)packetBuffer[1] << 24) |
478                             ((uint32_t)packetBuffer[2] << 16) |
479                             ((uint32_t)packetBuffer[3] << 8) |
480                             (uint32_t)packetBuffer[4];
481                         uint8_t id = packetBuffer[5];
482
483                         if (id < MAX_INSTRUMENTS && id != drumInstId) {
484                             isInstConnected[id] = true;
485                             if (isInstJoin[id] != true) isInstJoin[id] = true;
486                             int32_t diff = (int32_t)(globalTick - slaveTick);
487                             lastReceivedDiff[id] = diff;
488
489                             //作った
490                             sendRPacket(id, diff);
491
492                             Serial.print(F("[T]_id=")); Serial.print(id);

```

```

493         Serial.print(F("_diff=")); Serial.println(diff);
494     }
495 }
496 break;
497
498 case 'V': {
499     uint8_t step = packetBuffer[1];
500     uint8_t vol = (step >= 1 && step <= 5) ? (step * 10) : 30;
501     sendVolumeValue(vol);
502     break;
503 }
504
505 // -----
506 // 'E': 指揮機からのブロードキャスト終了指示
507 // 演奏を停止し、state4にして初期化する
508 // -----
509 case 'E':
510     isDrumPlaying = false;
511     curTick = 0;
512     scoreIdx = 0;
513     globalTick = 0;
514     elapsedPlayTick = 0;
515     state = 4;
516     memset(oSentCount, 0, sizeof(oSentCount));
517     memset(isInstConnected, 0, sizeof(isInstConnected));
518     memset(isInstJoin, 0, sizeof(isInstJoin));
519     // memset(pendingStartCommand, 0, sizeof(pendingStartCommand));
520     curBpm = 0;
521     tickIval = 0;
522
523     // 追加: BPM関連の初期化
524     memset(receivedBpmLevels, 0, sizeof(receivedBpmLevels));
525     currentBpmLevel = 0;
526
527     Serial.println(F("[E]演奏終了。state=4_に初期化"));
528     break;
529
530 // -----
531 // 'S': 指揮機からのブロードキャスト開始指示
532 // state0に戻して初期化する
533 // -----
534 case 'S':
535 {
536     static unsigned long lastSReceivedTime = 0;
537     unsigned long nowMs = millis();
538
539     // 1秒以内の連続したS通信 (UDP連送) は無視する
540     if (nowMs - lastSReceivedTime < 1000) {

```

```

541     Serial.println(F("[S]_連続受信のためスキップ"));
542     break;
543 }
544 lastSReceivedTime = nowMs;
545
546 // 楽譜IDのローテーション
547 currentSongId = (currentSongId + 1) % MAX_SONGS;
548
549 // 初期化处理
550 isDrumPlaying      = false;
551 curTick            = 0;
552 scoreIdx           = 0;
553 globalTick         = 0;
554 elapsedPlayTick    = 0;
555 currentJumpIdx     = 0; // ジャンプインデックスも初期化
556 state              = 0;
557 memset(oSentCount, 0, sizeof(oSentCount));
558 memset(isInstConnected, 0, sizeof(isInstConnected));
559 memset(isInstJoin, 0, sizeof(isInstJoin));
560 // memset(pendingStartCommand, 0, sizeof(pendingStartCommand));
561 curBpm = 0;
562 tickIval = 0;
563
564 // 追加: BPM関連の初期化
565 memset(receivedBpmLevels, 0, sizeof(receivedBpmLevels));
566 currentBpmLevel = 0;
567
568 syncManager.reset();
569
570 Serial.print(F("[S]_演奏開始。state=0_に初期化。曲変更。SongID:"));
571 Serial.println(currentSongId);
572
573 // ☒以下の数行を全て削除する！☒
574 // // delayがあるため、加速が終わった時点で、バッファに溜まっている2回め・3回
575 //   めのパケットを捨てる（ボタンが動くようにする）
576 // Serial.println("連射された残りのパケットをクリアします。");
577 // while(Udp.parsePacket() > 0) {
578 //   while(Udp.available()) {
579 //     Udp.read();
580 //   }
581 // }
582 break;
583 }
584
585 // -----
586 // 'J': 楽器からの演奏参加予告
587 //   同期に参加する楽器を読み取る
588 // -----

```



```

588     case 'J':
589     if(pktSize >= 2){
590         uint8_t instId = packetBuffer[1];
591         if (instId < MAX_INSTRUMENTS) {
592             isInstJoin[instId] = true;
593             Serial.print(F("[J]_参加予告受信:_instId=")); Serial.println(instId);
594         }
595     }
596     break;
597
598     // 追加：子機からのBPM提案を受信
599     case 'M':
600         if (pktSize >= 3) {
601             uint8_t bpmVal = packetBuffer[1];
602             uint8_t instId = packetBuffer[2];
603             if (instId < MAX_INSTRUMENTS) {
604                 receivedBpmLevels[instId] = bpmVal;
605                 Serial.print(F("[M]_BPM提案受信_instId=")); Serial.print(instId);
606                 Serial.print(F("_BPM=")); Serial.println(bpmVal);
607             }
608         }
609
610         break;
611
612
613         default: break;
614     }
615     // } else if (pktSize > 0) {
616     //     while (Udp.available()) Udp.read();
617     // } else {
618     //     Udp.read(); // 空読みして破棄
619     // }
620     pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
621 }
622
623 if (state == 4) return;
624
625 // ---- レーザー検知 ----
626 bool laser = syncManager.update(cdsValue, now);
627
628 // ---- ステートマシン ----
629 switch (state) {
630
631     // --- state0: BPM安定待ち ---
632     case 0:
633         if (laser) {
634             int s = syncManager.getStableCount();
635             uint8_t curLaserBpm = syncManager.getCorrectedBPM();

```

```

636 Serial.print(F("[S0]_BPM=")); Serial.print(curLaserBpm);
637 Serial.print(F("_stable=")); Serial.println(s);
638
639 // ドラム自身のレーザーBPMを配列に上書き
640 receivedBpmLevels[drumInstId] = curLaserBpm;
641
642 if (s >= MIN_STABLE_COUNT) {
643     state = 1;
644     lastTick = micros();
645
646     // 初回確定時に自身へ適用 & 即座にN通信で子機にばら撒く！
647     currentBpmLevel = curLaserBpm;
648     setBpm(currentBpmLevel);
649     sendNPacket(currentBpmLevel);
650
651     // setBpm(syncManager.getCorrectedBPM());
652     Serial.println(F("[S0→1]_BPM安定"));
653 }
654 }
655 break;
656
657 // --- state1: 楽器と同期待ち ---
658 case 1:
659     updateTick();
660     if (laser) {
661         // setBpm(syncManager.getCorrectedBPM());
662         receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM(); // 配列上書き
663         // のみ
664         if (checkSyncReady()) {
665             // state2へ移行。elapsedPlayTick は state3移行後に開始するが、
666             // 0通信の送信タイミング算出のためにカウントはここから始める
667             state = 2;
668             elapsedPlayTick = -O_ADVANCE_TICKS;
669             memset(oSentCount, 0, sizeof(oSentCount));
670             Serial.println(F("[S1→2]_同期完了"));
671         } else {
672             Serial.println(F("[S1]_同期待ち..."));
673         }
674     }
675     break;
676
677 // --- state2: 同期完了・演奏開始待ち ---
678 // elapsedPlayTick を進めながら 0通信タイミングを監視する。
679 // state3への移行は processOCommands() 内でドラム自身の開始タイミングが来たとき。
680 case 2:
681     updateTick();
682     if (laser) receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM();
683     processOCommands();

```

```

683     break;
684
685     // --- state3: 演奏中 ---
686     case 3:
687         updateTick();
688         if (laser) receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM();
689         processOCommands();
690         break;
691
692     // --- state4: システム停止中 ---
693     // case 4: break;
694 }
695 }
696
697 // =====
698 // 同期完了チェック
699 //   接続済み楽器が1台以上 かつ 全員のdiffが SYNC_READY_THRESHOLD以内
700 // =====
701 // bool checkSyncReady() {
702 //     bool any = false;
703 //     for (uint8_t i = 1; i < MAX_INSTRUMENTS; i++) {
704 //         if (isInstConnected[i]) {
705 //             any = true;
706 //             if (abs(lastReceivedDiff[i]) > SYNC_READY_THRESHOLD) return false;
707 //         }
708 //     }
709 //     return any;
710 // }
711 bool checkSyncReady() {
712     bool expectedToJoin = false;
713
714     for (uint8_t i = 1; i < MAX_INSTRUMENTS; i++) {
715         if (isInstJoin[i]) {
716             expectedToJoin = true; // 参加予定の楽器が少なくとも1台いる
717
718             float SYNC_READY_THRESHOLD = 50 / tickIval; // 50ms超えてたら同期ができていない
719                 とみなすから50
720
721             // 参加予定だが未接続、または同期のズレが大きい場合は「まだ準備未完了」
722             if (!isInstConnected[i] || abs(lastReceivedDiff[i]) > SYNC_READY_THRESHOLD) {
723                 return false;
724             }
725         }
726     }
727
728     // 参加予定の全楽器が条件をクリアしていれば true。

```

```

728 // ※もし「他楽器が0台（ドラムソロ）でも開始させたい」場合は、return true; にしてく
729 //      ださい。
730 return expectedToJoin;
731 }
732 // =====
733 // 0通信管理
734 //   各イベントについて「演奏開始elapsedTick - 0_ADVANCE_TICKS」になったら送信。
735 //   ドラム自身のイベント（ID=0）は演奏開始処理のみ行う（0通信不要）。
736 //
737 //   ・ elapsedPlayTick が負（=state2の最初96tick）の間も処理するため、
738 //     startSequenceの最初のイベントが elapsed=0 であっても対応できる。
739 //   ・ 受信側(notDrum)が演奏開始予定globalTick をもとにcurTickを計算するため、
740 //     「ドラムの globalTick + 0_ADVANCE_TICKS」を startGlobalTick として送る。
741 // =====
742 void processOCommands() {
743     if (currentSongId < 0 || currentSongId >= MAX_SONGS) return;
744
745     for (int i = 0; i < MAX_EVENTS; i++) {
746         int32_t evTick = startSequence[currentSongId][i][0];
747         int      instId = startSequence[currentSongId][i][1];
748
749         if (evTick == -1) break;
750         if (oSentCount[i] >= 3) continue; // 3回送信し終わっていたらスキップ
751
752         int32_t ticksUntilStart = evTick - elapsedPlayTick;
753
754         if (instId == drumInstId) {
755             if (elapsedPlayTick >= evTick) {
756                 if (!isDrumPlaying) {
757                     isDrumPlaying = true;
758                     curTick = songStartParams[currentSongId].startTick;
759                     scoreIdx = songStartParams[currentSongId].startNoteIdx;
760                     state = 3;
761                     Serial.println(F("[DRUM]_演奏開始"));
762                 }
763                 oSentCount[i] = 3; // ドラム自身は通信不要なので完了扱い
764             }
765         } else {
766             if (isInstJoin[instId] && isInstConnected[instId]) {
767                 uint32_t startGlobalTick = (uint32_t)((int32_t)globalTick + ticksUntilStart);
768
769                 // 0_ADVANCE_TICKS(96)を切っていて、まだ0回目なら 1回目の送信
770                 if (ticksUntilStart <= 0_ADVANCE_TICKS && oSentCount[i] == 0) {
771                     sendOPacket_Single(instId, currentSongId, startGlobalTick);
772                     oSentCount[i] = 1;
773                 }
774             }
775         }
776     }
777 }

```

```

774 // 2/3 (64Tick前) を切っていて、まだ1回目なら 2回目の送信
775 else if (ticksUntilStart <= 0_ADVANCE_TICKS * 2 / 3 && oSentCount[i] == 1) {
776     sendOPacket_Single(instId, currentSongId, startGlobalTick);
777     oSentCount[i] = 2;
778 }
779 // 1/3 (32Tick前) を切っていて、まだ2回目なら 3回目の送信
780 else if (ticksUntilStart <= 0_ADVANCE_TICKS / 3 && oSentCount[i] == 2) {
781     sendOPacket_Single(instId, currentSongId, startGlobalTick);
782     oSentCount[i] = 3;
783 }
784 }
785 }
786 }
787 // if (currentSongId < 0 || currentSongId >= MAX_SONGS) return;
788
789 // for (int i = 0; i < MAX_EVENTS; i++) {
790 //     if (startSequence[currentSongId][i][0] == -1) break;
791 //     if (oSentFlags[i]) continue;
792
793 //     int32_t evTick    = startSequence[currentSongId][i][0];
794 //     int      instId    = startSequence[currentSongId][i][1];
795 //     int32_t sendTick = evTick - 0_ADVANCE_TICKS; // このelapsedで送信する
796 //     //int32_t sendTick = evTick; // このelapsedで送信する
797
798 //     if (elapsedPlayTick < sendTick) continue; // まだ早い
799
800 //     // 送信タイミング到達
801 //     oSentFlags[i] = true;
802
803 //     if (instId == drumInstId) {
804 //         // ドラム自身: elapsedPlayTick が evTick に達したら演奏開始
805 //         // (0通信不要。sendTick < 0 の場合はstate2移行直後にここへ来る)
806 //         // ここでは「evTickに達したか」だけチェック
807 //         if (elapsedPlayTick >= evTick) {
808 //             if (!isDrumPlaying) {
809 //                 isDrumPlaying = true;
810 //                 // 曲IDに応じた初期値セット
811 //                 curTick    = songStartParams[currentSongId].startTick;
812 //                 scoreIdx    = songStartParams[currentSongId].startNoteIdx;
813 //                 state       = 3;
814 //                 Serial.println(F("[DRUM] 演奏開始"));
815 //             }
816 //         } else {
817 //             // まだ evTick に達していない (sendTick    elapsed < evTick) → 待機
818 //             oSentFlags[i] = false; // まだ送信扱いにしない
819 //         }
820 //     } else {
821 //         // 変更点：他楽器への0通信処理

```

```

822 //      if(!isInstJoin[instId]){
823 //          // 参加していない楽器は無視（フラグはtrueのままなので次回以降スキップされ
            る）
824 //      } else if (isInstConnected[instId]) {
825 //          // 他楽器へ 0通信を送信
826 //          // startGlobalTick = 現在のglobalTick + (evTick - elapsedPlayTick)
827 //          // 「あと何Tick後に演奏開始してほしいか」をglobalTick基準で伝える
828 //          int32_t ticksUntilStart = evTick - elapsedPlayTick;
829 //          uint32_t startGlobalTick = (uint32_t)((int32_t)globalTick +
            ticksUntilStart);
830 //          sendOPacket(instId, currentSongId, startGlobalTick);
831 //          Serial.print(F("[0送信] instId=")); Serial.print(instId);
832 //          Serial.print(F(" startGlobalTick=")); Serial.println(startGlobalTick);
833 //      }else {
834 //          // 参加予定だが未接続（基本ここには来ない設計ですが安全策として）
835 //          oSentFlags[i] = false; // 送信失敗として次回再試行
836 //      }
837 //  }
838 // }
839 // -----
840 // 5：×0_ADVANCE_TICKS2以内に演奏開始するドラム以外の楽器が
841 //     存在しない場合に、不必要な待機をスキップする。
842 //
843 //     state2 の最初の1回だけ判定する (isDrumPlaying が false かつ state==2) 。
844 //     ・ドラム以外で 0_ADVANCE_TICKS*2 以内に開始予定の楽器がある
845 //       → その最早開始Tick - 0_ADVANCE_TICKS に elapsedPlayTick を合わせる
846 //     ・そのような楽器が1台もない（ドラムだけ、またはドラムのみ0から始まる）
847 //       → elapsedPlayTick = 0 にして即ドラム演奏開始へ
848 // -----
849 // if (state == 2 && !isDrumPlaying) {
850 //     int32_t earliestNonDrum = INT32_MAX;
851 //     for (int i = 0; i < MAX_EVENTS; i++) {
852 //         int32_t evTick = startSequence[currentSongId][i][0];
853 //         int      instId = startSequence[currentSongId][i][1];
854 //         if (evTick == -1) break;
855 //         // 変更点：参加していない楽器(isInstJoin == false)は最短開始時間の計算から除
            外する
856 //         if (instId != drumInstId && isInstJoin[instId] && evTick < earliestNonDrum)
            {
857 //             earliestNonDrum = evTick;
858 //         }
859 //     }
860 //
861 //     if (earliestNonDrum == INT32_MAX || earliestNonDrum > 0_ADVANCE_TICKS * 2) {
862 //         // ドラム以外に 0_ADVANCE_TICKS*2 以内の開始予定がない → 即開始
863 //         if (elapsedPlayTick < 0) {
864 //             elapsedPlayTick = 0;

```

```

865 //      Serial.println(F("[SKIP] 近接非ドラム楽器なし: elapsedPlayTick=0 に即進め
      "));
866 //    }
867 //  } else {
868 //    // 最早の非ドラム楽器の 0通信送信タイミングに合わせる
869 //    int32_t targetElapsed = earliestNonDrum - 0_ADVANCE_TICKS;
870 //    if (elapsedPlayTick < targetElapsed) {
871 //      elapsedPlayTick = targetElapsed;
872 //      Serial.print(F("[SKIP] elapsedPlayTick を ")); Serial.print(targetElapsed)
      ;
873 //      Serial.println(F(" に調整"));
874 //    }
875 //  }
876 // }
877 // -----
878 // 5: 無駄な待機のスキップ処理 (修正版)
879 //   ドラム自身も含めた参加楽器の中で、最も早く演奏開始するTickを探し、
880 //   そこから 0_ADVANCE_TICKS 前までタイムラインを調整する。
881 // -----
882 if (state == 2 && !isDrumPlaying) {
883   int32_t earliestEvent = INT32_MAX;
884   for (int i = 0; i < MAX_EVENTS; i++) {
885     int32_t evTick = startSequence[currentSongId][i][0];
886     int     instId = startSequence[currentSongId][i][1];
887     if (evTick == -1) break;
888
889     // 修正点: ドラム自身 (drumInstId) も開始時間の計算に含める
890     if ((instId == drumInstId || isInstJoin[instId]) && evTick < earliestEvent) {
891       earliestEvent = evTick;
892     }
893   }
894
895   if (earliestEvent == INT32_MAX || earliestEvent > 0_ADVANCE_TICKS * 2) {
896     if (elapsedPlayTick < 0) {
897       elapsedPlayTick = 0;
898       Serial.println(F("[SKIP]_近接楽器なし:_elapsedPlayTick=0_に即進め"));
899     }
900   } else {
901     int32_t targetElapsed = earliestEvent - 0_ADVANCE_TICKS;
902     if (elapsedPlayTick < targetElapsed) {
903       elapsedPlayTick = targetElapsed;
904       Serial.print(F("[SKIP]_elapsedPlayTick_を_")); Serial.print(targetElapsed);
905       Serial.println(F("_に調整"));
906     }
907   }
908 }
909 }
910

```

```

911 // =====
912 // 0通信パケット送信
913 //   [0]='0' [1]=songId [2..5]=startGlobalTick(uint32 big-endian)
914 // =====
915 // void send0Packet(uint8_t instId, uint8_t songId, uint32_t startGlobalTick) {
916 //   for(int i=0;i<3;i++){
917 //     Udp.beginPacket(instIP(instId), LOCAL_PORT);
918 //     Udp.write('0');
919 //     Udp.write(songId);
920 //     Udp.write((uint8_t)((startGlobalTick >> 24) & 0xFF));
921 //     Udp.write((uint8_t)((startGlobalTick >> 16) & 0xFF));
922 //     Udp.write((uint8_t)((startGlobalTick >> 8) & 0xFF));
923 //     Udp.write((uint8_t)( startGlobalTick          & 0xFF));
924 //     Udp.endPacket();
925 //     delay(3); // ☒ Wi-Fiのバッファ溢れを防ぐための必須の遅延
926 //   }
927 // }
928 // 0通信を「1回だけ(delayなしで)」送る専用の関数を作成
929 void send0Packet_Single(uint8_t instId, uint8_t songId, uint32_t startGlobalTick) {
930   Udp.beginPacket(instIP(instId), LOCAL_PORT);
931   Udp.write('0');
932   Udp.write(songId);
933   Udp.write((uint8_t)((startGlobalTick >> 24) & 0xFF));
934   Udp.write((uint8_t)((startGlobalTick >> 16) & 0xFF));
935   Udp.write((uint8_t)((startGlobalTick >> 8) & 0xFF));
936   Udp.write((uint8_t)( startGlobalTick          & 0xFF));
937   Udp.endPacket();
938   // ★ delayなし。これなら演奏のタイミングはズレない！
939 }
940
941 // ☒☒ 5. ファイルの下の方、send0Packet() の下あたりに追加 ☒☒
942 // =====
943 // N通信パケット送信 (確定BPMを全参加楽器へブロードキャスト)
944 //   [0]='N' [1]=bpm(uint8)
945 // =====
946 void sendNPacket(uint8_t bpm) {
947   // for(int k=0;k<3;k++){
948   for (int i = 1; i < MAX_INSTRUMENTS; i++) {
949     // if (isInstJoin[i] && isInstConnected[i]) {
950     Udp.beginPacket(instIP(i), LOCAL_PORT);
951     Udp.write('N');
952     Udp.write(bpm);
953     Udp.endPacket();
954     // }
955     delay(1); // 2msならok
956   }
957   // }
958 }

```



```

959
960 //ついでにRも
961 void sendRPacket(uint8_t instId,int32_t diff){
962     // R通信: diffのみ返信 (0通信は別途 send0Packet で送る)
963     Udp.beginPacket(instIP(instId), LOCAL_PORT);
964     Udp.write('R');
965     Udp.write((uint8_t)((diff >> 24) & 0xFF));
966     Udp.write((uint8_t)((diff >> 16) & 0xFF));
967     Udp.write((uint8_t)((diff >> 8) & 0xFF));
968     Udp.write((uint8_t)( diff          & 0xFF));
969     Udp.endPacket();
970 }
971
972 // =====
973 // Tick管理
974 // =====
975 void updateTick() {
976     if (tickIval == 0) return;
977     uint32_t now = micros();
978     while (now - lastTick >= tickIval) {
979         uint32_t currentIval = tickIval; // ★ループ突入時の値を保存
980
981         globalTick++;
982
983         // --- 48TickごとにBPM段階の多数決とN通信送信 ---
984         if (state >= 1 && globalTick % 48 == 0) {
985             uint8_t newLevel = determineSystemBpmLevel();
986             if (newLevel != currentBpmLevel && newLevel > 0) {
987                 currentBpmLevel = newLevel;
988                 setBpm(currentBpmLevel);
989             }
990             sendNPacket(currentBpmLevel);
991             // Serial.print(F("[SYNC] 48Tick周期 N通信送信: 確定BPM = "));
992             // Serial.println(currentBpmLevel);
993         }
994
995         if (state >= 2) elapsedPlayTick++;
996         if (state == 3 && isDrumPlaying) {
997             if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
998                 currentSongId][currentJumpIdx][0]) {
999                 curTick = jumpTable[currentSongId][currentJumpIdx][1];
1000                 scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];
1001                 currentJumpIdx++;
1002
1003                 if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
1004                     ][0] == -1) {
1005                     currentJumpIdx = 0;
1006                 }
1007             }
1008         }
1009     }

```

```

1005         Serial.println(F("[JUMP]_楽譜をジャンプしました！"));
1006     }
1007
1008     sendPacket();
1009     curTick++;
1010 }
1011
1012     lastTick += currentIval; // ★書き換えられていても元の値を足す
1013 }
1014 }
1015
1016 // =====
1017 // BPM更新 → tickIval再計算
1018 // =====
1019 void setBpm(uint8_t bpm) {
1020     if (bpm == curBpm) return;
1021     curBpm = bpm;
1022     tickIval = 60000000UL / (uint32_t)bpm / 24;
1023     Serial.print('B'); Serial.print(','); Serial.print(bpm); Serial.print('\n');
1024 }
1025
1026 void sendVolumeValue(uint8_t vol) {
1027     if (vol == curVol) return;
1028     curVol = vol;
1029     Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n');
1030 }
1031
1032 // N通信で送信するためのBPM段階を決定する関数
1033 uint8_t determineSystemBpmLevel() {
1034     uint8_t validLevels[MAX_INSTRUMENTS];
1035     uint8_t validCount = 0;
1036
1037     // 1. 有効なデータ（参加中かつデータがある楽器）だけを抽出
1038     for (int i = 0; i < MAX_INSTRUMENTS; i++) {
1039         if ((i == drumInstId || isInstJoin[i]) && receivedBpmLevels[i] > 0) {
1040             validLevels[validCount] = receivedBpmLevels[i];
1041             validCount++;
1042         }
1043     }
1044
1045     if (validCount == 0) return currentBpmLevel; // データが一つもなければ前回維持
1046     if (validCount == 1) return validLevels[0]; // 1台だけならそれを採用
1047
1048     // 2. 配列を小さい順にソート（バブルソートで十分）
1049     for (int i = 0; i < validCount - 1; i++) {
1050         for (int j = 0; j < validCount - i - 1; j++) {
1051             if (validLevels[j] > validLevels[j + 1]) {
1052                 uint8_t temp = validLevels[j];

```

```

1053     validLevels[j] = validLevels[j + 1];
1054     validLevels[j + 1] = temp;
1055 }
1056 }
1057 }
1058
1059 // 3. 中央値の取得と、偶数時の「ユーザー提案ロジック（丸め）」
1060 if (validCount % 2 != 0) {
1061     // 奇数個なら、ど真ん中がそのまま中央値
1062     return validLevels[validCount / 2];
1063 } else {
1064     // 偶数個なら、中央を挟む2つの値を取得
1065     uint8_t mid1 = validLevels[validCount / 2 - 1];
1066     uint8_t mid2 = validLevels[validCount / 2];
1067
1068     // もし2つの値が同じなら、それが中央値
1069     if (mid1 == mid2) return mid1;
1070
1071     // 異なる場合、前回値（currentBpmLevel）と差が小さい方を採用！
1072     int diff1 = abs((int)mid1 - (int)currentBpmLevel);
1073     int diff2 = abs((int)mid2 - (int)currentBpmLevel);
1074
1075     if (diff1 <= diff2) {
1076         return mid1;
1077     } else {
1078         return mid2;
1079     }
1080 }
1081 }
1082
1083 // =====
1084 // ノート送信（Processingへ）
1085 // =====
1086 void sendPacket() {
1087     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
1088         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
1089         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
1090         Serial.print('N'); Serial.print(',');
1091         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
1092         Serial.print(Note[scoreIdx].duration); Serial.print(',');
1093         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
1094         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
1095         Serial.println(Note[scoreIdx].type);
1096         scoreIdx++;
1097     }
1098 }

```

F.2 flute_0706.ino

コード 75 flute_0706.ino (楽譜データ省略)

```
1 #include <WiFiS3.h>
2 #include <WiFiUdp.h>
3
4 const char SSID[] = "hackathon001-WPA2";
5 const char PASS[] = "hackathon001";
6
7 // ★楽器ごとに変更 (ピアノ=12, ストリングス=13, フルート=14)
8 IPAddress localIP(192, 168, 11, 14);
9 IPAddress drumIP (192, 168, 11, 11);
10 const uint16_t LOCAL_PORT = 5005;
11
12 // ★楽器ID (0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート)
13 const uint8_t MY_INST_ID = 3;
14
15 WiFiUDP Udp;
16 uint8_t packetBuffer[32];
17
18 const int CDS_PIN = A0;
19 const int CDS_ON_THRESHOLD = 1010; // 起動直後だけ使う仮しきい値
20 const int CDS_OFF_THRESHOLD = 980; // 起動直後だけ使う仮しきい値
21 const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
22 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
23 const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
24 const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
25 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
26 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
27 const unsigned long DEBOUNCE_MS = 20;
28
29 const int BPM_LIST[] = { 60, 90, 120, 150, 180 };
30 // 段階1~5(BPM_LISTと対応)の実測の1周あたり時間[ms]。理論値(30000/BPM)では観覧車の
31 // 実際の回転速度と合わないことが判明したため、実測値でBPMを判定する。
32 const float ROTATION_PERIOD_LIST[] = {230, 180, 140, 110, 80}; //pwmと大体対応させてい
33 // る
34
35 const int MIN_STABLE_COUNT = 3;
36
37 // =====
38 // 楽譜データ (★楽器ごとに書き換える★)
39 // =====
40
41 struct NoteData {
42     uint16_t timing;
43     uint16_t duration;
44     uint8_t pitch;
45     uint8_t startVelocity;
46     uint8_t endVelocity;
```

```

44     uint8_t type;
45 };
46
47 const NoteData Note[] = {
48
49     // 【1小節目】
50     {0, 24, 72, 80, 80, 0}, // [0] C5
51     {24, 24, 74, 80, 80, 0}, // [1] D5
52     {48, 24, 76, 80, 80, 0}, // [2] E5
53     {72, 24, 77, 80, 80, 0}, // [3] F5
54
55     // 【2小節目】
56     {96, 24, 76, 80, 80, 0}, // [4] E5
57     {120, 24, 74, 80, 80, 0}, // [5] D5
58     {144, 36, 72, 80, 80, 0}, // [6] C5
59
60     // ... (以下、同様の形式の楽譜データが続くため省略) ...
61 };
62
63 const uint16_t note_count = sizeof(Note) / sizeof(Note[0]); // 音符の数(配列に含まれて
    いる合計/音符1つあたりの要素数)
64 uint16_t scoreIdx = 0;
65
66 // =====
67 // Tick管理変数
68 // =====
69 uint32_t globalTick    = 0;
70 uint32_t curTick       = 0; // 楽譜用Tick
71 uint32_t lastTick      = 0;
72
73 uint32_t baseTickIval = 0; // BPMから計算した基本interval  $\mu$  [s/tick]
74 uint32_t tickIval     = 0; // 実際に使うinterval (同期補正あり)
75 int32_t  ticksToCorrect = 0; // 補正残りTick数
76 int32_t  lastDiff      = 0; // 直近のdiff (BPM変化時の再計算用)
77 uint32_t lastTPacketMicroTime = 0; // 直近のTパケット送信時の時刻Micro
78 uint32_t lastTPacketTick = 0; // 直近のTパケット送信時のglobalTick (drumNotStarted判
    定用)
79
80 // =====
81 // ドラム起動前同期フラグ
82 //   true  : ドラムがまだglobalTickを加算していない (-diff == globalTick が続いてい
    る)
83 //   false : ドラムが動き出した (初回のみ即時同期を実施済み)
84 // E通信を受信したらtrueに戻す
85 // =====
86 bool drumNotStarted = true;
87

```

```

88 uint8_t curBpm = 0;
89 uint8_t curVol = 50;
90
91 // 演奏管理
92 bool waitingForStart = false; // 0通信を受信して演奏開始globalTickを待っている
93 uint32_t startGlobalTick = 0; // ドラムから指定された演奏開始globalTick
94 uint8_t currentSongId = 0;
95
96 // =====
97 // SyncManager (レーザー検知・BPM安定判定)
98 // =====
99 class SyncManager {
100 public:
101     SyncManager()
102         : lastDetectTime(0), correctedBPM(0),
103           stableCount(0), prevCorrectedBPM(0), laserOn(false),
104           baseline(0), baselineReady(false),
105           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
106           pulsePeak(0), peakIndex(0), peakCount(0),
107           calibrated(false), startupCalibrated(false), waitingForRelease(false),
108           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
109         clearPeakHistory();
110     }
111
112     void reset() {
113         lastDetectTime = 0;
114         correctedBPM = 0;
115         stableCount = 0;
116         prevCorrectedBPM = 0;
117         laserOn = false;
118         baseline = 0;
119         baselineReady = false;
120         onThreshold = CDS_ON_THRESHOLD;
121         offThreshold = CDS_OFF_THRESHOLD;
122         pulsePeak = 0;
123         peakIndex = 0;
124         peakCount = 0;
125         calibrated = false;
126         startupCalibrated = false;
127         waitingForRelease = false;
128         calibrationStartMs = 0;
129         startupMin = 1023;
130         releaseThreshold = 0;
131         clearPeakHistory();
132     }
133
134     bool update(int cdsValue, unsigned long nowMs) {
135         if (calibrationStartMs == 0) calibrationStartMs = nowMs;

```

```

136     if (!detectLight(cdsValue, nowMs)) return false;
137     if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
138     if (lastDetectTime != 0) {
139         unsigned long interval = nowMs - lastDetectTime;
140         int newBPM = intervalToBpm((float)interval);
141         if (newBPM == prevCorrectedBPM) stableCount++;
142         else stableCount = 1;
143         prevCorrectedBPM = newBPM;
144         correctedBPM = newBPM;
145     }
146     lastDetectTime = nowMs;
147     return true;
148 }
149
150 int getCorrectedBPM() const { return correctedBPM; }
151 int getStableCount() const { return stableCount; }
152
153 private:
154     unsigned long lastDetectTime;
155     int correctedBPM;
156     int stableCount;
157     int prevCorrectedBPM;
158     bool laserOn;
159     int baseline;
160     bool baselineReady;
161     int onThreshold;
162     int offThreshold;
163     int pulsePeak;
164     int peakHistory[CDS_PEAK_HISTORY];
165     uint8_t peakIndex;
166     uint8_t peakCount;
167     bool calibrated;
168     bool startupCalibrated;
169     bool waitingForRelease;
170     unsigned long calibrationStartMs;
171     int startupMin;
172     int releaseThreshold;
173
174     bool detectLight(int v, unsigned long nowMs) {
175         if (!baselineReady) {
176             baseline = v;
177             startupMin = v;
178             baselineReady = true;
179             updateFallbackThresholds();
180         }
181
182         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
183         if (!startupCalibrated) {

```

```

184     if (v < startupMin) startupMin = v;
185     baseline = startupMin;
186     updateFallbackThresholds();
187     if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
188     startupCalibrated = true;
189     waitingForRelease = (v >= onThreshold);
190     releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
        : offThreshold;
191     return false;
192 }
193
194 if (waitingForRelease) {
195     if (v <= releaseThreshold) {
196         baseline = v;
197         updateFallbackThresholds();
198         waitingForRelease = false;
199     }
200     return false;
201 }
202
203 if (!laserOn) {
204     if (v < baseline) baseline = v;
205     else baseline = (baseline * 31 + v) / 32;
206     if (!calibrated) updateFallbackThresholds();
207 }
208
209 if (!laserOn && v >= onThreshold) {
210     laserOn = true;
211     pulsePeak = v;
212     return true;
213 }
214
215 if (laserOn) {
216     if (v > pulsePeak) pulsePeak = v;
217     if (v <= offThreshold) {
218         laserOn = false;
219         registerPeak(pulsePeak);
220     }
221 }
222 return false;
223 }
224
225 void registerPeak(int peak) {
226     if (peak - baseline < CDS_MIN_DELTA) return;
227
228     peakHistory[peakIndex] = peak;
229     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
230     if (peakCount < CDS_PEAK_HISTORY) peakCount++;

```



```

231     if (peakCount < 3) return;
232
233     long sum = 0;
234     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
235     int peakAvg = sum / peakCount;
236     int amplitude = peakAvg - baseline;
237     if (amplitude < CDS_MIN_DELTA) return;
238
239     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
240     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);

241
242     if (!calibrated) {
243         onThreshold = targetOn;
244         offThreshold = targetOff;
245         calibrated = true;
246     } else {
247         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
248         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
249     }
250     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
251 }
252
253 void updateFallbackThresholds() {
254     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
255     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
        );
256 }
257
258 int limitStep(int current, int target, int limit) {
259     if (target > current + limit) return current + limit;
260     if (target < current - limit) return current - limit;
261     return target;
262 }
263
264 void clearPeakHistory() {
265     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
266 }
267
268 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
269 int intervalToBpm(float intervalMs) {
270     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
271     for (int i = 1; i < 5; i++) {
272         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
273         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
274     }
275     return best;

```

```

276     }
277 };
278
279 SyncManager syncManager;
280
281 // =====
282 // state:
283 // 0 = 待機 (BPM安定待ち)
284 // 1 = BPM安定、ドラムと同期中 (Tパケット送信・diff収束待ち)
285 // 2 = 演奏中
286 // 4 = システム停止中
287 // =====
288 int state = 4;
289
290
291
292 // =====
293 // 楽譜遷移 (ジャンプ) 定義
294 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
295 //   scoreIdx]
296 // 終端は {-1, -1, -1}
297 // =====
298 const int MAX_SONGS = 5;
299 const int MAX_JUMPS = 20;
300 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
301     { // 曲ID: 0 かえるのうた
302         {1536, 0, 0},
303         {-1, -1, -1}
304     },
305     {
306         {2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728, 48},
307         {-1, -1, -1}
308     },
309     {
310         {5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856, 856}, {6216, 6456, 993},
311         {6816, 4608, 651}, {5760, 6816, 1046}, {8832, 4608, 651},
312         {-1, -1, -1}
313     },
314     {
315         {9240, 8856, 1399}, {10008, 9240, 1499}, {10392, 10008, 1669}, {10776, 10392,
316         1783}, {11544, 10776, 1895}, {11544, 10008, 1669}, {10392, 10008, 1669},
317         {10392, 8856, 1399}, {9240, 8856, 1399}, {10008, 11544, 2111}, {11928, 11544,
318         2111}, {11904, 11952, 2225}, {13632, 8856, 1399},
319         {-1, -1, -1}
320     },
321     {
322         {16488, 14952, 2973},
323         {-1, -1, -1}
324     }
325 };

```

```

319     }
320 };
321
322 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
323
324 // =====
325 // 曲ごとの初期設定データ
326 // =====
327
328 struct SongStartData {
329     uint32_t startTick;
330     uint16_t startNoteIdx;
331 };
332
333 // Processingの設定に合わせた初期値の定義
334 // インデックスがそのまま currentSongId に対応します
335 const SongStartData songStartParams[MAX_SONGS] = {
336     {0, 0}, // 曲ID: 0 (かえるのうた)
337     {1536, 29}, // 曲ID: 1 (酒場でブギウギ)
338     {4608, 651}, // 曲ID: 2 (王宮のメヌエット)
339     {8856, 1399}, // 曲ID: 3 (トルコ行進曲)
340     {13632, 2638} // 曲ID: 4 (序曲)
341 };
342
343 // =====
344 // 関数宣言
345 // =====
346 void updateTick();
347 void setBpm(uint8_t bpm);
348 void applyDiffCorrection(int32_t diff);
349 void sendTPacket();
350 void sendPacket();
351 void sendMPacket(uint8_t bpm);
352
353 // =====
354 // セットアップ
355 // =====
356 void setup() {
357     Serial.begin(115200);
358     WiFi.config(localIP);
359     WiFi.begin(SSID, PASS);
360     Serial.print("Connecting_WiFi...");
361     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
362     Serial.println("\nWiFi_Connected!");
363     Udp.begin(LOCAL_PORT);
364     Serial.println("state=0_BPM安定待ち");
365 }
366

```

```

367 // =====
368 // メインループ
369 // =====
370 void loop() {
371     unsigned long now      = millis();
372     int          cdsValue = analogRead(CDS_PIN);
373
374     // ---- UDP受信 ----
375     int pktSize = Udp.parsePacket();
376     while (pktSize > 0) {
377         if (pktSize >= 2) {
378             Udp.read(packetBuffer, sizeof(packetBuffer));
379             char header = (char)packetBuffer[0];
380             uint8_t payload = packetBuffer[1];
381
382             switch (header) {
383
384                 // -----
385                 // 'R': ドラムからのdiff返信
386                 //   [0]='R' [1..4]=diff(int32 big-endian)
387                 //
388                 //   ドラム起動前判定：
389                 //   -diff == globalTick (つまり globalTick + diff == 0)
390                 //   → ドラムはまだglobalTickを加算していない。自身もリセット。
391                 //   ドラムが動き出した初回：
392                 //   上記条件が初めてfalseになった → globalTick -= diff で即時同期。
393                 //   以降は通常のdiff補正 (applyDiffCorrection) へ。
394                 // -----
395                 case 'R':
396                     // 【追加】 停止中 (4) や安定待ち (0) に届いた過去のR通信は「遅延ゴミ (Stale
397                     //   Packet)」なので完全無視する！
398                     if (state == 0 || state == 4) break;
399
400                     if (pktSize >= 5) {
401                         int32_t rawDiff =
402                             ((int32_t)(int8_t)packetBuffer[1] << 24) |
403                             ((int32_t)packetBuffer[2] << 16) |
404                             ((int32_t)packetBuffer[3] << 8) |
405                             (int32_t)packetBuffer[4];
406
407                         // 遅延 (レイテンシ) 補正の計算
408                         uint32_t rtt = micros() - lastTPacketMicroTime;
409                         // 【安全対策】 往復時間が長すぎる (例:200ms以上) 場合は古いパケットとみなして
410                         //   破棄
411                         if (rtt > 1000 * 1000) {
412                             Serial.println(F("[R]_パケット遅延過大のためスキップ"));
413                             break;
414                         }
415                     }
416                 }
417             }

```

```

413
414 uint32_t latencyMicros = rtt / 2; // 片道遅延
415 int32_t latencyTicks = 0;
416
417 if (baseTickIval > 0) {
418     // 四捨五入してTick数に変換：(値 + 割る数の半分) / 割る数
419     latencyTicks = (int32_t)((latencyMicros + (baseTickIval / 2)) /
420                             baseTickIval);
421 }
422
423 // ドラム機の勘違い（遅延分）を差し引いて真のズレを計算
424 int32_t diff = rawDiff - latencyTicks;
425
426 Serial.print(F("[R]_rawDiff=")); Serial.print(rawDiff);
427 Serial.print(F("_latencyTicks=")); Serial.print(latencyTicks);
428 Serial.print(F("_trueDiff=")); Serial.println(diff);
429
430 // Serial.print(F("[R] diff=")); Serial.println(diff);
431
432 if (drumNotStarted) {
433     if ((int32_t)lastTPacketTick + diff == 0) {
434         // Tパケット送信時点でドラムはまだ0。自身もリセットして足並みを揃える
435         globalTick = 0;
436         lastTick = micros(); // Tick計測の基点もリセット
437         Serial.println(F("[R]_ドラム未起動:_globalTick_リセット"));
438     } else {
439         // ドラムが動き出した初回：即時同期
440         drumNotStarted = false;
441         globalTick = (uint32_t)((int32_t)globalTick + diff);
442         lastDiff = diff;
443         Serial.print(F("[R]_ドラム起動検知:_即時同期_globalTick="));
444         Serial.println(globalTick);
445         // 以降は通常補正も適用
446         if (state >= 1) {
447             // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
448             if (ticksToCorrect == 0 || abs(diff) >= 24) {
449                 applyDiffCorrection(diff);
450             } else {
451                 Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
452             }
453         }
454     } else {
455         // 通常の同期補正
456         lastDiff = diff;
457         if (state >= 1) {

```

```

458         // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ
        補正を更新
459         if (ticksToCorrect == 0 || abs(diff) >= 24) {
460             applyDiffCorrection(diff);
461         } else {
462             Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
463         }
464     }
465 }
466 }
467 break;
468
469 // -----
470 // '0': ドラムからの演奏開始指示
471 //   [0]='0' [1]=songId [2..5]=startGlobalTick(uint32 big-endian)
472 //
473 //   自身のglobalTickと startGlobalTick を比較して:
474 //   ・まだ到達していない → waitingForStart=true で待つ
475 //   ・すでに超過している → 超過分だけ curTick をオフセットして即演奏開始
476 // -----
477 case '0':
478     if (pktSize >= 6) {
479         if(waitingForStart || state == 2) return;
480
481         uint8_t songId = packetBuffer[1];
482         uint32_t sgt =
483             ((uint32_t)packetBuffer[2] << 24) |
484             ((uint32_t)packetBuffer[3] << 16) |
485             ((uint32_t)packetBuffer[4] << 8) |
486             (uint32_t)packetBuffer[5];
487
488         currentSongId = songId;
489         startGlobalTick = sgt;
490
491         Serial.print(F("[0]_songId=")); Serial.print(songId);
492         Serial.print(F("_startGlobalTick=")); Serial.println(sgt);
493
494         // 既に超過していたらスキップして即開始
495         if ((int32_t)(globalTick - sgt) >= 0) {
496             // 超過分だけ curTick を進める
497             uint32_t overrun = globalTick - sgt;
498             curTick = songStartParams[currentSongId].startTick + overrun;
499             scoreIdx = songStartParams[currentSongId].startNoteIdx;
500             // overrunより前のノートをスキップ
501             while (scoreIdx < note_count && Note[scoreIdx].timing < curTick) scoreIdx
                ++;
502             state = 2;
503             Serial.print(F("[0]_超過済み。overrun=")); Serial.print(overrun);

```

```

504         Serial.println(F("_Tick_→_即開始"));
505     } else {
506         waitingForStart = true;
507         Serial.println(F("[0]_開始待機中..."));
508     }
509 }
510 break;
511
512 case 'V': {
513     uint8_t vol = (payload >= 1 && payload <= 5) ? (payload * 10) : 30;
514     if (vol != curVol) {
515         curVol = vol;
516         Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n')
            ;
517     }
518     break;
519 }
520
521 // -----
522 // 'E': 演奏終了通知 (指揮機から)
523 //   drumNotStarted フラグをリセットし、次の演奏開始に備える
524 // -----
525 case 'E':
526     drumNotStarted = true;
527     waitingForStart = false;
528     state = 4; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
                // 1ではなく0
529     curTick = 0;
530     scoreIdx = 0;
531     globalTick = 0;
532     curBpm = 0; // 追加: BPMリセット
533     tickIval = 0; // 追加
534     baseTickIval = 0; // 追加
535     Serial.println(F("[E]_演奏終了。drumNotStarted_リセット"));
536     break;
537
538 // -----
539 // 'S': 演奏開始通知 (指揮機から)
540 //   ドラムに対して演奏への参加を申請する
541 // -----
542 case 'S':
543     drumNotStarted = true;
544     waitingForStart = false;
545     state = 0; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
                // 1ではなく0
546     curTick = 0;
547     scoreIdx = 0;
548     globalTick = 0;

```

```

549     currentJumpIdx = 0; // ジャンプインデックスも初期化
550     Serial.println(F("[S]_演奏開始。drumNotStarted_リセット"));
551
552     syncManager.reset();
553
554     sendJPacket();
555
556     Serial.println(F("[S]_ドラムに同期予告を送信。"));
557
558     //だーめ
559     // delayがあるため、加速が終わった時点で、バッファに溜まっている2回め・3回め
560     //   のパケットを捨てる（ボタンが動くようにする）
561     // Serial.println("連射された残りのパケットをクリアします。");
562     // while(Udp.parsePacket() > 0) {
563     //     while(Udp.available()) {
564     //         Udp.read();
565     //     }
566     // }
567     break;
568
569     // 追加： マスターからの確定BPMを受信
570     case 'N':
571         if (pktSize >= 2) {
572             uint8_t confirmedBpm = packetBuffer[1];
573             setBpm(confirmedBpm);
574             Serial.print(F("[N]_確定BPM受信:")); Serial.println(confirmedBpm);
575         }
576         break;
577
578     default: break;
579 }
580 // } else if (pktSize > 0) {
581 //     while (Udp.available()) Udp.read();
582 // } else {
583     Udp.read(); // 空読みして破棄
584 }
585 pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
586 }
587
588 if (state == 4) return;
589
590 // ---- レーザー検知 ----
591 bool laser = syncManager.update(cdsValue, now);
592
593 // ---- ステートマシン ----
594 switch (state) {
595

```



```

596 // --- state0: BPM安定待ち ---
597 case 0:
598     if (laser) {
599         int s = syncManager.getStableCount();
600         uint8_t curLaserBpm = syncManager.getCorrectedBPM();
601         Serial.print(F("[S0]_BPM=")); Serial.print(curLaserBpm);
602         Serial.print(F("_stable=")); Serial.println(s);
603
604         sendMPacket(curLaserBpm); // M通信でドラムに提案！
605
606         if (s >= MIN_STABLE_COUNT) {
607             state = 1;
608             lastTick = micros();
609             // setBpm(syncManager.getCorrectedBPM());
610
611             Serial.println(F("[S0→1]_BPM安定(N通信待ち)"));
612         }
613     }
614     break;
615
616 // --- state1: 同期中 ---
617 case 1:
618     updateTick();
619     if (laser) {
620         static unsigned long lastSentTime = 0;
621         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
622             // setBpm(syncManager.getCorrectedBPM());
623             // レーザー通過のたびにTパケットを送信してdiff返信を要求
624             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
625             lastTPacketTick = globalTick;
626             sendTPacket();
627             lastSentTime = now;
628         }
629     }
630
631     // waitingForStart かつ startGlobalTick に到達したら演奏開始
632     if (waitingForStart && (int32_t)(globalTick - startGlobalTick) >= 0) {
633         waitingForStart = false;
634         curTick = songStartParams[currentSongId].startTick;
635         scoreIdx = songStartParams[currentSongId].startNoteIdx;
636         state = 2;
637         Serial.println(F("[S1→2]_演奏開始！"));
638     }
639     break;
640
641 // --- state2: 演奏中 ---
642 case 2:

```

```

643     updateTick();
644     if (laser) {
645         static unsigned long lastSentTime = 0;
646         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
647             // setBpm(syncManager.getCorrectedBPM());
648             // レーザー通過のたびにTパケットを送信してdiff返信を要求
649             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
650             lastTPacketTick = globalTick;
651             sendTPacket();
652             lastSentTime = now;
653         }
654     }
655     break;
656
657     // --- state4: システム停止（笑）中 ---
658     // case 4: break;
659 }
660 }
661
662 // =====
663 // diff補正によるtickIval算出
664 // diff = drumGlobalTick - myGlobalTick
665 // diff > 0: 自分が遅れている → 速く進む（tickIvalを小さく）
666 // diff < 0: 自分が早すぎる → 遅く進む（tickIvalを大きく）
667 // =====
668 void applyDiffCorrection(int32_t diff) {
669     if (baseTickIval == 0) return;
670
671     float SYNC_READY_THRESHOLD = 50 / baseTickIval;
672
673     if (abs(diff) <= SYNC_READY_THRESHOLD) {
674         // 不感帯：ズレ±が2 Tick以内なら何もしない（ネットワーク遅延のブレを吸収）
675         tickIval = baseTickIval;
676         ticksToCorrect = 0;
677     } else if (diff <= -24) {
678         // 大幅に早い → ×2 で -diff Tick間遅らせる
679         tickIval = baseTickIval * 2;
680         ticksToCorrect = -diff;
681         Serial.println(F("[SYNC]_大幅早い: _×2"));
682     } else if (diff >= 24) {
683         // 大幅に遅い → ×2/3 で ×diff3 Tick間速める
684         tickIval = baseTickIval * 2 / 3;
685         ticksToCorrect = diff * 3;
686         Serial.println(F("[SYNC]_大幅遅い: _×2/3"));
687     } else {
688         // 小さいズレ → 次の48Tickでdiffを吸収する比率補正
689         tickIval = (uint32_t)((48UL * baseTickIval) / (48 + diff));

```

```

690     ticksToCorrect = 48 + diff;
691     Serial.print(F("[SYNC]_微補正_diff=")); Serial.println(diff);
692 }
693
694 uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
695 Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
    ('\n');
696 }
697
698 // =====
699 // BPM更新
700 // =====
701 void setBpm(uint8_t bpm) {
702     if (bpm == curBpm) return;
703     curBpm      = bpm;
704     uint32_t oldBaseTickIval = baseTickIval;
705     baseTickIval = 60000000UL / (uint32_t)bpm / 24;
706
707     // if (ticksToCorrect > 0) {
708     //     // BPM変化後も補正を継続する場合は lastDiff で再計算
709     //     applyDiffCorrection(lastDiff);
710     //     Serial.println(F("[BPM変化] 補正を再計算"));
711     if (oldBaseTickIval > 0 && ticksToCorrect > 0) {
712         // 昔のズレ(lastDiff)で計算し直すのではなく、現在の補正スピードを新しいBPMの比率
713         // に合わせてスケールさせるだけ
714         tickIval = (tickIval * baseTickIval) / oldBaseTickIval;
715         Serial.println(F("[BPM変化]_補正スピードを新BPMにスケール"));
716     } else {
717         tickIval = baseTickIval;
718     }
719
720     uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
721     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
722         ('\n');
723 }
724
725 // =====
726 // Tパケット送信 (自身のglobalTickをドラムへ)
727 //   [0]='T' [1..4]=globalTick(uint32 big-endian) [5]=MY_INST_ID
728 // =====
729 void sendTPacket() {
730     Udp.beginPacket(drumIP, LOCAL_PORT);
731     Udp.write('T');
732     Udp.write((uint8_t)((globalTick >> 24) & 0xFF));
733     Udp.write((uint8_t)((globalTick >> 16) & 0xFF));
734     Udp.write((uint8_t)((globalTick >> 8) & 0xFF));
735     Udp.write((uint8_t)( globalTick      & 0xFF));
736     Udp.write(MY_INST_ID);

```

```

735     Udp.endPacket();
736     lastTPacketMicroTime = micros();
737 }
738
739 void sendJPacket() {
740     for(int i=0;i<3;i++){
741         Udp.beginPacket(drumIP, LOCAL_PORT);
742         Udp.write('J');
743         Udp.write(MY_INST_ID);
744         Udp.endPacket();
745         delay(3); // ☒ Wi-Fiのバッファ溢れを防ぐための必須の遅延
746     }
747 }
748
749 // ☒☒ 4. ファイルの下の方、sendTPacket() の下あたりに追加 ☒☒
750 // =====
751 // M通信送信（自身のレーザーBPMをドラムへ提案）
752 //   [0]='M' [1]=bpm(uint8) [2]=MY_INST_ID
753 // =====
754 void sendMPacket(uint8_t bpm) {
755     // for(int i=0;i<3;i++){
756     Udp.beginPacket(drumIP, LOCAL_PORT);
757     Udp.write('M');
758     Udp.write(bpm);
759     Udp.write(MY_INST_ID);
760     Udp.endPacket();
761     // }
762 }
763
764 // =====
765 // Tick管理
766 // =====
767 void updateTick() {
768     if (tickIval == 0) return;
769     uint32_t now = micros();
770     while (now - lastTick >= tickIval) {
771         uint32_t currentIval = tickIval; // ★ループ突入時の値を保存
772
773         globalTick++;
774
775         // 補正カウントダウン
776         if (ticksToCorrect > 0) {
777             ticksToCorrect--;
778             if (ticksToCorrect == 0) {
779                 tickIval = baseTickIval;
780                 Serial.println(F("[SYNC]_補正完了"));
781             }
782         }

```

```

783
784     if (state == 2) {
785         // ジャンプ（ループ）処理の判定
786         if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
787             currentSongId][currentJumpIdx][0]) {
788             curTick = jumpTable[currentSongId][currentJumpIdx][1];
789             scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];
790             currentJumpIdx++;
791
792             if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
793                 ][0] == -1) {
794                 currentJumpIdx = 0;
795             }
796         }
797
798         sendPacket();
799         curTick++;
800     }
801 }
802 }
803
804 // =====
805 // ノート送信 (Processingへ)
806 // =====
807 void sendPacket() {
808     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
809         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
810         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
811         Serial.print('N'); Serial.print(',');
812         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
813         Serial.print(Note[scoreIdx].duration); Serial.print(',');
814         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
815         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
816         Serial.println(Note[scoreIdx].type);
817         scoreIdx++;
818     }
819 }

```

F.3 piano_0706.ino

コード 76 piano_0706.ino (楽譜データ省略)

```

1  #include <WiFiS3.h>
2  #include <WiFiUdp.h>
3
4  const char SSID[] = "hackathon001-WPA2";

```

```

5  const char PASS[] = "hackathon001";
6
7  // ★楽器ごとに変更（ピアノ=12, ストリングス=13, フルート=14）
8  IPAddress localIP(192, 168, 11, 12);
9  IPAddress drumIP (192, 168, 11, 11);
10 const uint16_t LOCAL_PORT = 5005;
11
12 // ★楽器ID (0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート)
13 const uint8_t MY_INST_ID = 1;
14
15 WiFiUDP Udp;
16 uint8_t packetBuffer[32];
17
18 const int CDS_PIN          = A0;
19 const int CDS_ON_THRESHOLD = 1010; // 起動直後だけ使う仮しきい値
20 const int CDS_OFF_THRESHOLD = 980; // 起動直後だけ使う仮しきい値
21 const int CDS_MIN_DELTA = 25;      // 暗い状態との差がこれ未満ならレーザー扱いしない
22 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
23 const uint8_t CDS_ON_RATIO = 65;   // baselineからピークまでの65%をONしきい値にする
24 const uint8_t CDS_OFF_RATIO = 35;  // baselineからピークまでの35%をOFFしきい値にする
25 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
26 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
27 const unsigned long DEBOUNCE_MS = 20;
28 const int BPM_LIST[] = { 60, 90, 120, 150, 180 };
29 // 段階1~5(BPM_LISTと対応)の実測の1周あたり時間[ms]。理論値(30000/BPM)では観覧車の
30 // 実際の回転速度と合わないことが判明したため、実測値でBPMを判定する。
31 const float ROTATION_PERIOD_LIST[] = {230, 180, 140, 110, 80}; //pwmと大体対応させてい
    る
32 const int MIN_STABLE_COUNT = 3;
33
34 // =====
35 // 楽譜データ（★楽器ごとに書き換える★）
36 // =====
37 struct NoteData {
38     uint16_t timing;
39     uint16_t duration;
40     uint8_t pitch;
41     uint8_t startVelocity;
42     uint8_t endVelocity;
43     uint8_t type;
44 };
45
46 const NoteData Note[] = {
47
48     // 【1小節目】
49     {0, 24, 60, 96, 80, 0}, // [0] C4
50     {24, 24, 62, 96, 80, 0}, // [1] D4
51     {48, 24, 64, 96, 80, 0}, // [2] E4

```

```

52 {72, 24, 65, 96, 80, 0}, // [3] F4
53
54 // 【2小節目】
55 {96, 24, 64, 96, 80, 0}, // [4] E4
56 {120, 24, 62, 96, 80, 0}, // [5] D4
57 {144, 36, 60, 96, 80, 0}, // [6] C4
58
59 // ... (以下、同様の形式の楽譜データが続くため省略) ...
60 };
61
62 const uint16_t note_count = sizeof(Note) / sizeof(Note[0]); // 音符の数(配列に含まれて
    いる合計/音符1つあたりの要素数)
63 uint16_t scoreIdx = 0;
64
65 // =====
66 // Tick管理変数
67 // =====
68 uint32_t globalTick = 0;
69 uint32_t curTick = 0; // 楽譜用Tick
70 uint32_t lastTick = 0;
71
72 uint32_t baseTickIval = 0; // BPMから計算した基本interval μ[s/tick]
73 uint32_t tickIval = 0; // 実際に使うinterval (同期補正あり)
74 int32_t ticksToCorrect = 0; // 補正残りTick数
75 int32_t lastDiff = 0; // 直近のdiff (BPM変化時の再計算用)
76 uint32_t lastTPacketMicroTime = 0; // 直近のTパケット送信時の時刻Micro
77 uint32_t lastTPacketTick = 0; // 直近のTパケット送信時のglobalTick (drumNotStarted判
    定用)

78
79 // =====
80 // ドラム起動前同期フラグ
81 // true : ドラムがまだglobalTickを加算していない (-diff == globalTick が続いてい
    る)
82 // false : ドラムが動き出した (初回のみ即時同期を実施済み)
83 // E通信を受信したらtrueに戻す
84 // =====
85 bool drumNotStarted = true;
86
87 uint8_t curBpm = 0;
88 uint8_t curVol = 50;
89
90 // 演奏管理
91 bool waitingForStart = false; // 0通信を受信して演奏開始globalTickを待っている
92 uint32_t startGlobalTick = 0; // ドラムから指定された演奏開始globalTick
93 uint8_t currentSongId = 0;
94
95 // =====

```

```

96 // SyncManager (レーザー検知・BPM安定判定)
97 // =====
98 class SyncManager {
99 public:
100     SyncManager()
101         : lastDetectTime(0), correctedBPM(0),
102           stableCount(0), prevCorrectedBPM(0), laserOn(false),
103           baseline(0), baselineReady(false),
104           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
105           pulsePeak(0), peakIndex(0), peakCount(0),
106           calibrated(false), startupCalibrated(false), waitingForRelease(false),
107           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
108         clearPeakHistory();
109     }
110
111     void reset() {
112         lastDetectTime = 0;
113         correctedBPM = 0;
114         stableCount = 0;
115         prevCorrectedBPM = 0;
116         laserOn = false;
117         baseline = 0;
118         baselineReady = false;
119         onThreshold = CDS_ON_THRESHOLD;
120         offThreshold = CDS_OFF_THRESHOLD;
121         pulsePeak = 0;
122         peakIndex = 0;
123         peakCount = 0;
124         calibrated = false;
125         startupCalibrated = false;
126         waitingForRelease = false;
127         calibrationStartMs = 0;
128         startupMin = 1023;
129         releaseThreshold = 0;
130         clearPeakHistory();
131     }
132
133     bool update(int cdsValue, unsigned long nowMs) {
134         if (calibrationStartMs == 0) calibrationStartMs = nowMs;
135         if (!detectLight(cdsValue, nowMs)) return false;
136         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
137         if (lastDetectTime != 0) {
138             unsigned long interval = nowMs - lastDetectTime;
139             int newBPM = intervalToBpm((float)interval);
140             if (newBPM == prevCorrectedBPM) stableCount++;
141             else stableCount = 1;
142             prevCorrectedBPM = newBPM;
143             correctedBPM = newBPM;

```



```

144     }
145     lastDetectTime = nowMs;
146     return true;
147 }
148
149 int getCorrectedBPM() const { return correctedBPM; }
150 int getStableCount() const { return stableCount; }
151
152 private:
153     unsigned long lastDetectTime;
154     int correctedBPM;
155     int stableCount;
156     int prevCorrectedBPM;
157     bool laserOn;
158     int baseline;
159     bool baselineReady;
160     int onThreshold;
161     int offThreshold;
162     int pulsePeak;
163     int peakHistory[CDS_PEAK_HISTORY];
164     uint8_t peakIndex;
165     uint8_t peakCount;
166     bool calibrated;
167     bool startupCalibrated;
168     bool waitingForRelease;
169     unsigned long calibrationStartMs;
170     int startupMin;
171     int releaseThreshold;
172
173     bool detectLight(int v, unsigned long nowMs) {
174         if (!baselineReady) {
175             baseline = v;
176             startupMin = v;
177             baselineReady = true;
178             updateFallbackThresholds();
179         }
180
181         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
182         if (!startupCalibrated) {
183             if (v < startupMin) startupMin = v;
184             baseline = startupMin;
185             updateFallbackThresholds();
186             if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
187             startupCalibrated = true;
188             waitingForRelease = (v >= onThreshold);
189             releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
190                             : offThreshold;
191             return false;

```

```

191     }
192
193     if (waitingForRelease) {
194         if (v <= releaseThreshold) {
195             baseline = v;
196             updateFallbackThresholds();
197             waitingForRelease = false;
198         }
199         return false;
200     }
201
202     if (!laserOn) {
203         if (v < baseline) baseline = v;
204         else baseline = (baseline * 31 + v) / 32;
205         if (!calibrated) updateFallbackThresholds();
206     }
207
208     if (!laserOn && v >= onThreshold) {
209         laserOn = true;
210         pulsePeak = v;
211         return true;
212     }
213
214     if (laserOn) {
215         if (v > pulsePeak) pulsePeak = v;
216         if (v <= offThreshold) {
217             laserOn = false;
218             registerPeak(pulsePeak);
219         }
220     }
221     return false;
222 }
223
224 void registerPeak(int peak) {
225     if (peak - baseline < CDS_MIN_DELTA) return;
226
227     peakHistory[peakIndex] = peak;
228     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
229     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
230     if (peakCount < 3) return;
231
232     long sum = 0;
233     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
234     int peakAvg = sum / peakCount;
235     int amplitude = peakAvg - baseline;
236     if (amplitude < CDS_MIN_DELTA) return;
237
238     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);

```

```

239     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
                                targetOn - 5);

240
241     if (!calibrated) {
242         onThreshold = targetOn;
243         offThreshold = targetOff;
244         calibrated = true;
245     } else {
246         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
247         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
248     }
249     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
250 }
251
252 void updateFallbackThresholds() {
253     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
254     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
                               );
255 }
256
257 int limitStep(int current, int target, int limit) {
258     if (target > current + limit) return current + limit;
259     if (target < current - limit) return current - limit;
260     return target;
261 }
262
263 void clearPeakHistory() {
264     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
265 }
266
267 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
268 int intervalToBpm(float intervalMs) {
269     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
270     for (int i = 1; i < 5; i++) {
271         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
272         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
273     }
274     return best;
275 }
276 };
277
278 SyncManager syncManager;
279
280 // =====
281 // state:
282 // 0 = 待機 (BPM安定待ち)
283 // 1 = BPM安定、ドラムと同期中 (Tパケット送信・diff収束待ち)

```

```

284 // 2 = 演奏中
285 // 4 = システム停止中
286 // =====
287 int state = 4;
288
289
290
291 // =====
292 // 楽譜遷移（ジャンプ）定義
293 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
// scoreIdx]
294 // 終端は {-1, -1, -1}
295 // =====
296 const int MAX_SONGS = 5;
297 const int MAX_JUMPS = 20;
298 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
299     { // 曲ID: 0 かえるのうた
300         {1536, 0, 0},
301         {-1, -1, -1}
302     },
303     {
304         {2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728, 48},
305         {-1, -1, -1}
306     },
307     {
308         {5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856, 856}, {6216, 6456, 993},
309         {6816, 4608, 651}, {5760, 6816, 1046}, {8832, 4608, 651},
310         {-1, -1, -1}
311     },
312     {
313         {9240, 8856, 1399}, {10008, 9240, 1499}, {10392, 10008, 1669}, {10776, 10392,
1783}, {11544, 10776, 1895}, {11544, 10008, 1669}, {10392, 10008, 1669},
10392, 8856, 1399}, {9240, 8856, 1399}, {10008, 11544, 2111}, {11928, 11544,
2111}, {11904, 11952, 2225}, {13632, 8856, 1399},
314         {-1, -1, -1}
315     },
316     {
317         {16488, 14952, 2899},
318         {-1, -1, -1}
319     }
320 };
321 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
322
323 // =====
324 // 曲ごとの初期設定データ
325 // =====
326

```

```

327 struct SongStartData {
328     uint32_t startTick;
329     uint16_t startNoteIdx;
330 };
331
332 // Processingの設定に合わせた初期値の定義
333 // インデックスがそのまま currentSongId に対応します
334 const SongStartData songStartParams[MAX_SONGS] = {
335     {0, 0}, // 曲ID: 0 (かえるのうた)
336     {1536, 29}, // 曲ID: 1 (酒場でブギウギ)
337     {4608, 651}, // 曲ID: 2 (王宮のメヌエット)
338     {8856, 1399}, // 曲ID: 3 (トルコ行進曲)
339     {13632, 2638} // 曲ID: 4 (序曲)
340 };
341
342
343
344 // =====
345 // 関数宣言
346 // =====
347 void updateTick();
348 void setBpm(uint8_t bpm);
349 void applyDiffCorrection(int32_t diff);
350 void sendTPacket();
351 void sendPacket();
352 void sendMPacket(uint8_t bpm);
353
354 // =====
355 // セットアップ
356 // =====
357 void setup() {
358     Serial.begin(115200);
359     WiFi.config(localIP);
360     WiFi.begin(SSID, PASS);
361     Serial.print("Connecting_WiFi...");
362     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
363     Serial.println("\nWiFi_Connected!");
364     Udp.begin(LOCAL_PORT);
365     Serial.println("state=0_BPM安定待ち");
366 }
367
368 // =====
369 // メインループ
370 // =====
371 void loop() {
372     unsigned long now = millis();
373     int cdsValue = analogRead(CDS_PIN);
374

```

```

375 // ---- UDP受信 ----
376 int pktSize = Udp.parsePacket();
377 while (pktSize > 0) {
378   if (pktSize >= 2) {
379     Udp.read(packetBuffer, sizeof(packetBuffer));
380     char header = (char)packetBuffer[0];
381     uint8_t payload = packetBuffer[1];
382
383     switch (header) {
384
385       // -----
386       // 'R': ドラムからのdiff返信
387       // [0]='R' [1..4]=diff(int32 big-endian)
388       //
389       // ドラム起動前判定:
390       //   -diff == globalTick (つまり globalTick + diff == 0)
391       //   → ドラムはまだglobalTickを加算していない。自身もリセット。
392       //   ドラムが動き出した初回:
393       //   上記条件が初めてfalseになった → globalTick -= diff で即時同期。
394       //   以降は通常のdiff補正 (applyDiffCorrection) へ。
395       // -----
396       case 'R':
397         // 【追加】停止中 (4) や安定待ち (0) に届いた過去のR通信は「遅延ゴミ (Stale
398         //   Packet)」なので完全無視する！
399         if (state == 0 || state == 4) break;
400
401         if (pktSize >= 5) {
402           int32_t rawDiff =
403             ((int32_t)(int8_t)packetBuffer[1] << 24) |
404             ((int32_t)packetBuffer[2] << 16) |
405             ((int32_t)packetBuffer[3] << 8) |
406             (int32_t)packetBuffer[4];
407
408           // 遅延 (レイテンシ) 補正の計算
409           uint32_t rtt = micros() - lastTPacketMicroTime;
410           // 【安全対策】往復時間が長すぎる (例:200ms以上) 場合は古いパケットとみなして
411           //   破棄
412           if (rtt > 1000 * 1000) {
413             Serial.println(F("[R]_パケット遅延過大のためスキップ"));
414             break;
415           }
416
417           uint32_t latencyMicros = rtt / 2; // 片道遅延
418           int32_t latencyTicks = 0;
419
420           if (baseTickIval > 0) {
421             // 四捨五入してTick数に変換: (値 + 割る数の半分) / 割る数

```

```

420         latencyTicks = (int32_t)((latencyMicros + (baseTickIval / 2)) /
421             baseTickIval);
422     }
423     // ドラム機の勘違い（遅延分）を差し引いて真のズレを計算
424     int32_t diff = rawDiff - latencyTicks;
425
426     Serial.print(F("[R]_rawDiff=")); Serial.print(rawDiff);
427     Serial.print(F("_latencyTicks=")); Serial.print(latencyTicks);
428     Serial.print(F("_trueDiff=")); Serial.println(diff);
429
430     // Serial.print(F("[R] diff=")); Serial.println(diff);
431
432     if (drumNotStarted) {
433         if ((int32_t)lastTPacketTick + diff == 0) {
434             // Tパケット送信時点でドラムはまだ0。自身もリセットして足並みを揃える
435             globalTick = 0;
436             lastTick = micros(); // Tick計測の基点もリセット
437             Serial.println(F("[R]_ドラム未起動:_globalTick_リセット"));
438         } else {
439             // ドラムが動き出した初回：即時同期
440             drumNotStarted = false;
441             globalTick = (uint32_t)((int32_t)globalTick + diff);
442             lastDiff = diff;
443             Serial.print(F("[R]_ドラム起動検知:_即時同期_globalTick="));
444             Serial.println(globalTick);
445             // 以降は通常補正も適用
446             if (state >= 1) {
447                 // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
448                 if (ticksToCorrect == 0 || abs(diff) >= 24) {
449                     applyDiffCorrection(diff);
450                 } else {
451                     Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
452                 }
453             }
454         }
455     } else {
456         // 通常の同期補正
457         lastDiff = diff;
458         if (state >= 1) {
459             // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
460             if (ticksToCorrect == 0 || abs(diff) >= 24) {
461                 applyDiffCorrection(diff);
462             } else {
463                 Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
464             }
465         }
466     }

```

```

465     }
466   }
467 }
468 break;
469
470 // -----
471 // '0': ドラムからの演奏開始指示
472 //   [0]='0' [1]=songId [2..5]=startGlobalTick(uint32 big-endian)
473 //
474 //   自身のglobalTickと startGlobalTick を比較して:
475 //   ・まだ到達していない → waitingForStart=true で待つ
476 //   ・すでに超過している → 超過分だけ curTick をオフセットして即演奏開始
477 // -----
478 case '0':
479   if (pktSize >= 6) {
480     if(waitingForStart || state == 2) return;
481
482     uint8_t  songId  = packetBuffer[1];
483     uint32_t sgt =
484       ((uint32_t)packetBuffer[2] << 24) |
485       ((uint32_t)packetBuffer[3] << 16) |
486       ((uint32_t)packetBuffer[4] << 8) |
487       (uint32_t)packetBuffer[5];
488
489     currentSongId  = songId;
490     startGlobalTick = sgt;
491
492     Serial.print(F("[0]_songId=")); Serial.print(songId);
493     Serial.print(F("_startGlobalTick=")); Serial.println(sgt);
494
495     // 既に超過していたらスキップして即開始
496     if ((int32_t)(globalTick - sgt) >= 0) {
497       // 超過分だけ curTick を進める
498       uint32_t overrun = globalTick - sgt;
499       curTick  = songStartParams[currentSongId].startTick + overrun;
500       scoreIdx = songStartParams[currentSongId].startNoteIdx;
501       // overrunより前のノートをスキップ
502       while (scoreIdx < note_count && Note[scoreIdx].timing < curTick) scoreIdx
503         ++;
504       state = 2;
505       Serial.print(F("[0]_超過済み。overrun=")); Serial.print(overrun);
506       Serial.println(F("_Tick_→_即開始"));
507     } else {
508       waitingForStart = true;
509       Serial.println(F("[0]_開始待機中..."));
510     }
511   }
512   break;

```



```

512
513     case 'V': {
514         uint8_t vol = (payload >= 1 && payload <= 5) ? (payload * 10) : 30;
515         if (vol != curVol) {
516             curVol = vol;
517             Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n')
518         };
519         break;
520     }
521
522     // -----
523     // 'E': 演奏終了通知 (指揮機から)
524     //   drumNotStarted フラグをリセットし、次の演奏開始に備える
525     // -----
526     case 'E':
527         drumNotStarted = true;
528         waitingForStart = false;
529         state = 4; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
530                 1ではなく0
531         curTick = 0;
532         scoreIdx = 0;
533         globalTick = 0;
534         curBpm = 0; // 追加: BPMリセット
535         tickIval = 0; // 追加
536         baseTickIval = 0; // 追加
537         Serial.println(F("[E]_演奏終了。drumNotStarted_リセット"));
538         break;
539
540     // -----
541     // 'S': 演奏開始通知 (指揮機から)
542     //   ドラムに対して演奏への参加を申請する
543     // -----
544     case 'S':
545         drumNotStarted = true;
546         waitingForStart = false;
547         state = 0; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
548                 1ではなく0
549         curTick = 0;
550         scoreIdx = 0;
551         globalTick = 0;
552         currentJumpIdx = 0; // ジャンプインデックスも初期化
553         Serial.println(F("[S]_演奏開始。drumNotStarted_リセット"));
554
555         syncManager.reset();
556
557         sendJPacket();

```

```

557     Serial.println(F("[S]_ドラムに同期予告を送信。"));
558
559     //だーめ
560     // delayがあるため、加速が終わった時点で、バッファに溜まっている2回め・3回め
        のパケットを捨てる（ボタンが動くようにする）
561     // Serial.println("連射された残りのパケットをクリアします。");
562     // while(Udp.parsePacket() > 0) {
563     //     while(Udp.available()) {
564     //         Udp.read();
565     //     }
566     // }
567     break;
568
569     // 追加： マスターからの確定BPMを受信
570     case 'N':
571         if (pktSize >= 2) {
572             uint8_t confirmedBpm = packetBuffer[1];
573             setBpm(confirmedBpm);
574             Serial.print(F("[N]_確定BPM受信:")); Serial.println(confirmedBpm);
575         }
576         break;
577
578
579         default: break;
580     }
581     // } else if (pktSize > 0) {
582     //     while (Udp.available()) Udp.read();
583     // } else {
584     //     Udp.read(); // 空読みして破棄
585     // }
586     pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
587 }
588
589 if (state == 4) return;
590
591 // ---- レーザー検知 ----
592 bool laser = syncManager.update(cdsValue, now);
593
594 // ---- ステートマシン ----
595 switch (state) {
596
597     // --- state0: BPM安定待ち ---
598     case 0:
599         if (laser) {
600             int s = syncManager.getStableCount();
601             uint8_t curLaserBpm = syncManager.getCorrectedBPM();
602             Serial.print(F("[S0]_BPM=")); Serial.print(curLaserBpm);
603             Serial.print(F("_stable=")); Serial.println(s);

```

```

604
605     sendMPacket(curLaserBpm); // M通信でドラムに提案！
606
607     if (s >= MIN_STABLE_COUNT) {
608         state = 1;
609         lastTick = micros();
610         // setBpm(syncManager.getCorrectedBPM());
611
612         Serial.println(F("[S0→1]_BPM安定(N通信待ち)"));
613     }
614 }
615 break;
616
617 // --- state1: 同期中 ---
618 case 1:
619     updateTick();
620     if (laser) {
621         static unsigned long lastSentTime = 0;
622         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
623         // setBpm(syncManager.getCorrectedBPM());
624         // レーザー通過のたびにTパケットを送信してdiff返信を要求
625         sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
626         lastTPacketTick = globalTick;
627         sendTPacket();
628         lastSentTime = now;
629     }
630 }
631
632 // waitingForStart かつ startGlobalTick に到達したら演奏開始
633 if (waitingForStart && (int32_t)(globalTick - startGlobalTick) >= 0) {
634     waitingForStart = false;
635     curTick = songStartParams[currentSongId].startTick;
636     scoreIdx = songStartParams[currentSongId].startNoteIdx;
637     state = 2;
638     Serial.println(F("[S1→2]_演奏開始！"));
639 }
640 break;
641
642 // --- state2: 演奏中 ---
643 case 2:
644     updateTick();
645     if (laser) {
646         static unsigned long lastSentTime = 0;
647         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
648         // setBpm(syncManager.getCorrectedBPM());
649         // レーザー通過のたびにTパケットを送信してdiff返信を要求

```

```

650         sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
651         lastTPacketTick = globalTick;
652         sendTPacket();
653         lastSentTime = now;
654     }
655 }
656 break;
657
658 // --- state4: システム停止（笑）中 ---
659 // case 4: break;
660 }
661 }
662
663 // =====
664 // diff補正によるtickIval算出
665 // diff = drumGlobalTick - myGlobalTick
666 // diff > 0: 自分が遅れている → 速く進む (tickIvalを小さく)
667 // diff < 0: 自分が早すぎる → 遅く進む (tickIvalを大きく)
668 // =====
669 void applyDiffCorrection(int32_t diff) {
670     if (baseTickIval == 0) return;
671
672     float SYNC_READY_THRESHOLD = 50 / baseTickIval;
673
674     if (abs(diff) <= SYNC_READY_THRESHOLD) {
675         // 不感帯：ズレ±が2 Tick以内なら何もしない（ネットワーク遅延のブレを吸収）
676         tickIval = baseTickIval;
677         ticksToCorrect = 0;
678     } else if (diff <= -24) {
679         // 大幅に早い → ×2 で -diff Tick間遅らせる
680         tickIval = baseTickIval * 2;
681         ticksToCorrect = -diff;
682         Serial.println(F("[SYNC]_大幅早い:×2"));
683     } else if (diff >= 24) {
684         // 大幅に遅い → ×2/3 で ×diff3 Tick間速める
685         tickIval = baseTickIval * 2 / 3;
686         ticksToCorrect = diff * 3;
687         Serial.println(F("[SYNC]_大幅遅い:×2/3"));
688     } else {
689         // 小さいズレ → 次の48Tickでdiffを吸収する比率補正
690         tickIval = (uint32_t)((48UL * baseTickIval) / (48 + diff));
691         ticksToCorrect = 48 + diff;
692         Serial.print(F("[SYNC]_微補正_diff=")); Serial.println(diff);
693     }
694
695     uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
696     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
        ('\n');

```

```

697 }
698
699 // =====
700 // BPM更新
701 // =====
702 void setBpm(uint8_t bpm) {
703     if (bpm == curBpm) return;
704     curBpm      = bpm;
705     uint32_t oldBaseTickIval = baseTickIval;
706     baseTickIval = 60000000UL / (uint32_t)bpm / 24;
707
708     // if (ticksToCorrect > 0) {
709     //     // BPM変化後も補正を継続する場合は lastDiff で再計算
710     //     applyDiffCorrection(lastDiff);
711     //     Serial.println(F("[BPM変化] 補正を再計算"));
712     if (oldBaseTickIval > 0 && ticksToCorrect > 0) {
713         // 昔のズレ(lastDiff)で計算し直すのではなく、現在の補正スピードを新しいBPMの比率
714         // に合わせてスケールさせるだけ
715         tickIval = (tickIval * baseTickIval) / oldBaseTickIval;
716         Serial.println(F("[BPM変化]_補正スピードを新BPMにスケール"));
717     } else {
718         tickIval = baseTickIval;
719     }
720
721     uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
722     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
723         ('\n');
724 }
725
726 // =====
727 // Tパケット送信 (自身のglobalTickをドラムへ)
728 // [0]='T' [1..4]=globalTick(uint32 big-endian) [5]=MY_INST_ID
729 // =====
730 void sendTPacket() {
731     Udp.beginPacket(drumIP, LOCAL_PORT);
732     Udp.write('T');
733     Udp.write((uint8_t)((globalTick >> 24) & 0xFF));
734     Udp.write((uint8_t)((globalTick >> 16) & 0xFF));
735     Udp.write((uint8_t)((globalTick >> 8) & 0xFF));
736     Udp.write((uint8_t)( globalTick      & 0xFF));
737     Udp.write(MY_INST_ID);
738     Udp.endPacket();
739     lastTPacketMicroTime = micros();
740 }
741
742 void sendJPacket() {
743     for(int i=0;i<3;i++){
744         Udp.beginPacket(drumIP, LOCAL_PORT);

```

```

743     Udp.write('J');
744     Udp.write(MY_INST_ID);
745     Udp.endPacket();
746     delay(3); // ☒ Wi-Fiのバッファ溢れを防ぐための必須の遅延
747 }
748 }
749
750 // ☒☒ 4. ファイルの下の方、sendTPacket() の下あたりに追加 ☒☒
751 // =====
752 // M通信送信（自身のレーザーBPMをドラムへ提案）
753 // [0]='M' [1]=bpm(uint8) [2]=MY_INST_ID
754 // =====
755 void sendMPacket(uint8_t bpm) {
756     // for(int i=0;i<3;i++){
757     Udp.beginPacket(drumIP, LOCAL_PORT);
758     Udp.write('M');
759     Udp.write(bpm);
760     Udp.write(MY_INST_ID);
761     Udp.endPacket();
762     // }
763 }
764
765 // =====
766 // Tick管理
767 // =====
768 void updateTick() {
769     if (tickIval == 0) return;
770     uint32_t now = micros();
771     while (now - lastTick >= tickIval) {
772         uint32_t currentIval = tickIval; // ★ループ突入時の値を保存
773
774         globalTick++;
775
776         // 補正カウントダウン
777         if (ticksToCorrect > 0) {
778             ticksToCorrect--;
779             if (ticksToCorrect == 0) {
780                 tickIval = baseTickIval;
781                 Serial.println(F("[SYNC]_補正完了"));
782             }
783         }
784
785         if (state == 2) {
786             // ジャンプ（ループ）処理の判定
787             if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
                currentSongId][currentJumpIdx][0]) {
788                 curTick = jumpTable[currentSongId][currentJumpIdx][1];
789                 scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];

```

```

790         currentJumpIdx++;
791
792         if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
793             ][0] == -1) {
794             currentJumpIdx = 0;
795         }
796     }
797     sendPacket();
798     curTick++;
799 }
800
801     lastTick += currentIval; // ★書き換えられていても元の値を足す
802 }
803 }
804
805 // =====
806 // ノート送信 (Processingへ)
807 // =====
808 void sendPacket() {
809     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
810         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
811         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
812         Serial.print('N'); Serial.print(',');
813         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
814         Serial.print(Note[scoreIdx].duration); Serial.print(',');
815         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
816         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
817         Serial.println(Note[scoreIdx].type);
818         scoreIdx++;
819     }
820 }

```

F.4 strings_0706.ino

コード 77 strings_0706.ino (楽譜データ省略)

```

1  #include <WiFiS3.h>
2  #include <WiFiUdp.h>
3
4  const char SSID[] = "hackathon001-WPA2";
5  const char PASS[] = "hackathon001";
6
7  // ★楽器ごとに変更 (ピアノ=12, ストリングス=13, フルート=14)
8  IPAddress localIP(192, 168, 11, 13);
9  IPAddress drumIP (192, 168, 11, 11);
10 const uint16_t LOCAL_PORT = 5005;
11

```

```

12 // ★楽器ID (0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート)
13 const uint8_t MY_INST_ID = 2;
14
15 WiFiUDP Udp;
16 uint8_t packetBuffer[32];
17
18 const int CDS_PIN = A0;
19 const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
20 const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
21 const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
22 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
23 const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
24 const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
25 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
26 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
27 const unsigned long DEBOUNCE_MS = 20;
28 const int BPM_LIST[] = { 60, 90, 120, 150, 180 };
29 // 段階1~5(BPM_LISTと対応)の実測の1周あたり時間[ms]。理論値(30000/BPM)では観覧車の
30 // 実際の回転速度と合わないことが判明したため、実測値でBPMを判定する。
31 const float ROTATION_PERIOD_LIST[] = {230, 180, 140, 110, 80}; //pwmと大体対応させてい
    る
32 const int MIN_STABLE_COUNT = 3;
33
34 // =====
35 // 楽譜データ (★楽器ごとに書き換える★)
36 // =====
37 struct NoteData {
38     uint16_t timing;
39     uint16_t duration;
40     uint8_t pitch;
41     uint8_t startVelocity;
42     uint8_t endVelocity;
43     uint8_t type;
44 };
45
46 const NoteData Note[] = {
47
48     // 【1小節目】
49     {0, 24, 72, 80, 80, 0}, // [0] C5
50     {24, 24, 74, 80, 80, 0}, // [1] D5
51     {48, 24, 76, 80, 80, 0}, // [2] E5
52     {72, 24, 77, 80, 80, 0}, // [3] F5
53
54     // 【2小節目】
55     {96, 24, 76, 80, 80, 0}, // [4] E5
56     {120, 24, 74, 80, 80, 0}, // [5] D5
57     {144, 36, 72, 80, 80, 0}, // [6] C5
58

```



```

59 // ... (以下、同様の形式の楽譜データが続くため省略) ...
60 };
61
62 const uint16_t note_count = sizeof(Note) / sizeof(Note[0]); // 音符の数(配列に含まれて
    いる合計/音符1つあたりの要素数)
63 uint16_t scoreIdx = 0;
64
65 // =====
66 // Tick管理変数
67 // =====
68 uint32_t globalTick = 0;
69 uint32_t curTick = 0; // 楽譜用Tick
70 uint32_t lastTick = 0;
71
72 uint32_t baseTickIval = 0; // BPMから計算した基本interval  $\mu$  [s/tick]
73 uint32_t tickIval = 0; // 実際に使うinterval (同期補正あり)
74 int32_t ticksToCorrect = 0; // 補正残りTick数
75 int32_t lastDiff = 0; // 直近のdiff (BPM変化時の再計算用)
76 uint32_t lastTPacketMicroTime = 0; // 直近のTパケット送信時の時刻Micro
77 uint32_t lastTPacketTick = 0; // 直近のTパケット送信時のglobalTick (drumNotStarted判
    定用)

78
79 // =====
80 // ドラム起動前同期フラグ
81 // true : ドラムがまだglobalTickを加算していない (-diff == globalTick が続いている)
82 // false : ドラムが動き出した (初回のみ即時同期を実施済み)
83 // E通信を受信したらtrueに戻す
84 // =====
85 bool drumNotStarted = true;
86
87 uint8_t curBpm = 0;
88 uint8_t curVol = 50;
89
90 // 演奏管理
91 bool waitingForStart = false; // 0通信を受信して演奏開始globalTickを待っている
92 uint32_t startGlobalTick = 0; // ドラムから指定された演奏開始globalTick
93 uint8_t currentSongId = 0;
94
95 // =====
96 // SyncManager (レーザー検知・BPM安定判定)
97 // =====
98 class SyncManager {
99 public:
100     SyncManager()
101         : lastDetectTime(0), correctedBPM(0),
102         stableCount(0), prevCorrectedBPM(0), laserOn(false),

```

```

103     baseline(0), baselineReady(false),
104     onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
105     pulsePeak(0), peakIndex(0), peakCount(0),
106     calibrated(false), startupCalibrated(false), waitingForRelease(false),
107     calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
108     clearPeakHistory();
109 }
110
111 void reset() {
112     lastDetectTime = 0;
113     correctedBPM = 0;
114     stableCount = 0;
115     prevCorrectedBPM = 0;
116     laserOn = false;
117     baseline = 0;
118     baselineReady = false;
119     onThreshold = CDS_ON_THRESHOLD;
120     offThreshold = CDS_OFF_THRESHOLD;
121     pulsePeak = 0;
122     peakIndex = 0;
123     peakCount = 0;
124     calibrated = false;
125     startupCalibrated = false;
126     waitingForRelease = false;
127     calibrationStartMs = 0;
128     startupMin = 1023;
129     releaseThreshold = 0;
130     clearPeakHistory();
131 }
132
133 bool update(int cdsValue, unsigned long nowMs) {
134     if (calibrationStartMs == 0) calibrationStartMs = nowMs;
135     if (!detectLight(cdsValue, nowMs)) return false;
136     if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
137     if (lastDetectTime != 0) {
138         unsigned long interval = nowMs - lastDetectTime;
139         int newBPM = intervalToBpm((float)interval);
140         if (newBPM == prevCorrectedBPM) stableCount++;
141         else stableCount = 1;
142         prevCorrectedBPM = newBPM;
143         correctedBPM = newBPM;
144     }
145     lastDetectTime = nowMs;
146     return true;
147 }
148
149 int getCorrectedBPM() const { return correctedBPM; }
150 int getStableCount() const { return stableCount; }

```

```

151
152 private:
153     unsigned long lastDetectTime;
154     int correctedBPM;
155     int stableCount;
156     int prevCorrectedBPM;
157     bool laserOn;
158     int baseline;
159     bool baselineReady;
160     int onThreshold;
161     int offThreshold;
162     int pulsePeak;
163     int peakHistory[CDS_PEAK_HISTORY];
164     uint8_t peakIndex;
165     uint8_t peakCount;
166     bool calibrated;
167     bool startupCalibrated;
168     bool waitingForRelease;
169     unsigned long calibrationStartMs;
170     int startupMin;
171     int releaseThreshold;
172
173 bool detectLight(int v, unsigned long nowMs) {
174     if (!baselineReady) {
175         baseline = v;
176         startupMin = v;
177         baselineReady = true;
178         updateFallbackThresholds();
179     }
180
181     // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
182     if (!startupCalibrated) {
183         if (v < startupMin) startupMin = v;
184         baseline = startupMin;
185         updateFallbackThresholds();
186         if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
187         startupCalibrated = true;
188         waitingForRelease = (v >= onThreshold);
189         releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
190             : offThreshold;
191         return false;
192     }
193
194     if (waitingForRelease) {
195         if (v <= releaseThreshold) {
196             baseline = v;
197             updateFallbackThresholds();
198             waitingForRelease = false;
199         }
200     }

```

```

198     }
199     return false;
200 }
201
202 if (!laserOn) {
203     if (v < baseline) baseline = v;
204     else baseline = (baseline * 31 + v) / 32;
205     if (!calibrated) updateFallbackThresholds();
206 }
207
208 if (!laserOn && v >= onThreshold) {
209     laserOn = true;
210     pulsePeak = v;
211     return true;
212 }
213
214 if (laserOn) {
215     if (v > pulsePeak) pulsePeak = v;
216     if (v <= offThreshold) {
217         laserOn = false;
218         registerPeak(pulsePeak);
219     }
220 }
221 return false;
222 }
223
224 void registerPeak(int peak) {
225     if (peak - baseline < CDS_MIN_DELTA) return;
226
227     peakHistory[peakIndex] = peak;
228     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
229     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
230     if (peakCount < 3) return;
231
232     long sum = 0;
233     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
234     int peakAvg = sum / peakCount;
235     int amplitude = peakAvg - baseline;
236     if (amplitude < CDS_MIN_DELTA) return;
237
238     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
239     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);
240
241     if (!calibrated) {
242         onThreshold = targetOn;
243         offThreshold = targetOff;

```

```

244     calibrated = true;
245 } else {
246     onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
247     offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
248 }
249 if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
250 }
251
252 void updateFallbackThresholds() {
253     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
254     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
        );
255 }
256
257 int limitStep(int current, int target, int limit) {
258     if (target > current + limit) return current + limit;
259     if (target < current - limit) return current - limit;
260     return target;
261 }
262
263 void clearPeakHistory() {
264     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
265 }
266
267 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
268 int intervalToBpm(float intervalMs) {
269     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
270     for (int i = 1; i < 5; i++) {
271         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
272         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
273     }
274     return best;
275 }
276 };
277
278 SyncManager syncManager;
279
280 // =====
281 // state:
282 // 0 = 待機 (BPM安定待ち)
283 // 1 = BPM安定、ドラムと同期中 (Tパケット送信・diff収束待ち)
284 // 2 = 演奏中
285 // 4 = システム停止中
286 // =====
287 int state = 4;
288
289
290

```

```

291 // =====
292 // 楽譜遷移（ジャンプ）定義
293 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
    scoreIdx]
294 // 終端は {-1, -1, -1}
295 // =====
296 const int MAX_SONGS = 5;
297 const int MAX_JUMPS = 20;
298 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
299     { // 曲ID: 0 かえるのうた
300         {1536, 0, 0},
301         {-1, -1, -1}
302     },
303     {
304         {2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728, 48},
305         {-1, -1, -1}
306     },
307     {
308         {5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856, 856}, {6216, 6456, 993},
309         {6816, 4608, 651}, {5760, 6816, 1046}, {8832, 4608, 651},
310         {-1, -1, -1}
311     },
312     {
313         {9240, 8856, 1399}, {10008, 9240, 1499}, {10392, 10008, 1669}, {10776, 10392,
314             1783}, {11544, 10776, 1895}, {11544, 10008, 1669}, {10392, 10008, 1669},
315             {10392, 8856, 1399}, {9240, 8856, 1399}, {10008, 11544, 2111}, {11928, 11544,
316             2111}, {11904, 11952, 2225}, {13632, 8856, 1399},
317         {-1, -1, -1}
318     },
319     {
320         {16488, 14952, 3053},
321         {-1, -1, -1}
322     }
323 };
324 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
325
326 // =====
327 // 曲ごとの初期設定データ
328 // =====
329 struct SongStartData {
330     uint32_t startTick;
331     uint16_t startNoteIdx;
332 };
333
334 // Processingの設定に合わせた初期値の定義
335 // インデックスがそのまま currentSongId に対応します

```

```

334 const SongStartData songStartParams[MAX_SONGS] = {
335     {0, 0},          // 曲ID: 0 (かえるのうた)
336     {1536, 29},      // 曲ID: 1 (酒場でブギウギ)
337     {4608, 651},     // 曲ID: 2 (王宮のメヌエット)
338     {8856, 1399},    // 曲ID: 3 (トルコ行進曲)
339     {13632, 2638}    // 曲ID: 4 (序曲)
340 };
341
342
343
344 // =====
345 // 関数宣言
346 // =====
347 void updateTick();
348 void setBpm(uint8_t bpm);
349 void applyDiffCorrection(int32_t diff);
350 void sendTPacket();
351 void sendPacket();
352 void sendMPacket(uint8_t bpm);
353
354 // =====
355 // セットアップ
356 // =====
357 void setup() {
358     Serial.begin(115200);
359     WiFi.config(localIP);
360     WiFi.begin(SSID, PASS);
361     Serial.print("Connecting_WiFi...");
362     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
363     Serial.println("\nWiFi_Connected!");
364     Udp.begin(LOCAL_PORT);
365     Serial.println("state=0_BPM安定待ち");
366 }
367
368 // =====
369 // メインループ
370 // =====
371 void loop() {
372     unsigned long now = millis();
373     int cdsValue = analogRead(CDS_PIN);
374     // Serial.println(cdsValue);
375
376     // ---- UDP受信 ----
377     int pktSize = Udp.parsePacket();
378     while (pktSize > 0) {
379         if (pktSize >= 2) {
380             Udp.read(packetBuffer, sizeof(packetBuffer));
381             char header = (char)packetBuffer[0];

```

```

382     uint8_t payload = packetBuffer[1];
383
384     switch (header) {
385
386         // -----
387         // 'R': ドラムからのdiff返信
388         //   [0]='R' [1..4]=diff(int32 big-endian)
389         //
390         //   ドラム起動前判定：
391         //     -diff == globalTick (つまり globalTick + diff == 0)
392         //     → ドラムはまだglobalTickを加算していない。自身もリセット。
393         //   ドラムが動き出した初回：
394         //     上記条件が初めてfalseになった → globalTick -= diff で即時同期。
395         //   以降は通常のdiff補正 (applyDiffCorrection) へ。
396         // -----
397     case 'R':
398         // 【追加】 停止中 (4) や安定待ち (0) に届いた過去のR通信は「遅延ゴミ (Stale
399         //   Packet)」なので完全無視する！
400
401         if (state == 0 || state == 4) break;
402
403         if (pktSize >= 5) {
404             int32_t rawDiff =
405                 ((int32_t)(int8_t)packetBuffer[1] << 24) |
406                 ((int32_t)packetBuffer[2] << 16) |
407                 ((int32_t)packetBuffer[3] << 8) |
408                 (int32_t)packetBuffer[4];
409
410             // 遅延 (レイテンシ) 補正の計算
411             uint32_t rtt = micros() - lastTPacketMicroTime;
412             // 【安全対策】 往復時間が長すぎる (例:200ms以上) 場合は古いパケットとみなして
413             //   破棄
414             if (rtt > 1000 * 1000) {
415                 // Serial.println(F("[R] パケット遅延過大のためスキップ"));
416                 break;
417             }
418
419             uint32_t latencyMicros = rtt / 2; // 片道遅延
420             int32_t latencyTicks = 0;
421
422             if (baseTickIval > 0) {
423                 // 四捨五入してTick数に変換: (値 + 割る数の半分) / 割る数
424                 latencyTicks = (int32_t)((latencyMicros + (baseTickIval / 2)) /
425                     baseTickIval);
426
427                 // ドラム機の勘違い (遅延分) を差し引いて真のズレを計算
428                 int32_t diff = rawDiff - latencyTicks;

```



```

427 // Serial.print(F("[R] rawDiff=")); Serial.print(rawDiff);
428 // Serial.print(F(" latencyTicks=")); Serial.print(latencyTicks);
429 // Serial.print(F(" trueDiff=")); Serial.println(diff);
430
431 if (drumNotStarted) {
432     if ((int32_t)lastTPacketTick + diff == 0) {
433         // Tパケット送信時点でドラムはまだ0。自身もリセットして足並みを揃える
434         globalTick = 0;
435         lastTick = micros(); // Tick計測の基点もリセット
436         // Serial.println(F("[R] ドラム未起動: globalTick リセット"));
437     } else {
438         // ドラムが動き出した初回：即時同期
439         drumNotStarted = false;
440         globalTick = (uint32_t)((int32_t)globalTick + diff);
441         lastDiff = diff;
442         // Serial.print(F("[R] ドラム起動検知: 即時同期 globalTick="));
443         // Serial.println(globalTick);
444         // 以降は通常補正も適用
445         if (state >= 1) {
446             // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
447             if (ticksToCorrect == 0 || abs(diff) >= 24) {
448                 applyDiffCorrection(diff);
449             } else {
450                 // Serial.println(F("[R] 補正中のため新規R通信の適用をスキップ"));
451             }
452         }
453     }
454 } else {
455     // 通常の同期補正
456     lastDiff = diff;
457     if (state >= 1) {
458         // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
459         if (ticksToCorrect == 0 || abs(diff) >= 24) {
460             applyDiffCorrection(diff);
461         } else {
462             // Serial.println(F("[R] 補正中のため新規R通信の適用をスキップ"));
463         }
464     }
465 }
466 }
467 break;
468
469 // -----
470 // '0': ドラムからの演奏開始指示
471 // [0]='0' [1]=songId [2..5]=startGlobalTick(uint32 big-endian)
472 //

```

```

473 // 自身のglobalTickと startGlobalTick を比較して：
474 // ・まだ到達していない → waitingForStart=true で待つ
475 // ・すでに超過している → 超過分だけ curTick をオフセットして即演奏開始
476 // -----
477 case '0':
478     if (pktSize >= 6) {
479         if(waitingForStart || state == 2) return;
480
481         uint8_t songId = packetBuffer[1];
482         uint32_t sgt =
483             ((uint32_t)packetBuffer[2] << 24) |
484             ((uint32_t)packetBuffer[3] << 16) |
485             ((uint32_t)packetBuffer[4] << 8) |
486             (uint32_t)packetBuffer[5];
487
488         currentSongId = songId;
489         startGlobalTick = sgt;
490
491         // Serial.print(F("[0] songId=")); Serial.print(songId);
492         // Serial.print(F(" startGlobalTick=")); Serial.println(sgt);
493
494         // 既に超過していたらスキップして即開始
495         if ((int32_t)(globalTick - sgt) >= 0) {
496             // 超過分だけ curTick を進める
497             uint32_t overrun = globalTick - sgt;
498             curTick = songStartParams[currentSongId].startTick + overrun;
499             scoreIdx = songStartParams[currentSongId].startNoteIdx;
500             // overrunより前のノートをスキップ
501             while (scoreIdx < note_count && Note[scoreIdx].timing < curTick) scoreIdx
502                 ++;
503             state = 2;
504             // Serial.print(F("[0] 超過済み。overrun=")); Serial.print(overrun);
505             // Serial.println(F(" Tick → 即開始"));
506         } else {
507             waitingForStart = true;
508             // Serial.println(F("[0] 開始待機中..."));
509         }
510     }
511     break;
512 case 'V': {
513     uint8_t vol = (payload >= 1 && payload <= 5) ? (payload * 10) : 30;
514     if (vol != curVol) {
515         curVol = vol;
516         Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n')
517         ;
518     }
519     break;

```

```

519     }
520
521     // -----
522     // 'E': 演奏終了通知（指揮機から）
523     //   drumNotStarted フラグをリセットし、次の演奏開始に備える
524     // -----
525     case 'E':
526         drumNotStarted = true;
527         waitingForStart = false;
528         state          = 4; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
529                             1ではなく0
530         curTick         = 0;
531         scoreIdx        = 0;
532         globalTick      = 0;
533         curBpm          = 0; // 追加: BPMリセット
534         tickIval        = 0; // 追加
535         baseTickIval    = 0; // 追加
536         // Serial.println(F("[E] 演奏終了。drumNotStarted リセット"));
537         break;
538
539     // -----
540     // 'S': 演奏開始通知（指揮機から）
541     //   ドラムに対して演奏への参加を申請する
542     // -----
543     case 'S':
544         drumNotStarted = true;
545         waitingForStart = false;
546         state          = 0; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
547                             1ではなく0
548         curTick         = 0;
549         scoreIdx        = 0;
550         globalTick      = 0;
551         currentJumpIdx  = 0; // ジャンプインデックスも初期化
552         // Serial.println(F("[S] 演奏開始。drumNotStarted リセット"));
553
554         syncManager.reset();
555
556         sendJPacket();
557
558         // Serial.println(F("[S] ドラムに同期予告を送信。"));
559
560         // だーめ
561         // delayがあるため、加速が終わった時点で、バッファに溜まっている2回め・3回め
562         //   のパケットを捨てる（ボタンが動くようにする）
563         // Serial.println("連射された残りのパケットをクリアします。");
564         // while(Udp.parsePacket() > 0) {
565         //     while(Udp.available()) {
566         //         Udp.read();

```

```

564         // }
565         // }
566         break;
567
568         // 追加: マスターからの確定BPMを受信
569     case 'N':
570         if (pktSize >= 2) {
571             uint8_t confirmedBpm = packetBuffer[1];
572             setBpm(confirmedBpm);
573             // Serial.print(F("[N] 確定BPM受信: ")); Serial.println(confirmedBpm);
574         }
575         break;
576
577     default: break;
578 }
579 // } else if (pktSize > 0) {
580 //     while (Udp.available()) Udp.read();
581 // } else {
582     Udp.read(); // 空読みして破棄
583 }
584 pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
585 }
586
587
588 if (state == 4) return;
589
590 // ---- レーザー検知 ----
591 bool laser = syncManager.update(cdsValue, now);
592
593 // ---- ステートマシン ----
594 switch (state) {
595
596     // --- state0: BPM安定待ち ---
597     case 0:
598         if (laser) {
599             int s = syncManager.getStableCount();
600             uint8_t curLaserBpm = syncManager.getCorrectedBPM();
601             // Serial.print(F("[S0] BPM=")); Serial.print(curLaserBpm);
602             // Serial.print(F(" stable=")); Serial.println(s);
603
604             sendMPacket(curLaserBpm); // M通信でドラムに提案！
605
606             if (s >= MIN_STABLE_COUNT) {
607                 state = 1;
608                 lastTick = micros();
609                 // setBpm(syncManager.getCorrectedBPM());
610
611                 // Serial.println(F("[S0→1] BPM安定(N通信待ち)"));

```

```

612     }
613 }
614 break;
615
616 // --- state1: 同期中 ---
617 case 1:
618     updateTick();
619     if (laser) {
620         static unsigned long lastSentTime = 0;
621         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
622             // setBpm(syncManager.getCorrectedBPM());
623             // レーザー通過のたびにTパケットを送信してdiff返信を要求
624             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
625             lastTPacketTick = globalTick;
626             sendTPacket();
627             lastSentTime = now;
628         }
629     }
630
631     // waitingForStart かつ startGlobalTick に到達したら演奏開始
632     if (waitingForStart && (int32_t)(globalTick - startGlobalTick) >= 0) {
633         waitingForStart = false;
634         curTick = songStartParams[currentSongId].startTick;
635         scoreIdx = songStartParams[currentSongId].startNoteIdx;
636         state = 2;
637         // Serial.println(F("[S1→2] 演奏開始！"));
638     }
639     break;
640
641 // --- state2: 演奏中 ---
642 case 2:
643     updateTick();
644     if (laser) {
645         static unsigned long lastSentTime = 0;
646         if (now - lastSentTime > 200) { // ☒ 200ms以内の連続送信（チャタリング）をブ
            ロック
647             // setBpm(syncManager.getCorrectedBPM());
648             // レーザー通過のたびにTパケットを送信してdiff返信を要求
649             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案！
650             lastTPacketTick = globalTick;
651             sendTPacket();
652             lastSentTime = now;
653         }
654     }
655     break;
656
657 // --- state4: システム停止（笑）中 ---

```

```

658     // case 4: break;
659 }
660 }
661
662 // =====
663 // diff補正によるtickIval算出
664 // diff = drumGlobalTick - myGlobalTick
665 // diff > 0: 自分が遅れている → 速く進む (tickIvalを小さく)
666 // diff < 0: 自分が早すぎる → 遅く進む (tickIvalを大きく)
667 // =====
668 void applyDiffCorrection(int32_t diff) {
669     if (baseTickIval == 0) return;
670
671     // if(state == 1) {
672     //     globalTick += diff;
673     //     return;
674     // }
675
676     float SYNC_READY_THRESHOLD = 60 * 1000 / baseTickIval;
677
678     if (abs(diff) <= SYNC_READY_THRESHOLD) {
679         // 不感帯：ズレ±が2 Tick以内なら何もしない（ネットワーク遅延のブレを吸収）
680         tickIval = baseTickIval;
681         ticksToCorrect = 0;
682     } else if (diff <= -12) {
683         // 大幅に早い → ×2 で -diff Tick間遅らせる
684         tickIval = baseTickIval * 2;
685         ticksToCorrect = -diff;
686         // Serial.println(F("[SYNC] 大幅早い: ×2"));
687     } else if (diff >= 24) {
688         // 大幅に遅い → ×1/2 で ×diff2 Tick間速める
689         tickIval = baseTickIval * 1 / 2;
690         ticksToCorrect = diff * 2;
691         // Serial.println(F("[SYNC] 大幅遅い: ×2/3"));
692     } else {
693         // 小さいズレ → 次の24Tickでdiffを吸収する比率補正
694         tickIval = (uint32_t)((24UL * baseTickIval) / (24 + diff));
695         ticksToCorrect = 24 + diff;
696         // Serial.print(F("[SYNC] 微補正 diff=")); Serial.println(diff);
697     }
698
699     uint8_t sendProcessingBpm = (uint8_t)(6000000UL / (tickIval * 24));
700     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print(
        '\n');
701 }
702
703 // =====
704 // BPM更新

```

```

705 // =====
706 void setBpm(uint8_t bpm) {
707     if (bpm == curBpm) return;
708     curBpm = bpm;
709     uint32_t oldBaseTickIval = baseTickIval;
710     baseTickIval = 60000000UL / (uint32_t)bpm / 24;
711
712     // if (ticksToCorrect > 0) {
713     //     // BPM変化後も補正を継続する場合は lastDiff で再計算
714     //     applyDiffCorrection(lastDiff);
715     //     Serial.println(F("[BPM変化] 補正を再計算"));
716     if (oldBaseTickIval > 0 && ticksToCorrect > 0) {
717         // 昔のズレ(lastDiff)で計算し直すのではなく、現在の補正スピードを新しいBPMの比率
718         // に合わせてスケールさせるだけ
719         tickIval = (tickIval * baseTickIval) / oldBaseTickIval;
720         // Serial.println(F("[BPM変化] 補正スピードを新BPMにスケール"));
721     } else {
722         tickIval = baseTickIval;
723     }
724
725     uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
726     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
727         ('\n');
728 }
729 // =====
730 // Tパケット送信 (自身のglobalTickをドラムへ)
731 // [0]='T' [1..4]=globalTick(uint32 big-endian) [5]=MY_INST_ID
732 // =====
733 void sendTPacket() {
734     Udp.beginPacket(drumIP, LOCAL_PORT);
735     Udp.write('T');
736     Udp.write((uint8_t)((globalTick >> 24) & 0xFF));
737     Udp.write((uint8_t)((globalTick >> 16) & 0xFF));
738     Udp.write((uint8_t)((globalTick >> 8) & 0xFF));
739     Udp.write((uint8_t)( globalTick & 0xFF));
740     Udp.write(MY_INST_ID);
741     Udp.endPacket();
742     lastTPacketMicroTime = micros();
743 }
744 void sendJPacket() {
745     for(int i=0;i<3;i++){
746         Udp.beginPacket(drumIP, LOCAL_PORT);
747         Udp.write('J');
748         Udp.write(MY_INST_ID);
749         Udp.endPacket();
750         delay(3); // ☒ Wi-Fiのバッファ溢れを防ぐための必須の遅延

```

```

751     }
752 }
753
754 // ☒☒ 4. ファイルの下の方、sendTPacket() の下あたりに追加 ☒☒
755 // =====
756 // M通信送信（自身のレーザーBPMをドラムへ提案）
757 //   [0]='M' [1]=bpm(uint8) [2]=MY_INST_ID
758 // =====
759 void sendMPacket(uint8_t bpm) {
760     // for(int i=0;i<3;i++){
761     Udp.beginPacket(drumIP, LOCAL_PORT);
762     Udp.write('M');
763     Udp.write(bpm);
764     Udp.write(MY_INST_ID);
765     Udp.endPacket();
766     // }
767 }
768
769 // =====
770 // Tick管理
771 // =====
772 void updateTick() {
773     if (tickIval == 0) return;
774     uint32_t now = micros();
775     while (now - lastTick >= tickIval) {
776         uint32_t currentIval = tickIval; // ★ループ突入時の値を保存
777
778         globalTick++;
779
780         // 補正カウントダウン
781         if (ticksToCorrect > 0) {
782             ticksToCorrect--;
783             if (ticksToCorrect == 0) {
784                 tickIval = baseTickIval;
785                 // Serial.println(F("[SYNC] 補正完了"));
786             }
787         }
788
789         if (state == 2) {
790             // ジャンプ（ループ）処理の判定
791             if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
                currentSongId][currentJumpIdx][0]) {
792                 curTick = jumpTable[currentSongId][currentJumpIdx][1];
793                 scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];
794                 currentJumpIdx++;
795
796                 if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
                    ][0] == -1) {

```



```

797         currentJumpIdx = 0;
798     }
799 }
800
801     sendPacket();
802     curTick++;
803 }
804
805     lastTick += currentIval; // ★書き換えられていても元の値を足す
806 }
807 }
808
809 // =====
810 // ノート送信 (Processingへ)
811 // =====
812 void sendPacket() {
813     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
814         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
815         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
816         Serial.print('N'); Serial.print(',');
817         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
818         Serial.print(Note[scoreIdx].duration); Serial.print(',');
819         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
820         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
821         Serial.println(Note[scoreIdx].type);
822         scoreIdx++;
823     }
824 }

```